

Riassunto Sistemi Operativi

Sommario

Capitolo 1 – Intro.....	7
Cos'è un Sistema Operativo	7
Il Sistema Operativo dal punto di vista Utente e Progettista	7
Obiettivi di un Sistema Operativo	8
Funzionamento di un Sistema Operativo	8
Intro ai Sistemi Operativi.....	10
Eventi e Interrupt	10
Utilizzo efficace di un computer e portabilità ed estendibilità dei SO	10
Gestire elaborazioni degli Utenti	10
Processi e thread.....	10
Sincronizzazione di processi.....	10
Scambio di messaggi.....	11
Scheduling.....	11
Deadlock	11
Gestione della Memoria.....	11
Memoria virtuale	11
Gestione dei file e dei dispositivi di I/O	12
Capitolo 2 - Il SO, il computer e i programmi utente.....	12
Principi fondamentali del funzionamento di un SO.....	12
Il Computer.....	13
La CPU.....	13
Memory Management Unit (MMU)	14
Gerarchia della memoria.....	14
Memoria cache.....	15
Memoria RAM	16
Protezione della memoria	16
Input/Output.....	17
Interrupt.....	17
Interrupt Annidati	19
Interazione del SO con il computer e i programmi utente	20
System Call	20
Capitolo 3 – Panoramica dei Sistemi Operativi	21
Ambienti di elaborazione e natura delle elaborazioni	21
Ambienti di elaborazione non interattivi	22
Ambienti di elaborazione interattivi	22
Ambienti real-time, distribuiti ed embedded	22
Moderni ambienti di elaborazione.....	22
Efficienza, prestazioni del sistema e servizio per l'utente	22
Efficienza.....	22
Prestazioni del sistema	23
Servizio per l'utente	23
Classi di Sistemi Operativi.....	23
Sistemi di Elaborazione Batch.....	23
Sistemi Multi programmati.....	24
Priorità dei programmi	24
Sistemi time-sharing	25
Swapping dei programmi	25
Sistemi Operativi Real-Time	26
Sistemi Hard e Soft real-time	27
Caratteristiche di un sistema operativo real-time	27

Sistemi Operativi Distribuiti	27
Tecniche speciali di sistemi operativi distribuiti.....	27
Moderni Sistemi Operativi.....	27
Capitolo 4 - Struttura dei Sistemi Operativi.....	28
Funzionamento di un SO.....	28
Struttura di un SO.....	29
Portabilità ed espandibilità dei SO.....	29
Portabilità	29
Espandibilità.....	29
Sistemi Operativi con struttura Monolitica	29
Sistemi Operativi strutturati a livelli.....	30
Sistemi Operativi basati su Kernel.....	31
Evoluzione della struttura basata su Kernel.....	32
Sistemi Operativi basati su microKernel	32
Capitolo 5 – Processi e Thread.....	33
Differenza tra processo e programma.....	33
Relazioni tra processi e programmi	33
Processi figli	34
Differenza tra parallelismo e concorrenza	35
Implementazione dei processi	35
Stati del processo e transizioni di stato.....	35
Cause delle transizioni di stato dei processi	36
Processi sospesi	36
Il contesto del processo e il process control block	37
Salvataggio del contesto, scheduling e dispatching	37
Commutazione tra processi e di contesto (context switch).....	38
Gestione degli eventi.....	38
Eventi relativi alla creazione, terminazione e prelazione del processo	38
Creazione di un processo	39
Terminazione di un processo	39
Prelazione di un processo.....	39
Eventi relativi all'utilizzo delle risorse.....	39
Event Control Block (ECB).....	39
Azioni per la gestione di un evento del Kernel	40
Condivisione, comunicazione e sincronizzazione tra processi	41
Segnali.....	41
Thread	41
Stati e transizioni di stato dei thread.....	42
Vantaggi dei thread rispetto ai processi.....	42
Codifica per usare i thread	42
I Thread in C: Thread Posix	43
Thread di livello Kernel, di livello Utente e Ibridi.....	43
Thread di livello Kernel	43
Thread di livello Utente	44
Modelli di Thread Ibridi	45
Casi Studio – Processi e Thread	46
Processi in UNIX	46
Tipi di processi	46
Creazione e terminazione di un processo	46
Attesa della terminazione di un processo	47
Attesa dell'occorrenza degli eventi.....	47
Gestione degli Interrupt	47
System call.....	47
Segnali	47
Stati del processo e transizioni di stato	48
Processi e thread in Linux.....	48

Strutture dati.....	48
Creazione di Processi e Thread.....	49
Terminazione di Processi e Thread.....	49
Miglioramenti con il kernel 2.6.....	49
Relazioni genitore-figlio	49
Stati di un processo	49
Thread in Solaris	50
Gestione dei segnali	50
Stati dei processi e thread kernel.....	50
Scheduler activation	51
Passaggio all'implementazione 1:1	51
Capitolo 6 – Sincronizzazione dei processi: Memoria Condivisa.....	51
Che cos'è la sincronizzazione dei processi?.....	52
Definizione Formale di Processi intercomunicanti.....	52
Requisiti di sincronizzazione nei processi intercomunicanti	52
Race Condition	52
Sincronizzazione per l'accesso ai dati	53
Sezioni Critiche.....	53
Configurazioni delle Sezioni Critiche	53
Effetti delle Sezioni Critiche sulle Prestazioni	54
Proprietà dell'implementazione di una Sezione Critica.....	54
Controllo della sincronizzazione e operazioni indivisibili.....	54
Sincronizzazione tra Processi e Segnalazione	54
Tentativo Maldestro di Sincronizzazione (utilizzo di variabili booleane).....	55
Operazioni Indivisibili come Soluzione	55
Approcci alla Sincronizzazione	56
Attesa Attiva (Busy Waiting).....	56
Blocco e Sospensione del Processo	56
Supporto Hardware per la sincronizzazione dei processi.....	56
Istruzioni indivisibili	56
Variabili di lock	56
Istruzione Test-and-Set (TS).....	56
Istruzione Swap.....	57
Approcci Algoritmici, primitive di sincronizzazione e costrutti di programmazione concorrente	58
Approcci Algoritmici	58
Primitive di sincronizzazione	58
Costrutti di programmazione concorrente.....	58
Problemi Classici di Sincronizzazione dei Processi	58
Produttori-Consumatori con buffer limitati.....	58
Produttore e consumatore con singolo buffer	59
Lettori e Scrittori	60
Filosofi a Cena.....	61
Approccio algoritmico per implementare le sezioni critiche	62
Algoritmo per due processi	62
Algoritmo di Dekker	64
Algoritmo di Peterson.....	65
Algoritmo del Panettiere (Bakery).....	65
Cosa accade se non usiamo l'array choosing?.....	67
Semafori	68
Uso dei semafori nei sistemi concorrenti	69
Semaforo binario	69
Produttori-Consumatori usando i semafori	70
Produttore-Consumatore con buffer singolo.....	71
Produttore-Consumatore ad N Buffer con Semafori	71
Lettori-Scrittori con Semafori.....	72
Lettori-Scrittori con Semafori con Preferenza ai Lettori.....	73
Implementazione dei Semafori.....	75
Implementazione dei semafori (NO valori negativi).....	76
Implementazione dei semafori (SI valori negativi)	76
Implementazione livello kernel, livello utente e ibrida	76

Problema del Barbiere Addormentato	77
Monitor	78
Monitor: Regole per le variabili di condizione	79
Struttura di un Monitor	79
Semaforo Binario con Monitor	80
Esempio di utilizzo di un monitor per la sincronizzazione di 3 processi	81
Esempio: Produttori-Consumatori con Monitor	82
Message Passing	84
Motivazioni	84
Spazio di indirizzamento non condiviso	85
Invio di un Messaggio	85
Primitive di comunicazione	85
Send e Receive Bloccanti e Non Bloccanti	85
Indirizzamento	86
Indirizzamento Diretto	86
Indirizzamento Indiretto	86
Mailbox	86
Creazione e utilizzo di una mailbox	87
Implementazione dello scambio di messaggi	87
Consegna dei messaggi tra processi	88
Strategie di Accodamento	89
Message Passing in UNIX	89
Pipe	90
Coda dei messaggi	90
Socket	90
Mutua Esclusione con Scambio di Messaggi (send non bloccante e receive bloccante)	91
Produttore-Consumatore con Scambio di Messaggi	92
Capitolo 7 – Scheduling	93
Introduzione	93
Terminologia legata allo scheduling	93
Riguardo alla richiesta	93
Riguardo al servizio per l'utente	93
Riguardo alle prestazioni complessive del sistema	93
Tecniche di Scheduling	94
Scheduling basato su priorità	94
Riordino delle richieste	94
Variazione del time slice	94
Politiche di scheduling senza prelazione (non preemptive)	94
Scheduling First-Come, First-Served (FCFS)	94
Scheduling Shortest Job First (SJF)	95
Scheduling Highest Response Ratio Next (HRN)	96
Politiche di Scheduling con prelazione (preemptive)	97
Round – Robin con time-slicing (RR)	97
Scheduling Least Completed Next (LCN)	98
Scheduling Shortest Time to Go (STG)	98
Tipi di Scheduler	99
Scheduler a lungo termine	99
Scheduler a medio termine	99
Scheduler a breve termine	99
Esempio di flusso in un sistema time-sharing	100
Funzionamento dello Scheduler a Breve Termine	100
Scheduling basato su priorità	101
Problemi comuni	101
Scheduling Round-Robin con Time Slice	101
Scheduling Multilivello	101
Scheduling Multilivello Adattivo	102
Scheduling Multi-Level Feedback Queue (MLFQ)	102
Struttura delle code e regole operative	102

Adattamento dinamico: il cuore dell'MLFQ	102
Meccanismo di modifica della priorità.....	102
Comportamenti tipici e interazioni	102
Problemi e criticità dell'MLFQ base	103
Strategie di correzione	103
Regolazione dei time slice	103
Scheduling Real-Time	103
Tipologie di sistemi real-time.....	103
Dipendenze tra processi	103
Approcci di Scheduling nei Sistemi Real-Time	104
Scheduling statico	104
Scheduling basato su priorità.....	104
Scheduling dinamico.....	104
Scheduling con deadline.....	104
Earliest Deadline First (EDF).....	105
Scheduling Rate Monotonic (SRC).....	106
Assegnazione della priorità.....	106
Esempio di esecuzione.....	107
Schedulabilità e limite di utilizzo CPU	107
Capitolo 8 - DEADLOCK	107
Deadlock di risorsa.....	107
Condizioni per il deadlock di risorsa	108
Modello dello stato di allocazione delle risorse	108
Modello a Grafo	108
Modello a Matrice	108
Gestione dei deadlock.....	109
Individuazione e risoluzione dei deadlock.....	109
Algoritmo Individuazione.....	110
Funzionamento pratico dell'algoritmo.....	111
Risoluzione.....	111
Prevenzione dei Deadlock	112
Allocazione Simultanea o Globale.....	113
Ranking delle risorse	113
Evitamento dei deadlock (Algoritmo del banchiere)	114
Schema dell'approccio.....	114
Algoritmo del banchiere	115
1)Inizializzazioni	115
2)Simulazione.....	115
3)Fattibilità.....	115
4)Verifica allocazione sicura	116
5)Allocazione effettiva.....	116
Capitolo 9 – Gestione della memoria	116
Gestione della Gerarchia di Memoria	117
Allocazione Statica e Dinamica della memoria.....	117
Esecuzione dei Programmi	118
Rilocazione.....	118
Linking.....	119
Allocazione della memoria a un processo	119
Stack	119
Heap	119
Il Modello di allocazione della memoria	120
Protezione della memoria	120
Gestione dell'Heap	121
Riuso della memoria	121
Gestione con free list	121
Mantenere una free list.....	121
Tecniche di allocazione mediante free list	122
Frammentazione della memoria.....	123

Unione delle aree di memoria libera.....	123
Boundary Tag.....	123
Compattazione della memoria	124
Buddy System e allocatori potenza del 2.....	125
Allocazione contigua della memoria	126
Allocazione non contigua della memoria.....	126
Indirizzo Logico, Fisico e Traduzione degli Indirizzi	126
Come funziona la traduzione dell'indirizzo	127
Paginazione.....	127
Vantaggi della paginazione.....	128
Svantaggi della paginazione	128
Notazione per la traduzione degli indirizzi:	129
Segmentazione.....	129
Vantaggi della segmentazione	130
Svantaggi della segmentazione	130
Segmentazione con paginazione	130
Capitolo 10 – Memoria Virtuale.....	131
Introduzione	131
Paginazione su richiesta.....	132
Concetti preliminari.....	133
Tabella delle pagine.....	133
Page fault e caricamento su richiesta delle pagine	134
Operazioni di page-in, page-out e sostituzione delle pagine	135
Tempo di accesso effettivo alla memoria (EAT).....	135
Sostituzione delle pagine e Località dei riferimenti	136
Allocazione della memoria per un processo	136
Thrashing	137
Dimensione di pagina ottimale	138
Supporto hardware alla paginazione	138
Translation Look-aside Buffer (TLB).....	139
Implementazione tabelle delle pagine	140
Tabella delle pagine invertita	140
Tabella delle pagine multilivello	142
Politiche di sostituzione delle pagine	143
Strategie di sostituzione: Ottimale, FIFO e LRU	143
Esempio	143
Proprietà dello stack	144
Esempio	145
Strategia Not Recently Used (NRU)	146
Algoritmo di Clock ad una Lancetta.....	146
Algoritmo di Clock a due Lancette.....	147
Thread demoni dell'MMU.....	148
Allocazione della memoria ad un processo	148
Working Set.....	148
Copy On Write.....	149
Capitolo 11 - File System.....	150
Introduzione	150
File	150
Directory.....	151
Allocazione di Spazio su Disco	151
Allocazione Concatenata	152
Allocazione indicizzata.....	152
Affidabilità del file system.....	153
Unix File System (UFS)	154
Berkeley Fast File System (FFS).....	154
Capitolo 12 – Input/Output Control System.....	155
Introduzione	155

Organizzazione dell'I/O.....	156
Operazioni di I/O	156
Dispositivi di I/O.....	157
Hard Disk.....	158
Scheduling del Disco.....	158
Esempio scheduling del Disco	159
First-Come, First Served.....	160
Shortest Seek Time First	161
LOOK.....	162
C-LOOK	163

Capitolo 1 – Intro

Cos'è un Sistema Operativo

Il **sistema operativo** (SO) è un intermediario tra gli utenti e il sistema di elaborazione. Esso fornisce servizi in base alle esigenze degli utenti, che hanno una visione astratta del computer.

I progettisti di SO devono bilanciare tre aspetti: **l'uso efficiente delle risorse, la convenienza per l'utente e la prevenzione delle interferenze.**

L'uso efficiente è cruciale per sistemi dedicati ad applicazioni specifiche, mentre la convenienza è prioritaria nei personal computer; nei sistemi condivisi, entrambi sono importanti. Il SO gestisce programmi e risorse per garantire efficienza e usabilità, mentre le sue funzioni di sicurezza prevengono interferenze.

Il Sistema Operativo dal punto di vista Utente e Progettista

La percezione del S.O dal punto di vista Utente (finale) e del Progettista è diversa, poiché l'utente percepisce un S.O come un mezzo per ottenere un utilizzo specifico di un sistema di elaborazione. Ad esempio, uno studente, potrebbe individuare il S.O come mezzo per navigare in internet; un programmatore o un tecnico, in maniera analoga,

identificano il S.O in base allo scopo per cui utilizzano il computer. Da ciò si intuisce che poiché i loro obiettivi sono differenti, anche la loro percezione del S.O è differente.

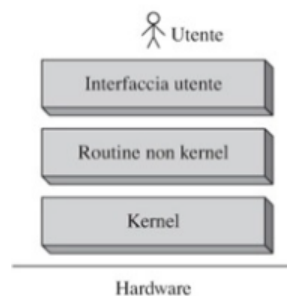
Il **Progettista** invece, ha una sua visione astratta del S.O, in termini di struttura e relazioni tra le sue parti costituenti che sono:

- **Interfaccia utente:** Accetta comandi per eseguire programmi e usa le risorse e i servizi forniti dal sistema operativo. Può essere a linea di comando (Linux o Unix) o una GUI (Windows).
- **Routine non di sistema:** Sono funzioni richiamate dall'interfaccia utente e servono a eseguire i comandi dell'utente relativi all'esecuzione dei programmi e all'uso delle risorse del computer. Non fanno parte del Kernel, ma operano a livello di interfaccia utente, consentendo attività come avviare o terminare programmi, gestire file e cartelle, accedere alle periferiche ed eseguire comandi di sistema.
Ad esempio, in Linux digitando sul terminale il comando

```
cp documento.txt backup.txt
```

cp permette di avviare una routine che esegue la copia del file.

- **Kernel:** È il cuore del S.O, controlla il computer e fornisce un insieme di funzioni e servizi per utilizzare la CPU, la memoria e le altre risorse del computer. Le funzioni Kernel sono richiamate dalle routine non-kernel e dai programmi utente.



Quindi, dal **punto di vista del progettista**, il S.O è una **collezione di routine** che facilitano l'esecuzione dei programmi utente e l'uso delle risorse del computer, organizzate gerarchicamente a livelli in cui ogni routine di livello superiore utilizza le operazioni delle routine di livello immediatamente inferiore, creando una sorta di astrazione tra i livelli adiacenti.

Obiettivi di un Sistema Operativo

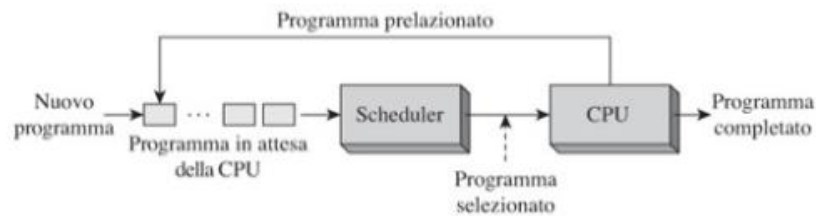
Gli obiettivi di un SO sono:

- **Uso efficiente:** Per garantire buone prestazioni, il SO deve ridurre al minimo lo spreco di risorse sia da parte dei programmi che del proprio overhead. L'**efficienza delle risorse** può essere migliorata monitorandone l'utilizzo e applicando azioni correttive quando necessario. Tuttavia, il monitoraggio stesso genera **overhead**. Per limitarlo, i SO si concentrano sull'**efficienza di poche risorse chiave**, come CPU e RAM, oppure evitano il monitoraggio diretto, gestendo programmi e risorse in modo da garantire comunque un'elevata efficienza.
- **Convenienza per l'utente:** Questo aspetto riguarda diversi fattori tra cui:
 - **Garantire il soddisfacimento della necessità**, permettendo di eseguire programmi e utilizzare il file system;
 - **Garantire un buon servizio**, offrendo velocità di risposta alle richieste di elaborazione;
 - **Fornire un'interfaccia user friendly**, preferibilmente una GUI con comandi intuitivi;
 - **Includere funzionalità orientate al Web** come la possibilità di configurare server Web;
 - **Capacità del SO di evolversi nel tempo** aggiungendo nuove caratteristiche e uso di nuove tecnologie.
- **Assenza di interferenze:** Il SO deve essere in grado di **prevenire interferenze** allocando risorse a uso esclusivo dei programmi utente e dei servizi del sistema stesso, **prevenendo l'accesso illegale alle risorse**. Deve inoltre **proteggere i file** di un utente da accessi illeciti da parte di altri utenti, garantendo però la possibilità di condividere determinati file in modo controllato.

Funzionamento di un Sistema Operativo

Le tre funzioni principali di un SO sono:

- **Gestione dei programmi:** Inizializza i programmi e li termina quando hanno completato l'esecuzione.



Gestisce l'utilizzo della CPU quando il sistema ha più programmi in esecuzione contemporaneamente, grazie allo **scheduling** che seleziona quale programma eseguire in un dato momento. Lo scheduler è implementato come routine del SO e mantiene una lista dei programmi in attesa di essere eseguiti dalla CPU, selezionandone uno per l'esecuzione. Un programma che viene prelazonato perde l'uso della CPU al termine del periodo di tempo specificato e lo concede ad un altro programma. Se il programma che è stato prelazonato non ha terminato la sua esecuzione, viene inserito nuovamente in lista pronto per essere schedulato successivamente. Esistono diverse **politiche di scheduling** che influenzano la qualità del servizio per l'utente. Ad esempio, se un programma viene prelazonato frequentemente, l'overhead dello scheduling sarà elevato a causa delle frequenti prelazioni, ma questo comporterà un buon servizio per l'utente perché ogni programma deve attendere solo un piccolo ritardo prima di essere schedulato nuovamente.

- **Gestione delle risorse:** Alloca risorse come CPU, RAM e dispositivi di I/O quando un programma li richiede e le dealloca quando termina, assegnandole successivamente ad altri programmi. **L'allocazione e deallocazione** delle risorse viene effettuata usando una **tabella delle risorse** creata in fase di boot, contenente nome, indirizzo di una risorsa e il suo stato attuale che indica se è disponibile o già allocata. Le strategie di allocazione delle risorse più utilizzate sono la strategia su **partizionamento delle risorse**, in cui il SO divide le risorse del sistema in partizioni, dove ogni partizione può includere ad esempio 1 MB di memoria RAM, 1000 blocchi di disco e 1 monitor. Questa strategia è semplice da implementare, soggetto a meno overhead ma pecca in flessibilità, poiché ci sarà uno spreco se un programma non utilizza tutte le risorse allocate, oppure se un programma eccede le risorse che gli sono state assegnate, non potrà andare in esecuzione. L'altra strategia per la gestione delle risorse è denominata **pool-based**, dove il SO **alloca dinamicamente** le risorse (se sono disponibili) ai programmi. Questo approccio è soggetto a maggior overhead dovuto all'allocazione e deallocazione delle risorse su richiesta, ma evita entrambi i problemi del metodo nel partizionamento delle risorse.

Risorse virtuali: Una risorsa virtuale è un'illusione creata dal sistema operativo (SO) utilizzando risorse reali, dando l'impressione di avere più risorse di quelle effettivamente disponibili. L'uso delle risorse virtuali è nato con i **dispositivi virtuali**, che permettono di assegnare a ogni programma un dispositivo esclusivo anche se le risorse fisiche sono limitate. Ad esempio, due dischi virtuali possono essere basati su un unico disco reale. Un esempio comune è il server di stampa, che gestisce le richieste di stampa mettendole in coda mentre i programmi continuano l'esecuzione come se la stampa fosse immediata. Un'altra risorsa virtuale diffusa è la memoria virtuale, che consente di eseguire programmi più grandi della memoria fisica disponibile. Infine, alcuni SO creano **macchine virtuali (VM)**, che permettono a ogni utente di avere un ambiente isolato con un proprio sistema operativo, eliminando interferenze e consentendo l'uso simultaneo di SO diversi sullo stesso computer.

- **Sicurezza e protezione:** Il SO non deve permettere a nessun utente l'utilizzo illegale di programmi e risorse o di interferire con il loro funzionamento. Ciò è garantito dalla **funzione di sicurezza**, mentre la **funzione di protezione** evita che utenti diversi dal proprietario di un file o dagli utenti autorizzati da quest'ultimo, abbiano accesso al file.

All'accensione, un computer effettua il **bootstrapping** (procedura di boot), ovvero viene creata una lista di tutte le risorse hardware nel sistema e successivamente viene passato il controllo al SO.

Alcune funzioni comuni eseguite dai SO:

- **Costruire una lista di risorse** durante la fase di boot;
- **Memorizzare le informazioni per la sicurezza:** Ciò avviene quando vengono registrati nuovi utenti da parte dell'amministratore di sistema. Il SO consente solo a questi utenti di autenticarsi per utilizzare le risorse e i servizi del computer;
- **Verificare l'identità di un utente** al momento del login;
- **Inizializzare l'esecuzione dei programmi** quando richiesto dall'utente;
- **Memorizzare le informazioni per l'autorizzazione** quando un utente specifica che i suoi collaboratori possono avere accesso ai suoi programmi o dati. Il SO annota questa informazione e la usa per implementare la protezione;
- **Eseguire l'allocazione delle risorse** quando richiesto dagli utenti o programmi;
- **Preservare lo stato corrente delle risorse** durante l'allocazione e deallocazione delle risorse;
- **Preservare lo stato corrente dei programmi ed effettuare lo scheduling** in maniera continuativa durante l'esecuzione del SO.

Intro ai Sistemi Operativi

Eventi e Interrupt

Mentre un programma utente è in esecuzione, esso stesso o altri programmi possono richiedere l'attenzione del SO, generando un **interrupt**. Questo può avvenire, ad esempio, per effettuare un'operazione di I/O oppure per richiedere una risorsa. Quando si verifica un interrupt il controllo della CPU passa al SO per eseguire le istruzioni necessarie per analizzare l'evento ed eseguire le azioni appropriate. Per questo motivo l'esecuzione del SO è detta **guidata dagli eventi** (event driven).



La conclusione di un'operazione di I/O o la richiesta di una risorsa da parte di un programma causa la **generazione di un interrupt**. La CPU è progettata per rilevarli e commutare (modalità Kernel) l'esecuzione verso il codice del SO per gestirli.

Utilizzo efficace di un computer e portabilità ed estendibilità dei SO

In passato ogni ambiente di elaborazione aveva una diversa concezione di **uso efficace**; quindi, il suo SO usava differenti tecniche per realizzarlo. Oggi, i **moderni ambienti di elaborazione** integrano caratteristiche e tecniche provenienti da diversi modelli, come i **sistemi non-interattivi**, i **sistemi time-sharing** e gli **ambienti di elaborazione distribuiti**. Inoltre, i primi SO erano sviluppati per essere strettamente integrati con l'architettura hardware del sistema, a differenza dei moderni SO come Unix e Windows, che sono portatili, quindi installabili su diverse architetture, ed estensibili così da potersi adattare ai nuovi requisiti imposti dall'evoluzione degli ambienti di elaborazione.

Gestire elaborazioni degli Utenti

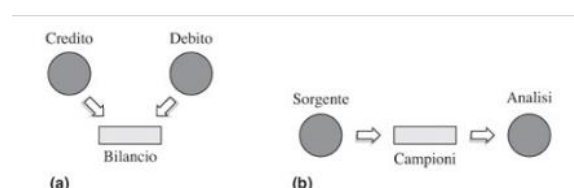
Processi e thread

Un **processo** è un programma in esecuzione. Un SO alloca risorse ai processi e li schedula per l'utilizzo della CPU. Il SO effettua la **commutazione di processo** (prelazionando il processo se questo non ha terminato l'elaborazione) quando è arrivato il momento di schedulare un altro processo. Quando avviene la commutazione di processo le informazioni relative al processo prelazionato vengono salvate e vengono caricate le informazioni riguardanti il processo schedulato. Tutto ciò comporta l'utilizzo della CPU e l'introduzione di overhead.

Il **thread** viene introdotto per ridurre l'overhead di un SO dato che la **commutazione di thread** richiede una minore quantità di informazioni da salvare rispetto alla commutazione di un processo.

Sincronizzazione di processi

La **sincronizzazione di processi** è indispensabile se si vogliono coordinare le attività di processi che hanno un obiettivo comune, poiché se due o più processi aggiornano senza sincronizzazione una variabile nello stesso istante, il risultato potrebbe essere inconsistente.



Scambio di messaggi

I processi possono comunicare tra loro attraverso lo **scambio di messaggi**, che il SO memorizza in una struttura interna finché il processo destinatario non effettua la richiesta di riceverlo.

La **differenza chiave** tra sincronizzazione e scambio di messaggi è che:

- la **sincronizzazione** garantisce che i processi accedano correttamente alle risorse condivise;
- lo **scambio di messaggi** permette ai processi di comunicare senza condividere risorse.

Scheduling

L'uso efficace di un SO è garantito da una opportuna **politica di scheduling appropriata**, che permette la condivisione della CPU tra più processi. Questo consente l'**esecuzione simultanea (apparente) dei processi**, migliorando la velocità del servizio e offrendo un'esperienza più efficiente per l'utente.

Deadlock

Il **deadlock** è una situazione in cui un insieme di processi rimane bloccato indefinitamente perché ciascuno di essi è in attesa di una risorsa detenuta da un altro processo. Questo accade quando le risorse del sistema sono condivise tra più processi e si verifica una **condizione circolare di attesa**: ad esempio, se il processo A detiene una risorsa richiesta dal processo B, mentre il processo B detiene una risorsa richiesta dal processo C, e così via fino a formare un ciclo chiuso, nessun processo può procedere, portando a uno **stallo**.

Gestione della Memoria

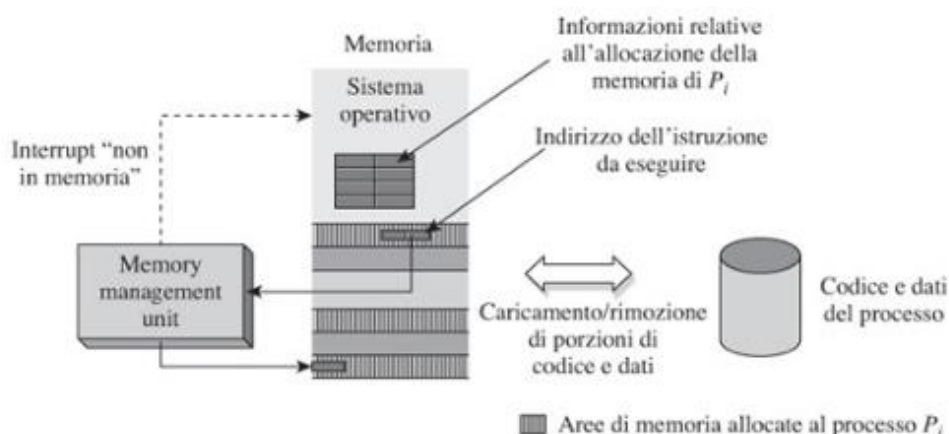
La memoria può essere gestita adoperando due modelli di gestione: il modello di allocazione della memoria contigua e il modello di allocazione della memoria non contigua.

- **Allocazione contigua**: In questo modello, un processo deve essere caricato in **un'unica area contigua di memoria**. Tuttavia, se lo spazio libero disponibile non è abbastanza grande per contenere un nuovo processo, non può essere riutilizzato direttamente. Per ottimizzare l'uso della memoria, il kernel può usare **tecniche di compattazione** per combinare frammenti di memoria liberi in un'unica area più ampia, generando un **notevole overhead**.
- **Allocazione non contigua** consente di assegnare a un processo più blocchi di memoria sparsi in diverse aree. Ciò permette un **utilizzo più efficiente della memoria**, riducendo il problema della frammentazione e facilitando il riutilizzo immediato di piccole aree disponibili.

Memoria virtuale

La **memoria virtuale** permette di eseguire processi più grandi rispetto alla memoria fisica (RAM) disponibile, memorizzando parte del codice e dei dati su disco e caricandoli in memoria solo quando necessario. Questo è possibile grazie all'**allocazione non contigua**, gestita dal sistema operativo tramite una tabella che registra la posizione delle parti di un processo in memoria.

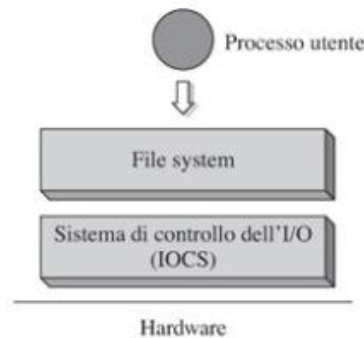
Durante l'esecuzione, la **Memory Management Unit (MMU)** traduce gli indirizzi richiesti dalla CPU e verifica se il codice o i dati sono in memoria. Se non lo sono, genera un **interrupt "non in memoria"**, richiedendo al SO di caricare i dati necessari dal disco rigido, eventualmente rimuovendo altre parti dalla memoria RAM. Questo meccanismo consente di eseguire processi più grandi della memoria fisica disponibile.



Un SO deve generare un tasso ridotto di interrupt “non in memoria” poiché questi rallentano il progresso di un processo; quindi, utilizza tecniche per decidere quante e quali porzioni di codice e dati del processo tenere in memoria RAM.

Gestione dei file e dei dispositivi di I/O

Un **file system** ha il compito di fornire accesso veloce a un file, proteggendoli da accessi di utenti non autorizzati e garantendo un uso efficiente dei dispositivi di I/O. Inoltre, deve assicurare affidabilità anche in caso di malfunzionamenti hardware o interruzioni di corrente, riducendo il rischio di perdita o corruzione dei dati.



Un file system è organizzato a livelli, dove il livello superiore rappresenta lo stesso file system che permette all'utente di condividere i suoi file con altri utenti, implementa la protezione dei file e fornisce affidabilità. Quando un utente esegue un'operazione su file, il file system richiama il livello inferiore, ovvero il **Sistema di controllo dell'I/O (IOCS)** che assicura a un processo un accesso veloce ai file e un uso efficiente dei dispositivi di I/O.

Capitolo 2 - Il SO, il computer e i programmi utente

Il Sistema Operativo esegue in modo ripetitivo un insieme di routine chiamate **funzioni di controllo** che prendono temporaneamente il controllo della CPU. La CPU può eseguire sia programmi utente che codice del SO. L'azione che forza la CPU a sospendere l'esecuzione di un programma utente per avviarne un altro, oppure per eseguire codice in **modalità Kernel**, prende il nome di **context switch** (Commutazione di Contesto).

Principi fondamentali del funzionamento di un SO

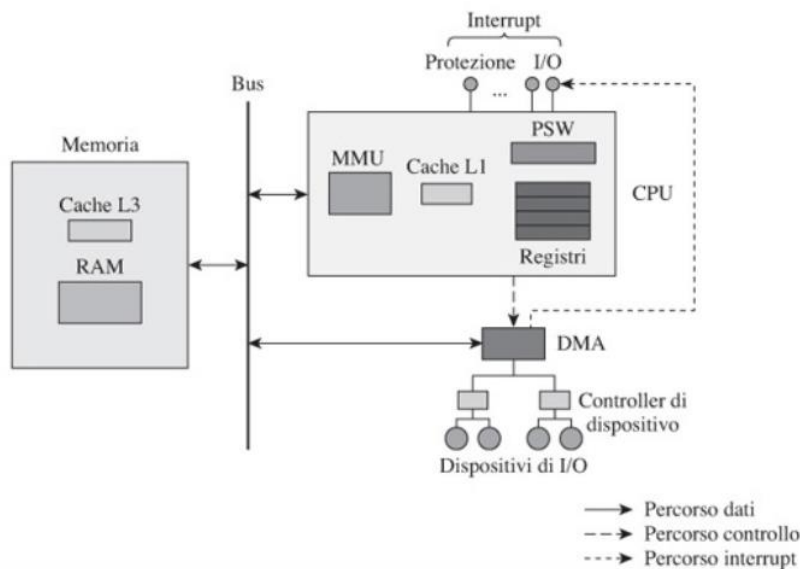
Il **Kernel** è il componente centrale del SO e si occupa di controllare il funzionamento del computer. Risiede in memoria RAM durante l'esecuzione del sistema e **gestisce le funzioni di controllo**, oltre che **fornire servizi ai programmi utente**.

La CPU viene utilizzata principalmente per eseguire programmi utente, ma può eseguire codice del Kernel quando si verifica un evento come la **generazione di un interrupt**. Questo può avvenire, ad esempio, quando un programma **richiede un servizio del Kernel**, come la creazione di un file (tramite un interrupt software) oppure quando termina un'operazione di I/O.

La CPU può operare in due modalità:

- **Modalità Utente:** Viene impostata in questa modalità per eseguire i programmi utente. Non può eseguire istruzioni privilegiate (Kernel) che, se usate in modo improprio, potrebbero compromettere la stabilità e la sicurezza del sistema interferendo con altri programmi o con il SO.
- **Modalità Kernel:** La CPU ha accesso completo a tutte le risorse del sistema e può eseguire qualsiasi tipo di istruzione, comprese quelle privilegiate (Kernel).

Il Computer



CPU e memoria RAM sono direttamente connessi al BUS, mentre le periferiche di I/O sono connesse al bus tramite un controller e il **DMA** (Direct Memory Access). Il DMA permette alle periferiche di I/O di accedere direttamente alla memoria senza coinvolgere la CPU. Tuttavia, CPU e periferiche di I/O non possono accedere simultaneamente alla memoria RAM poiché il BUS è progettato per gestire l'accesso in modo sequenziale, consentendo l'accesso a uno solo di questi dispositivi alla volta.

La CPU

La CPU possiede due tipologie di registri:

- **General-Purpose Registers (GPR):** Sono **registri visibili agli utenti** e vengono utilizzati per memorizzare dati, indirizzi, dati, indici e stack pointer durante l'esecuzione dei programmi.
- **Program Status Word (PSW):** Sono **registri di controllo** che contengono le informazioni necessarie a controllare il funzionamento della CPU. L'insieme dei registri di controllo è chiamato PSW mentre i SINGOLI registri di controllo verranno individuati come CAMPI della PSW.

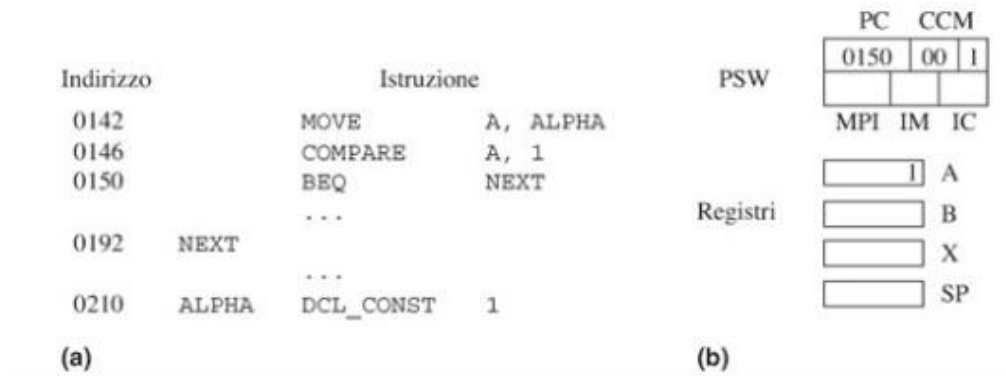
Program counter (PC)	Condition code (CC)	Modalità (M)	Informazioni di protezione della memoria (MPI)	Maschera degli interrupt (IM)	Codice interrupt (IC)
----------------------	---------------------	--------------	--	-------------------------------	-----------------------

I campi della PSW sono:

- **Program Counter (PC):** Contiene l'indirizzo della prossima istruzione da eseguire;
- **Condition Code (CC):** Indica alcune caratteristiche del risultato di un'istruzione aritmetica (<0 , $=0$, >0). Questo codice viene utilizzato nell'istruzione di salto condizionato.
- **Modalità (M):** Indica se la CPU è in esecuzione in modalità kernel (bit in modalità **0**) oppure in modalità utente (bit in modalità **1**).
- **Informazioni di protezione della memoria:** È un campo composto da sottocampi che contengono il **registro base** e il **registro limite**.
- **Maschera degli Interrupt:** Indica quali interrupt sono abilitati e quali "mascherati".
- **Codice Interrupt:** Descrive la condizione o l'evento che ha causato l'ultimo interrupt. Viene utilizzato da una routine di servizio dell'interrupt.

Le informazioni contenute nei registri General-Purpose Registers e nel Program Status Word costituiscono lo **stato della CPU**.

Ciò è utile quando un programma attualmente in esecuzione deve essere prelazionato. Per assicurare che il programma possa riprendere la sua esecuzione da dove si era fermato, il kernel **salva lo stato della CPU** quando viene sottratta al programma e lo ricarica quando viene ri-schedulato.



In figura viene mostrato un programma in linguaggio assembly. Il Kernel sottrae la CPU al programma dopo che questo ha eseguito l'istruzione COMPARE (che ha restituito come risultato "uguale", impostando il campo CC a 00). Il Kernel salva lo stato della CPU come mostrato nella figura (b) di destra, con il PC che contiene l'indirizzo 150 (l'indirizzo della prossima istruzione da eseguire). Quando lo stato della CPU viene caricato nuovamente, il programma riprenderà l'esecuzione dall'istruzione contenuta all'indirizzo 150, ovvero BEQ NEXT.

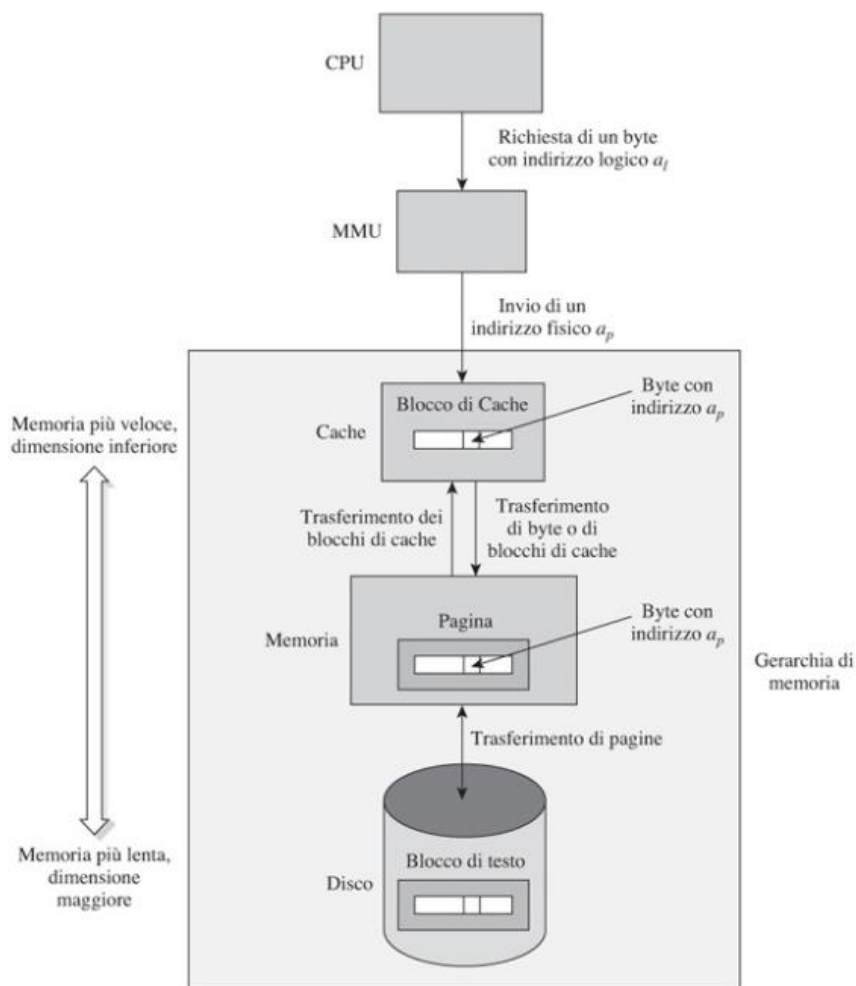
Memory Management Unit (MMU)

La **MMU** è un componente utilizzato per la **traduzione degli indirizzi logici in indirizzi di memoria fisici**.

La MMU insieme alla gestione dell'**allocazione della memoria non contigua** è cruciale per implementare la tecnica della **memoria virtuale**, che consiste nel creare l'illusione di una memoria più grande della reale memoria RAM installata.

Gerarchia della memoria

La CPU accede solo alla memoria più veloce, ovvero alla **memoria cache**. La cache però è molto costosa e quindi la soluzione è quella di adoperare una **gerarchia della memoria**, dove la memoria più veloce è la più costosa e meno capiente, e la memoria più lenta è la meno costosa ma più capiente. La gerarchia della memoria è composta da memoria cache che è la più veloce e piccola, poi dalla memoria RAM, più lenta e capiente e in ultimo dal disco rigido che è il più lento e più capiente. Se il dato o l'istruzione non è presente nella memoria cache (avviene un **miss**), viene copiato dalla memoria più lenta e così via. Il **tempo di accesso** alla memoria dipende da quanto frequentemente si verifica un miss in una memoria più veloce.



Memoria cache

Per **ottimizzare le prestazioni della cache** non viene trasferito un singolo byte dalla memoria RAM, ma un “blocco” di memoria di dimensioni standard chiamato **cache block** o **cache line**. Ogni blocco contiene più dati, e la logica è che, se un byte è stato appena accessibile, è probabile che anche i byte vicini siano necessari a breve. In questo modo l'accesso ai dati vicini (dati contigui) sarà molto più rapido, senza la necessità di fare un nuovo accesso alla memoria RAM.

Quando la CPU scrive un nuovo valore in un byte, viene scritto prima in memoria cache, ma successivamente questo valore deve essere anche scritto in memoria RAM per mantenerla aggiornata. Un metodo semplice per fare ciò è chiamato **write-through**, che implica che ogni modifica venga scritta contemporaneamente sia nella cache che in RAM, garantendo che questa resti sempre aggiornata con il contenuto della cache.

La CPU effettua una ricerca nella cache confrontando gli indirizzi dei byte richiesti con quelli presenti in cache. Si verifica un **hit** se i byte sono presenti e un **miss** se non lo sono. Se si verifica un miss i byte devono essere caricati in cache dalla memoria.

L'**Hit ratio** misura quante volte la CPU trova i dati nella cache invece che nella memoria principale. Un **valore alto di hit ratio** significa che la CPU accede frequentemente ai dati già presenti nella cache, velocizzando il programma. Questo accade anche grazie alla **località dei riferimenti**, che si basa su due principi:

- **Località spaziale:** i dati vicini tra loro sono usati insieme, ovvero quando bisogna trasferire un byte dalla RAM alla cache si trasferisce quel byte più un intorno di quel byte.
- **Località temporale:** i dati usati recentemente tendono a essere usati di nuovo.

Formula per calcolare il tempo effettivo di accesso alla memoria (EAT) considerando una gerarchia di memoria costituita da una cache e dalla RAM:

$$t_{EAT} = h \times t_{cache} + (1 - h) \times (t_{tra} + t_{cache}) = t_{cache} + (1 - h) \times t_{tra}$$

Dove:

- h = probabilità di successo
- $(1 - h)$ = probabilità di insuccesso
- t_{EAT} = tempo di accesso effettivo alla memoria

- t_{cache} = tempo di accesso alla cache
- t_{tra} = tempo necessario per trasferire in cache un blocco di memoria

Quindi, probabilità che si verifica un hit (h) moltiplicato il tempo di accesso alla cache (t_{cache}) + la probabilità che si verifichi un insuccesso, ovvero un "miss" (1-h) moltiplicato il tempo di accesso alla cache sommato al tempo di accesso alla memoria RAM per il trasferimento dalla RAM alla cache ($t_{tra} + t_{cache}$).

La sfida nelle moderne architetture è quello di aumentare la probabilità che si verifichi un hit adottando opportune tecniche, come l'uso del **principio di località dei riferimenti** visto precedentemente. Inoltre, sono state introdotte memorie cache intermedie tra la L1 e la RAM come le cache L2 e L3 per ridurre il più possibile il tempo di accesso ai dati.

È da notare che l'**hit ratio** in una cache è **basso all'inizio dell'esecuzione di un programma** perché poche istruzioni o dati sono stati trasferiti in cache. L'hit ratio è maggiore quando il programma rimane in esecuzione per lungo tempo.

Nota: Le memorie cache L1 e L2 sono on-chip mentre la L3 è esterna alla CPU.

Memoria RAM

Il trasferimento di un blocco di byte dal disco alla memoria RAM, chiamato **pagina**, è analogo al funzionamento della cache, con la differenza che il **trasferimento viene effettuato dal software**, diversamente dalla cache dove vengono effettuati dall'hardware.

Protezione della memoria

La protezione della memoria è un vincolo necessario a prevenire che un programma acceda illegalmente ad una porzione di memoria destinata all'esecuzione di un altro programma.

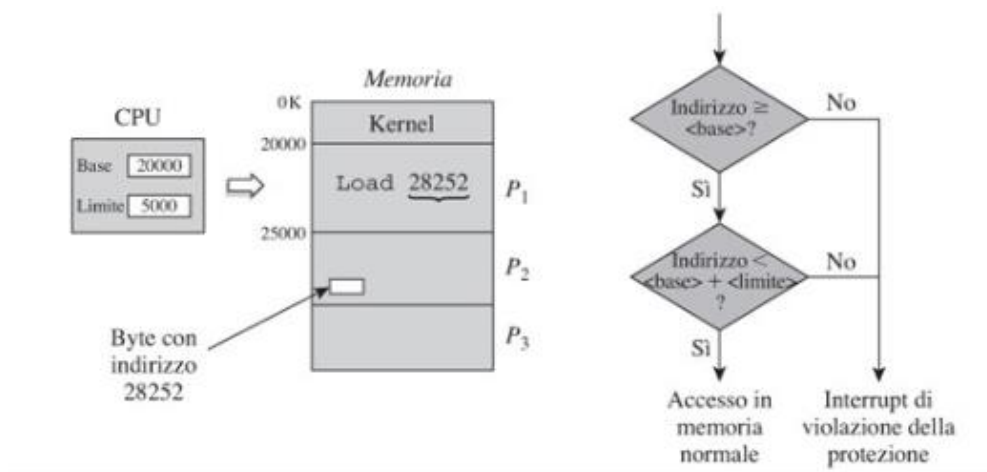
È implementata utilizzando **due registri di controllo**:

- il **registro base** che contiene l'indirizzo di partenza della memoria allocata al programma;
- il **registro limite** che contiene la dimensione massima della memoria allocata al programma.

L'indirizzo dell'ultimo byte allocato ad un programma si calcola:

$$\text{Indirizzo dell'ultimo byte} = \langle \text{registro base} \rangle + \langle \text{registro limite} \rangle - 1$$

Nel caso in cui un programma dovesse accedere ad una locazione di memoria al di fuori dell'indirizzo dell'ultimo byte assegnatogli, l'hardware di protezione della memoria **genera un interrupt per segnalare una violazione** e interrompe l'accesso alla memoria. Il Kernel in risposta all'interrupt termina il programma. Le informazioni riguardanti la protezione della memoria sono contenute nel campo del PSW della CPU, denominato MPI e vengono salvate e ricaricate quando il programma viene prelazionato e riportato in esecuzione.



Nell'esempio il Kernel alloca al Programma P1 5000 byte. Possiamo notare che l'indirizzo base è impostato a 20000 ed è caricato nel registro base e il limite è impostato a 5000 caricato nel registro limite. Se l'istruzione eseguita dalla CPU utilizzasse un indirizzo non compreso tra 20000 e 24999, ad esempio, 28252, verrebbe generato un interrupt di violazione della memoria.

Per sicurezza, le istruzioni per accedere o modificare qualunque parte della memoria possono essere eseguite solo quando la CPU si trova in modalità Kernel. Un programma utente non può modificare queste informazioni poiché quando è in esecuzione, la CPU è impostata in modalità Utente.

La protezione della memoria nella cache è complessa perché **più programmi possono usare gli stessi indirizzi logici** (da 0 a $n-1$), rendendo insufficiente un controllo basato solo sugli indirizzi.

Una **soluzione semplice sarebbe svuotare la cache** (flush) a ogni cambio di programma, in questo modo la cache non manterrebbe il contenuto delle aree di memoria allocate ad altri programmi, ma ciò **ridurrebbe le prestazioni iniziali** a causa di un basso hit ratio.

Un' **alternativa più efficiente** consiste nell'associare a ogni blocco della cache un **identificativo del programma proprietario**, consentendo l'accesso solo se l'identificativo corrisponde. Questo metodo evita di svuotare la cache e mantiene alte le prestazioni.

Input/Output

Nelle vecchie architetture Hardware un'operazione di I/O richiedeva l'intervento della CPU. Nei moderni computer l'organizzazione dell'I/O si è evoluta grazie all'introduzione di un processore Hardware dedicato alle operazioni di I/O chiamato **DMA** (Direct Access Memory). Oltre a consentire tassi di trasferimento più elevati tra la RAM e il dispositivo di I/O, il **DMA libera la CPU dalla gestione delle operazioni di I/O**, consentendogli di effettuare altre attività.

Modalità di I/O:

- **I/O Programmato** (obsoleto): Il trasferimento dati tra la periferica I/O e la RAM avviene grazie alla CPU che resta impegnata per tutta l'operazione, non potendo eseguire altro fino al suo completamento. Può essere effettuata solo un'operazione di I/O alla volta.
- **Interrupt di I/O**: Questa modalità è ugualmente lenta perché effettua il **trasferimento dei byte uno alla volta** con l'aiuto della CPU che esegue la routine di servizio dell'interrupt che viene generato quando un byte deve essere trasferito dalla periferica di I/O alla RAM. La CPU è libera di eseguire altre istruzioni dopo aver eseguito l'istruzione di I/O.
- **I/O basato sul DMA**: Vengono trasferiti blocchi di dati tra la RAM e la periferica di I/O **direttamente sul BUS senza coinvolgere la CPU**. Quando il trasferimento è completato il **controller DMA genera un interrupt di I/O**, il Kernel analizza la causa dell'interrupt e deduce che l'operazione di I/O è stata completata. In questa modalità si ottengono tassi di trasferimento elevati e **sono supportate operazioni concorrenti di CPU e dispositivi di I/O**. Ad esempio, quando viene eseguita un'istruzione di read sulla periferica d, **la CPU trasferisce i dettagli dell'operazione di I/O al DMA** delegando solo a questa il compito di gestirla.

Interrupt

Un **interrupt è un evento**, con diversa priorità, che il Sistema Operativo deve opportunamente gestire. Se diversi interrupt si verificano allo stesso tempo, la CPU seleziona quello a **priorità più alta**, mentre gli altri restano pendenti finché non verranno selezionati.

Durante il ciclo di esecuzione di un'istruzione della CPU, è recuperata, decodificata ed eseguita l'istruzione, il cui indirizzo è contenuta nel Program Counter.

Se nel frattempo si verifica un interrupt durante l'esecuzione dell'istruzione, è necessario eseguire un'azione di interrupt che **salva lo stato corrente della CPU** (PSW e GPR), e carichi al suo posto i nuovi dati. Successivamente la **CPU viene commutata in modalità Kernel** per eseguire le istruzioni di una **routine di servizio dell'interrupt**, chiamata ISR. Infine, il kernel può ripristinare l'esecuzione del programma precedentemente interrotto ricaricando lo stato salvato nei registri della CPU.

Esistono diverse **classi di interrupt**:

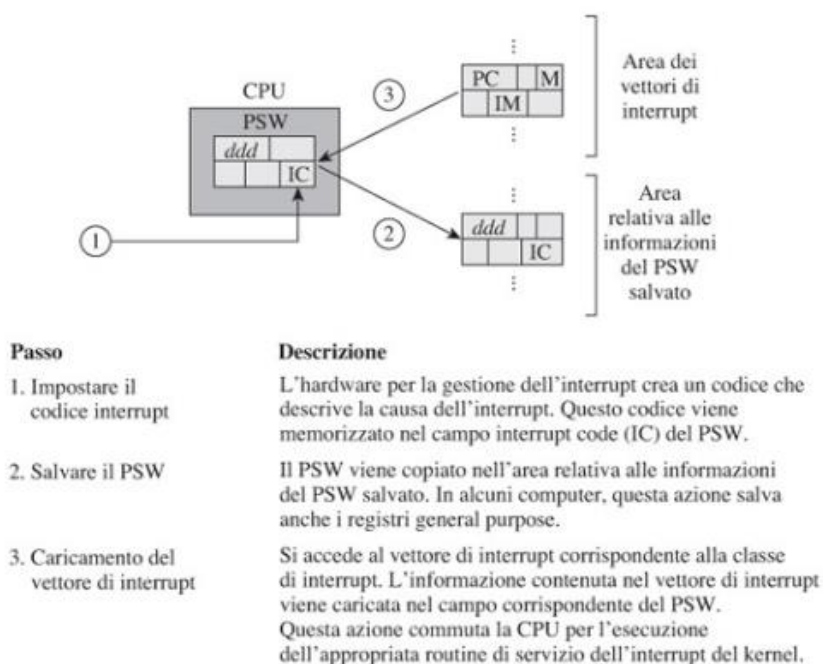
- **I/O Interrupt**: È causato dal completamento di un'operazione di I/O o il malfunzionamento delle periferiche di I/O.
- **Timer Interrupt**: Viene generato a intervalli di tempo, sia periodicamente dopo un predefinito numero di tick di clock, o all'occorrenza dopo un intervallo di tempo programmato, dove è necessario specificare il numero di tick di clock e salvarli nel **registro timer** mediante una istruzione privilegiata.
- **Program Interrupt**: Chiamato anche **trap** o **eccezione** è utilizzato sia per indicare una **condizione di eccezione** durante l'esecuzione di un'istruzione come un overflow o una violazione di protezione della memoria, oppure può essere **richiamata da programma utente** tramite una **istruzione speciale** (istruzione di **interrupt software** con operation code **SI**), il cui unico scopo è di generare un interrupt in modo tale da passare il controllo al kernel. Ciò è utile per richiedere al kernel risorse o servizi ai quali non hanno possibilità di accedere direttamente.

In base alla **classe di interrupt**, l'hardware imposta un **codice interrupt**, specifico per ogni architettura, nel **campo IC** del PSW per identificare quale specifico interrupt si è verificato.

Per **interrupt di I/O** solitamente il **codice corrisponde all'indirizzo della periferica** che l'ha causato, mentre per un **interrupt da programma** vengono assegnati **codici distinti** per identificare condizioni diverse di eccezioni come l'overflow o una violazione di protezione della memoria, e comprendono anche un insieme di codici per gli interrupt software. L'istruzione **SI** consente di **generare un interrupt software** accettando un intero come operando che viene interpretato come codice interrupt secondo le regole dell'architettura e del SO.

Nel PSW il **campo maschera degli interrupt (IM)** indica quali interrupt sono **rilevabili** (o abilitati) e quali sono **mascherati** (o disabilitati) in un dato momento. Può contenere sia un intero **m** che indica che verranno rilevati solo gli interrupt che hanno **priorità maggiore o uguale a m**, o alternativamente può contenere un valore codificato a bit, in cui ogni bit indica se l'interrupt può essere rilevato. Gli **interrupt mascherati** non vengono persi ma **restano pendenti** finché non vengono abilitati, per poter poi essere gestiti.

Azione di interruzione della CPU



Dopo l'esecuzione di ogni istruzione, la CPU controlla se sono stati generati interrupt e in caso affermativo esegue il **gestore dell'interrupt** che salva lo stato della CPU ed esegue quindi la **routine di servizio dell'interrupt** nel kernel.

In figura è possibile notare che il **gestore degli interrupt è composto da 3 fasi**:

1. Nella prima fase **viene impostato il codice dell'interrupt** nel campo IC del PSW in base alla causa dell'interrupt.
2. Successivamente **viene salvato lo stato della CPU** e quindi il contenuto del PSW in un **area di memoria riservata** oppure un'area sullo stack, così che successivamente, dopo la gestione dell'interrupt, il Kernel potrà ripristinare l'esecuzione del programma interrotto.
3. Viene **caricato nei campi del PSW il vettore degli interrupt** inerente alla classe di appartenenza dell'interrupt. Un **vettore degli interrupt** è composto dall'**indirizzo della routine di servizio dell'interrupt**, una **maschera degli interrupt** che indica gli interrupt che potranno essere gestiti durante l'elaborazione dell'interrupt corrente, e uno 0 o un 1 per indicare la modalità in cui deve trovarsi la CPU, solitamente 0 che corrisponde alla modalità Kernel.

Per semplicità si assume che il vettore degli interrupt abbia lo stesso formato del PSW e che gli elementi appena descritti (indirizzo della routine di servizio dell'interrupt, maschera e modalità) vengano caricati rispettivamente nei campi PC, maschera degli interrupt (IM) e modalità (M) del PSW della CPU quando viene richiamata la routine di servizio dell'interrupt.

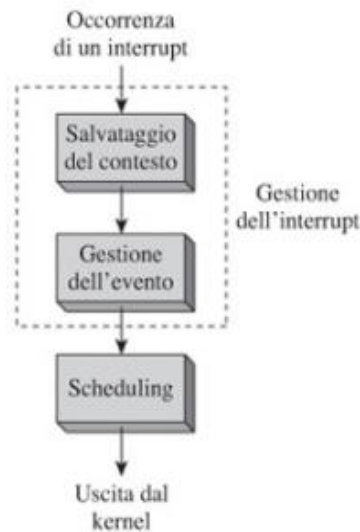
La semplificazione che il vettore degli interrupt ha lo stesso formato del PSW, rende l'idea più semplice, poiché basta copiare i valori preimpostati del vettore direttamente nei campi corrispondenti del PSW. In questo modo, la CPU può eseguire la routine di servizio senza bisogno di conversioni o operazioni extra. Nella realtà le cose sono leggermente diverse.

Il Kernel costruisce il vettore degli interrupt per le diverse classi di interrupt all'avvio del sistema operativo. Ogni vettore degli interrupt contiene le seguenti informazioni:

- Uno **0 nel campo modalità M**, poiché la CPU deve essere utilizzata in modalità kernel;

- L'**indirizzo della prima istruzione** della routine di servizio dell'interrupt nel campo program counter (PC);
- **Uno 0** e la **dimensione della memoria** nel campo memory protection information (MPI), in modo che la routine di servizio dell'interrupt possa avere accesso all'intera memoria;
- Una **maschera degli interrupt** nel campo maschera degli interrupt (IM) che serve ad impedire agli altri interrupt di potersi verificare o consente solo agli interrupt a priorità più alta di potersi verificare.

La figura di seguito contiene uno schema del funzionamento del kernel che prende il controllo quando si verifica un interrupt.



Il funzionamento è detto **interrupt-driven** (guidato dagli interrupt) e funziona in questo modo:

1. **Salvataggio del contesto:** La routine di servizio dell'interrupt salva nella tabella dei programmi il PSW, il contenuto dei GPR, le informazioni riguardanti la memoria e le risorse utilizzate del programma interrotto.
2. **Gestione dell'evento:** Il secondo step consiste nella gestione dell'evento che ha causato l'interrupt. Nel PSW il campo codice dell'interrupt fornisce informazioni utili a questo scopo.
3. **Scheduling:** Infine, terminata la gestione dell'interrupt, la routine di scheduling seleziona un programma che, in base all'evento che ha causato l'interrupt e allo stato degli altri programmi, può essere lo stesso programma che era in esecuzione o qualche altro programma.

Di seguito ci sono alcuni esempi di **interrupt** (a sinistra) e le **azioni** (a destra) adottate per la gestione di tali eventi:

- **Eccezioni aritmetiche** -> Terminazione del programma.
- **Violazione di protezione della memoria** -> Terminazione del programma.
- **Interrupt Software** -> Soddisfare la richiesta del programma se possibile, in caso contrario, annotarlo per un'azione futura.
- **Fine di un'operazione di I/O** -> Trovare quale programma aveva iniziato l'operazione di I/O e annotare che può ora essere preso in considerazione per essere schedato. Iniziare un'operazione di I/O pendente, se presente, sul dispositivo.
- **Timer interrupt** -> (1) Aggiornare l'orologio di sistema. (2) Effettuare l'azione appropriata nel caso in cui sia trascorso uno specificato intervallo di tempo.

Interrupt Annidati

Gli **interrupt annidati** sono un meccanismo in cui un'interruzione può essere temporaneamente sospesa da un'altra con priorità più alta. Questo permette di gestire eventi critici con maggiore reattività. Quando si verifica un'interruzione, la CPU salva il contesto corrente e avvia l'esecuzione del gestore dell'interrupt. Se nel frattempo arriva un'altra interruzione con priorità superiore, la CPU sospende l'interrupt attuale, salva il suo stato e inizia a gestire quello più urgente. Una volta terminato, ripristina il contesto del primo interrupt e prosegue la sua esecuzione. Tuttavia, questa tecnica, introduce maggiore complessità nella gestione del contesto e può aumentare il rischio di overflow dello stack se gli interrupt si accumulano in modo eccessivo.

I SO hanno utilizzato **due approcci** per la gestione degli interrupt annidati:

- Nei **Kernel non preemptive** (non prelazionabili) viene usato il campo della maschera degli interrupt (IM) nel vettore degli interrupt per mascherare tutti gli interrupt mentre è già in esecuzione una routine di gestione degli interrupt. Questo approccio **rende possibile solo la gestione di un interrupt alla volta** e ciò potrebbe causare dei ritardi nella gestione di interrupt a più alta priorità.
- Nei **Kernel preemptive** (prelazionabili), il Kernel **imposta la maschera degli interrupt in ogni vettore degli interrupt** per mascherare gli interrupt meno critici, continuando a poter servire gli interrupt più critici in maniera annidata.

Interazione del SO con il computer e i programmi utente

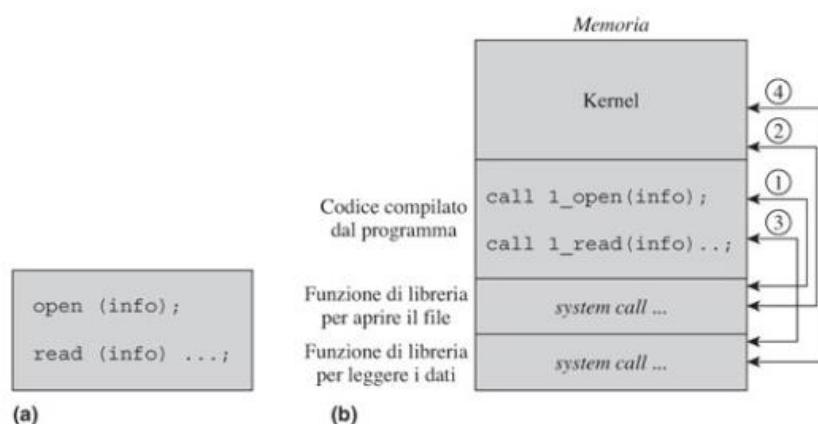
Nei Sistemi Operativi, alcuni eventi, come richieste hardware o condizioni eccezionali, causano un interrupt. I programmi hanno bisogno di accedere ai servizi messi a disposizione dal SO, come l'esecuzione di operazioni di I/O o la gestione dei file. Per questo motivo, esistono le **system call**, che permettono di trasferire le richieste al kernel, spesso utilizzando un meccanismo basato su interrupt o istruzioni speciali per il cambio di modalità.

System Call

Una **system call** è una richiesta che un programma fa al Kernel attraverso un interrupt software.

Le system call hanno lo scopo di facilitare la gestione delle risorse da parte dei programmi utente, come l'allocazione o l'accesso a risorse critiche.

I programmi utente non possono accedere direttamente alle risorse, ma possono farlo attraverso le system call che effettuano la richiesta al Kernel per loro conto.



Dal punto di vista di un programmatore, un programma utilizza le risorse del computer attraverso istruzioni fornite in un linguaggio di programmazione come il C. Il compilatore svolge il ruolo di tradurre il linguaggio di programmazione in linguaggio macchina, e durante la compilazione le istruzioni relative all'uso delle risorse del sistema vengono sostituite con chiamate a funzioni di libreria. Le **funzioni di libreria** rappresentano implementazioni predefinite che forniscono un'interfaccia standard per interagire con le risorse del computer.

In fase di esecuzione del programma, la funzione di libreria **non agisce direttamente sulla risorsa**, ma si avvale di un meccanismo fornito dal SO, ovvero effettua una chiamata alla/e system call.

Le chiamate di sistema o system call rappresentano il metodo attraverso il quale un programma utente può richiedere servizi al kernel del SO, il quale ha il controllo diretto sulle risorse hardware.

Una **system call** è implementata attraverso il metodo degli interrupt software. Si assume che l'istruzione di un interrupt software abbia la forma

$$SI < int_code >$$

Dove `int_code` è tipicamente un intero nell'intervallo 0-255 e indica quale servizio del kernel è richiesto.

L'interrupt software si verifica quando un programma esegue l'**istruzione SI**.

Successivamente viene copiato `<int_code>` nel campo codice dell'interrupt (IC) del PSW (in questo caso è `d1`).

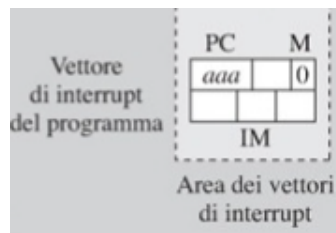
PC	M
bbb	0
	d ₁

IC La routine di servizio dell'interrupt per l'interrupt di programma analizza il campo IC nell'area delle informazioni salvate del PSW per conoscere la tipologia di richiesta fatta dal programma.

Una system call accetta parametri che forniscono informazioni importanti al servizio del kernel invocato. Ad esempio, la system call per aprire un file accetta come parametri il nome del file, il numero di byte da leggere e l'indirizzo in memoria dove i dati letti devono essere memorizzati. Per il passaggio dei parametri possono essere utilizzati diversi metodi: è possibile caricare i parametri nei registri prima che la system call sia invocata, possono essere copiati sullo stack o memorizzati in un'area di memoria.

Esempio di funzionamento System call in un ipotetico SO:

Sia 78 il codice per ottenere l'ora corrente. Quando un programma utente desidera conoscere l'ora corrente esegue l'istruzione SI <78>, che causa un interrupt software. Il numero 78 viene inserito nel campo IC (codice dell'interrupt) del PSW, prima questo sia salvato nell'area di memoria del PSW salvato.



Come mostrato in figura, il vettore degli interrupt per gli interrupt da programma contiene aaa nel suo campo PC. Questo significa che l'esecuzione della CPU verrà trasferita alla routine presente all'indirizzo aaa. La routine verifica che il codice interrupt è 78 e deduce che il programma utente vuole conoscere l'ora. L'informazione relativa all'ora di sistema viene restituita al programma utente e salvata in una locazione dati (registro standard).

I sistemi operativi come Linux mettono a disposizione diverse system call per vari scopi, richiamabili attraverso le funzioni libreria di linguaggi di programmazione come C, C++, Java, oppure attraverso l'istruzione SI utilizzando il linguaggio assembly. Di seguito troviamo alcuni esempi:

Numero della system call	Nome della system call	Descrizione
1	exit	Termina l'esecuzione di questo programma
3	read	Legge i dati da un file
4	write	Scrive i dati in un file
5	open	Apri un file
6	close	Chiude un file
7	waitpid	Attende che l'esecuzione di un programma termini
11	execve	Esegue un programma
12	chdir	Cambia la directory di lavoro
14	chmod	Cambia i permessi del file
39	mkdir	Crea una nuova directory
74	sethostname	Imposta il nome dell'host
78	gettimeofday	Recupera l'ora corrente
79	settimeofday	Imposta l'ora corrente

Capitolo 3 – Panoramica dei Sistemi Operativi

Nel corso della storia gli ambienti di elaborazione si sono evoluti con i cambiamenti delle architetture hardware e delle aspettative degli utenti. Per questo motivo è stata sviluppata una nuova classe di sistemi operativi che sfruttano molti concetti e tecniche utilizzate nei sistemi operativi classici.

Ambienti di elaborazione e natura delle elaborazioni

L'Hardware evolve di continuo per fornire una migliore qualità del servizio agli utenti; tuttavia, il SO deve svolgere compiti più complessi al crescere della potenza computazionale, della complessità delle interfacce con le periferiche di I/O e della richiesta da parte degli utenti di nuovi servizi.

Per **svolgere diverse attività simultaneamente** il SO deve fornire le risorse necessarie come la CPU, la RAM e dispositivi di I/O in modo tale che le attività progrediscano senza intoppi.

Ambienti di elaborazione non interattivi

L'obiettivo del SO in questa tipologia di ambiente è **garantire l'uso efficiente delle risorse**.

In questa tipologia di ambiente un utente fornisce al SO sia il programma che i dati e il SO restituisce i risultati, senza poter intervenire durante l'esecuzione.

Le elaborazioni erano basate su

- **programmi**, costituiti da un insieme di funzioni o moduli eseguibili indipendentemente;
- **job**, composti da una sequenza di programmi che insieme raggiungono un obiettivo comune.

Se durante l'esecuzione di un job, un programma andava in errore, il programma successivo non veniva eseguito.

Ambienti di elaborazione interattivi

L'obiettivo del SO è di **ridurre il tempo medio richiesto per implementare l'interazione tra un utente e la sua elaborazione**.

Negli ambienti di elaborazione interattivi **un utente può avviare un'elaborazione e interagire con essa mentre è in esecuzione**.

In questo contesto, il termine **processo** si riferisce all'esecuzione di un programma, mentre ogni interazione con esso viene chiamata **sotto richiesta al processo**. In base al tipo di sotto richiesta la risposta può consistere in un insieme di risultati o un insieme di azioni come operazioni su file o aggiornamenti di database.

Nota: Gli SO per l'elaborazione interattiva gestiscono esclusivamente l'esecuzione di processi (programmi) e non di job.

Ambienti real-time, distribuiti ed embedded

Un **ambiente di elaborazione real-time** utilizza tecniche speciali per assicurare che le elaborazioni **rispettino specifici vincoli temporali**. Un esempio è l'elaborazione periodica di campioni in un file, dove è necessario ultimare la memorizzazione del campione attuale prima di proseguire con il prossimo.

In un **ambiente di elaborazione distribuito** è possibile utilizzare risorse hardware presenti in diversi sistemi attraverso una rete.

Un **ambiente di elaborazione embedded** deve rispettare i vincoli temporali derivanti dalla natura del sistema controllato ed è tipico dei computer che fanno parte di uno specifico sistema hardware come un elettrodomestico o il computer di bordo di un'automobile.

Moderni ambienti di elaborazione

I **moderni ambienti di elaborazione** implementano le caratteristiche dei diversi ambienti visti in precedenza.

In questa tipologia di ambienti si adottano diverse e complesse strategie per ridurre il tempo medio richiesto per implementare l'interazione tra utente e applicazione e assicurare allo stesso tempo un uso efficiente delle risorse.

Efficienza, prestazioni del sistema e servizio per l'utente

L'**Efficienza**, le **prestazioni del sistema** e il **servizio per l'utente** sono parametri utilizzati per valutare specifici aspetti del funzionamento di un sistema operativo.

Efficienza

*L'**efficienza** misura il grado di utilizzo effettivo di una risorsa rispetto alla sua capacità totale, al netto di inattività e overhead di sistema.*

Per la CPU, essa corrisponde alla quota di tempo impiegata nell'esecuzione di processi utente, escludendo i periodi di inattività (ad esempio durante attese di I/O) e il tempo destinato alle operazioni di gestione del sistema operativo, quali interrupt e scheduling.

Analoga definizione si applica ad altre risorse, come memoria e dispositivi di I/O, considerando la parte realmente utilizzata rispetto a quella disponibile.

Prestazioni del sistema

Le **prestazioni del sistema** sono caratterizzate dalla **quantità di lavoro svolto per unità di tempo** e sono generalmente misurate dal **throughput**, che rappresenta il numero di job, programmi, processi e sotto richieste completati da un sistema in una unità di tempo.

Un SO può mettere in atto **azioni correttive per aumentare le prestazioni**. Ad esempio, se l'efficienza della memoria è bassa, l'SO può caricare più programmi utente in memoria, portando ad un incremento delle prestazioni del sistema aumentando il tasso con il quale il sistema completa le elaborazioni degli utenti.

Servizio per l'utente

Il **servizio per l'utente**, indica **quanto velocemente** un'elaborazione dell'utente è stata completata dal SO. Esistono due misure del servizio per l'utente:

Tempo di turnaround: Rappresenta il tempo trascorso dalla sottomissione da parte di un utente di un job, di un programma o di un processo fino al momento in cui i risultati sono restituiti all'utente.

Tempo di risposta: Rappresenta il tempo trascorso dalla sottomissione di una sotto richiesta da parte di un utente fino al momento in cui il processo risponde a essa.

Classi di Sistemi Operativi

Le classi di SO si sono evolute nel tempo con l'evoluzione dell'hardware e delle aspettative degli utenti.

Le cinque classi fondamentali di SO sono:

- Sistemi di elaborazione batch;
- Sistemi Multi programmati;
- Sistemi Time-Sharing;
- Sistemi Operativi Real-Time;

Classe del SO	Periodo	Obiettivo principale	Concetti chiave
Elaborazione batch	anni '60	Tempo idle della CPU	Automatizzare la transizione tra i job
Multiprogrammazione	anni '60	Utilizzo delle risorse	Proprietà del programma, prelazione
Time-sharing	anni '70	Buon tempo di risposta	Scheduling, time slice, round-robin
Real time	anni '80	Rispettare i vincoli temporali	Scheduling real-time
Distribuiti	anni '90	Condivisione delle risorse	Controllo distribuito, trasparenza

Poiché l'hardware era costoso, i primi sistemi di elaborazione come quelli basati **sull'elaborazione batch** e sui **SO multiprogrammati** erano sistemi **non interattivi** e progettati per **ottimizzare l'uso delle risorse**.

Successivamente l'hardware divenne più economico e **l'obiettivo si spostò dall'uso efficiente delle risorse alla produttività degli utenti**. Nacquero così i primi ambienti di elaborazione interattivi con l'introduzione dei **SO time-sharing** dove è stato possibile migliorare la produttività fornendo risposte rapide alle sotto richieste fatte ai processi. Negli anni '80 nacquero gli **SO real-time** che avevano come obiettivo il rispetto dei vincoli temporali di applicazioni nate con lo scopo di controllare e tracciare attività del mondo reale.

Infine, negli anni '90, nacquero i **SO distribuiti** grazie ai quali diversi computer condividevano le proprie risorse attraverso la rete.

Sistemi di Elaborazione Batch

In un sistema operativo per l'elaborazione batch, l'obiettivo principale è **l'uso efficiente della CPU**. Questo sistema **opera elaborando un job alla volta**; all'interno dei job esegue i programmi uno dopo l'altro, con un solo programma in esecuzione in un dato momento.

I primi computer degli anni '60 non erano interattivi e utilizzavano le **schede perforate** come mezzo di input. **Un job** era costituito da un insieme di schede perforate contenenti sia il programma che i dati necessari all'elaborazione, e veniva caricato manualmente da un operatore. Questo processo manuale causava un rallentamento nell'utilizzo della

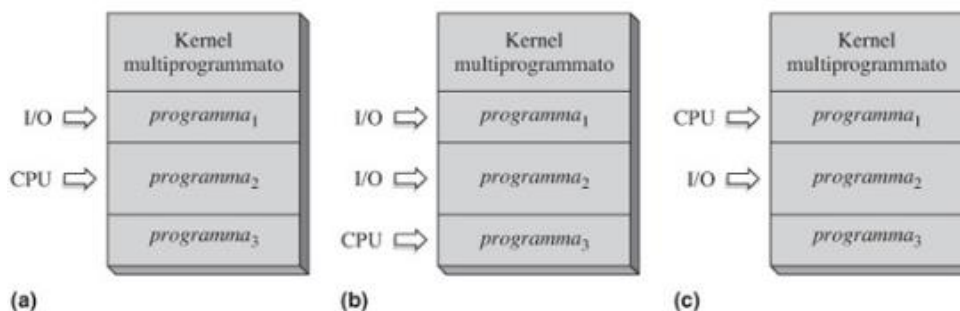
CPU. Per ottimizzare l'uso delle risorse e ridurre gli sprechi di tempo venne introdotta l'**elaborazione batch**. Il **batch** è una sequenza di job, separati da schede speciali che ne segnalavano l'inizio e la fine. L'elaborazione e l'esecuzione dei job erano gestiti dal **batching kernel**, riducendo la necessità di intervento manuale: l'operatore avviava il batch e interveniva solo alla fine per raccogliere i risultati. Per **aumentare il throughput** di questi sistemi, vennero introdotti **meccanismi di spooling**, che utilizzavano lettori di schede e unità di stampa virtuali per sovrapporre le operazioni di I/O con l'elaborazione della CPU. Questo permetteva di ridurre il collo di bottiglia rappresentato dai dispositivi di I/O e garantiva un flusso di elaborazione continuo. Per **evitare interferenze** tra job diversi, come il rischio che un job accedesse a schede non appartenenti ad esso, venivano inserite **istruzioni di controllo** all'interno del gruppo di schede che costituiva ciascun job. Era compito dell'**interprete dei comandi** verificare queste istruzioni ed effettuare le azioni appropriate per garantire il corretto svolgimento dell'elaborazione.

Sistemi Multi programmati

Un Sistema Operativo Multi programmato **si focalizza sull'uso efficiente** sia della CPU che dei dispositivi di I/O. Un **SO multi-programmato** consente l'esecuzione **concorrente** di più programmi utente mantenendoli in memoria e alternandone l'esecuzione sulla CPU. La particolarità di questo tipo di sistema è che le operazioni di I/O vengono gestite dal **DMA**, riducendo il coinvolgimento diretto della CPU che può così dedicarsi esclusivamente all'elaborazione dei programmi. Grazie a questa organizzazione si ottiene un **uso efficiente** sia dei dispositivi di I/O che della CPU, poiché entrambe le operazioni possono svolgersi in parallelo. Il kernel di un SO multi-programmato **gestisce lo scheduling dei processi, la memoria** (tramite allocazione basata su partizionamento o sul pool) e **le operazioni di I/O**.

Per prevenire interferenze tra programmi, viene implementata la **protezione della memoria**, che impedisce a un processo di accedere a locazioni al di fuori del proprio spazio di indirizzamento. Questo è controllato dal registro base e dal registro limite della CPU, permettendo la coesistenza sicura di più programmi in memoria. Un SO Multi programmato utilizza la modalità utente per eseguire i programmi utente e la modalità kernel per l'esecuzione di istruzioni privilegiate.

In un SO multi-programmato per misurare il **servizio per l'utente** si utilizza il tempo di **turnaround**, che misura il tempo totale necessario per completare l'esecuzione di un programma. Questo valore dipende dal numero totale di programmi in esecuzione e dalla strategia con cui il kernel condivide la CPU tra essi.



Priorità dei programmi

Per misurare le prestazioni di un SO multi-programmato si fa riferimento al **throughput**, che rappresenta il numero di job, programmi, processi e sotto richieste completati da un sistema in una unità di tempo. Il throughput dipende anche da quanta elaborazione e da quanto I/O eseguono i programmi, oltre che dall'efficienza con cui il kernel riesce a sovrapporre le loro attività nel tempo.

La formula del **throughput** è:

$$n/(t_f - t_0)$$

Dove t_0 e t_f rappresentano rispettivamente l'istante di tempo finale e iniziale e n il numero di programmi.

Un elevato grado di multiprogrammazione (numero di programmi mantenuti in memoria) non garantisce necessariamente un utilizzo efficiente della CPU e dei dispositivi di I/O. È fondamentale avere un giusto **mix di programmi** che sono sia **CPU-bound** (che richiedono molta elaborazione), e **I/O-bound** (che richiedono molte operazioni di I/O).

Un SO multi-programmato utilizza lo **scheduling a priorità con prelazione** per gestire la condivisione della CPU tra i programmi. La **priorità** è un criterio mediante il quale lo scheduler decide quale processo eseguire quando più processi sono in attesa.

Questo meccanismo viene implementato dal kernel assegnando priorità numeriche ai processi: in questo modo la CPU esegue sempre il processo a più alta priorità e, se necessario, interrompe (prelazione) quelli a priorità inferiore.

In genere, nei sistemi multi-programmati, **un processo I/O bound riceve una priorità più alta rispetto a un processo CPU-bound**, poiché questo permette di ridurre il tempo di attesa medio e migliorare l'utilizzo delle risorse del sistema.

Questo accade perché se i processi CPU-bound avessero sempre la priorità, potrebbero occupare la CPU per lunghi periodi senza lasciare spazio ai processi I/O-bound, che rimarrebbero bloccati in attesa di essere eseguiti. Questo causerebbe un sottoutilizzo dei dispositivi di I/O.

Al contrario, **se si dà priorità ai processi I/O-bound**, questi possono eseguire rapidamente le loro operazioni di I/O e poi passare allo stato di attesa, lasciando la CPU libera per altri processi.

Questo sovrappone meglio l'uso della CPU e dell'I/O, **migliorando l'efficienza complessiva**.

Ad esempio, in un sistema con un processo CPU-bound che esegue calcoli intensivi e un processo I/O-bound che legge dati da un disco, se il processo CPU-bound avesse la priorità, il processo I/O-bound dovrebbe aspettare prima di poter inviare la sua richiesta al disco.

Questo farebbe sì che il disco rimanesse inattivo per lunghi periodi. Invece, dando priorità al processo I/O-bound, il sistema può inviare subito la richiesta al disco, permettendo poi alla CPU di lavorare su altri processi mentre il disco è occupato.

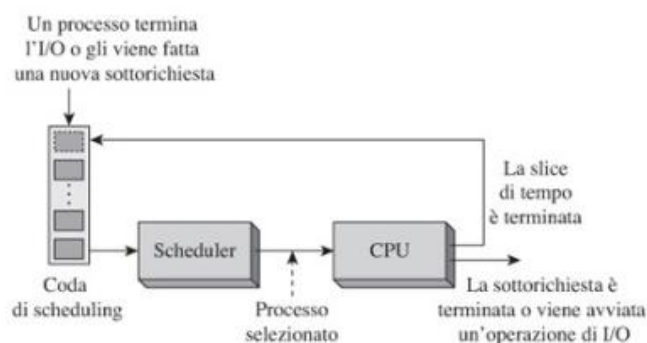
Sistemi time-sharing

Un **Sistema Operativo time-sharing** è progettato per garantire una condivisione equa della CPU tra più processi interattivi, migliorando la responsività e il tempo di risposta. Ciò è ottenuto attraverso lo **scheduling round-robin con time-slicing** (nei kernel time-sharing) che suddivide equamente il tempo di CPU tra i processi in esecuzione.

Il funzionamento di questo tipo di scheduling consiste nel mantenere una coda di processi pronti.

Il Kernel assegna la CPU al processo in testa alla coda e, se questo termina o avvia un'operazione di I/O, viene rimosso e spostato in uno stato di attesa. Se invece il processo supera il **time-slice** assegnato, un **interrupt timer** provoca la **prelazione**, sospendendolo e spostandolo alla fine della coda per permettere l'esecuzione di altri processi. Un processo può essere schedulato più volte prima di completare la propria esecuzione.

Nota: Il **time-slice** (δ) è la più grande porzione di tempo di CPU che ogni processo può utilizzare quando viene schedulato per essere eseguito dalla CPU.



Per misurare il **servizio per l'utente** si calcola il **tempo di risposta** (rt) che rappresenta il tempo utilizzato per eseguire una sotto richiesta, dato dalla formula:

$$rt = n \times (\delta + \sigma)$$

Dove:

- n è il numero di utenti che stanno utilizzando il sistema in un dato momento.
- δ secondi il tempo di CPU necessario per il completamento della sottorichiesta di ogni utente
- σ l'overhead dello scheduling, ovvero il tempo di CPU utilizzato dal kernel per eseguire la schedulazione.

Swapping dei programmi

Per misurare le prestazioni di un sistema operativo time-sharing si utilizza il **throughput delle sotto richieste**, che rappresenta il numero di sotto richieste completate per unità di tempo.

Consideriamo un sistema in cui la memoria può ospitare solo **due processi**. Supponiamo che questi due processi utilizzino la CPU per 45ms per eseguire una sotto richiesta, lasciando la CPU in idle, e successivamente eseguono operazioni di I/O fino a 125ms. Se dopo 125ms, gli utenti non inviassero immediatamente nuove sotto richieste per questi processi, il throughput del sistema si ridurrebbe, poiché la CPU rimarrebbe inutilizzata. Inoltre, anche il tempo di risposta delle sotto richieste inviate ad altri processi ne risentirebbe, in quanto il sistema sarebbe meno reattivo nella gestione delle richieste.

Per poter servire un numero maggiore di processi rispetto a quelli che possono essere effettivamente mantenuti in memoria, si utilizza la tecnica dello **swapping**. Lo Swapping consiste nel rimuovere temporaneamente un processo dalla memoria RAM per far spazio ad altri processi.

Se un processo non deve essere eseguito immediatamente, il kernel effettua un'operazione di **swap-out**, liberando la memoria che gli era stata assegnata. Quando il processo dovrà essere nuovamente schedato per eseguire una nuova sotto richiesta, il kernel lo ricaricherà in memoria tramite un'operazione di swap-in.

Considerando l'esempio precedente, i due processi che hanno completato la gestione delle loro sotto richieste possono essere **swappati su disco**, liberando memoria per caricare altri processi ai quali sono state fatte nuove sotto richieste e che hanno quindi bisogno di essere schedati.

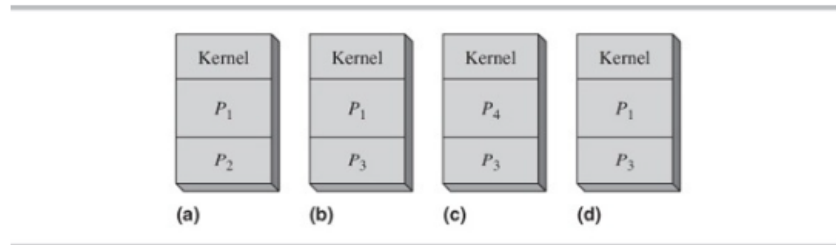


Figura 3.8 Swapping: (a) i processi in memoria tra 0 e 105 ms; (b) P_2 è sostituito da P_3 a 105 ms; (c) P_1 è sostituito da P_4 a 125 ms; (d) P_1 viene swappato per servire la successiva sottorichiesta.

Sistemi Operativi Real-Time

I SO real-time sono stati sviluppati per poter eseguire **applicazioni real-time**, ovvero applicazioni che devono rispondere agli eventi di un sistema esterno entro un tempo massimo stabilito da quel sistema.

Si usa il termine **requisito di risposta** di un sistema per indicare il più grande valore di tempo di risposta per il quale il sistema può funzionare correttamente.

Ad esempio, se un satellite invia 500 campioni al secondo, significa che l'applicazione real-time riceve un campione ogni 2ms (1000ms/500), essa deve necessariamente rispondere alla richiesta di memorizzazione in meno di 2ms, in caso contrario l'arrivo di un nuovo campione cancellerebbe il precedente dalla memoria.

Quindi, possiamo affermare che il **requisito di risposta** dell'applicazione in questione è di **1.99ms**, il che significa che il sistema ha al massimo questo tempo per salvare un campione prima che arrivi il successivo.

Con il termine **deadline** ci si riferisce al tempo massimo entro cui un'operazione deve essere completata per garantire il corretto funzionamento del sistema.

Formalmente, se un'operazione inizia al tempo t e deve essere completata entro un tempo massimo R (requisito di risposta), allora la sua deadline è:

$$Deadline = t + R$$

Dove:

- t è il momento in cui l'evento (es. ricezione di un campione) si verifica.
- R è il requisito di risposta, ovvero il tempo massimo concesso per completare l'operazione.

Quindi, in riferimento all'esempio precedente, se un campione arriva al tempo $t = 300ms$, il sistema ha 1.99ms per salvarlo e la sua **deadline** sarà:

$$300 + 1.99 = 301.99ms$$

Quindi, se il campione non viene salvato entro 301,99ms, verrà sovrascritto dal successivo, causando perdita di dati.

Sistemi Hard e Soft real-time

Un **sistema Hard Real-Time** è un sistema dedicato ad elaborare applicazioni real-time e soddisfa il requisito di risposta in ogni condizione. Ad esempio, un sistema di controllo per l'**airbag di un'automobile**: il gonfiaggio deve avvenire entro pochi millisecondi dall'impatto. Un ritardo, anche minimo, comprometterebbe la sicurezza e renderebbe il sistema inaccettabile.

Un **sistema Soft Real-Time** soddisfa i requisiti di risposta in maniera probabilistica (es. 98% delle volte) ma non può garantire che lo farà in ogni condizione. Ad esempio, un sistema di **streaming video**: i pacchetti devono essere consegnati in tempo per garantire la fluidità della riproduzione, ma occasionali ritardi o perdita di frame non compromettono la funzionalità globale del servizio, incidendo al massimo sulla qualità percepita.

Caratteristiche di un sistema operativo real-time

Un **applicazione real-time** può essere progettata affinché il SO possa eseguire alcune sue parti **concorrentemente** (contemporaneamente), ovvero come processi separati, e assegnare a ciascuno di essi delle **priorità**, impiegando uno **scheduling basato su priorità o deadline**. Lo **scheduling basato su deadline** è una tecnica che garantisce che i processi vengano eseguiti in modo da rispettare le loro scadenze temporali. Questo è particolarmente cruciale nei sistemi hard real-time, dove il mancato rispetto di una deadline può causare gravi malfunzionamenti. Se un processo avvia un'operazione di I/O, il SO può assegnare la CPU a un altro processo per ottimizzare l'uso delle risorse, proprio come avviene nella multiprogrammazione. In questo modo, le attività di CPU e I/O dell'applicazione possono essere sovrapposte contribuendo a ridurre il tempo di esecuzione.

Un SO real-time deve avere la possibilità di **associare azioni specifiche ad eventi**. Ciò consente di reagire prontamente a situazioni critiche.

Un altro requisito essenziale è la **predicibilità**, poiché è cruciale poter calcolare con precisione il tempo massimo di esecuzione di ogni operazione. Per questo motivo nei sistemi hard real-time si evita l'utilizzo della memoria virtuale, che introduce ritardi imprevedibili.

Per **garantire continuità** in caso di guasti un SO real-time utilizza tecniche come la **fault tolerance** che utilizza risorse ridondanti (es. 2 dischi) per mantenere il funzionamento del sistema anche in presenza di malfunzionamenti, e la **graceful degradation** che consente al sistema di ridurre gradualmente le prestazioni anziché interrompersi del tutto, assicurando che le funzioni più critiche continuino a poter essere eseguite.

Sistemi Operativi Distribuiti

Un **sistema distribuito** è composto da due o più nodi, dove ogni nodo è un computer con un proprio clock e memoria, collegato attraverso una rete. Ogni nodo può svolgere funzioni di controllo di un SO.

Un sistema operativo distribuito sfrutta la molteplicità delle risorse e la connessione tramite rete per offrire numerosi vantaggi:

- la **condivisione delle risorse** tra nodi;
- **maggiore affidabilità** grazie alla ridondanza;
- **l'incremento della velocità di elaborazione** grazie all'esecuzione parallela delle applicazioni su nodi differenti per velocizzarne l'elaborazione;
- la **comunicazione** tra gli utenti indipendentemente dalla loro posizione nel sistema.

Tecniche speciali di sistemi operativi distribuiti

I SO distribuiti utilizzano il **controllo distribuito**, in cui le **funzioni di gestione e controllo sono suddivise tra più nodi**, a differenza del controllo centralizzato, dove un solo nodo centrale gestisce tutto. Questo approccio riduce il rischio che un malfunzionamento di un singolo nodo comprometta l'intero sistema.

Grazie alla **trasparenza** gli utenti possono accedere alle risorse o ai servizi condivisi senza dover conoscere quale nodo li ospiti fisicamente.

Utilizzando la **Remote procedure call (RPC)** è possibile invocare una funzione o procedura su un computer remoto attraverso la rete. La RPC funziona in modo simile a una funzione o metodo in un linguaggio di programmazione, ma i parametri vengono inviati tramite la rete e i risultati restituiti allo stesso modo.

Moderni Sistemi Operativi

I moderni SO ereditano molti dei concetti e delle tecniche delle varie tipologie di SO discusse precedentemente.

L'**elaborazione batch** viene usata durante le transazioni su database, in elaborazioni di ufficio e scientifiche; È implementato lo **scheduling a priorità con prelazione**, che, anziché assegnare priorità fisse come nei sistemi multi programmati, assegna priorità variabili ai processi, aumentando la loro priorità se sono stati coinvolti in interazioni o operazioni di I/O e la riduce in caso contrario. Viene utilizzato il concetto di **Time-Slicing** per aumentare il tempo di

risposta dei processi e prevenire che un processo monopolizzi la CPU. Viene utilizzata la **memoria virtuale** e la tecnica dello **swapping dei processi** su disco che permette al Kernel di allocare dinamicamente memoria RAM ai processi. Infine, il Kernel permette la **creazione di processi multipli** in un'applicazione, la **condivisione delle risorse** come stampanti e servizi e il **plug-and-play** con la quale è possibile connettere periferiche di I/O al computer mentre è in funzione.

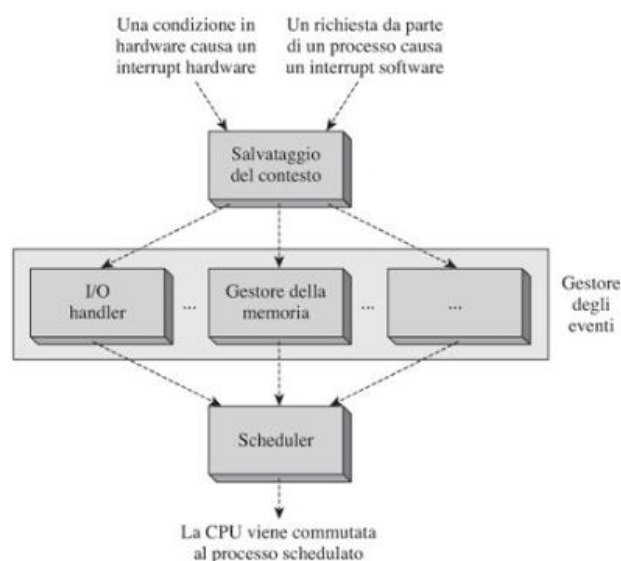
Capitolo 4 - Struttura dei Sistemi Operativi

I moderni Sistemi Operativi sono implementati nella forma di un nucleo, detto kernel o microkernel, e costruiscono il resto del sistema operativo utilizzando i servizi offerti dal nucleo.

Funzionamento di un SO

All'accensione del computer attraverso la **procedura di boot**, viene analizzata la configurazione del computer (CPU, memoria, dispositivi I/O), e successivamente **viene caricata una parte del SO in memoria** e inizializzate le strutture dati con le informazioni ottenute.

In figura una visione d'insieme del funzionamento di un SO.



Un evento di natura hardware o software genera un **interrupt**, causando la sospensione dell'esecuzione corrente. La CPU salva il contesto del processo interrotto e trasferisce il controllo alla corrispondente **Interrupt Service Routine (ISR)**, che gestisce l'evento. Una volta completata la gestione, lo **scheduler** sceglie il prossimo processo (può essere anche il processo interrotto precedentemente) da eseguire e **ripristina il contesto**, riprendendo l'esecuzione.

Molte funzioni di un SO sono attivate da interrupt e gestite da apposite routine. Tra queste:

- la **gestione dei processi** comprende la creazione, la terminazione, lo scheduling e la sincronizzazione dei processi;
- la **gestione dell'I/O** include il controllo dei dispositivi, il buffering, lo spooling e la gestione dei driver;
- la **gestione dei file** riguarda l'accesso, l'organizzazione del filesystem e la protezione dei dati;
- la **sicurezza e la protezione** comprendono meccanismi di autenticazione, controllo degli accessi e difesa da minacce.

Infine, il SO si occupa della **gestione delle comunicazioni**, supportando la comunicazione tra processi e la gestione della rete.

Struttura di un SO

Nel determinare come un SO esegue una funzione, il progettista deve considerare due livelli distinti:

- **Politica:** È un principio guida che stabilisce **cosa** il SO dovrebbe fare per svolgere una determinata funzione. Una politica definisce le decisioni da prendere ed è implementata come un modulo decisionale che seleziona quale meccanismo eseguire e in quali circostanze. Un esempio di politica è lo scheduling round-robin.
- **Meccanismo:** È **l'insieme delle azioni** specifiche necessarie per implementare una politica. Viene realizzato come un modulo che esegue operazioni concrete. Ad esempio, per implementare lo scheduling round-robin, i meccanismi coinvolti includono la gestione di una coda dei processi pronti e la commutazione della CPU per eseguire il processo selezionato (operazione di dispatching).

Portabilità ed espandibilità dei SO

Un SO deve adattarsi ai cambiamenti degli ambienti di elaborazione, nell'hardware e nei dispositivi di I/O. Per questo motivo, due caratteristiche fondamentali in questo contesto sono la **portabilità** e l'**espandibilità**.

Portabilità

Per **porting** si intende l'adattamento di un software per il suo utilizzo su un'altra piattaforma hardware.

La **portabilità** misura la facilità con cui un software può essere sottoposto a porting ed è inversamente proporzionale allo sforzo necessario per adattarlo a un nuovo ambiente.

Il **porting di un SO** implica la modifica delle parti del codice strettamente dipendenti dall'architettura hardware, affinché il sistema possa funzionare correttamente su un nuovo hardware. Alcuni esempi di componenti dipendenti dall'architettura HW sono il vettore delle interruzioni, le informazioni di protezione della memoria, istruzioni di I/O per effettuare le operazioni di I/O ecc...

La parte di codice di un SO che dipende dall'architettura riguarda principalmente i **meccanismi** piuttosto che le **politiche**. Per garantire **un'elevata portabilità**, è fondamentale che il codice strettamente dipendente dall'architettura sia ridotto al minimo. Per questo motivo, si è adottato un approccio modulare, separando il codice dipendente dall'hardware da quello indipendente e definendo interfacce ben precise tra le due parti.

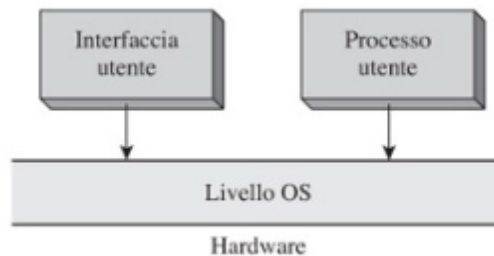
Espandibilità

Inizialmente i primi SO non prevedevano alcuna forma di **espandibilità**, ovvero la possibilità di aggiungere nuove funzionalità o supportare nuovo hardware. Tuttavia, con il tempo, questa esigenza è diventata sempre più importante per permettere l'integrazione di nuovi dispositivi e soddisfare le aspettative degli utenti. Nei SO successivi, il problema è stato affrontato introducendo, durante la fase di boot, un meccanismo per il rilevamento dell'hw. In caso di nuovo dispositivo, il sistema chiedeva all'utente di fornire il relativo driver, che veniva caricato e integrato nel Kernel per garantirne il corretto funzionamento.

Nei moderni SO è possibile aggiungere nuovo HW anche durante il funzionamento del sistema grazie alla tecnologia plug-and-play, che consente il riconoscimento e la configurazione automatica dei dispositivi senza richiedere un intervento manuale da parte dell'utente.

Sistemi Operativi con struttura Monolitica

I primi SO adottavano una **struttura monolitica** caratterizzata da un **singolo strato software** tra l'utente e l'hardware. L'interfaccia utente era costituita da un **interprete dei comandi (shell)** che permetteva di eseguire operazioni richiamando le funzionalità del sistema operativo tramite **system call**.



Essendo strettamente integrato con l'hardware, **il codice del sistema operativo risultava altamente dipendente dall'architettura specifica**, rendendo la portabilità verso altre piattaforme estremamente difficile. Inoltre, la mancanza di una chiara modularità rendeva il test e il debugging particolarmente complessi, aumentando significativamente i costi di manutenzione e aggiornamento. Per superare questi limiti, furono sviluppate architetture alternative, come la **struttura a livelli**, la **struttura basata su Kernel** e la **struttura basata su microKernel**.

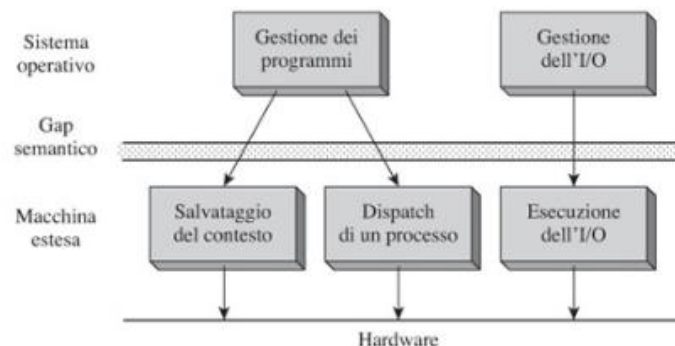
Sistemi Operativi strutturati a livelli

I sistemi operativi **strutturati a livelli** nascono per superare i limiti della struttura monolitica, in cui tutte le componenti devono interagire direttamente con l'hardware, aumentando complessità e costi di sviluppo.

Il problema principale è il **gap semantico**, ovvero la distanza tra le operazioni fornite dall'hardware e quelle richieste dal sistema operativo, o in altre parole, la differenza tra il livello a cui lavora l'hardware (operazioni primitive, es. "leggere un byte dalla memoria") e il livello a cui lavora il SO (es. "salva un file"). Per ridurre questo divario, si può utilizzare un **hardware con funzioni più avanzate** (soluzione costosa) o **simulare una macchina più potente** tramite software, creando una **macchina estesa** che funge da intermediario tra hardware e sistema operativo.

Ad esempio, invece di dover dire alla CPU come accendere ogni pixel dello schermo, il SO offre un comando "Mostra questa finestra" implementato tramite macchina estesa, simulata via software e creata attraverso vari livelli di astrazione nel SO.

Un esempio pratico di macchina estesa è la macchina virtuale Java: i programmi Java non comunicano direttamente con il processore fisico, ma con la JVM, che traduce il codice in istruzioni comprensibili dall'hardware del computer.



Nei **sistemi operativi a livelli**, ogni livello utilizza solo le funzioni del livello immediatamente inferiore, senza poter accedere direttamente agli altri. Questo approccio garantisce l'**information hiding**, impedendo l'uso improprio o la corruzione dei dati tra livelli. Inoltre, facilita il debugging e la manutenzione, poiché un errore a un livello può derivare solo da quel livello stesso.

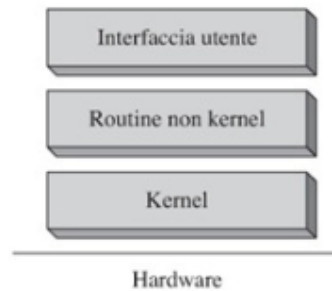
L'organizzazione a livelli semplifica la codifica e il test del sistema operativo, separando le funzioni dalla loro implementazione.

Gli SO progettati a livelli soffrono di tre problemi:

1. **Rallentamento dell'esecuzione:** Poiché ogni livello può interagire solo con il livello immediatamente inferiore, una richiesta di servizio deve attraversare più livelli prima di essere elaborata dall'hardware, generando un elevato overhead e rallentando il sistema.
2. **Difficoltà nello sviluppo:** L'ordine dei livelli deve essere attentamente pianificato affinché ciascun livello abbia accesso alle funzionalità necessarie. Questo può portare alla suddivisione di livelli esistenti o all'introduzione di nuovi livelli intermedi, rendendo più complessa la progettazione. Ad esempio, la gestione dei processi può essere separata in due livelli per risolvere dipendenze con la gestione della memoria.
3. **Stratificazione e scarsa espandibilità:** Le funzionalità del sistema devono essere suddivise tra livelli diversi, e la comunicazione tra di essi avviene tramite interfacce, come l'uso di interrupt. Questo può complicare il progetto e ridurre l'efficienza. Inoltre, l'aggiunta di nuove funzionalità richiede modifiche a più livelli, rendendo

difficile l'espansione e riducendo la portabilità del sistema operativo, poiché molte parti rimangono dipendenti dall'architettura hardware.

Sistemi Operativi basati su Kernel



I Sistemi Operativi basati su **kernel** hanno un **nucleo centrale** che fornisce servizi essenziali e gestisce le interazioni con l'hardware, mentre il resto del SO è composto da routine non kernel che gestiscono operazioni utente e interfacce grafiche. I **meccanismi** sono implementati nel Kernel, mentre le **politiche** sono implementate all'esterno.

Il kernel risponde agli **interrupt**, che segnalano eventi come il **completamento di un'operazione di I/O** o una **system call** (richiesta di servizio da parte di un processo o dell'utente). Quando un utente esegue un programma, ad esempio un file chiamato alpha, vengono effettuate più chiamate di sistema per allocare memoria, aprire il file, caricarlo in RAM e avviarne l'esecuzione.

Il **vantaggio** principale di questa struttura è la **portabilità**, perché le parti dipendenti dall'hardware sono concentrate nel kernel, rendendo più semplice l'adattamento del SO a diverse architetture. Tuttavia, il kernel è spesso monolitico per garantire efficienza, mentre la parte non Kernel può essere organizzata a livelli o in modo monolitico. L'altro vantaggio è la **semplicità di progettazione** e codifica delle routine non Kernel.

I problemi principali includono:

- **Overhead elevato:** L'uso di system call e interrupt per comunicare tra kernel e routine non kernel può rallentare il sistema. Alcuni SO riducono questo problema spostando nel kernel alcune funzioni non kernel, riducendo l'overhead ma compromettendo la portabilità.
- **Difficoltà di espansione:** Aggiungere nuove funzionalità spesso richiede modifiche al kernel stesso, rendendo il sistema meno flessibile ed estendibile.
- **Stratificazione complessa:** La separazione tra kernel e routine non kernel può complicare il design, simile ai problemi dei SO a livelli.

Nonostante questi limiti, i sistemi basati su kernel rimangono una soluzione diffusa per la loro efficienza e organizzazione strutturata.

Le funzioni e i servizi tipici offerti dal Kernel a supporto delle funzionalità del SO sono:

Funzionalità del SO	Esempi di funzioni e servizi del kernel
Gestione dei processi	Salva il contesto del programma interrotto, effettua il dispatch di un processo, manipola le liste di scheduling.
Comunicazioni tra processi	Invia e riceve messaggi dai processi.
Gestione della memoria	Imposta le informazioni relative alla protezione della memoria, effettua lo swap-in e lo swap-out, gestisce i page fault (ovvero l'interrupt "non presente in memoria", detto missing from memory, del Paragrafo 1.4).
Gestione dell'I/O	Avvia le operazioni di I/O, elabora l'interrupt generato dal completamento di un'operazione di I/O, effettua il recupero da errori di I/O.
Gestione dei file	Apri un file, legge/scrive dati.
Sicurezza e protezione	Aggiunge informazioni di autenticazione per un nuovo utente, conserva le informazioni per la protezione dei file.
Gestione della rete	Invia/riceve dati attraverso messaggi.

Evoluzione della struttura basata su Kernel

I SO basati su Kernel si sono evoluti per superare alcuni dei loro limiti, introducendo soluzioni che ne migliorano l'efficienza e la flessibilità. Una delle innovazioni più significative è stata l'introduzione del caricamento dinamico dei moduli, che ha permesso di strutturare il kernel come **un insieme di componenti indipendenti**, in grado di interagire tra loro attraverso interfacce ben definite.

Il **kernel di base** viene caricato durante l'avvio del sistema, mentre i **moduli aggiuntivi** possono essere caricati su richiesta e rimossi dalla memoria quando non sono più necessari. Questo approccio consente di ottimizzare l'uso della memoria e di rendere il SO più espandibile, poiché nuovi moduli possono essere facilmente integrati senza dover modificare il kernel principale.

Tuttavia, l'uso di moduli dinamici presenta anche alcuni **svantaggi**. Uno dei principali è la **frammentazione della memoria**, che può generare un overhead significativo dovuto alle operazioni di gestione della memoria necessarie per ridurre le richieste di spazio. Inoltre, un **bug** in un modulo del kernel può compromettere la stabilità del sistema, causando crash e riavvii improvvisi.

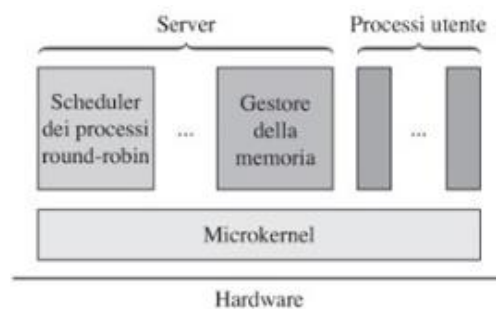
I moduli del kernel trovano applicazione in diversi ambiti, tra cui **l'implementazione di driver** per dispositivi di I/O, adattatori di rete, nuovi file system e persino l'aggiunta di nuove system call. Tuttavia, i driver che operano con privilegi del Kernel possono rappresentare un rischio per la stabilità del sistema: un malfunzionamento o un errore nel codice può causare blocchi improvvisi e frequenti riavvii.

Per ridurre questi rischi, una soluzione efficace è l'esecuzione dei **driver in modalità utente**. Questo approccio offre diversi vantaggi, tra cui una maggiore facilità di sviluppo, debugging semplificato, maggiore robustezza e una migliore espandibilità del SO. In passato, i driver in modalità utente soffrivano di problemi di prestazioni, ma questi sono stati progressivamente mitigati grazie ai progressi sia hardware che software, rendendo questa soluzione sempre più praticabile.

Sistemi Operativi basati su microKernel

I **microKernel** sono stati sviluppati per migliorare **portabilità, espandibilità e affidabilità** rispetto ai Kernel monolitici. Nei Kernel tradizionali, molte funzionalità sono integrate direttamente nel nucleo del sistema, rendendoli complessi, difficili da aggiornare e non agevolmente portabili. Al contrario, un microKernel include solo le **funzioni essenziali** (gestione della memoria, schedulazione dei processi e comunicazione tra processi), mentre altri servizi, come i driver e la gestione avanzata della memoria, operano nello spazio utente come **server separati**.

Nota: I server separati sono processi indipendenti eseguiti nello spazio utente che forniscono le funzionalità del SO. Ad esempio, un server per il file system gestisce la lettura/scrittura su disco, un server per la gestione dei processi si occupa della creazione e terminazione dei processi, i driver dei dispositivi possono essere eseguiti come server separati, invece che nel kernel.



Questa architettura offre diversi vantaggi:

- **Maggiore modularità**, perché i servizi possono essere aggiornati senza dover modificare il Kernel;
- **Maggiore stabilità**, dato che eventuali bug nei componenti esterni non compromettono l'intero sistema;
- **Migliore portabilità**, poiché il microKernel è meno dipendente dall'hardware specifico.
- **Espandibilità**, permettendo l'aggiunta di nuove funzionalità senza dover modificare il kernel.

I microKernel sono ideali per **sistemi embedded e distribuiti**, poiché permettono di adattare il SO a diversi ambienti.

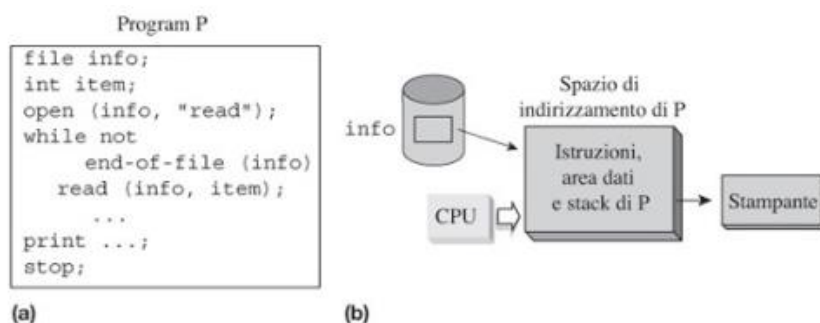
Tuttavia, la prima generazione di microKernel soffriva di problemi di prestazioni, con una perdita di throughput fino al 50% rispetto ai kernel monolitici. Questo era dovuto al costante scambio di messaggi tra kernel e server utente tramite IPC. Per risolvere il problema, i microkernel di seconda generazione, come L4, hanno ottimizzato il sistema di comunicazione riducendo il numero di controlli e migliorando l'integrazione con l'hardware, abbassando la perdita di prestazioni a circa il 5%.

Un'alternativa ancora più radicale ai microkernel è l'**exokernel**, che elimina le astrazioni e consente alle applicazioni di gestire direttamente le risorse hardware, ottenendo prestazioni molto elevate.

Capitolo 5 – Processi e Thread

Differenza tra processo e programma

La differenza tra programma e processo è che un programma è un'entità passiva che non effettua alcuna operazione da solo, mentre un **processo è un programma in esecuzione** che utilizza un insieme di risorse. Il Kernel alloca le risorse ai processi e li schedula per l'utilizzo della CPU.



La figura a sinistra mostra un programma e a destra una sua visione astratta. Le istruzioni, i dati e lo stack del programma costituiscono il suo **spazio di indirizzamento**, ovvero l'insieme di indirizzi di memoria che il programma può utilizzare durante la sua esecuzione. Affinché il programma possa essere eseguito, il SO **alloca la memoria** per lo spazio di indirizzamento, ed eventuali periferiche, dopodiché il processo verrà schedato per l'esecuzione.

Quando un utente avvia l'esecuzione di un programma, il SO **crea un nuovo processo**, gli **alloca una quantità sufficiente di risorse**, tra cui la memoria per contenere lo spazio di indirizzamento, e gli **assegna un identificativo univoco** (PID). Durante l'esecuzione, il processo può effettuare chiamate di sistema per richiedere ulteriori risorse.

Un processo è composto da sei componenti:

- **ID (PID)** è l'identificativo univoco assegnato dal SO.
- **codice** del programma.
- **dati** usati durante l'esecuzione, inclusi i dati contenuti nel file.
- **stack** contiene i parametri delle funzioni e delle procedure chiamate durante l'esecuzione del programma.
- **risorse allocate** dal SO.
- **stato della CPU** composto dal contenuto del PSW e dai registri General Purpose (GPR) della CPU (si assume che il puntatore allo stack venga conservato in uno dei GPR).

Program counter (PC)	Condition code (CC)	Mode (M)	Memory protection information (MPI)	Interrupt mask (IM)	Interrupt code (IC)
----------------------	---------------------	----------	-------------------------------------	---------------------	---------------------

Ricordiamo che PSW:

Relazioni tra processi e programmi

Si ha una relazione **uno a uno** tra un processo e un programma quando esiste **una singola istanza sequenziale** di un programma in esecuzione.

Una relazione **molti a uno** esiste tra molti processi e un programma in due casi:

- Molte istanze di un programma possono essere in esecuzione contemporaneamente;
- I processi che rappresentano queste istanze hanno una relazione molti a uno con il programma.

Durante l'esecuzione, un programma può effettuare una **system call** per richiedere che una parte specifica del suo codice venga eseguita in maniera **concorrente**, ovvero come una attività separata che viene eseguita simultaneamente ad altre attività. Il kernel gestisce questa nuova attività come un **processo separato** (o come un thread all'interno dello stesso processo).

Se viene creato un nuovo processo, questo è chiamato **processo figlio**, mentre il processo originario è chiamato **processo padre**. In questo caso, padre e figlio hanno una relazione uno a molti con il programma. Un programma che genera più processi o thread per eseguire attività contemporaneamente è chiamato **programma concorrente**. I processi concorrenti possono condividere **codice, dati e risorse** con altri processi e possono interagire tra loro durante l'esecuzione.

Processi figli

Il Kernel avvia l'esecuzione di un programma creando un processo, chiamato processo primario (o processo padre). Il **processo padre** può effettuare system call (**fork**) per creare altri processi. I processi appena creati prendono il nome di **processi figli**. Un processo figlio può a sua volta effettuare la stessa procedura, diventando in questo caso lui il processo padre dei processi figli creati. Un processo figlio può ereditare risorse dal processo padre e ottenere ulteriori risorse durante il suo funzionamento utilizzando le system call.

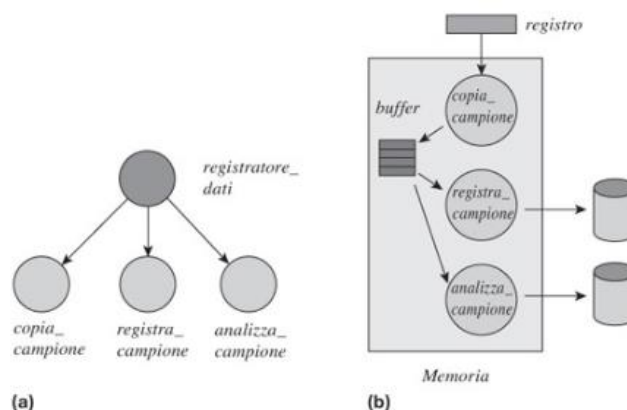
La relazione genitore-figlio che si viene a creare tra questi processi viene rappresentata come un **albero dei processi**, la cui radice è il processo primario.

La creazione di processi figli e quindi l'esecuzione concorrente porta **tre benefici**:

- **Aumento dello speedup dell'elaborazione:** Il Kernel può alternare l'esecuzione dei processi I/O bound e CPU bound dell'applicazione, diminuendo il tempo di esecuzione.
- **Priorità per le funzioni critiche:** Possibilità di soddisfare i requisiti **real time** di un'applicazione, assegnando una **priorità più alta** ad un processo figlio che deve effettuare una funzione critica.
- **Proteggere un processo genitore dagli errori:** Un errore come, ad esempio, una divisione per zero, porterebbe alla **terminazione del solo processo figlio** e non del padre. L'interprete dei comandi (la shell) del SO è anch'essa un processo, e utilizza questa tecnica (creazione di processi figli) quando deve eseguire un comando.

Inoltre, Il kernel fornisce funzioni per: **creare e terminare** un processo figlio, **assegnarli una priorità**, **determinare lo stato** di un processo figlio, **condivisione**, **comunicazione** e **sincronizzazione dei processi**.

Esempio: Processi figli in un'applicazione real-time



Esempio di applicazione real time. A sinistra c'è l'albero dei processi e a destra ci sono i processi.

L'applicazione real-time descritta riceve 500 campioni al secondo da un satellite e li salva in un file. Per gestire questi dati, il processo principale, chiamato **registratore_dati**, esegue tre operazioni: copiare il campione dalla memoria speciale in un buffer, registrarlo in un file e analizzarlo per salvarne il risultato in un altro file.

Per svolgere queste operazioni in **modo efficiente**, il processo principale crea tre processi figli: **copia_campione**, **registra_campione** e **analizza_campione**. Questi processi vengono eseguiti in **concorrenza**, cioè si alternano nell'uso della CPU grazie alla gestione del Kernel.

- Il processo **copia_campione** ha la **priorità più alta**, perché deve assicurarsi che ogni campione venga copiato prima dell'arrivo del successivo.

- Gli altri due processi, **registra_campione** e **analizza_campione**, lavorano sugli stessi dati salvati nel buffer e devono coordinarsi: **registra_campione** può scrivere un campione nel file solo dopo che **copia_campione** lo ha salvato nel buffer.

Differenza tra parallelismo e concorrenza

Il **parallelismo** consiste nell'eseguire **più operazioni simultaneamente**, mentre la **concorrenza** è un'illusione di **parallelismo**, in cui più task sembrano eseguirsi allo stesso tempo ma, in realtà, vengono eseguiti alternatamente.

Nei SO, la concorrenza si ottiene alternando l'esecuzione dei processi sulla CPU, mentre il parallelismo richiede più CPU per eseguire processi simultaneamente.

- La **concorrenza** migliora il **throughput** permettendo la **sovrapposizione tra operazioni di I/O e computazione di diversi processi**, come visto nei sistemi multi-programmati e time-sharing.
- Il **parallelismo**, invece, aumenta il **throughput** eseguendo più operazioni in parallelo su CPU multiple.

Lo **speedup** ottenuto dipende da:

- **Parallelismo intrinseco dell'applicazione**, ovvero la capacità di eseguire operazioni indipendenti;
- **Overhead della gestione della concorrenza e del parallelismo**, che non deve superare i benefici dell'esecuzione concorrente;
- **Supporto del SO**, ossia il modello adottato e il suo impatto sulle prestazioni.

Implementazione dei processi

Dal punto di vista del SO, un processo è un'istanza di un programma in esecuzione e rappresenta un'unità di elaborazione. Il Kernel si occupa di **allocare le risorse al processo**, **proteggerlo da interferenze** causate da altri processi e garantire che possa accedere alla CPU secondo le politiche di scheduling del SO.

Quando si verifica un **evento** nel sistema, come un **interrupt hardware** o una **chiamata di sistema**, il Kernel esegue una serie di operazioni fondamentali attraverso la **routine di gestione dell'evento**:

- **Salvataggio del contesto**: salva lo stato della CPU e le informazioni del processo interrotto;
- **Gestione dell'evento**: Analizza la causa dell'interrupt ed esegue le operazioni necessarie per gestirlo.
- **Scheduling**: seleziona il prossimo processo da far eseguire alla CPU;
- **Dispatching**: ripristina il contesto di esecuzione del processo selezionato e gli concede l'accesso alla CPU.

Dopo queste operazioni, il controllo torna al processo selezionato, che può proseguire l'esecuzione.

Per poter controllare l'esecuzione dei processi il SO deve monitorare costantemente lo **stato dei processi**.

Stati del processo e transizioni di stato



Lo **stato di un processo** è un parametro che descrive la natura dell'attività corrente di un processo.

Molti SO considerano **quattro stati** fondamentali:

- **Blocked**: Il processo è **in attesa** che venga soddisfatta una richiesta di risorsa o che si verifichi un evento specifico.
- **Ready**: il processo è **pronto per essere eseguito** dalla CPU non appena questa diventa disponibile. Raggiunge questo stato quando la richiesta di risorsa è stata soddisfatta o si è verificato l'evento atteso.
- **Running**: Il processo **sta attualmente utilizzando la CPU** ed eseguendo le sue istruzioni. Un processo può passare da Running a Ready se viene prelazionato. Se invece il processo richiede un'operazione di I/O o una risorsa non immediatamente disponibile, passa allo stato Blocked fino al completamento dell'operazione.
- **Terminated**: Il processo ha completato la sua esecuzione o è stato interrotto a causa di un errore.

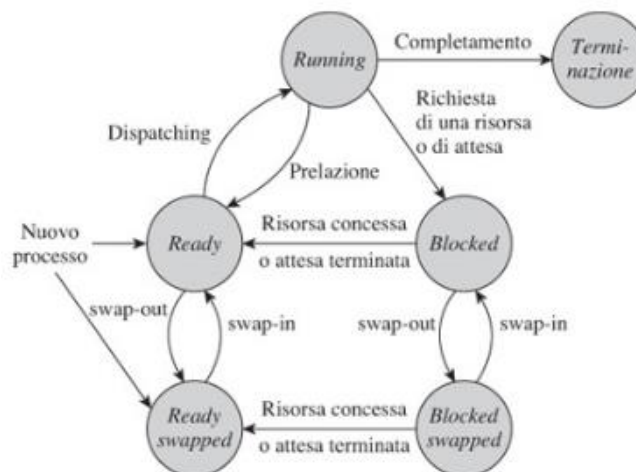
Cause delle transizioni di stato dei processi

Il SO gestisce le transizioni di stato di un processo durante la sua esecuzione. Le principali transizioni di stato sono:

- **Ready → Running:** Il processo viene eseguito dalla CPU dopo essere stato selezionato dallo scheduler.
- **Blocked → Ready:** Una richiesta del processo viene soddisfatta o si verifica un evento atteso, consentendogli di tornare pronto per l'esecuzione.
- **Running → Ready:** Il processo viene prelazionato dal Kernel per permettere l'esecuzione di un altro processo con priorità più alta o perché ha esaurito il suo time-slice.
- **Running → Blocked:** Il processo esegue una chiamata di sistema per attendere una risorsa o un evento. Le principali cause includono:
 - Richiesta di operazione di I/O.
 - Richiesta di una risorsa.
 - Necessità di essere sospeso per un certo tempo.
 - Attesa di un messaggio o azione da altri processi.
- **Running → Terminated:** Il processo termina la sua esecuzione. Le principali cause includono:
 - **Auto-terminazione:** Il processo completa il suo compito o incontra dati errati.
 - **Terminazione da un altro processo:** Un processo può terminare un suo processo figlio.
 - **Eccesso di utilizzo di risorse:** Il SO impone limiti sulle risorse, e il processo può essere terminato se li supera.
 - **Errori e condizioni anomale:** Esempi includono istruzioni illegali, overflow della memoria o violazioni di protezione.
 - **Errori hardware:** Malfunzionamenti della CPU o della memoria possono causare la terminazione.

Esempio 5.2 a pagina 122 del libro.

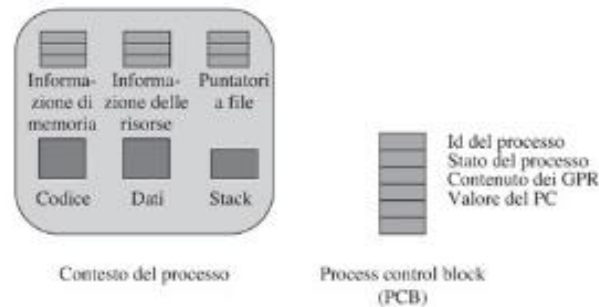
Processi sospesi



I **processi sospesi** sono stati aggiuntivi che il **kernel** utilizza per gestire quei processi che non si trovano nei quattro stati fondamentali. Un **processo** può essere sospeso se viene **swappato su disco** per **liberare memoria** o se l'utente richiede che **non venga schedato** per un certo periodo. In questi casi, il sistema operativo introduce due stati: **ready swapped** per i processi pronti ma spostati su disco e **blocked swapped** per quelli in attesa di una risorsa e swappati.

Un **processo** può passare dallo stato **ready** o **blocked** a quello **sospeso** tramite un'operazione di **swap-out**. Se la richiesta che lo bloccava viene soddisfatta, il processo rimane comunque su disco nello stato **ready swapped**, tornando in memoria solo quando le risorse lo permettono. Una volta riportato in memoria, rientra nello stato **ready** e compete per l'uso della CPU con gli altri processi pronti. Inoltre, un **nuovo processo** può essere creato direttamente nello stato **ready** o **ready swapped** in base alla disponibilità di memoria.

Il contesto del processo e il process control block



Il Kernel considera un processo composto da due parti:

1. Il **Contesto del processo**: è l'insieme di informazioni allocate dal SO per la sua esecuzione ed è composto da:
 - **Spazio di indirizzamento**: comprende codice, dati e stack del processo.
 - **Informazioni sulla memoria allocata**: utilizzate dalla MMU per gestire lo spazio di indirizzamento durante l'esecuzione del processo.
 - **Stato delle attività di elaborazione del file**: include informazioni sui file utilizzati, come la loro posizione e stato. Memorizza anche dettagli relativi alle operazioni eseguite dal processo sui file, come apertura, chiusura, creazione o cancellazione.
 - **Informazioni sulle Interazioni con altri processi**: memorizza dati relativi alla comunicazione tra processi, inclusi i messaggi inviati e ricevuti.
 - **Informazioni sulle risorse allocate**: registra le risorse assegnate al processo, come memoria e dispositivi di I/O e altre risorse necessarie alla sua esecuzione.
 - **Altre informazioni necessarie** per il corretto funzionamento del processo, tra cui parametri specifici dell'ambiente di esecuzione e dati utilizzati per il monitoraggio dello stato del processo.
2. **Process Control Block** è una **struttura dati** che il kernel utilizza per **memorizzare le informazioni essenziali** alla gestione del processo ed è composto da:
 - **Id del processo**: Id univoco assegnato al processo al momento della creazione.
 - **Id del genitore e dei figli**: Utilizzati per la sincronizzazione tra processi, permettendo al kernel di verificare se un processo figlio ha terminato la sua esecuzione.
 - **Priorità**: Valore numerico assegnato al processo al momento della creazione. Il kernel può cambiare la priorità del processo dinamicamente in base alla sua natura (CPU-bound o I/O bound), al tempo di attività e alle risorse utilizzate (tipicamente il tempo di CPU).
 - **Stato corrente del processo**: indica se il processo è ready, running, blocked o terminated.
 - **Contenuto PSW**: stato della CPU salvato l'ultima volta che il processo è stato prelazionato o bloccato. Quando il processo viene ripristinato, il PSW viene ricaricato per riprendere l'esecuzione dal punto in cui era stato interrotto.
 - **Contenuto GPR**: registri salvati l'ultima volta che il processo è stato prelazionato o bloccato. I GPR vengono ripristinati al momento della ripresa del processo.
 - **Informazioni sugli eventi**: Per un processo che si trova nello stato blocked, questo campo contiene la causa dell'attesa, come la richiesta di una risorsa o un'operazione di I/O. Il kernel utilizza questa informazione per effettuare la transizione blocked → ready non appena l'evento viene soddisfatto.
 - **Informazioni sui segnali**.
 - **Puntatore al PCB**: utilizzato dal kernel per gestire la lista dei PCB, necessaria per la schedulazione dei processi. Questo meccanismo consente al kernel di mantenere una coda di processi ready e di selezionare il prossimo processo da eseguire.

Salvataggio del contesto, scheduling e dispatching

Quando si verifica un evento, il **salvataggio del contesto** conserva lo stato della CPU del processo interrotto nel **PCB** e registra le informazioni sul suo contesto. Il processo interrotto, che era in **running**, viene spostato nello stato **ready**. Se viene richiesta un'operazione di I/O, il processo può essere successivamente spostato nello stato **blocked**.

Lo **scheduling** utilizza le informazioni memorizzate nel **PCB** per selezionare un processo **ready** da eseguire e ne passa l'ID alla funzione di **dispatching**. Il dispatching, a sua volta, imposta il contesto del processo selezionato, lo sposta nello stato **running**, ripristina il suo stato CPU e pulisce i buffer della **MMU** per garantire la protezione della memoria.

Esempio:

In un SO, quando un processo con **priorità più alta** (P2) è **bloccato** su un'operazione di I/O e un processo a **priorità inferiore** (P1) è in **running**, la commutazione avviene quando l'operazione di I/O di P2 si completa. Il **salvataggio del contesto** memorizza lo stato di P1, cambiandolo in **ready**, mentre **P2** passa da **blocked a ready**. Lo **scheduler** seleziona P2 in quanto processo a priorità più alta, e il **dispatching** imposta il suo stato a **running**, permettendone l'esecuzione.

Commutazione tra processi e di contesto (context switch)

La **commutazione tra processi avviene** anche quando un processo in **running** passa a **blocked** per una richiesta o viene prelazionato alla fine della time slice.

La **commutazione tra processi non si verifica nei seguenti casi**:

- **se l'evento cambia lo stato di un processo con priorità inferiore rispetto a quello attualmente in esecuzione** (ad esempio, un processo a priorità più bassa passa da blocked a ready). In questa situazione, la funzione di scheduling seleziona nuovamente lo stesso processo interrotto per il dispatching.
- **se l'evento non provoca alcuna transizione di stato** (ad esempio, il processo attualmente in esecuzione richiede una risorsa immediatamente disponibile). In questo caso, non è necessario eseguire lo scheduling e il processo continua direttamente la sua esecuzione.

Durante la commutazione, oltre al salvataggio e al caricamento dello stato della CPU, è necessario effettuare anche la **commutazione del contesto del processo**, che include tutte le informazioni utili alla ripresa della sua esecuzione. L'**overhead della commutazione** dipende dalla quantità di dati da salvare e ripristinare.

Alcuni sistemi forniscono istruzioni specializzate per ridurre questo overhead, gestendo automaticamente PSW, registri della CPU e buffer della MMU.

Inoltre, la commutazione genera un **overhead indiretto**: il nuovo processo selezionato potrebbe non avere dati in cache, riducendo temporaneamente le prestazioni. Anche i buffer di traduzione degli indirizzi della Memory Management Unit (MMU) devono essere aggiornati, causando inizialmente un rallentamento nella gestione della memoria virtuale.

Gestione degli eventi

Durante il funzionamento di un SO si possono verificare i seguenti eventi:

1. **Evento per la creazione di un processo**: viene creato un nuovo processo.
2. **Evento per la terminazione di un processo**: un processo termina la sua esecuzione.
3. **Evento timer**: si verifica un interrupt del timer
4. **Evento per la richiesta di una risorsa**: un processo effettua la richiesta per una risorsa.
5. **Evento per il rilascio di una risorsa**: un processo rilascia una risorsa.
6. **Evento per la richiesta di avvio di I/O**: un processo richiede l'avvio di un'operazione di I/O.
7. **Evento per il completamento di I/O**: un'operazione di I/O è completata.
8. **Evento per l'invio di un messaggio**: un processo invia un messaggio a un altro processo.
9. **Evento per la ricezione di un messaggio**: un processo riceve un messaggio.
10. **Evento per l'invio di un segnale**: un processo invia un segnale a un altro processo.
11. **Evento per la ricezione di un segnale**: un processo riceve un segnale.
12. **Un interrupt da programma**: l'istruzione corrente nel processo running non è stata eseguita correttamente.
13. **Evento per il malfunzionamento dell'hardware**: una unità hardware del computer ha causato un malfunzionamento.

Questi eventi sono causati da azioni nell'ambito del processo ad eccezione degli eventi relativi al **timer**, al **completamento dell'I/O** e al **malfunzionamento dell'hardware** che sono causati da situazioni esterne al processo running.

Eventi relativi alla creazione, terminazione e prelazione del processo

Creazione di un processo

Quando un utente lancia un comando per eseguire un programma, oppure quando un processo intende creare un processo figlio, viene effettuata la chiamata di sistema `create_process`, passando come parametro il nome del programma da eseguire. In risposta, il kernel **crea un PCB** per il nuovo processo, assegnandogli un **identificatore univoco** e una **priorità**. Successivamente **determina la quantità di memoria** necessaria per contenere il codice del programma, i suoi dati e lo stack, e le alloca al processo. Al processo vengono assegnate anche alcune **risorse standard**, quali una tastiera e dei file dedicati allo standard input e allo standard output. Infine, queste informazioni relative alla memoria allocata e alle risorse vengono inserite nel **contesto del processo**, il suo stato viene impostato su **ready**, e il processo viene inserito nella **lista appropriata dei PCB**.

Terminazione di un processo

Quando un processo termina sé stesso o un processo figlio attraverso una chiamata di sistema, il kernel attende prima che siano completate eventuali operazioni di I/O ancora in corso. Dopodiché **libera la memoria e le risorse utilizzate** dal processo e imposta il suo stato su **"terminated"**. Tuttavia, il **PCB del processo terminato non viene eliminato immediatamente** perché il processo genitore potrebbe voler controllare il suo stato di uscita. Per questo motivo **il PCB viene mantenuto** fino a quando il genitore recupera lo stato del figlio o termina a sua volta. Inoltre, se il genitore è in attesa della conclusione del processo figlio, il kernel lo riattiva appena la terminazione avviene.

Prelazione di un processo

Se il processo attualmente in esecuzione **esaurisce la sua time slice**, deve essere prelazonato. In questo caso, la funzione che salva il contesto imposta automaticamente lo stato del processo da **running** a **ready** prima di chiamare il gestore degli eventi associato agli interrupt del timer. Il gestore si limita quindi ad aggiungere il PCB del processo alla relativa lista dei processi pronti.

La prelazione avviene anche quando un **processo con priorità più alta diventa pronto**, ma ciò accade implicitamente tramite lo scheduler, senza che il gestore degli eventi debba eseguire operazioni aggiuntive.

Eventi relativi all'utilizzo delle risorse

Quando un processo **richiede una risorsa tramite system call**, possono verificarsi due situazioni:

- **Se la risorsa è subito disponibile**, il kernel assegna immediatamente la risorsa senza modificare lo stato del processo e invoca direttamente il dispatcher.
- **Se invece la risorsa non è disponibile**, il processo viene interrotto e posto in stato **blocked**, annotando nel campo **informazioni relative agli eventi** del PCB, l'identificativo della risorsa in attesa.

Quando un processo **rilascia una risorsa** attraverso una system call, il suo stato non cambia. Il **gestore degli eventi** controlla se ci sono **processi bloccati in attesa di quella stessa risorsa** e, se presenti, ne sblocca uno assegnandogli la risorsa e impostandolo in stato **ready**.

Le **chiamate di sistema per iniziare operazioni di I/O o comunicazioni tra processi** seguono una procedura comune: il processo che avvia l'operazione passa allo **stato blocked** e registra nel campo **informazioni relative agli eventi** del suo PCB la causa del blocco. Al termine dell'operazione (segnalata tramite interrupt), lo stato del processo viene riportato a **ready**. Questo schema vale analogamente anche per le richieste di invio e ricezione di messaggi tra processi.

Event Control Block (ECB)

L' **Event Control Block (ECB)** è una struttura dati utilizzata dal kernel di un SO per gestire eventi che coinvolgono **processi in attesa**. Quando avviene un evento, ad esempio il completamento di un'operazione di I/O, il kernel deve rapidamente identificare il processo associato che era in attesa di quell'evento.

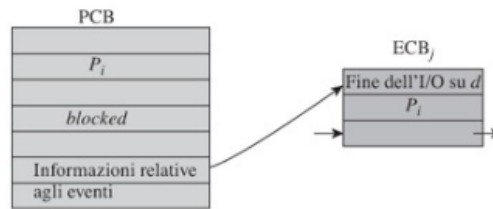
Descrizione dell'evento
Id del processo
Puntatore all'ECB

Poiché **la ricerca diretta tra le informazioni relative agli eventi contenute nei PCB è lenta**, si utilizza l'**ECB**, che contiene tre campi principali:

- **Descrizione dell'evento**, descrive un evento.
- **Id del processo**, contiene l'id del processo in attesa dell'evento.
- **Puntatore all'ECB**.

Quando un processo si blocca aspettando un evento, il kernel crea un ECB corrispondente, lo inserisce nella lista appropriata (il SO costruisce una lista per ogni classe di eventi), e in caso di evento verifica rapidamente la lista per trovare il processo da sbloccare.

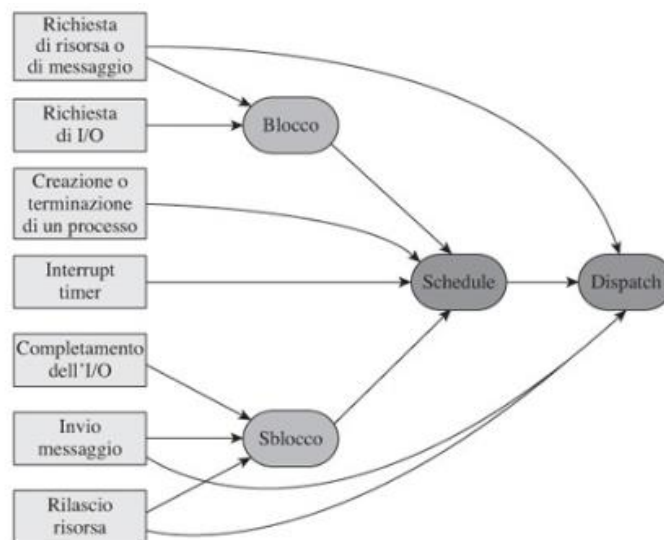
Pur avendo l'ECB, il campo eventi del PCB viene comunque conservato, anche se ridondante, perché potrebbe essere necessario conoscere l'evento per il quale un processo si è bloccato, ad esempio per la gestione della terminazione del processo stesso.



L'immagine mostra che il campo Informazioni relative agli eventi del PCB punta all'ECB.

Esempio 5.4 a pagina 128 del libro.

Azioni per la gestione di un evento del Kernel



Il diagramma in esame riassume le azioni intraprese in risposta al verificarsi di specifici eventi all'interno di un SO:

- **Richiesta di una risorsa o ricezione di un messaggio (Inter-Process Communication):** Qualora la risorsa o il messaggio richiesto siano immediatamente disponibili, il processo prosegue la sua esecuzione senza subire alcun blocco, accedendo direttamente alla fase di dispatch. In caso contrario, il processo viene sospeso e posto in stato di attesa. Lo scheduler seleziona quindi un altro processo pronto per l'esecuzione, avviando la fase di dispatch.
- **Richiesta di operazioni di I/O:** Quando un processo effettua una richiesta di I/O, viene sospeso fino al completamento dell'operazione. Una volta terminata, si procede alla schedulazione di un nuovo processo seguito dal dispatch.
- **Creazione o terminazione di un processo:** Alla creazione di un nuovo processo, il suo stato viene inizialmente impostato su ready. Lo scheduler, tuttavia, seleziona per l'esecuzione un processo dallo stato ready, che non necessariamente coincide con quello appena creato. In caso di terminazione di un processo, lo scheduler provvede alla selezione di un nuovo processo da eseguire.
- **Interrupt del timer:** Quando si verifica un interrupt generato dal timer, si accede immediatamente alla fase di schedulazione. Questo meccanismo è tipico degli scheduler round-robin basati su time-slice, nei quali l'evento determina la selezione di un nuovo processo da eseguire.
- **Completamento di un'operazione di I/O:** Una volta completata un'operazione di I/O, il processo che era in stato di attesa viene sbloccato e reso idoneo all'esecuzione.
- **Invio di un messaggio o rilascio di una risorsa:** Qualora un processo invii un messaggio o rilasci una risorsa, si verificano due possibili scenari. Se un altro processo era in attesa del messaggio o della risorsa rilasciata, esso viene sbloccato. In assenza di processi in attesa, il sistema procede direttamente alla fase di dispatch, consentendo la prosecuzione del processo attualmente in esecuzione.

Condivisione, comunicazione e sincronizzazione tra processi

I processi di un'applicazione possono interagire tra di loro per un obiettivo comune. Di seguito vengono riassunte le caratteristiche più importanti di quattro tipi di interazione:

- **Condivisione dei dati:** L'accesso simultaneo di più processi a dati condivisi può generare valori incoerenti. Per prevenire tale problema, è necessario garantire che, in ogni istante di tempo, un solo processo possa accedere alla variabile condivisa. Questo principio è noto come **mutua esclusione**.
- **Scambio di messaggi:** Un processo può trasmettere informazioni ad un altro processo sotto forma di messaggio. Il processo destinatario può archiviare le informazioni ricevute in strutture dati appropriate per un utilizzo successivo. La comunicazione tra il processo mittente e quello ricevente è regolata da specifici protocolli che ne garantiscono il corretto funzionamento.
- **Sincronizzazione:** Se un programma richiede che un'azione a_i venga eseguita solo dopo il completamento di un'azione a_j , e tali azioni sono eseguite da due processi distinti, è necessario sincronizzarli affinché il processo che deve eseguire a_i attenda il completamento di a_j . La **sincronizzazione tra processi** è fondamentale per evitare condizioni di competizione (race conditions) e garantire l'ordine corretto di esecuzione delle operazioni.
- **Segnali:** Un segnale è un meccanismo utilizzato per notificare a un processo il verificarsi di un evento eccezionale, consentendogli di adottare le opportune contromisure. **I segnali si basano sul meccanismo degli interrupt.** Quando un segnale viene inviato a un processo, **il kernel interrompe temporaneamente la sua esecuzione** ed esegue un **signal handler**, ovvero una routine di gestione dell'evento definita dal programmatore. In assenza di una gestione personalizzata, viene eseguita un'azione predefinita dal SO. Un esempio pratico di utilizzo dei segnali si verifica quando un processo entra in un ciclo infinito: in ambiente Windows, per interromperne l'esecuzione, è possibile premere Ctrl + Alt + Canc, selezionare il processo e terminarlo manualmente.

Segnali

Un **segnale** è un meccanismo utilizzato per notificare a un processo il verificarsi di una situazione di **eccezione**, consentendogli di adottare le opportune contromisure. Il SO mantiene una lista di segnali associati a specifiche eccezioni, come **overflow della CPU**, **gestione dei processi figli**, **utilizzo delle risorse** o **comunicazioni urgenti**. I segnali possono essere inviati dal kernel in risposta a eventi del sistema o dai processi utente ad altri processi.

Esistono due categorie di segnali:

- **Segnali sincroni**, generati da condizioni interne al processo (es. overflow della CPU).
- **Segnali asincroni**, derivanti da eventi esterni al processo (es. interruzioni manuali da parte dell'utente).

Un processo può registrare una routine di gestione (signal handler) per specifici segnali tramite una chiamata di sistema. Se un signal handler non è definito, il kernel esegue un default handler, che può includere azioni standard come la terminazione del processo.

Per **inviare un segnale**, un processo utilizza la **funzione signal**, specificando l'ID del processo destinatario e il numero del segnale.

Il **SO gestisce i segnali in modo simile agli interrupt**, utilizzando una **tabella di vettori dei segnali** che associa ogni segnale al relativo handler.

Se il **processo destinatario di un segnale è in stato blocked**, il sistema può temporaneamente metterlo in **ready** per permettere l'esecuzione del **signal handler**, per poi riportarlo allo stato originale. Alcuni SO, invece, preferiscono registrare il segnale e gestirlo quando il processo torna in stato ready e viene schedulato.

Esempio 5.6 a pagina 131

Thread

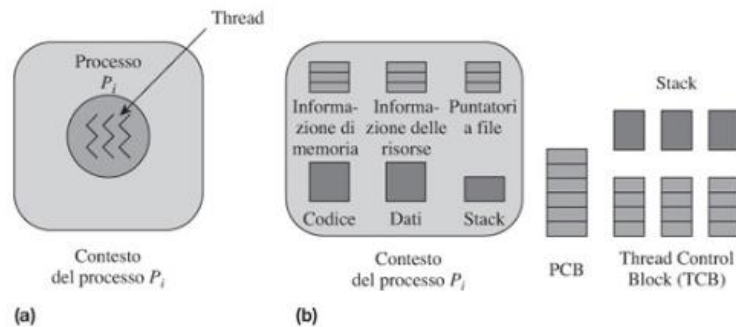
I thread rappresentano un modello di esecuzione alternativo ai processi, che consente di **migliorare la concorrenza** all'interno di un'applicazione **riducendo l'overhead** associato alla commutazione tra processi.

La commutazione tra processi comporta due tipi di overhead:

- **Overhead relativo all'esecuzione:** riguarda il salvataggio dello stato della CPU del processo in esecuzione e il caricamento dello stato del nuovo processo (**context switch**). Questo overhead è **inevitabile**.
- **Overhead dovuto all'uso delle risorse:** coinvolge il **contesto del processo**, comprese le **risorse allocate** (memoria, file, interazioni con altri processi). Questa operazione aggiunge ulteriore overhead alla commutazione.

Se si considerano due processi figli, **P_i** e **P_j**, dello stesso processo primario, essi ereditano il contesto del loro genitore e, se non possiedono risorse proprie, la loro differenza si limita allo stato della CPU e ai rispettivi stack. In questi casi, molte operazioni di salvataggio e ripristino risultano ridondanti.

I **thread** sfruttano questa caratteristica per **ridurre l'overhead**. A differenza dei processi, un thread **non possiede risorse proprie**, ma **utilizza il contesto del processo che lo ha creato**, condividendo la memoria e le risorse del processo principale. La **creazione di un thread** avviene tramite una **system call** e consente l'esecuzione parallela di parti di codice, evitando la duplicazione del contesto di esecuzione e migliorando così l'efficienza del sistema.



Nella figura di sinistra, è rappresentata la visione astratta di un processo con tre thread (linee ondulate). La figura di destra mostra il contesto del processo **P_i** e il suo **PCB**. Ogni thread di **P_i** è un'istanza del programma e possiede un proprio stack e un **Thread Control Block (TCB)**, analogo al PCB, che memorizza:

- **Informazioni per la schedulazione del thread:** thread id, priorità e stato;
- **Stato della CPU**, ovvero il contenuto del PSW e dei GPR;
- **Puntatore al PCB** del processo genitore;
- **Puntatore al TCB**, usato per creare la lista necessaria alla schedulazione.

L'uso dei thread suddivide lo **stato di un processo** in due parti:

- **Stato delle risorse** (memoria, file) → **condiviso** tra tutti i thread del processo.
- **Stato dell'esecuzione** (stato CPU, stack) → **separato** per ogni thread.

Questo riduce il costo della concorrenza, poiché durante la commutazione tra thread viene **scambiato solo lo stato di esecuzione**, mentre le risorse del processo restano invariate. Questo rende la commutazione tra thread più efficiente rispetto alla commutazione tra processi.

Stati e transizioni di stato dei thread

Gli **stati** e le **transizioni di stato** dei thread sono analoghi a quelli dei processi. Lo stato di un thread appena creato è impostato su **ready** ed entra nello stato **running** quando sottoposto a dispatching. Entra nello stato **blocked** a causa dei requisiti di sincronizzazione del processo.

Vantaggi dei thread rispetto ai processi

L'uso dei thread rispetto ai processi offre diversi vantaggi:

- **Minore overhead di creazione e commutazione:** lo stato del thread riguarda **solo l'elaborazione**, mentre risorse e comunicazioni restano nel processo principale, riducendo il costo di gestione.
- **Comunicazione più efficiente:** I thread condividono i dati e la memoria dello stesso processo, evitando l'overhead delle chiamate di sistema necessarie per la comunicazione tra processi. Per contro c'è la **necessità di sincronizzare i thread** per evitare inconsistenze sui dati condivisi.
- **Progettazione semplificata:** L'uso dei thread semplifica lo sviluppo e la gestione delle applicazioni concorrenti, riducendo la complessità del codice e migliorando le prestazioni.

Codifica per usare i thread

Per utilizzare i thread in modo corretto, è fondamentale impiegare **librerie thread-safe**, che garantiscono una gestione sicura della condivisione dei dati tra i thread.

La **gestione dei segnali** può essere definita sia dal **kernel** sia dall'**applicazione**, a seconda delle necessità. Nel caso di un **segnale associato a un'eccezione hardware**, la gestione avviene direttamente all'interno del thread in cui si è verificato l'evento. Gli **altri segnali**, invece, possono essere consegnati a qualsiasi thread del processo, ma idealmente dovrebbero essere gestiti dal **thread con la priorità più alta**, per garantire una risposta più rapida ed efficiente.

I Thread in C: Thread Posix

Lo standard ANSI/IEEE Portable Operating System Interface (POSIX) definisce l'API `pthread`, per l'utilizzo nei programmi scritti in C. Questa API fornisce 60 routine per:

- **Gestione dei thread:** mediante chiamate alla routine della libreria è possibile creare e gestire i thread, recuperare il loro stato, terminarli in modo normale o anormale, attendere la loro terminazione, impostare attributi di schedulazione e dimensionare lo stack.
- **Condivisione dei dati:** la libreria fornisce il meccanismo dei **mutex** (mutua esclusione) per garantire che i dati condivisi siano accessibili da un solo thread alla volta, prevenendo **race condition**.
- **Assistenza per la sincronizzazione:** sono disponibili le variabili di condizione, che consentono di coordinare l'esecuzione dei thread e di gestire eventi in modo controllato.

Un pthread è creato mediante la chiamata:

```
pthread_create(<id_thread>, <attributi>, <routine_di_avvio>, <argomenti>)
```

dove:

- `<id_thread>` è un puntatore a una variabile di tipo `pthread_t`, che identifica il thread creato.
- `<attributi>` è un puntatore a una struttura `pthread_attr_t` che specifica attributi come la priorità e le opzioni di sincronizzazione (se impostato a `NULL`, vengono usati gli attributi predefiniti).
- `<routine_di_avvio>` è un puntatore alla funzione che sarà eseguita dal thread. Questa funzione deve restituire un valore di tipo `void *`.
- `<argomenti>` è un puntatore che può essere passato alla funzione di avvio del thread.

Per **terminare un thread** si effettua la chiamata alla routine

```
pthread_exit(<stato>)
```

che prende come parametro lo stato del thread, dove `<stato>` è un valore di ritorno che può essere recuperato da altri thread.

La **sincronizzazione tra il thread genitore e un thread figlio** è effettuata mediante la chiamata alla routine

```
pthread_join(<id_thread>, <valore_di_ritorno>)
```

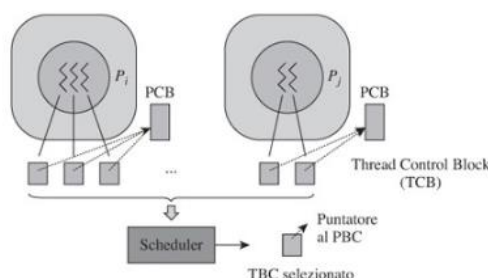
che prende come parametri:

- `id_thread` di cui si attende la terminazione.
- `valore_di_ritorno` è un puntatore a un valore che conterrà il risultato restituito da `pthread_exit`. Se non è necessario recuperare il valore, si può passare `NULL`.

Quando `pthread_join` viene chiamata, il thread chiamante si blocca fino alla terminazione del thread specificato.

Thread di livello Kernel, di livello Utente e Ibridi

Thread di livello Kernel



I **thread di livello kernel** sono gestiti direttamente dal kernel, che si occupa della loro **creazione**, **terminazione** e **gestione dello stato** tramite system call. Quando un processo crea un thread, il kernel gli assegna un **TCB**, che **punta al PCB** del processo contenitore.

Durante la **schedulazione**, se un **evento interrompe un thread**, il kernel **salva lo stato della CPU** nel suo **TCB**. Dopo aver gestito l'evento, lo **scheduler analizza i TCB** dei thread in stato **ready** e seleziona uno da eseguire. Il

dispatcher, sfruttando il **puntatore al PCB**, determina se il thread appartiene allo stesso processo di quello interrotto. Se sì, il contesto viene salvato e il thread viene avviato; altrimenti, il kernel deve anche cambiare il contesto del processo, il che comporta un overhead maggiore. Tuttavia, la commutazione tra thread dello stesso processo è circa dieci volte più veloce rispetto a quella tra processi distinti.

Sintesi processo di gestione dei thread di livello kernel

1. Quando si verifica un evento, il **kernel salva lo stato della CPU** del thread interrotto nel suo **TCB**.
2. Dopo aver gestito l'evento, lo **scheduler seleziona un thread ready** dalla lista dei TCB.
3. Il **dispatcher verifica** se il thread selezionato appartiene allo stesso processo di quello interrotto.
4. Se **appartiene allo stesso processo**, non avviene alcun salvataggio o caricamento del contesto del processo.
5. Se **appartiene a un processo diverso**, viene effettuato il **salvataggio del contesto** del processo interrotto e il **caricamento del contesto** del nuovo processo, seguito dal **dispatch del thread selezionato**.

Vantaggi e Svantaggi dei thread di livello kernel

- **Vantaggi:** Offrono **maggiore parallelismo** nei sistemi multiprocessore, poiché i thread appartenenti allo stesso processo possono essere eseguiti simultaneamente su diverse CPU. Inoltre, la loro gestione è più simile a quella dei processi, facilitando la programmazione per gli sviluppatori e **migliorando lo speedup dell'elaborazione**.
- **Svantaggi:** La gestione richiede il coinvolgimento del Kernel, aumentando l'overhead rispetto ai thread di livello utente. Inoltre, la commutazione tra thread dello stesso processo, pur essendo più veloce rispetto a quella tra processi, è comunque più onerosa rispetto ai thread di livello utente.

Thread di livello Utente

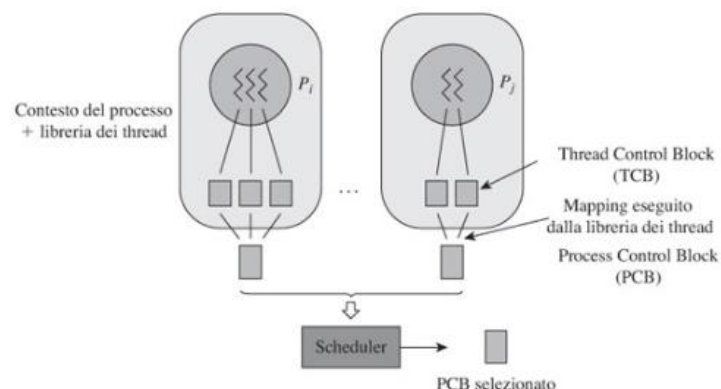
I **thread di livello utente** sono gestiti direttamente dal programma attraverso una **libreria di thread** (ad esempio pthread nei sistemi POSIX), senza coinvolgere il kernel. Questo significa che il **kernel non è a conoscenza dei singoli thread**, ma vede solo il processo come un'unica entità.

Quando un processo vuole creare un thread, utilizza una funzione della libreria. La libreria assegna al nuovo thread un **TCB** e lo aggiunge alla lista dei thread gestiti all'interno del processo.

Se un thread sta eseguendo e chiama una funzione della libreria per **sincronizzarsi** (ad esempio, aspettare un'operazione di I/O), la libreria manda in esecuzione un altro thread che si trova nello stato ready, senza coinvolgere il kernel. Questo rende lo **switch tra thread molto veloce**, perché non richiede una chiamata di sistema.

Tuttavia, se **non ci sono altri thread pronti all'esecuzione**, la libreria **chiede al kernel di sospendere l'intero processo**. Il kernel mette in pausa il processo fino a quando un evento (come il completamento di un'operazione di I/O) sblocca almeno un thread. A quel punto, il processo viene riattivato, la libreria riprende il controllo e sceglie quale thread far partire.

Schedulazione dei thread di livello utente

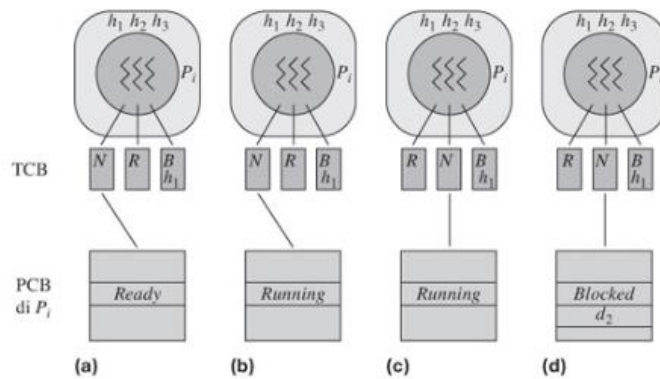


La **schedulazione** avviene grazie all'alternanza tra thread, realizzata attraverso una **mappatura** tra il **TCB** del thread selezionato e il **PCB** del processo. Per mappatura, si intende il meccanismo con cui la libreria **aggiorna il contesto del processo** con quello del thread da eseguire, simulando un cambio di thread senza coinvolgere il kernel.

Durante la commutazione, la libreria aggiorna lo **stack pointer** per puntare allo **stack del thread attivo** e modifica il **program counter (PC)** per riprendere l'esecuzione del thread. Poiché opera in **modalità utente**, la libreria utilizza **istruzioni non privilegiate**, evitando interventi del kernel e rendendo il passaggio tra thread più rapido.

Esempio Scheduling Thread di livello utente

La figura illustra come la **libreria di thread** gestisce più thread all'interno di un processo P_i , alternandoli tra gli stati di **running (N)**, **ready (R)** e **blocked (B)** senza coinvolgere direttamente il kernel.



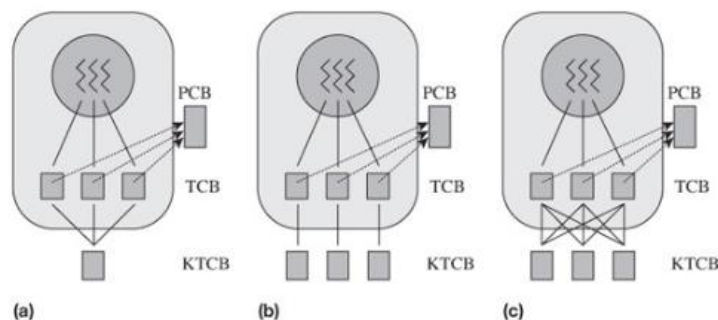
Inizialmente, il **processo P_i** è in esecuzione e la libreria seleziona il **thread h_1** , che passa nello stato **running (N)**. Nel frattempo, il **thread h_3** è **bloccato (B)**, in attesa di un'azione di sincronizzazione da parte di h_1 figura(b). Successivamente, la libreria **sospende h_1** e lo sposta nello stato **ready (R)**, selezionando h_2 per l'esecuzione, che passa quindi nello stato **running(N)** figura(c). Infine, il **kernel interviene** perché il thread h_2 ha eseguito una **system call bloccante** per avviare un'operazione di I/O. Poiché il kernel non distingue i thread di livello utente, **sospende l'intero processo P_i** , che passa nello stato **blocked (B)** figura(d). Tuttavia, all'interno del processo, la **libreria di thread continua a gestire i thread**, mantenendo il loro stato attuale (come se fossero congelati) in attesa della ripresa del processo.

Vantaggi e Svantaggi dei thread di livello kernel

- **Vantaggi:** I thread di livello utente offrono diversi vantaggi, tra cui una **schedulazione più veloce** grazie alla gestione da parte della libreria di thread, che evita il coinvolgimento del kernel e **riduce l'overhead delle system call**. Inoltre, permettono a ogni processo di **adottare una politica di scheduling personalizzata**, come lo scheduling a priorità per applicazioni real-time o il round-robin per server multithread.
- **Svantaggi:** Poiché il kernel non distingue i thread dai processi, **se un thread esegue una system call bloccante, tutto il processo viene sospeso**, impedendo agli altri thread di continuare l'esecuzione (cosa che non accade con i thread di livello kernel). Inoltre, i thread **non possono sfruttare il parallelismo**, poiché il kernel assegna le CPU ai processi e non ai singoli thread, **limitando l'esecuzione a un solo thread per volta** per ciascun processo.

Modelli di Thread Ibridi

Il modello dei thread ibrido combina sia i thread di livello utente che quelli di livello kernel, gestendo l'associazione tra di essi. A seconda del metodo di associazione utilizzato, si possono ottenere configurazioni con **diverso overhead di commutazione e differenti livelli di concorrenza e parallelismo**.



La figura mostra tre modelli di associazione:

- Molti a Uno:** Tutti i **thread di livello utente** sono associati a un **unico thread di livello kernel**. Questo metodo permette concorrenza tra i thread utente, ma **non parallelismo**, poiché il kernel gestisce solo un thread. La commutazione tra thread ha **basso overhead**, ma il blocco di un thread provoca il **blocco dell'intero processo**. (Effetto complessivo simile ai thread livello utente)
- Uno a Uno:** Ogni **thread di livello utente** è mappato su un **thread di livello Kernel**. Questo modello **supporta parallelismo**, permettendo ai thread di essere eseguiti su CPU diverse in sistemi multiprocessore. Tuttavia, la commutazione avviene a livello kernel, generando un **elevato overhead**. Il blocco di un thread non influisce sugli altri.
- Molti a Molti:** I thread di livello utente possono essere **mappati dinamicamente** su differenti **thread di livello kernel**, permettendo **parallelismo** e **basso overhead di commutazione**. Tuttavia, questo metodo richiede un'**implementazione più complessa**.

Possibile domanda di esame: Linux che tipo di thread utilizza? Utilizza una soluzione ibrida con corrispondenza 1:1.

Anche la libreria pthreads su linux utilizza una soluzione ibrida con corrispondenza 1:1.

Casi Studio – Processi e Thread

Processi in UNIX

Unix utilizza due strutture dati per gestire i processi:

- **Struttura `proc`**: Contiene informazioni relative a:
 - ID del processo;
 - lo stato;
 - la priorità;
 - le relazioni con altri processi;
 - descrittore per gli eventi in attesa;
 - maschera per gestione segnali;
 - info relative alla gestione della memoria.
- **`u-area` (User Area)**: Contiene un PCB che memorizza:
 - lo stato della CPU per i processi bloccati;
 - il puntatore alla struttura `proc`;
 - gli ID utente e del gruppo;
 - gestori dei segnali;
 - file aperti;
 - directory corrente;
 - il terminale collegato al processo;
 - utilizzo della CPU del processo.

La struttura `proc` memorizza i **dati relativi alla schedulazione** ed è sempre in memoria, mentre la `u-area` contiene i **dati relativi all'allocazione delle risorse e gestione dei segnali** ed è caricata solo quando il processo è in esecuzione.

Tipi di processi

In Unix esistono due tipi di processi:

- **Processi Utente** sono eseguiti per l'utente e associati a un terminale.
- **Processi Daemon** vengono eseguiti in background per funzioni di sistema (es. gestione rete).
- **Processi Kernel** eseguono il codice del kernel e operano in background (es. swapping) e sono creati all'avvio del sistema. Possono richiamare le funzioni del kernel o far riferimento alle strutture dati del kernel senza dover effettuare una system call.

Creazione e terminazione di un processo

Dopo la fase di boot, Unix avvia il processo **init**, che crea un processo figlio per ogni terminale connesso al sistema. Quando un utente esegue un programma da terminale, la shell genera un nuovo processo che utilizza la system call **exec()** per eseguire il file specificato, rendendolo il **processo principale** del programma.

Questo processo principale è un figlio della shell, la quale esegue la system call **wait()** per attendere la sua terminazione, rimanendo bloccata fino al completamento dell'esecuzione del programma.

Per creare un nuovo processo si utilizza la system call **fork()**, che alloca una struttura **proc** per il figlio, imposta il suo stato a **ready** e assegna una **u-area**. Il contesto utente del figlio è una copia esatta di quello del genitore, il che gli permette di eseguire lo stesso codice, a meno che non effettui una chiamata **exec()** per caricare un altro programma. L'unica differenza tra i due processi è che **fork()** restituisce 0 al figlio, mentre al genitore fornisce l'ID del nuovo processo.

Un processo **Pi** può terminare tramite la system call **exit(status_code)**, dove **status_code** indica lo stato di terminazione. Quando un processo termina, il kernel chiude i file aperti, rilascia la memoria e cancella la sua **u-area**, mantenendo però la sua struttura **proc** finché il genitore non ne richiede l'eliminazione. Durante questo periodo, il processo viene definito **zombie**, poiché tecnicamente è terminato ma ancora presente nel sistema fino a quando il suo stato non viene gestito dal processo genitore.

Quando un processo termina, il kernel invia un segnale al suo processo genitore per notificargli la terminazione. Tuttavia, **se il processo terminato Pi ha ancora figli attivi**, questi non vengono eliminati, ma vengono **riassegnati** al processo **init**, che diventa il loro nuovo genitore. In questo modo, quando **uno dei figli di Pi termina**, **init** riceve un segnale e può rilasciare la sua struttura **proc**. Questo meccanismo impedisce che i processi figli rimangano nello **stato zombie** e garantisce che il sistema gestisca correttamente la terminazione dei processi orfani.

Attesa della terminazione di un processo

La system call **wait()** permette a un processo padre (**Pi**) di attendere la terminazione di un processo figlio. Se un figlio termina prima della chiamata, **wait()** restituisce immediatamente il suo ID e memorizza lo stato di terminazione in una variabile specificata dal padre. Se più figli sono terminati, il padre deve chiamare **wait()** più volte per ottenere i rispettivi stati.

Se **Pi** ha figli ancora in esecuzione e nessuno di loro è terminato, la chiamata **wait()** lo mette nello stato **blocked**, sospendendone l'esecuzione fino a quando almeno uno dei figli non termina. Una volta che un figlio termina, il sistema operativo **sblocca Pi**, permettendogli di raccogliere il suo stato di terminazione. Se invece **Pi** non ha figli al momento della chiamata, **wait()** restituisce **-1**, segnalando che non c'è nessun processo da attendere. Questo meccanismo impedisce che i processi terminati rimangano **zombie** e garantisce una gestione ordinata della loro terminazione.

Attesa dell'occorrenza degli eventi

In Unix, un processo che attende un evento entra nello stato **sleep** su quell'evento, come accade per le operazioni di I/O in attesa di completamento. Invece di usare gli **Event Control Block (ECB)**, Unix associa ogni evento a un **indirizzo riservato nel kernel**, e ogni processo in attesa memorizza questo indirizzo nella propria struttura. Questo indirizzo viene utilizzato come descrizione dell'evento atteso dal processo. Quando un processo deve attendere un evento, il sistema calcola l'indirizzo corrispondente, imposta il processo in **stato blocked** e lo riattiva quando l'evento si verifica.

Questo metodo, però, può generare **overhead**. Se più processi attendono lo stesso evento, il kernel li attiva tutti, ma solo uno ottiene l'accesso alla risorsa, mentre gli altri devono tornare in attesa, creando un comportamento simile a **busy wait**. Inoltre, lo schema di **hashing** utilizzato per mappare gli eventi può assegnare lo stesso indirizzo a eventi diversi, causando l'attivazione di processi non interessati all'evento specifico. Per evitare sprechi di risorse, ogni processo deve verificare se l'evento atteso si è effettivamente verificato prima di continuare l'esecuzione.

Gestione degli Interrupt

Unix assegna a ogni interrupt un **livello di priorità (IPL)** per determinare quando può essere gestito. **Anche la CPU ha un livello di priorità**, e un interrupt viene servito solo se la sua priorità è superiore a quella attuale della CPU. In caso contrario, l'interrupt viene sospeso fino a quando la CPU riduce il proprio livello di priorità.

Il kernel sfrutta questo meccanismo per prevenire **incoerenze nei dati**, aumentando temporaneamente la priorità della CPU durante operazioni sensibili, come l'aggiornamento delle strutture dati, e abbassandola solo dopo aver completato l'operazione.

System call

Quando un processo invoca una **system call**, il kernel identifica la funzione richiesta attraverso un numero specifico e consulta una tabella interna per recuperare l'indirizzo del gestore corrispondente. Poiché i parametri della chiamata si trovano nello **stack utente**, vengono copiati nella **u-area** del processo prima di trasferire il controllo al gestore della system call. Questo passaggio garantisce un'esecuzione più efficiente e semplifica la gestione delle chiamate di sistema.

Segnali

Un segnale può essere inviato a un processo o a un gruppo di processi utilizzando la system call `kill(<pid>, <signal>)`. Il parametro `<pid>` determina il destinatario: se è un valore positivo, il segnale viene inviato a uno specifico processo identificato dal suo ID, se è zero, il segnale raggiunge tutti i processi che condividono lo stesso ID di gruppo del mittente; se è negativo, il segnale può essere indirizzato a processi esterni all'albero di esecuzione del mittente.

L'assegnazione dell'ID di gruppo avviene al momento della chiamata `fork`, con il processo figlio che eredita l'ID del genitore, ma può essere modificato successivamente con la system call `setgrp`.

Un processo può gestire i segnali specificando un'azione personalizzata attraverso la funzione `signal(<signal>, <function>)`. Questa funzione associa un segnale a una particolare funzione definita nel processo stesso. Se si utilizza `SIG_DFL`, all'occorrenza del segnale viene eseguita l'azione predefinita stabilita dal kernel, come la generazione di un core dump o la terminazione del processo. Se invece viene specificato `SIG_IGN`, il segnale viene completamente ignorato.

Il kernel **tiene traccia dei segnali** attraverso una **struttura dati (la u-area)** associata a ogni processo. Quando un segnale viene inviato, il bit corrispondente viene attivato all'interno della struttura **proc** del processo destinatario. Il

kernel verifica se il segnale debba essere ignorato o gestito e, in quest'ultimo caso, lo inoltra al processo. Se il processo si trova in stato bloccato, il segnale rimane pendente fino alla sua riattivazione. Il controllo dei segnali pendenti avviene nei momenti in cui un processo riprende l'esecuzione dopo una chiamata di sistema o un interrupt, quando viene sbloccato o prima che si blocchi su un evento.

Esistono alcune anomalie nella gestione dei segnali. Se un segnale viene inviato ripetutamente, il Kernel non registra il numero di occorrenze, ma solo il fatto che si sia verificato almeno una volta. Di conseguenza, il gestore del segnale può essere eseguito una o più volte a seconda del momento in cui il processo viene schedato. Un'altra particolarità riguarda i processi bloccati a causa di una system call: dopo l'esecuzione del gestore del segnale, il processo non riprende automaticamente la chiamata interrotta, ma vi ritorna, e se necessario deve ripeterla esplicitamente.

Segnale	Descrizione
SIGCHLD	Un processo figlio ha concluso l'esecuzione o è sospeso
SIGFPE	Errore aritmetico
SIGILL	Istruzione non valida (illegal instruction)
SIGINT	Tty interrupt (Control-C)
SIGKILL	Kill del processo
SIGSEGV	Segmentation fault
SIGSYS	System call non valida
SIGXCPU	Superato il limite del tempo di CPU
SIGXFSZ	Superato il limite della dimensione del file

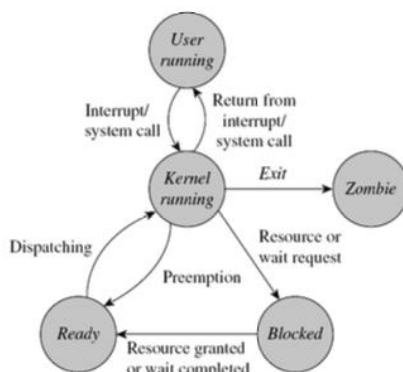
Tabella 5.9 Segnali interessanti in Unix.

Stati del processo e transizioni di stato

Nei sistemi Unix, la gestione dei processi segue un modello diverso rispetto a quello generale descritto nei paragrafi precedenti.

Nel **modello Unix**, quando un processo effettua una **system call**, esegue direttamente il codice del kernel con privilegi elevati, passando così dalla modalità utente alla modalità kernel. Una volta completata l'esecuzione della chiamata, un cambio di modalità riporta il processo nuovamente in modalità utente.

Il **kernel di Unix è rientrante**, il che significa che più processi possono eseguire codice kernel in modo concorrente senza interferire tra loro. Per rendere possibile questa gestione, ogni processo dispone di uno **stack kernel separato**, che conserva la cronologia delle chiamate effettuate mentre il processo è in esecuzione all'interno del kernel.



Il modello Unix distingue due stati di esecuzione: **running utente**, in cui un processo esegue codice utente, e **running kernel**, in cui viene eseguito codice del kernel.

Un processo passa da running utente a running kernel quando effettua una **system call** o quando viene **generato un interrupt**.

Se un processo si trova nello stato **running kernel**, può essere **bloccato**, ad esempio se è in attesa di un'operazione di I/O o di una risorsa. Quando la risorsa diventa disponibile o l'operazione viene completata, il processo ritorna in running kernel per concludere l'esecuzione del codice di sistema e successivamente torna in modalità utente, riprendendo lo stato di running utente.

Un processo **non può essere bloccato o interrotto** se si trova nello stato **running utente**, ma deve prima passare nello stato **running kernel**. Ad esempio, un processo che si trova nello stato running utente, effettua una chiamata alla system call exit, che lo porta a running kernel. Da qui il processo può essere terminato e diventa uno zombie come conseguenza di questa operazione.

Processi e thread in Linux

Strutture dati

Dalla versione 2.6 del kernel, Linux supporta il modello di **thread ibridi 1:1**, cioè ogni thread utente corrisponde ad un thread di livello kernel. Per gestire processi e thread, il kernel utilizza una struttura dati chiamata `task_struct`, che memorizza tutte le informazioni necessarie.

Per un processo, questa struttura include:

- lo stato del processo;

- le info riguardo il processo genitore e i figli;
- il terminale usato dal processo;
- la directory corrente;
- i file aperti;
- la memoria allocata;
- i segnali e i gestori dei segnali.

Il Kernel suddivide queste informazioni in sottostrutture specifiche (per terminale, directory, file, memoria e segnali) e memorizza i relativi puntatori dentro **task_struct**. Questo approccio ottimizza l'uso della memoria e riduce l'overhead nella creazione di nuovi thread.

Creazione di Processi e Thread

In Linux, processi e thread vengono creati tramite le system call `fork` e `vfork`, basate sulla chiamata di sistema `clone`, che è trasparente ai programmi.

La `clone` accetta quattro parametri: **l'indirizzo base del nuovo processo o thread**, **parametri da passare al nuovo processo o thread**, **flag di configurazione**, **stack del nuovo thread** (se specificato).

Alcuni flag importanti sono:

- `CLONE_VM` → Condivide la gestione della memoria.
- `CLONE_FS` → Condivide la directory root e la directory corrente.
- `CLONE_FILES` → Condivide i file aperti.
- `CLONE_SIGHAND` → Condivide i gestori dei segnali.

Un **thread** viene creato con **tutte le flag attivate**, così da condividere con il thread genitore lo **spazio di indirizzamento**, i **file** e i **signal handler**.

Un **processo** viene creato con **tutte le flag disattivate**, evitando la condivisione di queste risorse.

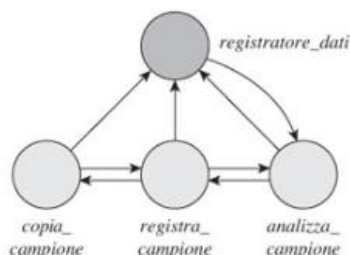
Terminazione di Processi e Thread

La system call `exit_group()` (introdotta nel kernel 2.6) permette di terminare un processo e tutti i suoi thread in circa 2 secondi, molto più velocemente rispetto alla versione 2.4 che impiegava circa 15 minuti per 100.000 thread.

Miglioramenti con il kernel 2.6

È stato introdotto il **supporto per la Native POSIX Threading Library (NPTL)** che permette di gestire fino a 2 miliardi di thread, rispetto ai 8192 thread per CPU del kernel 2.4. Inoltre, è stata introdotta la **gestione dei segnali in kernel space**, grazie alla quale i segnali vengono consegnati a uno dei thread disponibili. I segnali di blocco/sblocco coinvolgono l'intero processo, mentre quelli di errore lo terminano. Infine, è stato introdotto **l'uso di futex (fast user-space mutex)** che migliora la sincronizzazione tra thread riducendo l'overhead delle system call.

Relazioni genitore-figlio



Le relazioni tra processi o thread genitori e figli sono gestite tramite la struttura `task_struct`, che **conserva l'albero dei processi**. Questa struttura include:

- puntatori al processo genitore;
- puntatori al processo responsabile della gestione della terminazione (se il genitore è già terminato);
- puntatori all'ultimo figlio creato e ai fratelli più giovani e più vecchi.

Stati di un processo

Lo stato di un processo è indicato in un flag nel suo descrittore. Esistono cinque stati principali:

- **TASK_RUNNING**: Il processo è in esecuzione o pronto per essere schedato.
- **TASK_INTERRUPTIBLE**: Il processo è in attesa di un evento e può essere svegliato da un segnale.
- **TASK_UNINTERRUPTIBLE**: Il processo è in attesa di un evento ma NON può essere svegliato da segnali.

- **TASK_STOPPED**: Il processo è stato interrotto da un segnale (es. SIGSTOP).
- **TASK_ZOMBIE**: Il processo ha terminato l'esecuzione, ma il suo genitore non ha ancora raccolto il suo stato di uscita.

Lo stato TASK_RUNNING corrisponde allo stato Running/Ready.

Gli stati TASK_INTERRUPTIBLE e TASK_UNINTERRUPTIBLE corrispondono allo stato Blocked, con la distinzione sulla gestione dei segnali.

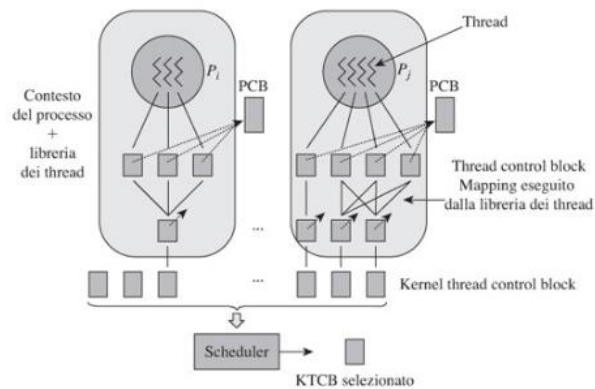
Un processo entra in TASK_STOPPED quando riceve segnali come SIGSTOP, SIGTTIN o SIGTTOU.

Thread in Solaris

Solaris, un sistema operativo basato su Unix 5.4, inizialmente utilizzava un **modello di thread ibrido** chiamato **M x N**, che supportava diverse modalità di associazione tra thread utente e thread kernel (Molti a Uno, Uno a Uno, Molti a Molti). Solaris 8 mantenne questo modello e introdusse anche il **modello 1:1, equivalente ai thread Kernel**. Tuttavia, Solaris 9 abbandonò il modello M x N per semplificare la gestione dei thread.

Il modello M x N prevedeva tre livelli di entità per la gestione della concorrenza e del parallelismo:

- **Thread utente**: creati e gestiti dalla libreria dei thread, non visibili al kernel.
- **Processi lightweight (LWP)**: entità intermedie tra thread utente e thread kernel. Ogni thread utente era mappato su un LWP, con un'associazione decisa dal programmatore (uno a uno, molti a uno, molti a molti).
- **Thread kernel**: creati per ogni LWP, eseguibili direttamente dalla CPU.



Gli LWP possono funzionare in parallelo poiché ognuno di essi è associato a un thread kernel. Ogni LWP aveva un **LWP control block**, mentre i thread kernel avevano un **kernel thread control block (KTCB)**. Lo **scheduler** selezionava un **thread kernel ready** per ogni CPU disponibile, assegnando il relativo LWP all'esecuzione. La libreria dei thread poteva gestire più thread utente all'interno di un LWP per massimizzare la concorrenza. Uno dei limiti del modello **M x N** era che un thread utente bloccato su una chiamata di sistema poteva bloccare l'intero LWP su cui era mappato, riducendo il parallelismo. Solaris mitigava questo problema con le **scheduler activation**, un meccanismo che permetteva al sistema di gestire in modo più efficiente i thread bloccati. Tuttavia, la complessità del modello e la gestione dei control block portarono al suo abbandono in Solaris 9 a favore del **modello 1:1**, più semplice ed efficiente.

Gestione dei segnali

Nel modello M x N di Solaris ogni processo ha un **LWP dedicato** alla gestione dei segnali.

- **Segnali interni** (es. eccezioni aritmetiche, violazioni di memoria) vengono consegnati direttamente al thread che li ha generati.
- **Segnali esterni** (es. timer) devono essere gestiti da un thread abilitato.

Quando viene generato un segnale il LWP dedicato ai segnali riceve la notifica del segnale dal kernel e verifica se esiste un thread abilitato alla sua gestione in esecuzione su un LWP del processo. Se trova un thread appropriato, richiede al kernel di consegnare il segnale sospeso all'LWP corrispondente.

Stati dei processi e thread kernel

Il Kernel gestisce solo gli stati dei processi e dei thread Kernel, ma non dei thread utente poiché non ne conosce l'esistenza.

Un processo può trovarsi negli stati:

- **SIDL**: stato transitorio durante la creazione.
- **SRUN**: processo eseguibile (ready).
- **SONPROC**: processo in esecuzione su una CPU.
- **SSLEEP**: processo in attesa (blocked).

- **SSTOP**: processo fermo.
- **SZOMB**: processo terminato (zombie).

I thread kernel hanno stati equivalenti a quelli dei processi:

- **TS_RUN** (eseguibile),
- **TS_ONPROC** (in esecuzione),
- **TS_SLEEP** (in attesa),
- **TS_STOPPED** (fermo),
- **TS_ZOMB** (terminato).

Un thread kernel non assegnato si trova nello stato **TS_FREE**.

Scheduler activation

Lo **scheduler activation** viene utilizzato per **gestire in modo efficiente i thread utente** in sistemi con associazione **molti a molti** tra thread utente e LWP. Serve a **prevenire il blocco dell'intero processo** quando un **LWP si blocca** a causa di una system call bloccante.

Il **kernel** interviene creando uno **scheduler activation**, notificando la libreria dei thread che può spostare i thread eseguibili su un nuovo LWP, evitando interruzioni nell'esecuzione. Questo meccanismo migliora la **responsività e l'efficienza** del sistema, consentendo ai thread di continuare a essere eseguiti anche quando uno di essi è temporaneamente bloccato.

Passaggio all'implementazione 1:1

Il **passaggio all'implementazione 1:1** è avvenuto perché il **modello MxN**, sebbene progettato per ridurre l'overhead dei context switch tramite la gestione dei thread a livello utente, funzionava in modo efficiente solo quando lo **scheduler della libreria dei thread e quello del kernel erano ben coordinati**.

Solaris 8 ha introdotto un **context switch efficiente a livello kernel nel modello 1:1**, in cui ogni thread utente ha un corrispondente thread kernel. Questo approccio ha semplificato la **gestione dei segnali**, eliminato la necessità dello **scheduler activation** e migliorato la **scalabilità**. Per questi vantaggi, Solaris 9 ha abbandonato il modello MxN in favore dell'implementazione 1:1.

Capitolo 6 – Sincronizzazione dei processi: Memoria Condivisa

I **processi intercomunicanti** sono processi concorrenti che condividono dati o che coordinano le loro attività. Per realizzare la **sincronizzazione dei processi** è necessario distinguere tra:

- **Sincronizzazione per l'accesso ai dati**, che garantisce ai processi l'accesso ai dati in **mutua esclusione**, assicurando la consistenza dei dati condivisi quando vengono aggiornati da processi intercomunicanti.
- **Sincronizzazione per il controllo**, che garantisce che i processi eseguano le loro attività nell'ordine corretto.

Nell'ambito della **programmazione concorrente**, uno dei problemi principali è la **race condition** (o **data race**), che si verifica quando il risultato di un programma dipende dall'ordine di esecuzione dei thread. Questo comportamento rende la computazione **non deterministica**, ovvero con un esito imprevedibile: lo stesso codice può produrre risultati diversi a ogni esecuzione.

Un caso tipico in cui si manifesta una race condition è l'accesso concorrente a una **sezione critica**, ovvero una porzione di codice che interagisce con **variabili o risorse condivise** tra più thread. Per evitare problemi di consistenza dei dati, l'esecuzione della sezione critica deve avvenire in **mutua esclusione**, ossia garantendo che,

mentre un thread è attivo all'interno di essa, nessun altro thread possa accedervi simultaneamente. Questo meccanismo assicura il **corretto funzionamento del programma** e previene errori legati alla concorrenza.

Che cos'è la sincronizzazione dei processi?

La **sincronizzazione dei processi** riguarda le tecniche per ritardare, ripristinare e coordinare i **processi intercomunicanti**.

Il termine **processo** verrà utilizzato come termine generico per indicare sia i processi, sia i thread.

Le applicazioni utilizzano i **processi concorrenti** sia per ottenere un'elaborazione più veloce, sia per semplificare la progettazione del software, come nel caso dei server multithread.

Con:

- **$read_set_i$** → Si indica l'insieme di **dati letti** dal processo P_i e messaggi interprocesso o segnali ricevuti da P_i .
- **$write_set_i$** → Si indica l'insieme di **dati modificati** dal processo P_i e messaggi interprocesso o segnali inviati da P_i .

Definizione Formale di Processi intercomunicanti

Due processi P_i e P_j sono definiti **intercomunicanti** se il $write_set$ di uno si sovrappone con il $write_set$ o con il $read_set$ dell'altro.

Il $write_set$ di un processo si sovrappone al $read_set$ di un altro quando il primo processo modifica il valore di una variabile mentre il secondo processo la sta leggendo.

I $write_set$ si sovrappongono quando due processi effettuano contemporaneamente operazioni di scrittura su un dato condiviso.

Requisiti di sincronizzazione nei processi intercomunicanti

Quando più processi devono collaborare, è importante sincronizzarli correttamente per evitare errori o comportamenti imprevisti. I due principali requisiti di sincronizzazione sono:

1. Attesa delle condizioni necessarie

Un processo può eseguire un'operazione solo se certe condizioni sui dati condivisi sono soddisfatte. Se queste condizioni non sono ancora presenti, il processo deve aspettare finché non lo saranno.

2. Ordine di esecuzione tra processi

Un processo deve eseguire un'operazione solo dopo che un altro processo ha completato una determinata azione. Questo viene gestito aggiornando variabili condivise che indicano lo stato delle operazioni e sospendendo temporaneamente il processo in attesa del momento giusto.

Esempio pratico:

- Il processo P_1 esegue un'operazione e aggiorna una variabile condivisa (come un flag booleano) per segnalare che ha terminato.
- Il processo P_2 controlla questa variabile: se indica che P_1 ha finito, allora può procedere con la sua operazione.
- Se la condizione non è soddisfatta, P_2 **aspetta** finché non viene notificato che può continuare.
- Una volta che il requisito è soddisfatto, P_2 **riprende** l'esecuzione.

Race Condition

La **Race condition** è una condizione in cui il valore di un dato condiviso d_s , risultante dall'esecuzione delle operazioni a_i eseguita da P_i e a_j eseguita da P_j su d_s nell'ambito di processi intercomunicanti, può essere diversa sia da $f_i(f_j(d_s))$ che da $f_j(f_i(d_s))$.

Una race condition si verifica quando **più processi accedono contemporaneamente a una variabile condivisa d_s** e il risultato finale dipende dall'ordine di esecuzione dei processi.

Se i processi **modificano la variabile uno alla volta** in modo sequenziale, il valore viene aggiornato correttamente. Tuttavia, se lavorano **contemporaneamente**, potrebbero **leggere e scrivere dati vecchi o non aggiornati**, causando errori e risultati incoerenti.

Il problema si verifica perché le operazioni come **lettura-modifica-scrittura** non sono eseguite in modo **atomico** (indivisibile).

Ad esempio, se un processo legge un valore mentre un altro lo sta modificando, entrambi potrebbero lavorare con dati sbagliati.

Per evitare race conditions, è necessario usare **meccanismi di sincronizzazione**, come la **mutua esclusione**, che garantisce che un **solo processo alla volta** possa accedere alla variabile condivisa. Questo permette di controllare l'ordine delle operazioni ed evitare errori.

Esempio 6.1 a pagina 161.

Sincronizzazione per l'accesso ai dati

Le **race condition** possono essere evitate garantendo che le operazioni concorrenti su dati condivisi non vengano eseguite simultaneamente. Questo principio è chiamato **mutua esclusione**: a ogni istante, solo un processo può accedere ai dati condivisi, se un processo sta usando una variabile, gli altri **devono aspettare** prima di poter accedere alla stessa risorsa.

La **sincronizzazione per l'accesso ai dati** consiste nel **coordinare i processi** per implementare la **mutua esclusione**. Questo significa:

- **Ritardare** un processo se un altro sta già accedendo ai dati condivisi.
- **Ripristinare** l'esecuzione del processo una volta che il primo ha terminato l'accesso.

Per verificare se due processi P_i e P_j necessitano di sincronizzazione, si utilizzano le **condizioni di Bernstein**, che analizzano se i due processi operano sugli stessi dati condivisi.

A questo scopo, definiamo:

- **$update_set_i$** : l'insieme dei dati aggiornati dal processo P_i , ossia le variabili lette, modificate e riscritte.
- **$update_set_j$** : l'insieme dei dati aggiornati dal processo P_j , ossia le variabili lette, modificate e riscritte.

Se l'intersezione tra questi due insiemi **non è vuota**:

$$update_set_i \cap update_set_j \neq \emptyset$$

allora i due processi **modificano la stessa variabile** e si genera una race condition, quindi è necessaria la sincronizzazione.

Un esempio pratico è un'applicazione di prenotazione aerea, dove due processi aggiornano la stessa variabile condivisa che rappresenta il numero del prossimo posto assegnato (`nextseatno`). Se entrambi i processi leggessero il valore `nextseatno` prima che uno di loro lo aggiorni, potrebbero assegnare lo stesso posto a due passeggeri diversi, causando incoerenza nei dati.

Sezioni Critiche

Una **sezione critica** è una parte del codice in cui un processo accede a un **dato condiviso** d_s e che non può essere eseguita contemporaneamente da più processi.

Per evitare problemi di concorrenza, si applica il principio della **mutua esclusione**, che garantisce che solo un processo alla volta possa eseguire una sezione critica su quel dato.

Se un processo P_i sta accedendo alla sezione critica su d_s , tutti gli altri processi che desiderano accedervi devono attendere finché P_i non ha terminato.

In questo modo, le sezioni critiche **proteggono i dati condivisi**, impedendo che più processi li modifichino simultaneamente, evitando così errori o incoerenze.

Configurazioni delle Sezioni Critiche

All'interno di un'applicazione, le sezioni critiche possono manifestarsi in **due configurazioni principali**:

- **Se i processi condividono un'unica copia del codice contenente una sezione critica.** In questo caso l'applicazione presenta **un'unica sezione critica** per d_s .
- **Se ogni processo possiede una copia indipendente del codice con la sezione critica.** In questo caso, possono esistere **più sezioni critiche** per d_s , ma devono comunque rispettare la mutua esclusione per garantire la correttezza dell'esecuzione concorrente.

Effetti delle Sezioni Critiche sulle Prestazioni

L'uso delle **sezioni critiche** introduce **ritardi nell'esecuzione dei processi**, pertanto sia i **processi** che il **kernel** devono collaborare per ridurli:

- Un **processo** non deve rimanere **troppo a lungo** in una sezione critica né eseguire **chiamate di sistema bloccati** mentre vi si trova.
- Il Kernel **non deve interrompere un processo** che sta eseguendo una sezione critica, il che implica che debba **essere a conoscenza** di quando un processo vi si trova.
- Il **Kernel deve sempre essere informato** quando un processo si trova in una sezione critica, per garantire che non venga prelazionato.

Proprietà dell'implementazione di una Sezione Critica

Quando più processi devono accedere a una sezione critica per un dato condiviso d_s , l'implementazione della sezione critica deve garantire l'accesso ai processi nel rispetto dei principi di correttezza ed equità.

Esistono **tre proprietà** fondamentali che un'implementazione di una sezione critica deve soddisfare per garantire questi requisiti.

1. **Mutua Esclusione:** In ogni istante, al massimo un processo può eseguire una sezione critica su d_s .

Questa proprietà assicura che due o più processi **non possano trovarsi simultaneamente** all'interno della sezione critica per lo stesso dato.

2. **Progresso:** Se nessun processo sta eseguendo una sezione critica su d_s , allora uno dei processi in attesa otterrà l'accesso.

Questa proprietà impedisce che una sezione critica rimanga **bloccata inutilmente**, evitando che l'accesso possa essere "riservato" a un processo che in quel momento **non è interessato**. Tuttavia, da sola non previene la **starvation**, poiché un processo potrebbe essere costantemente messo in attesa se l'implementazione favorisce sempre altri processi.

3. **Attesa Limitata:** Se un processo P_i ha richiesto di entrare in una sezione critica, il numero di volte in cui altri processi possono accedere prima di lui è limitato da un valore finito.

Questa proprietà impedisce la **starvation**, garantendo che ogni processo ottenga l'accesso alla sezione critica in un tempo finito. Tuttavia, non fornisce un limite specifico al ritardo che un processo potrebbe subire prima di entrare nella sezione critica.

Progresso e **Attesa Limitata** lavorano insieme per garantire che ogni processo abbia accesso alla sezione critica in modo equo, **prevenendo la starvation**.

Ad esempio, in un SO con **scheduling basato sulla priorità**, i processi con **priorità più alta** vengono eseguiti prima di quelli con **priorità più bassa**. Se processi ad alta priorità (P_1 e P_2) continuano a essere aggiunti alla coda di esecuzione, un processo a priorità bassa (P_3) potrebbe **non essere mai eseguito**, subendo **starvation** perché viene costantemente messo in attesa e non ottiene mai l'accesso alla CPU.

Controllo della sincronizzazione e operazioni indivisibili

Sincronizzazione tra Processi e Segnalazione

I processi intercomunicanti devono coordinare la loro esecuzione per garantire che le operazioni avvengano nell'ordine corretto. Questo avviene attraverso il **controllo della sincronizzazione**, che può essere implementato tramite **segnali**.

```

var
    operazione_aj_eseguita : boolean;
    pi_bloccato : boolean;
begin
    operazione_aj_eseguita := false;
    pi_bloccato := false;

Parbegin
    ...
    if operazione_aj_eseguita = false
    then
        pi_bloccato := true;
        block (Pi);
        {esegui l'operazione ai}
    ...
    ...
Parend;
end.

```

Processo P_i Processo P_j

L'esempio mostrato utilizza due processi P_i e P_j , in cui P_i deve eseguire un'operazione a_i solo dopo che P_j ha completato l'operazione a_j . Per garantire questa sincronizzazione, vengono impiegati segnali di controllo. Se P_j ha già eseguito la sua operazione, P_i procede immediatamente; altrimenti, deve attendere bloccandosi. Dopo aver completato la propria operazione, P_j segnala P_i per riprendere l'esecuzione.

Tentativo Maldestro di Sincronizzazione (utilizzo di variabili booleane)

Un'implementazione errata della sincronizzazione mostrata nell'esempio utilizza **variabili booleane** per segnalare lo stato delle operazioni:

- **operazione_aj_eseguita**: indica se P_j ha completato a_j .
- **pi_bloccato**: indica se P_i è in attesa che a_j venga completata.

Tuttavia, questo metodo può portare a un **blocco indefinito**. La tabella di seguito mostra un esempio in cui il processo P_i si blocca permanentemente a causa della prelazione del SO, che cambia l'ordine di esecuzione e causa un'interferenza tra le operazioni di controllo.

Tempo	Azioni del processo P_i	Azioni del processo P_j
t_1	if operazione_aj_eseguita = false	
t_2		{esegui operazione a_j }
t_3		if pi_bloccato = true
t_4		operazione_aj_eseguita = true
...		
t_{20}	pi_bloccato = true;	
t_{21}	blocca (P_i);	

Questo è un caso di race condition, in cui il comportamento del sistema dipende dall'ordine di esecuzione dei processi e dal loro stato al momento della schedulazione.

La causa principale della **race condition** è che P_i e P_j possono essere **prelazionati in momenti critici**, portando a uno stato incoerente nelle variabili di controllo, ovvero il processo P_i può essere prelazionato dopo aver verificato che operazione_aj_eseguita = false ma prima di impostare pi_bloccato a true oppure il processo P_j può essere prelazionato dopo aver verificato che pi_bloccato = false ma prima di impostare operazione_aj_eseguita a true.

Operazioni Indivisibili come Soluzione

Per evitare problematiche come race condition e blocchi indefiniti, si introducono **operazioni indivisibili (operazioni atomiche)**, che non possono essere interrotte o eseguite concorrentemente da più processi.

Un'operazione Indivisibile (o operazione atomica) è un'operazione che viene eseguita **in un solo passo**, senza possibilità di interruzione da altri processi o thread.

Non richiede necessariamente meccanismi di sincronizzazione perché la sua esecuzione è garantita come un'unità unica e non può essere divisa in sotto parti. Tipicamente le operazioni atomiche sono supportate a livello hardware (es. istruzioni CPU come test-and-set, compare-and-swap).

Approcci alla Sincronizzazione

La sincronizzazione tra processi assicura che più processi non possano eseguire contemporaneamente la stessa sezione critica o operazioni indivisibili, prevenendo race condition. Per evitare condizioni di gara esistono diversi approcci.

Attesa Attiva (Busy Waiting)

Un processo in attesa di accedere a una sezione critica controlla ripetutamente se la risorsa è libera, eseguendo un ciclo di controllo (`while`). Questo metodo, però, ha diverse problematiche. Innanzitutto, causa uno **spreco di CPU**, poiché il processo rimane in esecuzione anche se non sta facendo nulla. Inoltre, **viola la proprietà di attesa limitata**, poiché non si può controllare quale processo entrerà per primo nella sezione critica. Infine, può creare **problemi di Scheduling**, in particolare in sistemi time-sharing e a priorità.

Blocco e Sospensione del Processo

Invece di usare l'attesa attiva, un processo in attesa di entrare in una sezione critica viene **bloccato dal kernel** fino a quando la risorsa non è disponibile. Questo metodo **evita lo spreco di CPU** e migliora l'efficienza. Il kernel gestisce lo stato dei processi, riattivando quelli bloccati solo quando è sicuro che possano entrare nella sezione critica.

Per implementare correttamente le sezioni critiche evitando condizioni di gara, si possono adottare due strategie:

- un **approccio algoritmico**, che utilizza strutture di controllo software;
- un **approccio hardware**, che sfrutta istruzioni atomiche della CPU.

Supporto Hardware per la sincronizzazione dei processi

La **sincronizzazione dei processi** richiede l'esecuzione di alcune istruzioni in modo **mutuamente esclusivo**. Nei sistemi monoprocesso, questo può essere ottenuto **disabilitando gli interrupt** mentre un processo esegue tali istruzioni, evitando che venga prelazionato. Tuttavia, questo metodo **introduce un overhead** dovuto alle chiamate di sistema necessarie per gestire gli interrupt e può **ritardare l'elaborazione di operazioni critiche del sistema**, influenzando negativamente sulle prestazioni e sul servizio dell'utente. Inoltre, non è applicabile ai sistemi multiprocessore, perché disabilitare gli interrupt su un processore non impedisce a un altro processore di accedere alla stessa risorsa.

Per superare questi limiti, i SO implementano le sezioni critiche e le operazioni indivisibili utilizzando **istruzioni hardware atomiche** (istruzioni indivisibili) e **variabili di lock**.

Istruzioni indivisibili

Le **istruzioni indivisibili** sono **operazioni atomiche** fornite dall'hardware per garantire la **mutua esclusione** nell'accesso alle risorse condivise. Eseguite in un'unica operazione, impediscono interferenze tra processi e sono utilizzate per implementare meccanismi di sincronizzazione come le **variabili di lock**. Tuttavia, da sole **non garantiscono un accesso equo** alle risorse, motivo per cui devono essere integrate con strategie di sincronizzazione più avanzate. Tecniche come **Test-and-Set**, **Swap**, e **Compare-and-Swap** permettono di gestire l'accesso alle sezioni critiche in modo efficiente, evitando soluzioni meno performanti come la disabilitazione degli interrupt.

Variabili di lock

Per implementare sezioni critiche, si utilizzano **variabili di lock**, che possono assumere due stati: **aperto** (risorsa disponibile, accesso libero) e **chiuso** (risorsa non disponibile, accesso bloccato). Quando un processo desidera entrare in una sezione critica, verifica lo stato della variabile di lock. Se la risorsa è libera, imposta la variabile a **chiuso(1)** e procede con l'esecuzione. Al termine dell'operazione, riporta la variabile allo stato **aperto (0)**, consentendo ad altri processi di accedere alla risorsa. Per evitare **race condition** durante la modifica dello stato della variabile, è fondamentale che l'operazione di aggiornamento sia **atomica**, cioè eseguita senza interruzioni. Questo viene garantito dall'uso di **istruzioni hardware indivisibili**, come `test-and-set`, `swap` e `compare-and-swap`, che assicurano che solo un processo alla volta possa modificare il valore della variabile di lock.

Esempio 6.8 a pagina 169.

Istruzione Test-and-Set (TS)

L'istruzione **Test-and-Set** è una **istruzione atomica** messa a disposizione da alcuni processori per gestire la **mutua esclusione** tra processi o thread. La sua funzione principale è consentire che un solo processo alla volta entri in una **sezione critica**, evitando conflitti nell'accesso a risorse condivise.

Quando viene eseguita su una variabile di **lock**, l'istruzione compie in un unico passo due operazioni: legge il valore corrente della variabile e contemporaneamente lo imposta a **1**. Questa operazione non può essere interrotta da altri processi, e quindi garantisce l'atomicità.

Il comportamento è il seguente: se la variabile di lock era **0**, l'istruzione restituisce **0** e la imposta a **1**, segnalando che il processo ha acquisito il lock ed è quindi autorizzato ad accedere alla sezione critica. Se invece la variabile era già **1**, l'istruzione restituisce **1** (e la variabile rimane a **1**), indicando che la risorsa è occupata da un altro processo.

Il meccanismo di utilizzo tipico consiste in un ciclo di **attesa attiva** (*busy waiting*) in cui i processi chiamano ripetutamente Test-and-Set finché non riescono a ottenere il valore desiderato. Una volta entrati nella sezione critica, al termine dell'esecuzione è necessario rilasciare il lock riportandolo a **0**, in modo che altri processi possano entrare.

Esempio

TEST AND SET	CODICE THREAD (pseudocodice)
<pre>bool TestAndSet (bool &target) { bool retValue = target; target = True; return retValue; }</pre>	<pre>while test_and_set(lock): do nothing # critical section lock = False</pre>

La funzione **TestAndSet** prende in ingresso una variabile booleana che rappresenta il **lock**. All'inizio salva il valore corrente della variabile in una variabile temporanea chiamata `retValue`. Subito dopo imposta il lock a **True**, cioè nello stato "occupato". Infine, restituisce `retValue`, che contiene il valore precedente del lock. In questo modo, se il lock era **False** significa che era libero, quindi la funzione restituisce **False** ma nello stesso tempo lo ha impostato a **True**. Se invece il lock era già **True**, la funzione restituisce **True** lasciandolo nello stato occupato. La forza di questa istruzione sta nell'essere **atomica**, cioè nell'eseguire lettura e scrittura senza permettere che altri thread possano intervenire nel frattempo.

Nel codice del thread si usa questa funzione in un ciclo **while**. Finché la chiamata a `test_and_set(lock)` restituisce **True**, il thread resta bloccato in attesa, perché il lock è occupato da qualcun altro. Quando invece la funzione restituisce **False**, significa che il lock era libero e che il thread è riuscito a occuparlo, quindi può entrare nella **sezione critica**. Al termine dell'esecuzione della sezione critica, il lock viene riportato a **False** per permettere ad altri thread di accedere.

Istruzione Swap

L'istruzione **Swap** è una primitiva atomica usata per implementare la **mutua esclusione**. La sua operazione consiste nello **scambiare i valori** tra una variabile locale (ad esempio `key`) e una variabile condivisa che rappresenta il **lock**. Il thread inizializza `key` a **True** per segnalare l'intenzione di entrare nella **sezione critica**. Quando esegue `swap(key, lock)`, i due valori vengono scambiati in un unico passo indivisibile.

Se il lock era **False**, dopo lo scambio `key` diventa **False** e `lock` diventa **True**: il thread ha ottenuto il lock ed entra nella sezione critica. Se invece il lock era già **True**, dopo lo scambio `key` resta **True** e `lock` rimane **True**: il thread non ottiene il lock e deve ripetere il tentativo.

Esempio

SWAP	CODICE THREAD (pseudocodice)
<pre>void swap (bool &a, bool &b) { bool temp = a; a = b; b = temp; }</pre>	<pre>while(1) { key = true; while(key) swap(lock, key); # critical section lock = false; }</pre>

Parte sinistra: definizione della funzione `swap`

Questa funzione scambia i valori delle due variabili booleane `a` e `b`. L'uso del riferimento `&` fa sì che lo scambio avvenga direttamente sulle variabili passate, senza copie locali. L'operazione è **atomica**: ciò significa che nessun altro thread può intervenire nel mezzo dello scambio.

Parte destra: codice del thread

Un thread che vuole entrare nella **sezione critica** inizia impostando `key = true`. Poi entra in un ciclo `while(key)` che continua a ripetere `swap(lock, key)` finché `key` resta true.

Se il lock era libero (`lock = false`), lo scambio produce `key = false` e `lock = true`. In questo caso, dopo la `swap`, la condizione del ciclo `while(key)` non è più verificata e il thread può uscire dal ciclo per entrare nella sezione critica.

Se invece il lock era già occupato (`lock = true`), lo scambio lascia `key = true` e `lock = true`. In questo caso la condizione del ciclo resta vera e il thread continua ad attendere in **busy waiting**.

Alla fine della sezione critica, il thread rilascia la risorsa riportando `lock = false`, in modo che altri possano accedervi. L'intero ciclo esterno `while(1)` fa sì che il thread ripeta continuamente questo comportamento.

Approcci Algoritmici, primitive di sincronizzazione e costrutti di programmazione concorrente

Approcci Algoritmici

Gli **approcci algoritmici** furono i primi metodi per garantire la **mutua esclusione**, basandosi su schemi logici complessi senza dipendere dall'hardware o dal kernel. Tuttavia, la loro correttezza era difficile da dimostrare e si affidavano all'**attesa attiva**, con un **elevato consumo di risorse**.

Primitive di sincronizzazione

Per migliorare la gestione della sincronizzazione, furono introdotte le primitive di sincronizzazione, come i **semafori**, con operazioni come **wait** e **signal**, che sfruttavano **operazioni atomiche supportate dall'hardware e dal kernel**. Queste permettevano di evitare l'attesa attiva e garantire la mutua esclusione, ma il loro utilizzo errato poteva portare a problemi complessi come **deadlock** e **starvation**.

Costrutti di programmazione concorrente

Infine, i costrutti di programmazione concorrente fornirono un livello di astrazione più alto, integrando meccanismi di sincronizzazione nei linguaggi di programmazione con un controllo più rigoroso. Grazie a questi strumenti, come i **monitor** in java, la gestione della concorrenza divenne più sicura ed efficace, facilitando lo sviluppo di applicazioni di grandi dimensioni.

Problemi Classici di Sincronizzazione dei Processi

Una buona soluzione di sincronizzazione dovrebbe incorporare una combinazione adeguata di questi elementi:

- **Correttezza**: i processi devono accedere ai dati condivisi seguendo le regole stabilite, così da evitare errori o conflitti.
- **Massima concorrenza**: ogni processo dovrebbe poter lavorare liberamente, tranne quando deve aspettare che un altro completi una certa operazione di sincronizzazione.
- **Nessuna attesa attiva**: per non rallentare il sistema, i processi dovrebbero aspettare tramite meccanismi di blocco (come semafori o sleep), e non controllando continuamente (in modo attivo) se possono procedere.

Produttori-Consumatori con buffer limitati

Il sistema è composto da **N processi produttori**, **M processi consumatori** e un **numero finito di buffer**, ognuno dei quali può contenere un solo elemento. I **produttori** inseriscono dati nei buffer, mentre i **consumatori** li prelevano.

I vincoli da rispettare sono i seguenti:

1. Un produttore **non deve sovrascrivere** un buffer già pieno;
2. Un consumatore **non deve prelevare** da un buffer vuoto;
3. I produttori ed i consumatori devono accedere ai buffer in mutua esclusione (cioè uno alla volta);

4. I dati devono essere consumati nell'ordine in cui sono stati prodotti (FIFO – First In, First Out).

Soluzione:

```
begin
  Parbegin
    var prodotto : boolean;
    repeat
      prodotto := false;
      while prodotto = false
        if esiste un buffer vuoto
        then
          { Inserisci nel buffer }
          prodotto := true;
        { Parte restante del ciclo }
      forever;
    Parend;
  end.
  Produttore

  var consumato : boolean;
  repeat
    consumato := false;
    while consumato = false
      if esiste un buffer pieno
      then
        { Estrai dal buffer }
        consumato := true;
      { Parte restante del ciclo }
    forever;
  end.
  Consumatore
```

Un **produttore** entra nella **sezione critica** e controlla se esiste un **buffer vuoto**. Se sì, scrive un elemento nel buffer e imposta la variabile `prodotto = true`, usata per uscire dal ciclo `while`.

Allo stesso modo, un **consumatore** entra nella sezione critica per verificare se c'è un **buffer pieno** da cui prelevare dati, e imposta la variabile `consumato = true` per uscire dal ciclo `while`.

Questa soluzione presenta **due problemi** principali:

1. **Scarsa concorrenza:** L'uso dell'istruzione `if(esiste un buffer vuoto)` implica che ogni produttore deve controllare tutti i buffer per trovarne uno libero. Questo controllo richiede l'accesso in mutua esclusione a tutti i buffer, bloccando gli altri processi. Di fatto, si finisce per sequenzializzare l'accesso dei produttori (e lo stesso vale per i consumatori), riducendo la concorrenza.
2. **Attesa attiva (busy waiting):** Sia i produttori che i consumatori usano un ciclo `while` per controllare ripetutamente se esiste un buffer disponibile (vuoto o pieno). Questo comportamento consuma risorse CPU inutilmente, riducendo l'efficienza.

Produttore e consumatore con singolo buffer

Una **soluzione migliorata** per il caso di un solo produttore, un solo consumatore e un singolo buffer consiste nell'utilizzare un **meccanismo di segnalazione**:

- Dopo che il produttore ha inserito un elemento nel buffer, deve segnalarlo al consumatore.
- Dopo che un consumatore ha prelevato l'elemento dal buffer deve segnalarlo al produttore.

```
var
  buffer : ...;
  buffer_full : boolean;
  producer_blocked, consumer_blocked : boolean;

begin
  buffer_full := false;
  producer_blocked := false;
  consumer_blocked := false;

  Parbegin
    repeat
      check_b_empty;
      { Produce in the buffer }
      post_b_full;
      { Remainder of the cycle }
    forever;
  Parend;
  Producer

  repeat
    check_b_full;
    { Consume from the buffer }
    post_b_empty;
    { Remainder of the cycle }
  forever;
  Consumer
```

check_b_empty blocca il produttore se è vera
check_b_full blocca il consumatore se vera
N.B.: Indivisibili!

Uno schema migliorato per un sistema produttori-consumatori con singolo buffer usando la segnalazione

Per poter implementare il **meccanismo di segnalazione** si ha la necessità di avere delle **operazioni indivisibili**.

Si utilizza una variabile booleana `buffer_full` per sapere se il buffer è pieno, e altre due (`producer_blocked` e `consumer_blocked`) per sapere se uno dei due processi è bloccato.

All'inizio, tutte queste variabili sono impostate a false perché il buffer è vuoto e nessun processo è bloccato. Il **produttore** chiama una **funzione atomica** `check_buffer_empty` per vedere se può scrivere. Se il buffer è pieno, si blocca. Altrimenti, produce e poi chiama `post_buffer_full` per segnalare al consumatore che può leggere. Se il consumatore era bloccato, viene sbloccato.

Il **consumatore** fa lo stesso: chiama `check_buffer_full` per verificare se c'è qualcosa da leggere. Se il buffer è vuoto, si blocca. Se può leggere, consuma e poi chiama `post_buffer_empty` per avvisare il produttore che può produrre di nuovo.

```

procedure check_b_empty
begin
  if buffer_full = true
  then
    produttore_bloccato := true;
    block (produttore);
  end;

  procedure post_b_full
  begin
    buffer_full := true;
    if consumatore_bloccato = true
    then
      consumatore_bloccato := false;
      activate (consumatore);
    end;
  end;

```

Operazioni del produttore

```

procedure check_b_full
begin
  if buffer_full = false
  then
    consumatore_bloccato := true;
    block (consumatore);
  end;

  procedure post_b_empty
  begin
    buffer_full := false;
    if produttore_bloccato = true
    then
      produttore_bloccato := false;
      activate (produttore);
    end;
  end;

```

Operazioni del consumatore

Lettori e Scrittori

Un sistema lettori-scrittori è composto da **dati condivisi**, un numero indefinito di **processi lettori** che possono esclusivamente leggere i dati, e un numero indefinito di **processi scrittori** che possono modificarli o aggiornarli. Una soluzione al problema dei lettori-scrittori deve soddisfare le seguenti condizioni:

1. Molti lettori possono leggere concorrentemente;
2. La lettura è proibita mentre uno scrittore sta scrivendo;
3. Solo uno scrittore può eseguire la scrittura in un dato momento;
4. (opzionale) Un lettore ha una **priorità non prelazionabile** sugli scrittori, ovvero **un lettore ottiene l'accesso ai dati condivisi prima di uno scrittore in attesa**, ma non può prelazionare uno scrittore in esecuzione (si dice che è un sistema lettori-scrittori con preferenza ai lettori).

```

Parbegin
repeat
  If uno scrittore sta scrivendo
  then
    { attendi };
  { leggi }
  If nessun lettore sta leggendo
  then
    if uno scrittore/i è in attesa
    then
      attiva uno scrittore in attesa;
    forever;
  Parend;
end.

```

Lettore/i

```

repeat
  If un lettore/i sta leggendo, o uno
  scrittore sta scrivendo
  then
    { attendi };
  { scrivi }
  If un lettore/i o scrittore/i sono in attesa
  then
    attiva uno scrittore
    o un lettore in attesa;
  forever;

```

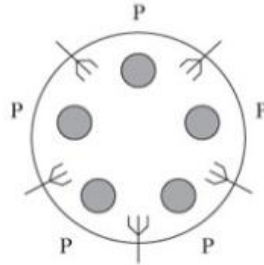
Scrittore/i

- Le **scritture** dei dati vengono eseguite in **mutua esclusione**. Quando uno scrittore termina un'operazione di scrittura, si dovrebbe riattivare un altro scrittore in modo tale che possa entrare nella sua sezione critica per scrivere, oppure attivare tutti i lettori che sono in attesa di leggere.
- Se i lettori stanno leggendo, uno scrittore in attesa può iniziare a scrivere solo quando **l'ultimo lettore ha terminato**. Ciò richiede di mantenere un **conteggio dei lettori attivi**, in modo da sapere quando è possibile concedere l'accesso allo scrittore.
- Le letture **non vengono eseguite in sezione critica** perché ciò limiterebbe la concorrenza tra i lettori. Per la gestione del blocco e attivazione dei lettori e scrittori si fa uso di **segnali** e di **contatori** per monitorare i lettori attivi o in attesa.

Questa soluzione non garantisce **attesa limitata**, ma fornisce comunque **massima concorrenza**. È una soluzione che non favorisce né lettori né scrittori.

Filosofi a Cena

Cinque filosofi siedono attorno a un tavolo, ognuno con un piatto di spaghetti e una forchetta condivisa tra ogni coppia di filosofi. Per mangiare, ciascun filosofo ha bisogno di **entrambe le forchette** alla sua destra e sinistra.



Soluzione:

Ogni processo rappresenta un filosofo. Ciascuno di essi deve poter mangiare (prendendo una forchetta per volta) quando ha fame. Nessuno dovrebbe **morire di inedia** (cioè soffrire di **starvation**, ovvero attesa indefinita). Una buona soluzione deve evitare il **deadlock** (tutti i processi si bloccano in attesa l'uno dell'altro), e **livelock** (i processi sono attivi ma continuano a cedere risorse agli altri senza mai progredire).

```
repeat
  if la forchetta di sinistra non è disponibile
  then
    block ( $P_i$ );
  prendi la forchetta di sinistra;
  if la forchetta di destra non è disponibile
  then
    block ( $P_i$ );
  prendi la forchetta di destra;
  { mangia }
  posa entrambe le forchette
  if il vicino di sinistra in attesa della sua forchetta di destra
  then
    activate (vicino di sinistra);
  if il vicino di destra in attesa della sua forchetta di sinistra
  then
    activate (vicino di destra);
  { pensa }
forever
```

Nota: **Livelock** → il thread è attivo, ma compie solo azioni che non portano mai a progresso.

Struttura e problemi del processo filosofo P_i :

Il filosofo può tentare di prendere prima la forchetta di sinistra, poi quella di destra (può prenderle in qualsiasi ordine). Tuttavia, se tutti i filosofi fanno lo stesso contemporaneamente, ognuno preleva la propria forchetta sinistra, nessuno sarebbe in grado di ottenere la forchetta destra: si genera così un **deadlock**. Inoltre, possono generarsi **race condition** tra filosofi vicini che si contendono una forchetta condivisa.

Per evitare il deadlock, si può modificare il comportamento in modo che se la forchetta destra non è disponibile, il filosofo posa quella sinistra e riprova più tardi, dando priorità al vicino di sinistra. Questo approccio però può portare a **livelock**, perché tutti i filosofi potrebbero continuare a cedere le forchette senza riuscire mai a mangiare.

Soluzione migliore:

```

var    successful : boolean;
repeat
    successful := false;
    while (not successful)
        if entrambe le forchette sono disponibili then
            prendi le forchette una alla volta;
            successful := true;
        if successful = false
            then
                block (Pi);
            { mangia }
            posa entrambe le forchette;
        if il vicino di sinistra è in attesa della sua forchetta di destra
            then
                activate (vicino di sinistra);
        if il vicino di destra è in attesa della sua forchetta di sinistra
            then
                activate (vicino di destra);
        { pensa }
    all'infinito

```

Struttura del processo filosofo P_i:

Questa struttura **previene il verificarsi di deadlock**. Se un filosofo non riesce ad ottenere entrambe le forchette contemporaneamente (controllo effettuato in sezione critica tramite il ciclo while), si **blocca** e verrà **riattivato** quando uno dei suoi vicini rilascia una forchetta condivisa. Lo scopo del ciclo while è quello di **controllare nuovamente la disponibilità delle forchette**, ma questo comporta una forma di **attesa attiva**.

Approccio algoritmico per implementare le sezioni critiche

L'approccio algoritmico alla gestione delle sezioni critiche **non impiega**

- i **servizi del kernel** per il blocco e l'attivazione dei processi;
- le **istruzioni indivisibili HW** per evitare le race condition, rendendo la sincronizzazione indipendente dal SO e dall'hardware.

Tuttavia, **si basa sull'attesa attiva** e su **condizioni logiche complesse** per prevenire le **race condition**, rendendo difficile la verifica della correttezza. Per questo motivo, è poco usato nella pratica.

Algoritmo per due processi

Prima soluzione

<pre> var turn : 1..2; begin turn := 1; Parbegin repeat while turn = 2 do { niente }; { Sezione critica } turn := 2; { Resto del ciclo } forever; Parend; end. </pre> Processo P₁	<pre> repeat while turn = 1 do { niente }; { Sezione critica } turn := 1; { Resto del ciclo } forever; </pre> Processo P₂
--	---

La variabile `turn` è condivisa tra i due processi e può assumere valore 1 o 2. Essa viene inizializzata ad 1 prima della creazione dei due processi e serve ad indicare quale dei due ha il permesso di entrare in **sezione critica**. Se `turn` è uguale a 1, allora P₁ può accedere immediatamente alla sezione critica, altrimenti attende finché non arriva il suo turno, e lo stesso vale per P₂ quando `turn` è uguale a 2. Ogni processo, una volta terminata l'esecuzione nella propria sezione critica, modifica il valore della variabile `turn` assegnandolo all'altro processo, consentendogli di entrare a sua volta. Il meccanismo si basa su un'**attesa attiva**: il processo che non ha il turno resta bloccato in un ciclo while fino a quando la variabile non assume il valore che gli consente l'accesso.

Questo approccio **viola la condizione di progresso**, poiché un processo potrebbe rimanere bloccato anche se l'altro non ha necessità di entrare in sezione critica. Ad esempio, se P1 entra e completa la sezione critica, ponendo `turn = 2` e desidera entrarci nuovamente, e P2 non intende accedervi, P1 resterà comunque bloccato in attesa attiva, non potendo procedere fino a che P2 non modifica `turn`.

Seconda soluzione

```
var  c1, c2 : 0..1;
begin
  c1 := 1;
  c2 := 1;
Parbegin
  repeat
    while c2 = 0
    do { niente };
    c1 := 0;
    { Sezione critica }
    c1 := 1;
    { Resto del ciclo }
  forever;
Parend;
end.
```

Processo P₁

```
repeat
  while c1 = 0
  do { niente };
  c2 := 0;
  { Sezione critica }
  c2 := 1;
  { Resto del ciclo }
forever;
```

Processo P₂

c_i variabili di stato.
c_i = 0 indica quando P_i è in SC
c_i = 1 quando è fuori dalla SC

P1 imposta `c1=0` quando desidera entrare in sezione critica, e lo stesso vale per il processo P2 con la variabile `c2`. Prima di entrare in sezione critica, ciascun processo controlla se l'altro vi si trovi già. In caso affermativo, attende (con un ciclo di attesa attiva) finché l'altro processo non esce dalla sezione critica. Questa soluzione **elimina la violazione della condizione di progresso** dell'algoritmo precedente, poiché i processi non sono costretti ad alternarsi rigidamente nell'accesso alla sezione critica.

Tuttavia, questa soluzione presenta un problema di **violazione della mutua esclusione** quando entrambi i processi tentano di entrare contemporaneamente in sezione critica: poiché entrambi impostano i rispettivi flag (`c1` e `c2`) a 1 prima del controllo, finendo potenzialmente per accedervi insieme.

Una possibile correzione alla violazione della mutua esclusione è **invertire l'ordine delle istruzioni**,



cioè posizionare i flag `c1` e `c2` prima del ciclo `while` e impostare entrambe le variabili a 0. In questo modo si evita l'accesso simultaneo, ma si introduce un nuovo problema: se entrambi i processi fanno lo stesso, i flag risultano entrambi a 0, e ciascun processo a causa del ciclo di controllo, entrerà in attesa, creando così una situazione di stallo (**deadlock**).

I problemi relativi alla **violazione della mutua esclusione** e **deadlock** possono essere evitati se un processo favorisce l'altro quando riscontra la volontà di quest'ultimo di entrare in sezione critica. Ad esempio, se P1 riscontra che P2 vuole entrare, può rinunciare temporaneamente, impostando `c1` a 1, permettendo così a P2 di entrare. Dopo un breve tempo di attesa, P1 può provare a entrare nella sezione critica, reimpostando `c1` a 0. P2 può fare lo stesso in modo simmetrico.

Tuttavia, questo approccio può portare a una situazione di **livelock**, in cui entrambi i processi continuano a favorirsi reciprocamente senza che nessuno dei due riesca a entrare effettivamente nella sezione critica.

Algoritmo di Dekker

```
var  turn : 1..2;
     c1, c2 : 0..1;
begin
  c1 := 1;
  c2 := 1;
  turn := 1;
Parbegin
  repeat
    c1 := 0;
    while c2 = 0 do
      if turn = 2 then
        begin
          c1 := 1;
          while turn = 2
            do { niente };
          c1 := 0;
        end;
        { Sezione critica }
        turn := 2;
        c1 := 1;
        { Resto del ciclo }
      forever;
    Parend;
  end.
  Processo P1

  repeat
    c2 := 0;
    while c1 = 0 do
      if turn = 1 then
        begin
          c2 := 1;
          while turn = 1
            do { niente };
          c2 := 0;
        end;
        { Sezione critica }
        turn := 1;
        c2 := 1;
        { Resto del ciclo }
      forever;
    end.
  Processo P2
```

Questo algoritmo combina le soluzioni adottate nei due algoritmi precedenti per **evitare il livelock**.

Combina **due meccanismi**: l'uso di **variabili booleane** come flag (c_1 e c_2) per indicare l'intenzione dei processi di entrare nella sezione critica, e una **variabile ausiliaria** $turn$ che regola l'accesso in caso di conflitto.

Quando un solo processo desidera accedere alla sezione critica, può farlo senza alcun ostacolo. Se invece entrambi i processi tentano di entrare contemporaneamente, il valore di $turn$ stabilisce a quale dei due venga data la precedenza. Il processo che non ha il turno attende che l'altro completi la propria esecuzione prima di proseguire. Questa gestione consente di evitare situazioni di blocco reciproco o attese indefinite.

La variabile $turn$ viene aggiornata opportunamente affinché i processi si alternino in modo equo nel caso in cui vi sia competizione, ma non influisce sul comportamento quando uno solo dei due richiede l'accesso. In questo modo l'algoritmo garantisce la **mutua esclusione**, **evita il deadlock** e **previene il livelock**, assicurando sia **correttezza** che **progresso**.

Esempio di funzionamento:

Supponiamo che entrambi i processi, P_1 e P_2 , vogliano entrare nella loro sezione critica nello stesso momento. P_1 imposta c_1 a 0, segnalando la sua intenzione di entrare. Simultaneamente, anche P_2 imposta c_2 a 0 con lo stesso scopo. A questo punto, entrambi entrano nella rispettiva istruzione `while`, dove ciascuno controlla se l'altro ha già espresso la stessa intenzione.

Poiché entrambi vedono che l'altro ha $c = 0$, ciascun processo verifica il valore della variabile $turn$. Supponiamo che $turn$ sia inizialmente impostato a 1. In questo caso, P_1 ha il permesso di procedere e ignora il blocco `if`, accedendo direttamente alla sezione critica. P_2 , invece, rileva che $turn$ è uguale a 1 e quindi deve favorire P_1 . Per questo motivo entra in attesa attiva nel ciclo `while turn = 1`.

Una volta che P_1 ha completato la propria sezione critica, reimposta c_1 a 1, aggiorna $turn$ a 2, e prosegue con il resto del ciclo. A questo punto, P_2 nota che $turn$ è cambiato e può uscire dal ciclo d'attesa per entrare nella propria sezione critica.

Algoritmo di Peterson

```
var   flag : array [0..1] of boolean;
      turn : 0..1;
begin
    flag[0] := false;
    flag[1] := false;
Parbegin
    repeat
        flag[0] := true;
        turn := 1;
        while flag[1] and turn = 1
            do {nothing};
        { Critical Section }
        flag[0] := false;
        { Remainder of the cycle }
    forever;
Parend;
end.

                                     Process P0

                                     repeat
                                     flag[1] := true;
                                     turn := 0;
                                     while flag[0] and turn = 0
                                     do {nothing};
                                     { Critical Section }
                                     flag[1] := false;
                                     { Remainder of the cycle }
                                     forever;

                                     Process P1
```

L'**algoritmo di Peterson** è una soluzione più semplice rispetto a quella proposta da Dekker per garantire la **mutua esclusione** tra due processi. Utilizza un **array di flag booleane**, dove ciascun processo imposta la propria flag a `true` per indicare la volontà di entrare nella sezione critica, e la riporta a `false` una volta uscita.

Ogni processo, prima di entrare, assegna alla variabile `turn` l'identificatore dell'altro processo, **concedendogli la precedenza in caso di conflitto**. Tuttavia, se l'altro processo non è interessato a entrare (ovvero la sua flag è `false`), il processo corrente può procedere direttamente.

Nel caso in cui entrambi i processi desiderino accedere contemporaneamente, il valore di `turn` risolve la contesa decidendo quale dei due può entrare per primo nella sezione critica. L'algoritmo si comporta correttamente anche se i due processi operano a velocità diverse: il primo che imposta `turn` all'altro e verifica che la flag dell'altro sia `false`, ottiene l'accesso.

In questo modo, viene **garantita la mutua esclusione**, si **evita il deadlock** e si **impedisce il livelock**.

Esempio di funzionamento:

Supponiamo di avere due processi, P0 e P1, che vogliono entrare nella propria sezione critica. All'inizio, entrambi hanno la propria flag impostata a `false`, e la variabile `turn` ha un valore arbitrario.

Supponiamo che P0 sia leggermente più veloce di P1, e quindi inizia per primo. P0 imposta `flag[0]` a `true`, segnalando la sua intenzione di entrare, e poi assegna `turn = 1`, concedendo temporaneamente la precedenza a P1. Subito dopo, P0 controlla se P1 vuole entrare: se `flag[1]` è ancora `false`, la condizione del `while` non è verificata (devono essere `true` sia `flag[1]` che `turn=1`) e allora P0 non attende ed entra direttamente nella sezione critica.

Nel frattempo, anche P1 vuole entrare. Imposta `flag[1] = true` e `turn = 0`. Ora P1 verifica le condizioni del `while`: vede che `flag[0]` è ancora `true`, ma `turn` è 0, cioè sta cedendo il turno a P0. Perciò P1 aspetta in un ciclo di attesa attiva, finché P0 non esce dalla sezione critica. Una volta che P0 è uscito dalla sezione critica, imposta `flag[0] = false`.

Una volta che P0 ha finito, P1 nota che `flag[0]` è diventato `false` e può così entrare nella propria sezione critica.

Algoritmo del Panettiere (Bakery)

```

const n = ...;
var  choosing : array [0 .. n - 1] of boolean;
      number : array [0 .. n - 1] of integer;
begin
  for j := 0 to n - 1 do
    choosing[j] := false;
    number[j] := 0;
  Parbegin
    process Pi :

      repeat
        choosing[i] := true;
        number[i] := max (number[0], ..., number[n - 1]) + 1;
        choosing[i] := false;
        for j := 0 to n - 1 do
          begin
            while choosing[j] do { nothing };
            while number[j] ≠ 0 and (number[j], j) < (number[i], i)
              do { nothing };
          end;
          { Sezione critica }
          number[i] := 0;
          { Resto del ciclo }
        forever;
      processo Pj : ...
    Parend;
  end.

```

L'**algoritmo del panettiere** risolve il problema della **mutua esclusione** tra più processi che desiderano accedere a una **sezione critica**, ispirandosi al sistema dei numeri in coda nei panifici. Ogni processo, prima di entrare nella sezione critica, sceglie un **numero progressivo** più alto rispetto a quelli già scelti dagli altri processi, stabilendo così un **ordine di accesso**.

L'array `choosing[0..n-1]`, dove `choosing[i]` indica se P_i è impegnato nella scelta.

L'array `number[0..n-1]`, dove `number[i]` contiene il numero scelto da P_i . `number[i] = 0` indica che P_i non ha scelto il numero.

Per decidere chi entra per primo tra due processi P_i e P_j , si confrontano le coppie formate da $(\text{number}[i], i) < (\text{number}[j], j)$.

La regola è: **Il processo con la coppia minore entra per primo.**

Questa relazione è definita così:

$(\text{number}[j], j) < (\text{number}[i], i)$ se $\text{number}[j] < \text{number}[i]$

Oppure se $\text{number}[j] == \text{number}[i]$ e $j < i$.

In altre parole, il numero scelto dal processo ha la precedenza, ma se due processi scelgono lo stesso numero, entra prima quello con ID più basso.

Quando un processo P_i vuole entrare, imposta `choosing[i] = true` per indicare che sta scegliendo un numero. Subito dopo, assegna a `number[i]` il valore massimo tra i numeri attualmente scelti dagli altri processi +1:

`number[i] = max(number[0], ..., number[n-1]) + 1.`

Una volta assegnato il numero, imposta `choosing[i] = false` per indicare che ha terminato la fase di scelta.

A questo punto, prima di entrare nella sezione critica, il processo P_i deve attendere che ogni altro processo P_j abbia terminato la propria scelta. Questo è garantito dal ciclo `while choosing[j] do {nothing};` che impone a P_i di aspettare finché P_j è ancora nella fase di selezione del numero. Ga

Dopodiché, P_i verifica se ci sono processi che hanno scelto un numero più basso. La condizione

`while number[j] ≠ 0 and (number[j], j) < (number[i], i) do {nothing};` assicura che P_i entri solo quando non esistono altri processi P_j con un numero più basso, oppure in caso di parità di numero, con un ID

minore. Questo confronto tra le coppie $(\text{number}[j], j)$ e $(\text{number}[i], i)$ definisce un ordine totale e risolve eventuali conflitti di accesso.

Quando tutte queste condizioni sono soddisfatte, P_i può finalmente accedere alla sezione critica. Una volta completata, esce resettando il proprio numero a zero con $\text{number}[i] = 0$, permettendo agli altri processi in attesa di rivalutare la priorità.

Cosa accade se non usiamo l'array choosing?

```
const n = ...;
var choosing : array [0..n-1] of boolean;
    number : array [0..n-1] of integer;
begin
  for j := 0 to n-1 do
    choosing[j] := false;
    number[j] := 0;
  Parbegin
    process Pi :
      repeat
        choosing[i] := true;
        number[i] := max (number[0]..number[n-1])+1;
        choosing[i] := false;
        for j := 0 to n-1 do
          begin
            while choosing[j] do { nothing };
            while number[j] ≠ 0 and (number[j], j) < (number[i], i)
              do { nothing };
          end;
          { Critical Section }
          number[i] := 0;
          { Remainder of the cycle }
        forever;
    process Pj : ...
  Parend;
end.
```

cosa accade
se non usiamo
l'array choosing ?

Se si rimuove l'array `choosing` dall'algoritmo del panettiere, si compromette la sua correttezza, in particolare la mutua esclusione. Questo array serve a segnalare quando un processo sta ancora scegliendo il proprio numero, cioè quando il valore in `number[i]` non è ancora definitivo.

Con l'array `choosing`, ogni processo prima setta `choosing[i] = true`, poi sceglie `number[i]`, poi setta `choosing[i] = false`.

Gli altri processi, prima di confrontare `number[i]`, aspettano che `choosing[i] == false`, così sono sicuri che il numero è definitivo.

Esempio pratico:

Supponiamo di avere due processi, P_0 e P_1 , che vogliono entrare nella sezione critica quasi nello stesso momento. Supponiamo che viene schedato P_0 e che inizi a scegliere il proprio numero; quindi, sta eseguendo `number[0] = max(...) + 1`, ma non ha ancora terminato quando viene prelazionato. Intanto viene schedato P_1 e vede che `number[0]` ha ancora il valore iniziale, ad esempio 0, e considera che P_0 non voglia entrare. Così P_1 prosegue e potrebbe entrare nella sezione critica. Subito dopo, viene schedato P_0 che termina la sua scelta, e ottiene `number = 1`, e non vedendo P_1 nella sezione critica, entra anche lui. In questo modo entrambi i processi si trovano nella sezione critica contemporaneamente, violando la mutua esclusione.

ESEMPIO — senza choosing

Inizialmente:

- `number[0] = 0`, `number[1] = 0` (entrambi fuori dalla sezione critica)

1. `P0` viene schedulato:

- Inizia a scegliere `number[0]`
- Fa:

```
cpp
```

[Copia](#)[Modifica](#)

```
number[0] = max(number) + 1 → number[0] = 1
```

- Ma viene interrotto prima di scrivere il valore (magari legge `max = 0`, ma non ha ancora fatto `number[0] = 1`).
- Quindi `number[0]` è ancora `0`.

2. Viene schedulato `P1`:

- Calcola `number[1] = max(0, 0) + 1 = 1`
- Entra nel ciclo `for` e guarda `P0`
 - Vede `number[0] = 0`, quindi **non aspetta** (`while` salta)
- `P1` entra nella **sezione critica**

3. `P0` riprende:

- Completa l'assegnazione: `number[0] = 1`
- Ora entra anche lui nel ciclo `for` e guarda `P1`
 - `number[1] = 1`, `number[0] = 1`, ma `P1` ha `j=1` e `P0` ha `i=0`
 - `(1,1) < (1,0)` è **falso**, quindi il `while` salta di nuovo
- Anche `P0` entra nella **sezione critica**

Semafori

Un semaforo è una variabile intera condivisa a valori non negativi, che può essere soggetta solo alle seguenti operazioni:

1. **Inizializzazione**, specificata come parte della sua dichiarazione.
2. Le operazioni indivisibili **wait** e **signal**, la cui indivisibilità è garantita dal linguaggio di programmazione o dal SO che le implementa, assicurando che non si possano verificare race condition sul semaforo.

I processi utilizzano le operazioni `wait` e `signal` per **sincronizzare la loro esecuzione** rispetto agli altri. Il valore iniziale di un semaforo determina quanti processi possono effettuare l'operazione `wait` senza essere bloccati. Un processo che non supera l'operazione `wait` viene **bloccato** sul semaforo.

L'operazione `wait` su un semaforo verifica se il valore del semaforo è maggiore di 0. In caso affermativo, decrementa il valore del semaforo e consente al processo di proseguire la sua esecuzione; altrimenti, blocca il processo sul semaforo.

```
procedure wait (S)
begin
  if S > 0
  then S := S-1;
  else blocca il processo su S;
end;
```

L'operazione `signal` su un semaforo **risveglia** un processo bloccato sul semaforo, se ve ne sono; in caso contrario, incrementa il valore del semaforo di una unità.


```

procedure signal (S)
begin
  if qualche processo è bloccato su S
  then attiva un processo bloccato;
  else S := S+1;
end;

```

Quando un semaforo ha valore zero e un processo è bloccato perché ha precedentemente eseguito un'operazione `wait` trovando il semaforo a zero, una successiva operazione `signal` non incrementa il valore del semaforo, ma risveglia un processo in attesa. Questo processo riprende l'esecuzione consumando immediatamente il "permesso" ricevuto, quindi il valore del semaforo resta comunque zero.

Uso dei semafori nei sistemi concorrenti

Nei sistemi concorrenti i semafori possono essere utili per implementare:

- **Mutua esclusione:** Si utilizza un semaforo inizializzato a **1**. Un processo, prima di entrare in una **sezione critica (CS)**, esegue un'operazione `wait` sul semaforo, che ne decrementa il valore da 1 a 0. Se, mentre il primo processo è nella sezione critica, un secondo processo tenta di entrare anch'esso, la sua `wait` troverà il semaforo a 0 e il processo verrà **bloccato**. In questo modo, si garantisce che un solo processo alla volta possa accedere alla sezione critica.

Esempio:

```

var sem_CS : semaforo := 1;
Parbegin
  repeat
    wait (sem_CS);
    { Sezione critica }
    signal (sem_CS);
    { Parte restante del ciclo }
  all'infinito;
Parend;
end.
Processo Pi

```

```

  repeat
    wait (sem_CS);
    { Sezione critica }
    signal (sem_CS);
    { Parte restante del ciclo }
  all'infinito;
Processo Pj

```

- **Concorrenza limitata:** La concorrenza limitata si verifica quando una risorsa o una funzione può essere utilizzata simultaneamente da un numero limitato di processi compreso tra 1 e una costante c , $1 \leq n \leq c$. In questo caso un semaforo inizializzato a c può controllare l'accesso concorrente: ogni processo esegue una `wait` prima di accedere alla risorsa, decrementando il semaforo, e una `signal` quando ha terminato, incrementandolo. I semafori utilizzati in questo contesto sono detti **semafori contatore**.
- **Segnalazione:** La segnalazione è usata quando un **processo P_i** deve eseguire un'operazione **ai** solo dopo che un **processo P_j** ha effettuato una operazione **aj**. Questo meccanismo può essere implementato con un semaforo inizializzato a **0**. Il **processo P_i** esegue una `wait` prima di procedere con l'operazione **ai** e trovando il semaforo a 0, **si blocca**. Dopo aver completato l'operazione **aj**, il **processo P_j** esegue una `signal`, che sblocca **P_i** , permettendogli di proseguire con **ai**.

Esempio:

```

var sync : semaforo := 0;
Parbegin
  ...
  wait (sync);
  { Esegui l'azione ai }
Parend;
end.
Processo Pi

```

```

  ...
  { Esegui l'azione aj }
  signal (sync);
Processo Pj

```

Semaforo binario

Un **semaforo binario** è una variante dei semafori (NB: Non è un'altra tipologia di semaforo! L'implementazione è la stessa!) che può assumere solo i valori **0** e **1**. Viene principalmente utilizzato per garantire la **mutua esclusione**,

ovvero per assicurare che **un solo processo alla volta** possa accedere a una **risorsa condivisa**. Proprio per questo motivo, un semaforo binario è spesso chiamato **mutex**.

```
waitB(semaphore *S){
    if (S->value == 1)

        S->value = 0
    else {
        aggiungi P a S->list;
        block();
    }

    signalB(semaphore *S){
        if S->list vuota
            S->value = 1;
        else {
            toglì P da S->list;
            wakeup(P);
        }
    }
}
```

In fase di inizializzazione, il semaforo binario viene impostato a **1**, indicando che la risorsa è libera. Quando un processo desidera accedere alla risorsa, esegue l'operazione `wait`. Se il semaforo è a 1, il processo può entrare nella sezione critica e il valore del semaforo viene **impostato a 0**, segnalando che la risorsa è ora occupata. Se invece il semaforo è già a 0, il processo viene **bloccato** finché la risorsa non sarà disponibile.

Quando il processo ha terminato l'utilizzo della risorsa, esegue l'operazione `signal`, che **riporta il valore del semaforo a 1**, rendendo nuovamente disponibile la risorsa per altri processi.

A differenza dei **semafori contatori**, che possono assumere valori maggiori di uno e che utilizzano le istruzioni $S = S - 1$ in `wait` e $S = S + 1$ in `signal`, i semafori binari semplificano queste operazioni: nella `wait`, il valore viene semplicemente impostato a **0**, mentre nella `signal` viene riportato a **1**.

Produttori-Consumatori usando i semafori

Il problema dei **produttori-consumatori** è un classico esempio di **sincronizzazione tramite segnalazione**. Un **produttore** segnala al **consumatore** che un elemento è stato prodotto ed è pronto per essere consumato. Allo stesso modo, il consumatore segnala al produttore che ha consumato un elemento, rendendo il buffer nuovamente disponibile.

Questo meccanismo **evita l'attesa attiva**, poiché l'accesso al buffer è regolato attraverso l'uso dei **semafori**, che controllano se il buffer è pieno o vuoto. Nel caso di un **buffer singolo**, la **concorrenza effettiva è limitata a uno**, poiché se un processo sta producendo, l'altro è bloccato, e viceversa.

Produttore-Consumatore con buffer singolo

```
type item = . . . ;
var
  full : Semaforo := 0; { Inizializzazione }
  empty : Semaforo := 1;
  buffer : array [0] of item;
begin
  Parbegin
    repeat
      wait (vuoto);
      buffer [0] := . . . ;
      { i.e. produci }
      signal (pieno);
      { Parte restante dei ciclo }
    all'infinito;
  Parend;
end.

                                     Produttore

                                     repeat
                                     wait (full);
                                     x := buffer [0];
                                     { i.e. consuma }
                                     signal (vuoto);
                                     { Parte restante dei ciclo }
                                     all'infinito;

                                     Consumatore
```

In questa versione semplificata del problema, si utilizza un array con un solo elemento, ovvero un **buffer di dimensione 1**. Si impiegano **due semafori**:

- `empty`, inizializzato a **1**, indica che il buffer è vuoto e può essere riempito;
- `full`, inizializzato a **0**, indica che il buffer contiene un elemento pronto per essere consumato.

Il **produttore**, prima di produrre, esegue una `wait (empty)`. Dato che `empty` inizialmente vale 1, può accedere al buffer e scrivere il nuovo elemento. Al termine della produzione, esegue `signal (full)` per segnalare che un elemento è disponibile.

Se nel frattempo il **consumatore** è stato schedato prima che il produttore abbia eseguito `signal (full)`, troverà il semaforo `full` a zero ed eseguendo `wait (full)` resterà **bloccato**. Verrà **risvegliato** solo dopo che il produttore esegue `signal (full)`.

Il consumatore, una volta sbloccato, accede al buffer, consuma l'elemento e infine esegue `signal (empty)` per indicare che il buffer è libero e può essere riutilizzato dal produttore.

Questo ciclo si ripete continuamente, garantendo che il **produttore e il consumatore non accedano mai contemporaneamente al buffer**, evitando condizioni di competizione e mantenendo il sistema sincronizzato.

Produttore-Consumatore ad N Buffer con Semafori

Una variante del problema prevede un **produttore** e un **consumatore** che condividono un **buffer di N elementi**. Il produttore può continuare a produrre finché c'è almeno uno slot libero nel buffer. Il buffer è implementato come un **array circolare**, per cui si utilizzano **due indici**: uno per il produttore e uno per il consumatore.

```

const      n = . . . ;
type      item = . . . ;
var
    buffer : array [0..n - 1] of item;
    full : Semaforo := 0; { Inizializzazioni }
    empty : Semaforo := n;
    prod_ptr, cons_ptr : integer;

begin
    prod_ptr := 0;
    cons_ptr := 0;

Parbegin
    repeat
        wait (vuoto);
        buffer [prod_ptr] := . . . ;
        { i.e. produci }
        prod_ptr := prod_ptr + 1 mod n;
        signal (pieno);
        { Parte restante del ciclo }
    all'infinito;
Parend;
end.

```

Produttore

```

repeat
    wait (pieno);
    x := buffer [cons_ptr];
    { i.e. consuma }
    cons_ptr := cons_ptr + 1 mod n;
    signal (vuoto);
    { Parte restante del ciclo }
all'infinito;

```

Consumatore

Il **produttore** scrive nella posizione indicata dal proprio indice, poi lo aggiorna per indicare la prossima posizione disponibile. Il consumatore fa lo stesso, aggiornando il proprio indice dopo ogni consumo. Poiché l'array è circolare, si utilizza l'operazione **modulo N** per evitare di uscire dai limiti dell'array.

Il **buffer** è rappresentato da un array di N elementi. Il semaforo `full` tiene conto del numero di slot **pieni** ed è inizializzato a 0, mentre il semaforo `empty` conta il numero di slot **vuoti** ed è inizializzato a N . Inoltre, sono presenti due indici, `prod_ptr` e `cons_ptr`, inizializzati entrambi a 0, che indicano rispettivamente la prossima posizione in cui produrre e quella da cui consumare.

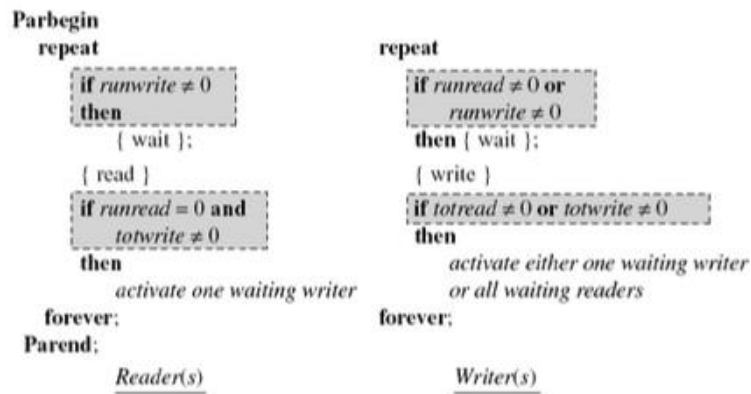
Il **produttore**, prima di accedere al buffer, esegue una `wait(empty)`. Dato che inizialmente il buffer è vuoto, ci sono slot disponibili; quindi, la `wait` ha successo e il produttore può scrivere nel buffer. Dopo aver prodotto un elemento, incrementa `prod_ptr` (utilizzando il modulo N per rimanere all'interno dell'array) e poi esegue `signal(full)`, segnalando che è presente un nuovo elemento pronto per il consumo.

Il **consumatore**, se viene schedato, esegue `wait(full)`. Se non ci sono elementi da consumare (cioè se `full == 0`), si blocca fino a quando il produttore non produce qualcosa. Una volta che `wait(full)` ha successo, il consumatore legge l'elemento dal buffer, aggiorna `cons_ptr` e infine esegue `signal(empty)` per indicare che uno slot è stato liberato e può essere riutilizzato dal produttore.

Se il **produttore** viene continuamente schedato, potrà produrre fino a riempire tutto il buffer, momento in cui il semaforo `empty` raggiunge 0 e il produttore si blocca. A quel punto, il **consumatore** può essere schedato, iniziare a consumare e, liberando spazio, permettere al produttore di riprendere la produzione.

Questa soluzione, se correttamente implementata, garantisce **mutua esclusione** nell'accesso al buffer ed evita **race condition**. Inoltre, permette che i processi possano procedere in modo **concorso**, anche se in momenti alternati, a seconda dello stato del buffer e dei semafori.

Lettori-Scrittori con Semafori



Nel problema dei lettori-scrittori, **N processi lettori** accedono in lettura a una **risorsa condivisa**, mentre **N processi scrittori** vi accedono in scrittura. I lettori **possono leggere simultaneamente**, senza causare race condition, a patto che nessuno stia scrivendo. Al contrario, uno scrittore **deve avere accesso esclusivo** alla risorsa: **non può scrivere** se ci sono altri scrittori attivi o se **almeno un lettore** sta leggendo. I semafori vengono usati per proteggere la sezione critica e per bloccare/sbloccare i lettori e gli scrittori.

Per gestire correttamente l'accesso alla risorsa, si utilizzano alcune **variabili di stato**:

- `totread`: numero totale di lettori in attesa.
- `runread`: numero di lettori che stanno attualmente leggendo.
- `totwrite`: numero totale di scrittori in attesa.
- `runwrite`: numero di scrittori attivi (può essere solo 0 o 1).

Funzionamento

Lettori

Quando un lettore vuole accedere alla risorsa, entra prima in una **sezione critica** per verificare se c'è uno scrittore attivo. Se `runwrite ≠ 0`, il lettore **si blocca**. Se invece non ci sono scrittori attivi, può iniziare a leggere: in tal caso, incrementa `runread`.

Dopo la lettura, decrementa `runread`. Se al termine della lettura, `runread = 0` e ci sono scrittori in attesa (`totwrite ≠ 0`), viene attivato **uno scrittore**.

Scrittori

Uno scrittore può accedere alla risorsa **solo se nessun lettore sta leggendo** (`runread = 0`) e **nessun altro scrittore sta scrivendo** (`runwrite = 0`). Se queste condizioni non sono soddisfatte, lo scrittore si blocca.

Quando uno scrittore inizia a scrivere, imposta `runwrite = 1`.

Al termine della scrittura, imposta `runwrite = 0` e controlla se ci sono **lettori o scrittori in attesa** (`totread ≠ 0` o `totwrite ≠ 0`). A seconda delle politiche di priorità adottate, può decidere se **risvegliare uno scrittore** o **tutti i lettori**.

Lettori-Scrittori con Semafori con Preferenza ai Lettori

In questa soluzione vengono utilizzate le variabili intere `totread`, `runread`, `totwrite` e `runwrite`, tutte inizializzate a 0, per tenere traccia rispettivamente del numero totale di lettori in attesa, lettori attivi, scrittori in attesa e scrittori attivi.

Si usano inoltre tre **semafori**:

- `sem_CS`, inizializzato a 1, per garantire **mutua esclusione** durante l'accesso alle variabili condivise;
- `reading` e `writing`, entrambi inizializzati a 0, per **bloccare** o **risvegliare** lettori e scrittori.

```

var
    totread, runread, totwrite, runwrite : integer;
    reading, writing : semaforo := 0;
    sem_CS : semaforo := 1;

begin
    totread := 0;
    runread := 0;
    totwrite := 0;
    runwrite := 0;

Parbegin
    repeat
        wait (sem_CS);
        totread := totread + 1;
        if runwrite = 0 then
            runread := runread + 1;
            signal (reading);
        signal (sem_CS);
        wait (reading);
        { Leggi }
        wait (sem_CS);
        runread := runread - 1;
        totread := totread - 1;
        if runread = 0 and
            totwrite > runwrite
        then
            runwrite := 1;
            signal (writing);
        signal (sem_CS);
    forever;

Parent;
end.

    repeat
        wait (sem_CS);
        totwrite := totwrite + 1;
        if runread = 0 and runwrite = 0 then
            runwrite := 1;
            signal (writing);
        signal (sem_CS);
        wait (writing);
        { Scrivi }
        wait (sem_CS);
        runwrite := runwrite - 1;
        totwrite := totwrite - 1;
        while (runread < totread) do
            begin
                runread := runread + 1;
                signal (reading);
            end;
        if runread = 0 and
            totwrite > runwrite then
            runwrite := 1;
            signal (writing);
        signal (sem_CS);
    forever;

    Scrittore/i

```

Funzionamento dei Lettori

Il lettore, per prima cosa, esegue una `wait(sem_CS)` per accedere in mutua esclusione alla sezione critica. Incrementa `totread`, indicando che vuole leggere.

Se nessuno scrittore sta scrivendo (`runwrite = 0`), può iniziare a leggere: incrementa `runread`, invoca `signal(reading)` per permettere a sé stesso di procedere con la lettura, e rilascia la sezione critica con `signal(sem_CS)`.

Subito dopo, esegue `wait(reading)` per aspettare eventualmente l'autorizzazione a leggere, e poi legge la risorsa condivisa.

Terminata la lettura, esegue di nuovo `wait(sem_CS)` per entrare nella sezione critica e decrementa sia `runread` sia `totread`.

A questo punto, se non ci sono più lettori attivi (`runread = 0`) e ci sono scrittori in attesa (`totwrite > runwrite`), allora autorizza uno scrittore: imposta `runwrite = 1` e invoca `signal(writing)`. Infine, rilascia il semaforo `sem_CS`.

Funzionamento degli Scrittori

Lo scrittore entra nella sezione critica con `wait(sem_CS)` e incrementa `totwrite`.

Se non ci sono lettori attivi (`runread = 0`) e nemmeno scrittori attivi (`runwrite = 0`), può iniziare a scrivere: imposta `runwrite = 1`, invoca `signal(writing)` e rilascia `sem_CS`.

Dopodiché, esegue `wait(writing)` per accedere effettivamente alla scrittura. Una volta terminato, torna nella sezione critica con `wait(sem_CS)`, decrementa `runwrite` e `totwrite`.

Ora controlla attraverso un ciclo se ci sono **lettori in attesa**: quindi, finchè `while runread < totread`, li attiva uno per uno. Per ogni lettore incrementa `runread` e invoca `signal (reading)` affinché possano leggere in parallelo.

Infine, **se non ci sono più lettori attivi** (`runread = 0`) e ci sono ancora scrittori in attesa (`totwrite > runwrite`), consente a uno scrittore di procedere (`runwrite = 1`) e invoca `signal (writing)`.

In ogni caso, termina rilasciando il semaforo `sem_CS`.

Implementazione dei Semafori

L'**implementazione dei semafori** prevede la definizione di un **tipo di dato** che include il **valore del semaforo**, una **lista** per memorizzare gli identificativi dei **processi bloccati** su di esso e una **variabile di lock** (es. Test and Set oppure Swap), utilizzata per garantire che le operazioni `wait` e `signal` siano eseguite in modo **indivisibile**. Le due operazioni sono realizzate come **procedure** che prendono in ingresso una variabile semaforo, e vengono utilizzate dai **processi concorrenti** per coordinarsi sull'**accesso alle risorse condivise**.

Dichiarazione del tipo semaforo

```
type
  semaforo = record
    value : integer;  { valore del semaforo }
    list : ...       { lista dei processi bloccati }
    lock : boolean;   { variabile di lock per le operazioni su questo semaforo }
  end;
```

Procedure per implementare le operazioni wait e signal

```
procedura wait (sem)
begin
  Close_lock (sem.lock);
  if sem.value > 0
  then
    sem.value := sem.value - 1;
    Open_lock (sem.lock);
  else
    Aggiunge l'id del processo alla lista dei processi bloccati su sem;
    block_me (sem.lock);
  end;

procedura signal (sem)
begin
  Close_lock (sem.lock);
  if qualche processo è bloccato su sem
  then
    proc_id := id di un processo bloccato su sem;
    activate (proc_id);
  else
    sem.value := sem.value + 1;
    Open_lock (sem.lock);
  end;
```

Per evitare **race condition** durante la modifica del valore del semaforo, le procedure `wait` e `signal` acquisiscono inizialmente un **lock** tramite la funzione `Close_lock`, che si basa su un'**istruzione indivisibile** (es. Test and Set oppure Swap) e su un'**attesa attiva**.

Quest'ultima è molto breve, poiché le operazioni sui semafori sono rapide. Al termine dell'operazione, viene invocata la funzione `Open_lock` per rilasciare il lock. Nei sistemi che utilizzano una **pianificazione a priorità**, l'attesa attiva può portare all'**inversione della priorità** (Priority Inheritance Protocol, o PIP). In un **sistema time-sharing**, l'attesa attiva può introdurre **ritardi**, ma non causa problemi gravi.

La procedura `wait` controlla se il **valore del semaforo** è maggiore di zero. In tal caso, lo **decrementa** e termina immediatamente. Se invece il valore è pari a zero, aggiunge l'**ID del processo** alla lista dei **bloccati** e invoca la chiamata di sistema `block_me`, che effettua due operazioni in una: **sospende** il processo e contemporaneamente **rilascia il lock**. È importante notare che questa sequenza deve essere gestita con attenzione: se il lock venisse **rilasciato prima** di bloccare il processo, si potrebbero verificare **race condition**; se invece il processo venisse **bloccato prima** del rilascio del lock, si rischierebbe un **deadlock**.

La procedura `signal` verifica se ci sono **processi bloccati** sul semaforo. Se presenti, ne **seleziona uno** e lo riattiva con una chiamata `activate`. Se non ci sono processi in attesa, **incrementa** semplicemente il valore del semaforo. È consigliabile mantenere la lista dei bloccati come una **coda**, in modo da risvegliare sempre il **primo processo** in ordine di arrivo, garantendo così la proprietà **dell'attesa limitata**. Tuttavia, la semantica dell'operazione `signal` non impone un ordine specifico, lasciando all'implementazione la libertà di scegliere come gestire il risveglio dei processi.

Implementazione dei semafori (NO valori negativi)

```
typedef struct{
    int value;
    struct process *list;
} semaphore;

wait(semaphore *S) {
    if (S->value > 0)
        S->value--;
    else {
        aggiungi P a S->list;
        block();
    }
}

signal(semaphore *S) {
    if (qualche P bloccato su S){
        toglì P da S->list;
        wakeup(P);
    }
    else
        S->value++;
}
```

Implementazione dei semafori (SI valori negativi)

```
typedef struct{
    int value;
    struct process *list;
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        aggiungi P a S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        toglì P da S->list;
        wakeup(P);
    }
}
```

Implementazione livello kernel, livello utente e ibrida

Nell'**implementazione a livello Kernel**, le procedure `wait` e `signal` sono gestite direttamente dal **kernel**. I semafori possono essere condivisi tra tutti i processi del sistema, ma ogni invocazione di `wait` o `signal` comporta una **chiamata di sistema**, con conseguente **overhead elevato**. In un sistema **monoprocessore** con un kernel **non prelazionabile**, non è necessario utilizzare una **variabile di lock**, quindi è possibile eliminare l'overhead causato dalle operazioni di **chiusura e apertura del lock**.

Nell'**implementazione a livello utente**, le operazioni `wait` e `signal` sono fornite come **procedure di libreria**, collegate all'applicazione. I semafori vengono gestiti direttamente nell'ambiente utente, e le operazioni di blocco e risveglio (`block_me` e `activate`) sono anch'esse gestite dalla **libreria** stessa, che ricorre al **kernel solo quando necessario**. Questo approccio è particolarmente adatto per i **thread di livello utente**, poiché la libreria dei thread può gestire autonomamente blocco, attivazione e schedulazione. Una chiamata di sistema per bloccare il processo avverrebbe solo se **tutti i thread** risultassero bloccati.

L'implementazione ibrida combina i due approcci. Anche qui `wait` e `signal` sono definite come **procedure di libreria**, ma in questo caso le chiamate `block_me` e `activate` sono **fornite dal kernel**. Le chiamate di sistema vengono eseguite **solo quando strettamente necessario**, cioè quando un processo deve essere bloccato o risvegliato. Poiché l'operazione `wait` fallisce raramente, le system call vengono effettuate di rado, permettendo così di ottenere **prestazioni migliori** rispetto a un'implementazione completamente a livello kernel, mantenendo al contempo la **correttezza** del comportamento.

Problema del Barbiere Addormentato

In un negozio di barbiere c'è:

- Un barbiere;
- N sedie per i clienti in attesa;
- 1 poltrona da barbiere;

Specifiche:

- Se **non ci sono clienti**, il barbiere si **addormenta sulla poltrona**;
- Quando **arriva, un cliente** sveglia il barbiere e si fa **tagliare i capelli** sulla poltrona;
- Se arriva un cliente mentre il barbiere sta tagliando i capelli ad un altro cliente, il **cliente attende** su una delle sedie libere;
- Se tutte le sedie sono **occupate**, il cliente, contrariato, **se ne va!**

Soluzione tramite semafori:

```
sem_CS: semaphore := 1;
clienteDisponibile, poltrona : semaphore := 0;
inAttesa : integer;
SEDIE : const;

begin
    SEDIE := N;
    inAttesa := 0;
Parbegin
    repeat

        wait(ClienteDisponibile);
        wait(sem_CS);
        inAttesa := inAttesa - 1;
        signal(poltrona);
        signal(sem_CS);
        {taglia capelli};

    forever

        wait(sem_CS);
        if inAttesa < SEDIE then
            inAttesa := inAttesa + 1;
            signal(ClienteDisponibile);
            signal(sem_CS);
            wait(poltrona);
            {taglio dei capelli}
        else signal(sem_CS);
            {lascia il negozio};

    Parent
    Barbiere
    Clienti
```

La sincronizzazione è gestita con:

- Un semaforo `sem_CS` inizializzato a **1**, per la **mutua esclusione**;
- Due semafori inizializzati a **0**: `ClienteDisponibile`, che segnala l'arrivo di un cliente, e `Poltrona`, che consente al cliente di sedersi sulla poltrona quando il barbiere è pronto;
- Una variabile intera `inAttesa`, inizializzata a 0, che conta i clienti seduti in attesa;
- Una costante `SEDIE`, che rappresenta il numero di posti disponibili in sala d'attesa.

Comportamento del Barbiere

Il barbiere esegue inizialmente una `wait(ClienteDisponibile)` e si **blocca** finché non arriva un cliente (simula il fatto che sta dormendo). Quando un cliente lo sveglia, il barbiere entra nella **sezione critica** con `wait(sem_CS)`,

decrementa `inAttesa` (poiché un cliente è passato dalla sala d'attesa alla poltrona), poi esegue una `signal(Poltrona)` per permettere al cliente di sedersi sulla poltrona e infine rilascia il **mutex** con `signal(sem_CS)`. A questo punto, procede con il **taglio dei capelli**.

Comportamento dei Clienti

Quando un cliente arriva, esegue `wait(sem_CS)` per accedere alla sezione critica. Se `inAttesa < SEDIE`, significa che c'è almeno una sedia libera: in tal caso, **incrementa** `inAttesa`, segnala la propria presenza con `signal(ClienteDisponibile)` (svegliando eventualmente il barbiere), poi rilascia la sezione critica con `signal(sem_CS)`. Subito dopo, esegue `wait(Poltrona)`, bloccandosi finché il barbiere non lo invita sulla poltrona. Una volta sbloccato, può procedere con il **taglio dei capelli**.

Se invece `inAttesa >= SEDIE`, non ci sono sedie libere. In questo caso, il cliente **rilascia il mutex** con `signal(sem_CS)` e **lascia il negozio** senza attendere.

Monitor

Un **monitor** è una struttura ad alto livello che incapsula **dati condivisi**, **strutture dati di supporto** e **procedure** che ne permettono l'accesso in modo controllato. I processi non possono accedere direttamente ai dati interni del monitor, ma solo tramite le procedure che esso espone. Il monitor garantisce **mutua esclusione implicita**: anche se più processi possono invocare le procedure del monitor, solo uno alla volta può eseguire una procedura attivamente all'interno.

Per la **sincronizzazione**, il monitor fornisce un **meccanismo di lock implicito** e **variabili di condizione** (zero o più) associate a **code FIFO**. Il lock garantisce che un solo processo per volta sia attivo nel monitor, mentre le **variabili di condizione** permettono ai processi di sospendersi e attendere determinate condizioni.

Un processo può interagire con una **variabile di condizione** `cond` attraverso **due operazioni**:

- `cond.wait()`: il processo che la invoca viene **sospeso** e inserito in una coda FIFO associata a `cond`. Il monitor **rilascia il lock** e consente ad altri processi di entrare.
- `cond.signal()`: **risveglia** il primo processo nella coda associata a `cond`, se presente. Se la coda è vuota, il segnale viene perso (non accumulato come accade nei semafori).

Esempio di sincronizzazione tra processi mediante monitor:

Si considerino due processi concorrenti, P1 e P2, che devono eseguire rispettivamente le istruzioni S1 e S2. Si impone la seguente **restrizione di precedenza**: l'istruzione S1 deve necessariamente essere eseguita prima di S2, indipendentemente dall'ordine in cui i due processi vengano schedulati.

Per garantire il vincolo di ordinamento tra S1 e S2, si implementa un monitor contenente due procedure:

- F1, invocata da P1, che esegue S1;
- F2, invocata da P2, che esegue S2;

All'interno del monitor si definiscono:

- Una variabile di condizione `x`, utilizzata da P2 per attendere l'esecuzione di S1.
- Una variabile booleana `done`, inizializzata a `false`, che rappresenta lo stato dell'esecuzione di S1.

Le procedure sono implementate nel seguente modo:

```
F1 () {
    S1;
    done = true;
    x.signal();
}
```

```
F2 () {
```

```

if (done == false)
    x.wait();
S2;
}

```

Descrizione del funzionamento

- **Caso 1: il processo P1 viene schedulato prima di P2**
In questo scenario, P1 accede al monitor, esegue S1, imposta `done = true` e invoca `x.signal()`. Quando P2 accede successivamente al monitor ed esegue S2, verifica che `done == true` e quindi prosegue direttamente con l'esecuzione di S2, senza blocchi.
- **Caso 2: il processo P2 viene schedulato prima di P1**
P2 accede al monitor, verifica che `done == false` e invoca `x.wait()`, sospendendosi sulla variabile di condizione `x` e rilasciando il lock del monitor. Quando P1 viene successivamente schedulato, esegue S1, imposta `done = true` e invoca `x.signal()`, risvegliando così il processo P2. Quest'ultimo può quindi riprendere l'esecuzione da dove era stato sospeso, ed eseguire S2.

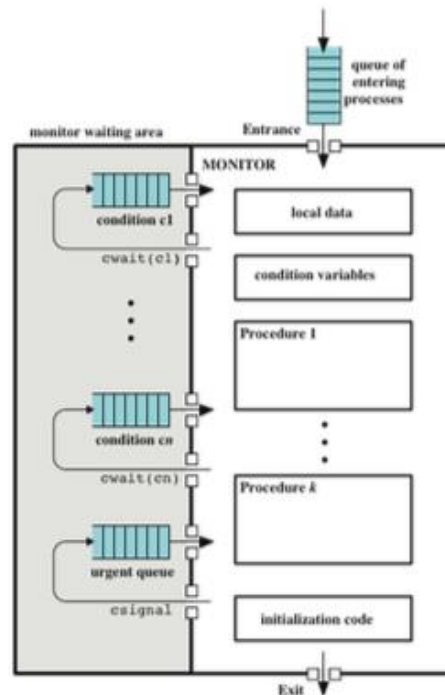
In tal modo si assicura che l'istruzione S2 non venga mai eseguita prima di S1, indipendentemente dall'ordine di scheduling dei processi.

Monitor: Regole per le variabili di condizione

Nei monitor, per garantire la mutua esclusione, è previsto che al massimo un processo possa essere attivo all'interno del monitor in ogni istante. Quando un processo invoca `cond.signal()` su una variabile di condizione, è necessario stabilire quale processo continuerà l'esecuzione nel monitor. A tale scopo, sono state definite diverse **semantiche operative**:

- **Hoare**: Si esegue il processo appena risvegliato, sospendendo l'altro (che ha invocato `cond.signal()`). Questa semantica garantisce che la **condizione associata** alla `wait` **sia ancora valida** nel momento in cui il processo risvegliato riprende l'esecuzione.
- **Brinch Hansen**: il processo che invoca `cond.signal()` **deve uscire immediatamente dal monitor**; e la `cond.signal()` può pertanto essere invocata solo come ultima istruzione di una procedura. Lo scheduler del SO è responsabile della scelta del prossimo processo da attivare: questo può essere il processo risvegliato oppure un altro in coda d'attesa.
Dato che non è garantito che la condizione su cui il processo si era bloccato sia ancora valida al momento del risveglio, è necessario che la `cond.wait()` sia sempre inserita all'interno di un ciclo `while`, in modo da ricontrollare la condizione ed evitare risvegli spuri.
- **Mesa**: adottata in sistemi come Unix e nelle principali librerie di sistema, prevede che il **processo che esegue `cond.signal()` continui la propria esecuzione**, mentre il **processo risvegliato** potrà accedere al monitor **solo dopo che il segnalante sarà uscito**. In questo modello **non è garantita la validità della condizione** al momento del risveglio. Di conseguenza, è necessario che la `cond.wait()` **sia racchiusa in un ciclo `while`** per verificarne nuovamente la validità prima di procedere.

Struttura di un Monitor



Possiamo immaginare un monitor come composto da una **prima coda**, in cui si accumulano i processi che desiderano accedere al monitor per invocare una delle sue procedure. Questa è chiamata **queue of entering processes**, e serve a gestire l'accesso concorrente garantendo che **solo un processo per volta possa entrare**, in accordo con il principio di **mutua esclusione**.

All'interno del monitor possono essere definite una o più **variabili di condizione**, ciascuna delle quali è associata a una propria **coda di attesa**. Queste variabili servono per sincronizzare i processi sulla base di certe condizioni logiche.

I processi possono sospendere la propria esecuzione tramite l'operazione `wait(c)`, bloccandosi nella **coda della variabile di condizione** `c`, fino a quando un altro processo non esegue `signal(c)`.

Quando un processo esegue `signal(c)`, il processo risvegliato **non entra immediatamente** nel monitor: deve attendere che il processo corrente termini. Il processo risvegliato viene temporaneamente messo in una **urgent queue** (coda urgente), **che ha priorità rispetto alla coda dei processi** che vogliono entrare nel monitor dall'esterno. Solo quando il monitor si libera, il processo risvegliato può riprendere la propria esecuzione, rispettando le regole del modello di monitor utilizzato (es. Hoare o Mesa).

Semaforo Binario con Monitor

```
monitor Sem_Mon_type
```

```
var
```

```
    busy: boolean;  
    non_busy: condition;
```

```
procedure sem_wait;
```

```
begin  
    if busy = true then  
        non_busy.wait;  
    busy := true;  
end;
```

```
procedure sem_signal;
```

```
begin  
    busy := false;  
    non_busy.signal;  
end;
```

```
begin // initialization
```

```
    busy := false;  
end;
```

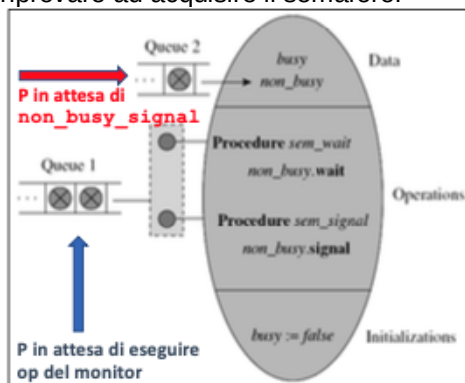

In questo esempio viene mostrata l'implementazione di un **semaforo binario** utilizzando un **monitor**. L'obiettivo è realizzare il comportamento tipico delle operazioni `sem_wait` e `sem_signal`, ma senza ricorrere ai semafori come primitivi, bensì sfruttando le potenzialità di sincronizzazione offerte dai monitor e dalle variabili di condizione.

All'interno del monitor vengono dichiarate due **variabili**. La prima è `busy`, di tipo **booleano**, che rappresenta lo stato del semaforo: se il suo valore è `true`, significa che il semaforo è **occupato**, ovvero un processo ha già invocato `sem_wait` e non ha ancora eseguito `sem_signal`; se il valore è `false`, il semaforo è **libero**. La seconda variabile è `non_busy`, che è una **variabile di condizione**. Questa serve per **sospendere** i processi che tentano di acquisire il semaforo quando esso risulta già occupato. In altre parole, un processo che invoca `sem_wait` mentre `busy` è `true` dovrà sospendersi in attesa che un altro processo esegua `sem_signal` e liberi il semaforo.

Nella sezione di **inizializzazione** del monitor, la variabile `busy` viene impostata inizialmente a `false`, in modo da indicare che il semaforo è libero. In questo modo, il primo processo che invoca `sem_wait` potrà procedere senza blocchi.

La procedura `sem_wait` viene utilizzata dai processi per **acquisire il semaforo**. Al suo interno, viene verificato il valore della variabile `busy`: se essa è `true`, significa che il semaforo è già stato acquisito da un altro processo e quindi il processo chiamante si blocca eseguendo `non_busy.wait`. Se invece `busy` è `false`, il semaforo è disponibile e il processo può proseguire, impostando `busy = true` per indicare che il semaforo è ora occupato.

La procedura `sem_signal` viene invece utilizzata per **rilasciare il semaforo**. Essa imposta il valore di `busy` a `false`, segnalando quindi che il semaforo è libero. Subito dopo viene eseguita un'operazione di `cond_signal` sulla variabile `non_busy`, che ha l'effetto di **risvegliare** un eventuale processo in attesa su quella condizione, permettendogli di riprovare ad acquisire il semaforo.



L'immagine rappresenta la struttura interna di un **monitor** divisa in tre sezioni: **Data**, **Operations** e **Initializations**. Nella sezione Data sono presenti la variabile booleana `busy`, che indica lo stato del semaforo, e la variabile di condizione `non_busy`, usata per sospendere i processi in attesa. Le operazioni `sem_wait` e `sem_signal` gestiscono rispettivamente l'acquisizione e il rilascio del semaforo, usando `wait` e `signal` sulla **variabile di condizione**. All'inizio, `busy` viene inizializzato a `false`.

Sono presenti **due code di attesa**:

- **Queue 2**: coda dei processi che vogliono entrare nel monitor.
- **Queue 1**: coda dei processi sospesi su `non_busy`, in attesa che il semaforo venga rilasciato.

Esempio di utilizzo di un monitor per la sincronizzazione di 3 processi

```
var
    binary_sem: Sem_Mon_type;
```

process P1:

```
begin
    repeat
        binary_sem.sem_wait;
        // Sezione critica
        binary_sem.sem_signal;
        // Resto del ciclo
    forever;
end;
```

process P2:

```
begin
    repeat
        binary_sem.sem_wait;
        // Sezione critica
        binary_sem.sem_signal;
        // Resto del ciclo
    forever;
end;
```

process P3:

```
begin
    repeat
        binary_sem.sem_wait;
        // Sezione critica
        binary_sem.sem_signal;
        // Resto del ciclo
    forever;
end;
```

Nel codice mostrato viene implementato un classico scenario di sincronizzazione tra processi mediante un **semaforo binario** realizzato con un **monitor**. Ogni processo, prima di accedere alla **sezione critica** (SC), deve invocare la procedura `sem_wait` per tentare di acquisire il semaforo. Se il semaforo risulta libero (`busy = false`), il processo può entrare nella sezione critica. Una volta terminata, il processo rilascia il semaforo invocando `sem_signal`. Questo schema viene ripetuto ciclicamente da tutti i processi.

Supponiamo ora un esempio di esecuzione: il **processo P1** viene schedulato per primo e invoca `sem_wait`. Poiché il semaforo è inizialmente libero, P1 lo acquisisce correttamente e accede alla sezione critica. Durante l'esecuzione della sezione critica da parte di P1, il **processo P2** tenta anch'esso di eseguire `sem_wait`. In questa fase, P2 riesce a entrare nel monitor (dato che P1, una volta entrato nella sezione critica, ha rilasciato il monitor), ma trova la variabile `busy = true`. Di conseguenza, P2 si sospende invocando `non_busy.wait` e viene inserito nella **coda della variabile di condizione**.

Nel frattempo, il **processo P3** tenta anch'esso di eseguire `sem_wait`, ma lo fa mentre P1 è uscito dalla sezione critica e sta eseguendo la procedura `sem_signal`. In questo momento il monitor è occupato da P1, che deve completare l'operazione `sem_signal` (che include l'aggiornamento di `busy` e la segnalazione della condizione). Pertanto, **P3 non può ancora entrare nel monitor** e viene posto nella **coda d'ingresso (Queue 2)** in attesa di poter accedere.

Esempio: Produttori-Consumatori con Monitor

```

type Bounded_buffer_type = monitor
  const
    n = ...;                                { Numeri di buffer }
  type
    item = ...;
  var
    buffer : array [0..n-1] of item;
    full, prod_ptr, cons_ptr : integer;
    buff_full : condition;
    buff_empty : condition;
  procedura produci (produced_info : item);
  begin
    if full = n then buff_empty.wait;
    buffer [prod_ptr] := produced_info;      { i.e. produci }
    prod_ptr := prod_ptr + 1 mod n;
    full := full + 1;
    buff_full.signal;
  end;
  procedura consuma (for_consumption : item);
  begin
    if full = 0 then buff_full.wait;
    for_consumption := buffer[cons_ptr];     { i.e. consuma }
    cons_ptr := cons_ptr + 1 mod n;
    full := full - 1;
    buff_empty.signal;
  end;
  begin { inizializzazione }
    full := 0;
    prod_ptr := 0;
    cons_ptr := 0;
  end;
end;

```

Dichiarazione del monitor e struttura dati

```

type Bounded_buffer_type = monitor
const
  n = ...;
type
  item = ...;
var
  buffer : array [0..n-1] of item;

```

```

full, prod_ptr, cons_ptr : integer;
buff_full : condition;
buff_empty : condition;

```

All'interno del monitor viene definito un buffer circolare di dimensione N, rappresentato come un array. Le variabili intere prod_ptr e cons_ptr sono utilizzate rispettivamente per individuare la prossima posizione in cui il produttore scriverà un elemento e la prossima da cui il consumatore leggerà. La variabile full tiene traccia del numero di slot attualmente occupati nel buffer. Le due variabili di condizione, buff_empty e buff_full, sono impiegate per sospendere e risvegliare i processi produttori e consumatori in caso di buffer pieno o vuoto.

Procedura produci

```

procedura produci (produced_info : item);
begin
    if full = n then buff_empty.wait;
    buffer[prod_ptr] := produced_info;
    prod_ptr := prod_ptr + 1 mod n;
    full := full + 1;
    buff_full.signal;
end;

```

La procedura produci è invocata dai **processi produttori**. Quando il buffer è pieno, ovvero quando `full = n`, il produttore non può inserire nuovi elementi e si **sospende** tramite la `wait` sulla variabile di condizione `buff_empty`. Una volta che c'è spazio disponibile, il produttore inserisce l'elemento nella posizione indicata da `prod_ptr`, aggiornando la variabile utilizzando l'**aritmetica modulare** per mantenere il comportamento circolare, e incrementa il contatore `full`. Infine, viene invocata una `signal` sulla variabile di condizione `buff_full` per **risvegliare** un eventuale consumatore in attesa.

Procedura consuma

```

procedura consuma (for_consumption : item);
begin
    if full = 0 then buff_full.wait;
    for_consumption := buffer[cons_ptr];
    cons_ptr := cons_ptr + 1 mod n;
    full := full - 1;
    buff_empty.signal;
end;

```

La procedura consuma è eseguita dai **processi consumatori**. Se il buffer è vuoto, cioè se `full = 0`, il consumatore non può leggere e si **blocca** invocando la `wait` su `buff_full`. Quando viene risvegliato o se il buffer contiene già elementi, il consumatore legge il dato dalla posizione `cons_ptr`, aggiorna l'indice in modulo n, e decrementa il valore di `full` per indicare che un elemento è stato rimosso dal buffer. Infine, invoca `buff_empty.signal` per **notificare** un produttore eventualmente in attesa.

Inizializzazione

```

begin { inizializzazione }
    full := 0;
    prod_ptr := 0;
    cons_ptr := 0;
end;

```

Durante la fase di **inizializzazione**, le variabili vengono impostate in modo che il buffer risulti completamente vuoto. `full` è inizializzato a zero, e gli indici `prod_ptr` e `cons_ptr` vengono entrambi impostati alla prima posizione dell'array, pronti rispettivamente a scrivere e leggere i primi elementi.

Esempio di utilizzo con un unico buffer e 3 processi (2 produttori e 1 consumatore)

```

begin
var B_buf : Bounded_buffer_type;

Parbegin
  var info : item;
  repeat
    info := ...
    B_buf.produce (info);
    { Parte restante
      del ciclo }
  all'infinito;
Parend;
end.

      var info : item;
      repeat
        info := ...
        B_buf.produce (info);
        { Parte restante
          del ciclo }
      all'infinito;

      var area : item;
      repeat
        B_buf.consume (area);
        { Area di consumo }
        { Parte restante
          del ciclo }
      all'infinito;

Produttore P1      Produttore P2      Consumatore P3

```

Viene definita una variabile `B_buf` di tipo `Bounded_buffer_type`, cioè un monitor che incapsula un buffer circolare.

Successivamente vengono creati **tre processi concorrenti**: due produttori (P_1 e P_2) e un consumatore (P_3), ciascuno eseguito all'interno di un ciclo infinito.

I processi P_1 e P_2 generano un valore da produrre e invocano il metodo `produce (info)` del monitor. L'accesso al monitor è **esclusivo**, pertanto se un produttore si trova al suo interno, l'altro deve **attendere** che venga rilasciato. Analogamente, il processo P_3 , il consumatore, richiama `consume (area)` per accedere al buffer, anch'esso soggetto alle stesse regole di **mutua esclusione**.

Nel caso illustrato, si ipotizza che tutti e tre i processi vengano avviati simultaneamente. Supponiamo che P_1 venga **schedulato per primo**. Questo processo accede al monitor, produce un elemento e poi esce per continuare con il codice restante. Durante l'esecuzione di P_1 , i processi P_3 e P_2 tentano di accedere al monitor: P_3 per consumare e P_2 per produrre. Tuttavia, poiché il monitor è occupato da P_1 , entrambi restano **bloccati all'ingresso** del monitor, in attesa di ottenere l'accesso.

Una volta che P_1 termina la produzione e **rilascia il monitor**, il sistema può schedulare un altro processo. Se viene scelto P_2 , quest'ultimo entra nel monitor, ma nel caso in cui il **buffer sia pieno** (ad esempio, se la dimensione è uno e l'unico slot è già stato occupato da P_1), P_2 si blocca sulla **variabile di condizione** `buff_empty`, poiché non può produrre fino a che il consumatore non libera uno slot.

A questo punto, P_3 può essere schedulato. Poiché il monitor è stato liberato da P_2 nel momento in cui si è sospeso, P_3 ottiene accesso al monitor, consuma l'unico elemento disponibile e, così facendo, **risveglia** P_2 segnalando la variabile di condizione `buff_empty`.

Message Passing

Quando i processi interagiscono tra loro, devono essere soddisfatti due requisiti fondamentali: **sincronizzazione** e **comunicazione** tramite lo scambio di informazioni.

Lo **scambio di messaggi** è un approccio che permette di ottenere entrambe le funzionalità ed è particolarmente adatto a **sistemi distribuiti**, **multiprocessore con memoria condivisa** e **monoprocessore**.

Motivazioni

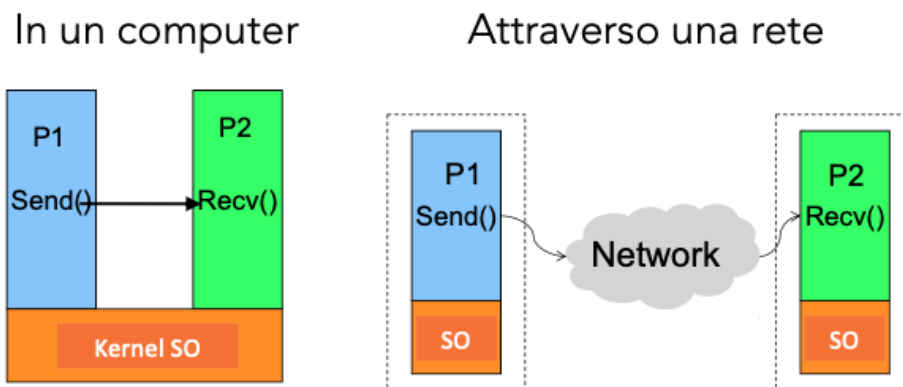
Lock, **semafori** e **monitor** funzionano solo quando è presente uno **spazio di indirizzamento condiviso**, ossia nel caso di thread all'interno dello stesso processo o di processi che condividono lo stesso spazio. Finora si è assunto che i processi o thread comunichino tramite **dati condivisi**, come un **contatore** o un **buffer produttore/consumatore**. Tuttavia, quando i processi hanno **spazi di indirizzamento separati**, è necessario ricorrere a meccanismi di **Inter-Process Communication (IPC)** per ottenere **sincronizzazione** e **comunicazione**.

È possibile implementare questi meccanismi attraverso l'**utilizzo di primitive** che funzionino in ogni situazione: in presenza di un solo SO su una macchina singola, con più macchine che eseguono lo stesso sistema operativo o addirittura con più SO distribuiti su più macchine.

Spazio di indirizzamento non condiviso

In questo contesto, **non è necessaria la mutua esclusione**, poiché i processi non possono accedere direttamente agli stessi dati. La **comunicazione** avviene tramite **invio e ricezione espliciti** dei messaggi. La **sincronizzazione è implicita** nello scambio dei messaggi, non richiede quindi meccanismi espliciti di **mutua esclusione** e consente di **ordinare gli eventi** tramite l'invio e la ricezione. Questo approccio risulta **più portabile** in ambienti diversi, anche se introduce una certa **complessità** che va gestita. Nella maggior parte dei casi, la comunicazione si realizza attraverso un **invio e una ricezione corrispondente**.

Invio di un Messaggio



Il **message passing** può avvenire sia tra processi sullo stesso computer e quindi condividono l'accesso al kernel, sia tra processi su macchine diverse collegate in rete.

Primitive di comunicazione

In linea generale esistono due primitive fondamentali:

- `send(destination, message)` consente l'invio di un messaggio a un destinatario specificato;
- `receive(source, message)` permette di ricevere un messaggio proveniente da un mittente.



Quando un processo P_i invia un messaggio a P_j , il kernel verifica se P_j è **bloccato** su una chiamata `receive()`. Se sì, **consegna immediatamente** il messaggio e **sblocca** P_j . Se no, il messaggio viene messo in attesa fino a quando P_j invoca `receive()`.

Send e Receive Bloccanti e Non Bloccanti

Entrambe le primitive devono **prevedere un meccanismo di sincronizzazione**, che è intrinseco all'invocazione stessa della funzione. Le due system call possono essere **bloccanti** o **non bloccanti** e la loro combinazione dà origine a diversi scenari di sincronizzazione:

- `send()` e `receive()` **bloccanti**: in questo caso sia il **mittente** che il **destinatario si bloccano** fino al completamento dello scambio del messaggio. Questo tipo di sincronizzazione prende il nome di **rendez-vous**, ed è utile quando è necessario garantire un allineamento preciso (sincronizzazione stretta) tra i due processi.

- `send()` **non bloccante** e `receive()` **bloccante**: il mittente invia il messaggio e prosegue senza attendere conferma, mentre il destinatario rimane in attesa finché non riceve il messaggio. Questa modalità è particolarmente utile quando il mittente deve inviare più messaggi a destinatari diversi, ad esempio in un contesto server-client. Tuttavia, può causare problemi al destinatario in caso di perdita del messaggio, tipica delle connessioni non affidabili.
- `send()` e `receive()` **non bloccanti**: in questa configurazione nessuno dei due processi si blocca. Il mittente invia il messaggio e continua la sua esecuzione, mentre il destinatario invoca `receive()` indipendentemente dalla presenza effettiva del messaggio. In questo caso, la **sincronizzazione è a carico del programmatore**, che deve assicurarsi che i due eventi (invio e ricezione) avvengano in modo coerente, evitando situazioni in cui il messaggio arrivi prima che il destinatario sia pronto, o viceversa.

Indirizzamento

Gli schemi utilizzati per specificare i processi nelle primitive `send` e `receive` si suddividono in due categorie principali: **Indirizzamento diretto** e **Indirizzamento indiretto**.

Indirizzamento Diretto

In questo schema, il **mittente specifica esplicitamente il processo destinatario** nella chiamata `send()`, e il **ricevente specifica il mittente** nella `receive()`.

```
send (<processo_destinazione>, <lunghezza_messaggio>,
      <indirizzo_messaggio>);
receive (<processo_sorgente>, <message_area>);
```

Dove `<processo_destinazione>` e `<processo_sorgente>` rappresentano i nomi o identificatori ID dei processi (tipicamente gestiti dal kernel), `<indirizzo_messaggio>` è l'indirizzo dell'area di memoria nello spazio di indirizzamento di `Pi` che contiene il messaggio da inviare, e `<message_area>` è l'area di memoria nello spazio di indirizzamento di `Pj` dove il messaggio viene consegnato.

Nel caso della `receive()`, il mittente può essere:

- **esplicitamente specificato** (naming **simmetrico**), oppure
- **implicito**, accettando messaggi da qualsiasi mittente (naming **asimmetrico**).

L'indirizzamento asimmetrico è utile in scenari come un **pool di processi**, dove il ricevente non deve essere legato a un singolo mittente.

Indirizzamento Indiretto

Con l'**indirizzamento indiretto**, i messaggi vengono inviati a un **contenitore condiviso** tra mittente e ricevente, come una **mailbox**, una **coda**, una **pipe** o una **socket**.

In questo caso, mittente e destinatario non si indirizzano tra loro direttamente, ma tramite un identificatore comune (es. nome della mailbox).

L'**ordine di consegna dei messaggi** dipende dalla struttura del contenitore (FIFO, LIFO, ecc).

Questo tipo di indirizzamento consente un **disaccoppiamento** tra mittente e destinatario, migliorando la **flessibilità** e la **scalabilità** della comunicazione.

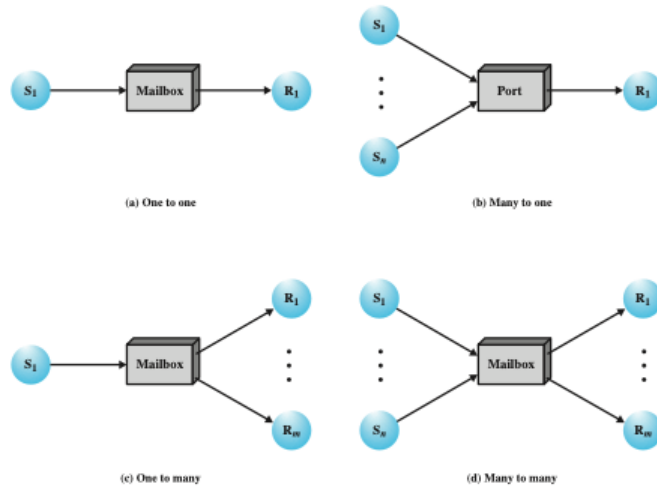
Mailbox

Una **mailbox** è un contenitore per messaggi identificato da un **nome univoco**, che deve essere specificato nelle chiamate `send()` e `receive()`.

La comunicazione tramite mailbox può avvenire secondo diverse modalità di **associazione tra mittente e destinatario**.

L'associazione può essere:

- **Statica**, quando la mailbox è collegata a un **identificatore fisso**, come un numero di porta, tipicamente in comunicazioni 1 a 1. La porta viene creata e assegnata in modo permanente al processo che la crea.
- **Dinamica**, quando più mittenti e destinatari si collegano e scollegano alla mailbox durante l'esecuzione, utilizzando primitive come `connect()` e `disconnect()`. Questa modalità è utile nei casi in cui è presente più di un mittente o più di un destinatario.

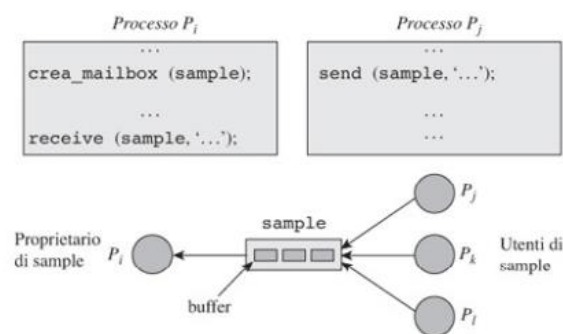


Sono possibili diversi schemi di comunicazione: 1 a 1, 1 a molti, molti a 1, e molti a molti, a seconda del numero di processi coinvolti nella comunicazione attraverso la stessa mailbox.

Creazione e utilizzo di una mailbox

La **creazione della mailbox** può essere gestita:

- Direttamente dal **processo ricevente**, in tal caso viene eliminata alla sua terminazione,
- oppure dal **kernel**, che la rende **indipendente dal ciclo di vita** dei processi, richiedendo una **connessione esplicita** alla mailbox prima del suo utilizzo, e **rimuoverla**, attraverso una **chiamata esplicita**, quando nessun processo vi è più connesso.



La figura mostra un esempio di uso di una **mailbox** chiamata `sample`. Il processo P_i (processo ricevente) la crea e riceve messaggi, mentre altri processi come P_j vi inviano dati tramite `send()`. Se il processo ricevente non ha ancora invocato `receive()`, il **kernel** conserva temporaneamente il messaggio in un **buffer**, che può essere **dedicato** alla mailbox o **condiviso** tra più mailbox.

Le mailbox permettono **alcuni vantaggi**, come:

- **Anonimato del destinatario**, poiché il mittente può inviare messaggi senza conoscere l'identità esatta del ricevente. Questo è particolarmente utile in scenari con **servizi dinamici** o **astrazione dei ruoli**.
- **Classificare i messaggi** in base al tipo, rendendo più semplice la loro gestione.
- **Trasferire funzioni** tra processi, come la gestione dello scheduler o della stampante, modificando il proprietario della relativa mailbox, grazie al **meccanismo di trasferimento di proprietà**, per una maggiore dinamicità e flessibilità.

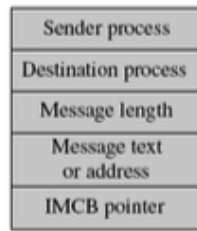
Quando un **processo mittente** si aspetta una risposta, può indicare la propria identità **includendola nel messaggio**, oppure lasciare che sia il **kernel** a gestirla.

In quest'ultimo caso, la chiamata `send()`, diventa **bloccante**, il processo resta in attesa finché non riceve la risposta, rendendo lo scambio **sincrono**.

Implementazione dello scambio di messaggi

Quando un processo P_i invia un messaggio a P_j utilizzando una **send non bloccante**, il kernel non può consegnarlo immediatamente se il destinatario non è pronto a riceverlo. In questi casi, il kernel costruisce una struttura dati

chiamata **IMCB (Interprocess Message Control Block)**, utilizzata per memorizzare tutte le informazioni necessarie allo scambio del messaggio in un momento successivo.



Interprocess message control block (IMCB).

L'IMCB include i dati fondamentali del messaggio, come l'**identificativo del mittente e del destinatario**, la **lunghezza del messaggio**, il **contenuto del messaggio** (oppure un riferimento alla sua posizione in memoria o su disco), e un campo **puntatore** che consente di organizzare più IMCB in una lista di messaggi in attesa.

Il messaggio viene quindi **bufferizzato**, cioè conservato temporaneamente dal kernel fino a quando il processo destinatario invoca `receive()`. A quel punto, il kernel recupera l'IMCB corrispondente e copia il contenuto del messaggio nello spazio di memoria del destinatario, completando così la consegna.

L'organizzazione interna degli IMCB può seguire due modalità:

- Nella **denominazione simmetrica** (indirizzamento diretto), esiste una **lista separata di IMCB per ogni coppia di processi comunicanti**; quindi, ogni mittente ha una lista distinta per ciascun destinatario. Quando un processo P_j esegue una `receive()`, è usata la lista di IMCB per la coppia P_i - P_j per consegnare il messaggio. Questo tipo di denominazione è utile quando i **processi comunicano in modo fisso e prevedibile**, per esempio in sistemi dove **ogni mittente ha un destinatario specifico**.

Ad esempio:

Se P_1 manda messaggi a P_2 , si crea una lista IMCB $P_1 \rightarrow P_2$

Se P_3 manda messaggi a P_2 , si crea un'altra lista IMCB $P_3 \rightarrow P_2$

Se P_1 manda messaggi a P_3 , si crea una terza lista IMCB $P_1 \rightarrow P_3$.

- Nella **denominazione asimmetrica** (indirizzamento indiretto), invece, **ogni processo ricevente mantiene una sola lista di messaggi**, indipendentemente dal mittente, e sarà il kernel a selezionare l'IMCB corretto quando `receive()` viene invocata. Questa soluzione è più **flessibile**, perché il ricevente **non deve sapere chi lo ha contattato** e può ricevere da **più mittenti in modo dinamico**. È utile in scenari tipo **server con molti client**.

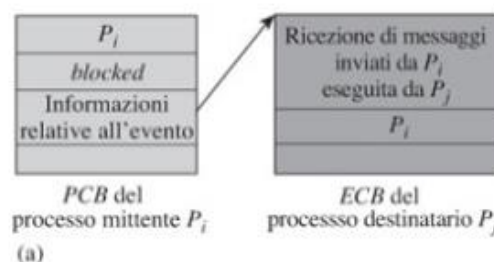
Ad esempio:

Il processo P_2 avrà una sola lista, e tutti i messaggi in arrivo (da P_1 , P_3 , P_4) finiranno lì. Quando P_2 esegue `receive()`, sarà il kernel a scegliere quale messaggio consegnargli dalla lista.

Nel caso in cui la `send()` sia **bloccante**, la gestione è più diretta. Il processo mittente resta fermo finché il destinatario non riceve il messaggio, e non è necessario copiare il messaggio in un buffer. Il kernel può semplicemente annotare un riferimento al messaggio già presente in memoria, evitando duplicazioni. Questo rende il sistema più efficiente, ma introduce il vincolo che il mittente non può proseguire finché la comunicazione non è completata.

Consegna dei messaggi tra processi

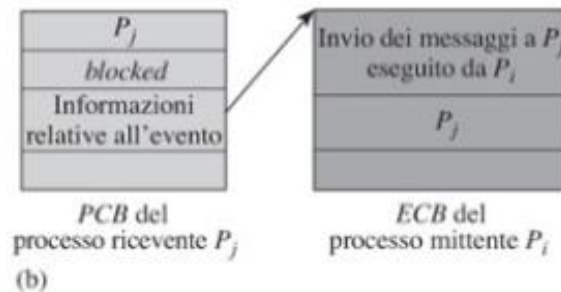
Si ricorda che l'ECB (Event Control Block) è una struttura usata dal kernel per tenere traccia di eventi che ancora non si sono verificati, ma che dovranno attivare un certo processo quando accadranno. In questo caso, l'evento è la consegna di un messaggio tra processi. Ogni ECB contiene la **descrizione dell'evento** (es. "ricezione di un messaggio da P_i a P_j), l'**identificativo del processo in attesa dell'evento**, un **puntatore per collegare più ECB in una lista**.



La figura in alto mostra cosa accade durante una **send() bloccante** e **naming simmetrico**.

Quando P_i invoca `send( $P_j$ , messaggio)`, il kernel verifica se P_j ha già chiamato `receive()` per ricevere da P_i . Se P_j è pronto, allora il kernel **consegna subito il messaggio**, **sblocca** P_j e lo rende ready per l'esecuzione, senza la necessità di creare un Evento.

Ma se P_j non ha ancora invocato `receive()`, allora il kernel non può completare subito la comunicazione. Quindi **crea un ECB** per rappresentare l'evento " P_j riceverà da P_i ". Poi inserisce nel PCB (Process Control Block) di P_i il riferimento a questo ECB. Infine, cambia lo stato di P_i in bloccato, perché deve aspettare che P_j chiami `receive()`.



La figura in alto mostra la situazione inversa, ovvero cosa accade se `receive()` viene invocata prima che P_i invochi `send()`.

Se invece è P_j a invocare `receive( $P_i$ )` prima che P_i faccia `send()`, allora la logica si inverte:

Il kernel **crea un ECB** per rappresentare l'evento " P_i invierà a P_j ". Quindi salva l'identificativo di P_j nell'ECB, così il kernel saprà chi sbloccare quando l'evento `send()` si verificherà. Infine, cambia lo stato di P_j in bloccato, in attesa che il messaggio arrivi.

Quindi, nel **primo caso** (figura a) se `send()` viene invocata prima di `receive()`, **si blocca il mittente** (P_i).

Nel **secondo caso** se `receive()` viene invocata prima di `send()`, **si blocca il destinatario** (P_j). In entrambi i casi, il kernel crea un ECB per rappresentare l'evento mancato. Quando l'evento si verifica, il Kernel usa l'ECB per riattivare il processo bloccato.

Strategie di Accodamento

La strategia di accodamento più semplice per la gestione dei messaggi è quella **FIFO** (First-In First Out), dove i messaggi vengono ricevuti nell'ordine in cui sono stati inviati. Tuttavia, questo approccio può risultare **inadeguato** nei casi in cui alcuni messaggi siano **più urgenti** di altri.

Per superare questa limitazione, esistono alcune **alternative più flessibili**. Una possibilità è quella di **assegnare una priorità ai messaggi**, basandosi sul loro tipo o su un'indicazione esplicita fornita dal mittente. Un'altra opzione consiste nel permettere al **destinatario di ispezionare la coda dei messaggi** e selezionare direttamente quale messaggio ricevere successivamente, invece di accettare automaticamente il primo disponibile.

Message Passing in UNIX

In sistema UNIX offre tre principali meccanismi per la **comunicazione tra processi**: **pipe**, **code dei messaggi** e **socket**. Tutti consentono lo scambio di dati tra processi senza che questi debbano conoscere direttamente le rispettive identità, e si differenziano per ambito di applicazione e modalità di utilizzo.

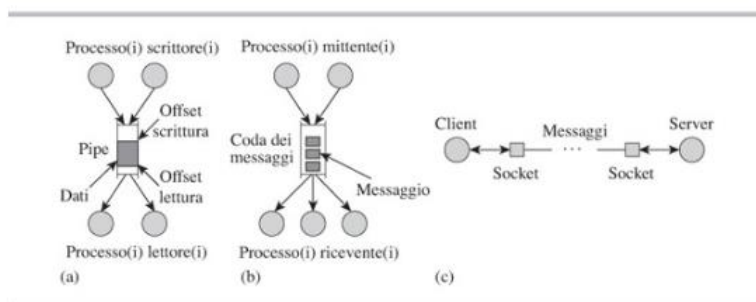


Figura 9.10 Comunicazione interprocesso in Unix: (a) pipe; (b) coda dei messaggi; (c) socket.

Pipe

La **pipe** è un meccanismo di tipo **FIFO** utilizzato per trasferire dati tra un processo **scrittore** e uno **lettore**. Esistono pipe **con nome** e **senza nome**. Le prime sono visibili nel file system e possono essere utilizzate da qualsiasi processo, secondo i permessi Unix; le seconde sono accessibili solo tra processi che fanno parte dello **stesso albero di processo** e vengono eliminate automaticamente quando non ci sono più processi collegati. Una pipe viene gestita dal kernel tramite **due offset**: uno per la scrittura e uno per la lettura. La scrittura avviene in coda, la lettura in testa, e se la pipe è piena o vuota, il processo scrittore o lettore viene messo in attesa. I dati scritti vengono letti una sola volta e poi rimossi.

Coda dei messaggi

La coda dei messaggi è simile a una **mailbox**: viene creata e posseduta da un processi, mentre altri possono inviare o ricevere messaggi secondo i **permessi di accesso definiti**. Ogni messaggio è composto da un **tipo** e da un **testo**, ed è gestito dal kernel attraverso una struttura con header e buffer. Le chiamate di sistema `msgsnd()` e `msgrcv()` gestiscono l'invio e la ricezione, consentendo anche di filtrare i messaggi in base al tipo. Se la coda è piena o non contiene il messaggio desiderato, i processi vengono **bloccati** finché le condizioni non cambiano. La gestione della coda avviene tramite la system call `msgget()`, e il kernel mantiene un array con tutte le code, accessibili tramite una **chiave**.

Socket

Il **socket** rappresenta un'estremità di un canale di comunicazione, ed è usato per realizzare **connessioni tra processi**, sia nel dominio **Unix locale** che in quello **Internet**. Nella comunicazione client-server, il **server** crea un socket e lo associa a un indirizzo, rendendolo visibile nel dominio. I **client** utilizzano questo indirizzo per connettersi, e una volta avvenuta la connessione, il kernel crea un nuovo socket dedicato per lo scambio dati. Il server può gestire più connessioni contemporaneamente, creando un nuovo processo per ciascuna. La comunicazione effettiva avviene tramite le chiamate `send()` e `receive()` oppure `read()` e `write()`. La connessione si chiude con `close()` o `shutdown()`.

Caratteristica	Pipe	Code dei Messaggi	Socket
Tipo di comunicazione	Flusso continuo di byte (FIFO)	Scambio di messaggi strutturati (tipo + testo)	Comunicazione orientata alla connessione (client/server)
Modalità	Unidirezionale (una pipe = una direzione)	Bidirezionale , ma gestita con più code	Bidirezionale , supporta dialoghi completi
Visibilità nel sistema	Pipe senza nome : solo processi imparentati Pipe con nome : visibili nel file system	Visibili e gestite dal kernel tramite chiavi e permessi	Ogni socket ha un indirizzo e può essere visibile nel dominio Unix o Internet
Persistenza	Le pipe con nome rimangono fino a <code>unlink</code> Le unnamed pipe scompaiono quando i processi terminano	Rimangono finché non eliminate dal proprietario	Il socket è attivo finché non viene chiuso o terminato
Permessi di accesso	Gestiti come file (solo per pipe con nome)	Simili ai permessi dei file (lettura/scrittura)	Basati sull'indirizzo del socket e sul dominio di visibilità
Gestione del kernel	Due offset per lettura e scrittura , gestiti nell'inode	Il kernel gestisce una lista di messaggi con header	Il kernel gestisce connessioni, accettazioni e canali dinamici
Blocking	Il processo può bloccarsi se la pipe è vuota o piena	Mittenti/receiver possono bloccarsi in base allo stato della coda	La connessione può bloccarsi in attesa del completamento
Uso tipico	Comunicazione rapida tra processi imparentati (es. padre-figlio)	Scambio strutturato e asincrono di dati tra processi dello stesso sistema	Comunicazione client-server tra processi anche su host diversi

Mutua Esclusione con Scambio di Messaggi (send non bloccante e receive bloccante)

```
int n /* numero di processi */
void P(int i)
{
    message msg;
    while (true) {
        receive (box, msg);
        /* sezione critica */;
        send (box, msg);
        /* resto del codice */;
    }
}

create mailbox (box);
send (box, null);
parbegin (P(1), P(2), ..., P(n))
```

Una forma di **interprocess communication** che consente anche **sincronizzazione** può essere utilizzata per implementare la **mutua esclusione**, sfruttando il comportamento delle primitive `send()` e `receive()`. In particolare, ipotizzando una **send non bloccante** e una **receive bloccante**, si può realizzare un meccanismo semplice ed efficace per garantire che un solo processo alla volta entri nella **sezione critica**.

Il sistema funziona creando inizialmente una **mailbox condivisa**, mediante `create_mailbox(box)`, e inserendo al suo interno un **messaggio iniziale nullo** con `send(box, null)`. Questo messaggio fungerà da **token**, ovvero da permesso per accedere alla sezione critica.

Vengono quindi avviati **N processi concorrenti**, ognuno dei quali esegue un ciclo in cui, per prima cosa, invoca `receive(box, msg)`. Poiché la `receive` è **bloccante**, se non ci sono messaggi nella mailbox, il processo resta in attesa. Solo il processo che riesce a ricevere il token potrà procedere e accedere alla **sezione critica**.

Una volta completata la sezione critica, il processo **rilascia il token** eseguendo `send(box, msg)`. A quel punto, uno degli altri processi eventualmente bloccati sulla `receive()` verrà **sbloccato** e potrà a sua volta entrare in sezione critica.

In questo schema, il **token viaggia** da processo a processo tramite **l'invio e la ricezione del messaggio**. La mutua esclusione è garantita dal fatto che **può esserci al massimo un solo token alla volta nella mailbox**; quindi, **un solo processo alla volta può accedere alla sezione critica**. Il sistema si basa sulla **sincronizzazione implicita** offerta da `receive()` e `send()`, senza necessità di lock o semafori.

Produttore-Consumatore con Scambio di Messaggi

```
int dimensione = /* capacità del buffer */
null = /* messaggio vuoto */ ;

void produttore() {
    message pmsg;
    while (true) {
        receive (possoProdurre, pmsg);

        pmsg = produci();
        send (possoConsumare, pmsg);
    }
}

void consumatore() {
    message cmsg;
    while (true) {
        receive (possoConsumare, cmsg);
        consuma (cmsg);
        send (possoProdurre, null);
    }
}

create_mailbox (possoProdurre);
create_mailbox (possoConsumare);

for (int i = 1; i <= dimensione; i++)
    send (possoProdurre, null);
parbegin (producer, consumer)
```

Nel problema classico del **produttore-consumatore** con un **buffer di capacità N**, è possibile implementare la sincronizzazione usando **message passing** e **due mailbox**: `possoProdurre` e `possoConsumare`.

Nel `main`, si inizializzano le due mailbox con le chiamate `create_mailbox(possoProdurre)` e `create_mailbox(possoConsumare)`. La variabile `dimensione` rappresenta la **capacità del buffer**, ovvero il numero massimo di elementi che possono essere prodotti prima che il produttore venga bloccato.

Per abilitare inizialmente i produttori alla produzione, si inviano **N messaggi vuoti** alla mailbox `possoProdurre`, ciascuno dei quali rappresenta **uno spazio disponibile nel buffer**. Questo avviene tramite un ciclo `for` che esegue `send(possoProdurre, null)` per `i` da 1 a `dimensione`.

Dopo l'inizializzazione, si avviano in parallelo i processi `produttore()` e `consumatore()` tramite `parbegin`.

Nel ciclo di vita del **produttore**, questo esegue `receive(possoProdurre, pmsg)` per attendere che ci sia spazio nel buffer. Se il buffer è pieno, non ci sono messaggi in `possoProdurre`, quindi il produttore si **blocca**. Quando riceve un messaggio (anche nullo), produce un elemento e lo invia al consumatore tramite `send(possoConsumare, pmsg)`.

Il **consumatore**, invece, chiama `receive(possoConsumare, cmsg)`. Se non ci sono elementi prodotti, si **blocca** fino a che il produttore non inserisce un messaggio. Quando riceve un messaggio, lo consuma e infine invia un messaggio vuoto a `possoProdurre` tramite `send(possoProdurre, null)`, segnalando che è stato liberato uno slot nel buffer.

In questo schema, i **messaggi fungono da risorsa** per controllare l'accesso al buffer: `possoProdurre` tiene traccia degli **slot liberi**, mentre `possoConsumare` tiene traccia degli **elementi disponibili**.

Capitolo 7 – Scheduling

Introduzione

Lo **scheduling** è l'attività che consiste nel selezionare quale richiesta servire successivamente da parte di un server. In un **SO**, il server principale è la **CPU**, mentre le richieste corrispondono ai **job** o ai **processi**. Lo scheduling influisce in modo significativo sulle **prestazioni del SO**, e ha l'obiettivo di bilanciare **efficienza del sistema** e **qualità del servizio per l'utente**.

Le principali operazioni dello scheduler sono **ammissione**, **schedulazione** e **prelazione**.

L'**ammissione** consiste nell'inserire in una coda le richieste (processi o job) che l'utente ha sottomesso. Nei sistemi con risorse limitate, le richieste vengono inizialmente collocate in una coda separata, e solo successivamente ammesse allo scheduling vero e proprio.

La **schedulazione** è l'operazione che seleziona una richiesta tra quelle ammesse, decidendo quale processo sarà **assegnato alla CPU** secondo una **politica di scheduling** definita dal sistema.

La **prelazione** avviene quando un processo viene **interrotto prima del termine della sua esecuzione**, ad esempio perché deve attendere una risorsa momentaneamente non disponibile. In questo caso, il processo viene reinserito nella coda delle richieste ammesse per poter riprendere in seguito.

Terminologia legata allo scheduling

Riguardo alla richiesta

- **Tempo di arrivo**: istante in cui l'utente invia la richiesta al sistema.
- **Tempo di ammissione**: istante in cui il SO inizia a considerare la richiesta per lo scheduling.
- **Tempo di completamento**: istante in cui l'esecuzione della richiesta termina.
- **Deadline**: termine entro il quale la richiesta deve essere completata, fondamentale nei sistemi real-time.
- **Tempo di servizio**: tempo totale di CPU e I/O richiesto per completare la richiesta.
- **Prelazione**: deallocazione forzata di un processo dalla CPU, senza che esso abbia terminato.
- **Priorità**: criterio che il sistema utilizza per decidere quale richiesta schedulare prima tra quelle in attesa.

Riguardo al servizio per l'utente

- **Deadline overrun**: differenza tra il tempo effettivo di completamento e la deadline. Può essere positivo (ritardo) o negativo (anticipo).
- **Condivisione equa**: misura in cui il tempo CPU è distribuito in modo bilanciato tra processi o gruppi.
- **Response ratio**: calcolato come (tempo di attesa + tempo di servizio) / tempo di servizio. Valori prossimi a 1 indicano risposta rapida.
- **Tempo di risposta**: intervallo tra la sottomissione di una sotto-richiesta e il completamento della sua elaborazione, esclusi tempi di attesa dovuti a prelazione o I/O.
- **Tempo medio di risposta**: media dei tempi di risposta di tutte le sotto-richieste.
- **Turnaround time** (tempo di completamento): tempo totale tra sottomissione e completamento, includendo tempi di attesa, I/O e prelazione.
- **Turnaround pesato**: rapporto tra turnaround time e tempo di servizio, utile per confrontare efficienza relativa.
- **Tempo medio di turnaround**: media dei turnaround time per tutte le richieste completate.

Riguardo alle prestazioni complessive del sistema

- **Durata della schedulazione**: Tempo necessario per completare un insieme di richieste.
- **Throughput**: numero medio di richieste completate nell'unità di tempo.

Tecniche di Scheduling

Scheduling basato su priorità

In questa tecnica, la CPU viene assegnata al processo con la **priorità più alta**. Nei SO, i processi **I/O bound** hanno generalmente priorità più alta rispetto a quelli **CPU-bound**, poiché il loro comportamento favorisce un **elevato throughput** del sistema (rilasciano la CPU rapidamente e attendono operazioni di I/O).

La priorità può essere **statica**, quando rimane invariata durante l'esecuzione, oppure **dinamica**, quando varia in base a parametri come **tempo di attesa**, **utilizzo della CPU** o **numero di richieste servite**.

Uno svantaggio di questo approccio è la possibile **starvation** dei processi a bassa priorità, che rischiano di non essere mai eseguiti. Per mitigare questo problema si applica una tecnica chiamata **aging**, che consiste nell'aumentare progressivamente la priorità dei processi in attesa da molto tempo, garantendo così equità nel lungo periodo.

Riordino delle richieste

Questa tecnica prevede l'esecuzione delle richieste **in un ordine diverso** da quello di arrivo. Il riordino può essere basato su criteri come la **priorità** o il **tempo stimato di esecuzione**. Ad esempio, è possibile anticipare i processi **più brevi**, migliorando così il **tempo medio di risposta** e il **throughput** del sistema. Il riordino è spesso utilizzato nei sistemi **time-sharing** o nei contesti con **prelazione**, dove può contribuire a ottimizzare la distribuzione delle risorse. Inoltre, può influire sulla **ridefinizione dinamica delle priorità** in base al nuovo ordine.

Variazione del time slice

Il **time slice**, ovvero il tempo di esecuzione assegnato a ciascun processo può essere **variato dinamicamente** per migliorare l'efficienza dello scheduling.

L'efficienza della CPU può essere espressa dalla formula $\eta = \frac{\delta}{\delta + \sigma}$ dove

- η è l'efficienza della cpu,
- δ è la durata del time slice,
- σ è l'overhead introdotto dal cambio di contesto.

Time slice più brevi migliorano il **tempo di risposta**, ma aumentano l'**overhead** dovuto alle frequenti commutazioni di processo. Per bilanciare prestazioni e servizio, si può adottare una politica adattiva: ai processi **I/O-bound** si assegna un time slice **più breve**, poiché tendono a rilasciare la CPU rapidamente, mentre ai processi **CPU-bound** si assegna un time slice **più lungo**, per ridurre le interruzioni e migliorare l'utilizzo della CPU.

Politiche di scheduling senza prelazione (non preemptive)

Nello **scheduling senza prelazione**, un processo che riceve la CPU continua la sua esecuzione **fino al completamento**, senza poter essere interrotto da altri processi. In questo modello, **nessuna richiesta può essere sospesa** per fare spazio a un'altra più urgente: una volta che un processo è stato schedulato, mantiene il controllo della CPU fino al termine.

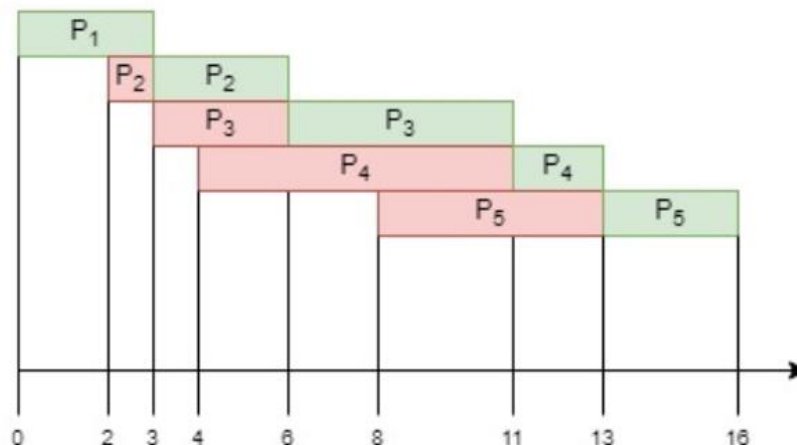
Lo **scheduler** ha quindi il solo compito di **decidere l'ordine di esecuzione dei processi in attesa**, poiché non può intervenire su quelli già in esecuzione. Il suo intervento si limita a riordinare le richieste in modo da **ottimizzare l'efficienza del sistema** o **ridurre i tempi di attesa** per l'utente. Se una nuova richiesta arriva mentre la CPU è occupata, **dovrà attendere** in coda fino a che la CPU non si libera.

Scheduling First-Come, First-Served (FCFS)

Nello scheduling **First-Come, First-Served**, le richieste vengono inserite in una **coda FIFO** (First-In, First-Out) e vengono schedulate **nell'ordine esatto di arrivo**. Questo significa che la **prima richiesta che arriva è anche la prima a essere eseguita**.

Processo	P_1	P_2	P_3	P_4	P_5
Tempo di ammissione	0	2	3	4	8
Tempo di servizio	3	3	5	2	3

Tempo	Processo completato			Processi nel sistema (in ordine FCFS)	Processo schedulato
	<i>id</i>	<i>ta</i>	<i>w</i>		
0	—	—	—	P_1	P_1
3	P_1	3	1.00	P_2, P_3	P_2
6	P_2	4	1.33	P_3, P_4	P_3
11	P_3	8	1.60	P_4, P_5	P_4
13	P_4	9	4.50	P_5	P_5
16	P_5	8	2.67	—	—

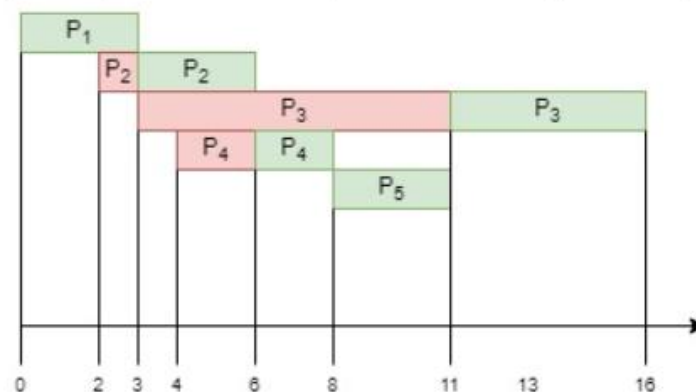


Scheduling Shortest Job First (SJF)

Nel **Shortest Job First (SJF)** lo scheduler seleziona sempre la richiesta con il **minor tempo di servizio**, cioè il processo che può essere completato più rapidamente. Sebbene questo approccio sia efficiente in termini di **tempo medio di completamento**, può causare **starvation** dei processi che richiedono **tempi di esecuzione più lunghi**.

Processo	P_1	P_2	P_3	P_4	P_5
Tempo di ammissione	0	2	3	4	8
Tempo di servizio	3	3	5	2	3

Tempo	Processo completato			Processi nel sistema (in ordine FCFS)	Processo schedulato
	<i>id</i>	<i>ta</i>	<i>w</i>		
0	—	—	—	$\{P_1\}$	P_1
3	P_1	3	1.00	$\{P_2, P_3\}$	P_2
6	P_2	4	1.33	$\{P_3, P_4\}$	P_4
8	P_4	4	2.00	$\{P_3, P_5\}$	P_5
11	P_5	3	1.00	$\{P_3\}$	P_3
16	P_3	13	2.60	$\{\}$	—



Nel diagramma mostrato, si osserva che il processo **P3**, pur essendo arrivato come **terzo**, viene **schedulato per ultimo**. Questo avviene perché il suo tempo di servizio (equivalente a 5) è maggiore rispetto a quello degli altri processi già presenti o arrivati successivamente. In particolare, prima che **P2** termini, arriva **P4**, che ha un tempo di servizio minore; a sua volta, mentre **P4** è in esecuzione, arriva anche **P5**, che anch'esso ha un tempo più breve di **P3**.

Di conseguenza, **P3 viene continuamente posticipato**, venendo **scavalcato** da altri processi con tempi di servizio inferiori. Questo è un esempio tipico di **starvation**, che può essere mitigata introducendo una tecnica detta "**aging**", la quale aumenta **gradualmente la priorità** di un processo in attesa in funzione del tempo trascorso nella coda, evitando così che venga ignorato indefinitamente.

Scheduling Highest Response Ratio Next (HRN)

Lo scheduling **HRN (Highest Response Ratio Next)** seleziona il processo con il **rapporto di risposta più alto**, calcolato secondo la seguente formula:

$$\frac{(\text{tempo di attesa} + \text{tempo di servizio})}{\text{tempo di servizio}}$$

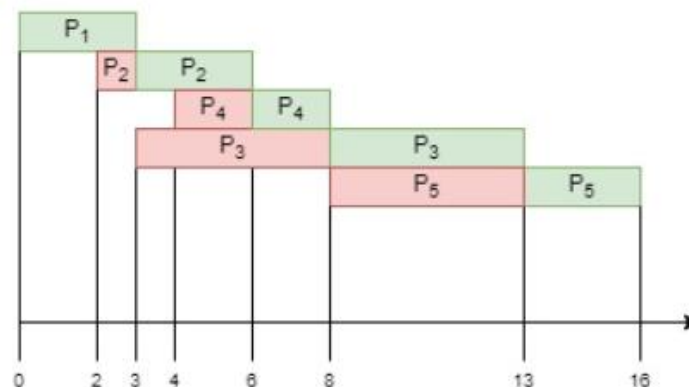
Questo rapporto tiene conto sia del **tempo di attesa** accumulato dal processo, sia del **tempo di esecuzione richiesto**, bilanciando quindi efficienza e giustizia.

Poiché un processo appena arrivato ha tempo di attesa pari a 0, il suo **response ratio** iniziale è 1. Invece, un processo che attende da più tempo vedrà **aumentare il numeratore** della formula, ottenendo così un valore di response ratio più elevato e quindi maggiore probabilità di essere selezionato. Inoltre, a parità di tempo di attesa, viene favorito il processo con **tempo di servizio minore**, poiché ha un **denominatore più piccolo**, e quindi un rapporto maggiore.

Questa politica **evita la starvation** grazie a un meccanismo implicito di **aging**: più un processo resta in attesa, più cresce il suo response ratio, fino a diventare il prossimo ad essere schedulato.

Processo	P_1	P_2	P_3	P_4	P_5
Tempo di ammissione	0	2	3	4	8
Tempo di servizio	3	3	5	2	3

Tempo	Processo completato			Response ratio dei processi					Processo schedulato
	id	ta	vr	P_1	P_2	P_3	P_4	P_5	
0	–	–	–	1.00					P_1
3	P_1	3	1.00		1.33	1.00			P_2
6	P_2	4	1.33			1.60	2.00		P_4
8	P_4	4	2.00			2.00		1.00	P_3
13	P_3	10	2.00					2.67	P_5
16	P_5	8	2.67						–



Nel diagramma, si nota come il processo **P4**, pur arrivando dopo **P3**, venga schedulato prima grazie al suo **tempo di servizio inferiore**, che gli conferisce un rapporto di risposta più alto. In seguito, **P3** viene selezionato nonostante il tempo di servizio più lungo, poiché il **tempo di attesa accumulato** ha aumentato il suo response ratio, evitando che venga ignorato.

Esempio: Momento del confronto all'istante 6.

Calcolo per P3:

$$\frac{(\text{tempo di attesa} + \text{tempo di servizio})}{\text{tempo di servizio}} = \frac{(6 - 3) + (5)}{5} = \frac{3 + 5}{5} = \frac{8}{5} = 1,6$$

Calcolo per P4:

$$\frac{(\text{tempo di attesa} + \text{tempo di servizio})}{\text{tempo di servizio}} = \frac{(6 - 4) + (2)}{2} = \frac{2 + 2}{2} = \frac{4}{2} = 2$$

P4 ha response ratio maggiore quindi viene schedulato prima di P3.

Politiche di Scheduling con prelazione (preemptive)

Nello **scheduling con prelazione**, lo scheduler ha la possibilità di **interrompere l'esecuzione di un processo anche se non è stato completato**, per assegnare la CPU a un altro processo ritenuto più opportuno (ad esempio con priorità più alta o tempo rimanente inferiore).

Il processo interrotto viene **reinserito nella coda dei processi pronti** (ready queue) e potrà essere nuovamente schedulato in seguito.

Questo approccio consente di ottenere un **servizio utente più reattivo**, ma introduce anche un **maggiore overhead** a causa delle **frequenti commutazioni di contesto** (context switch), che consumano tempo e risorse del sistema.

Round – Robin con time-slicing (RR)

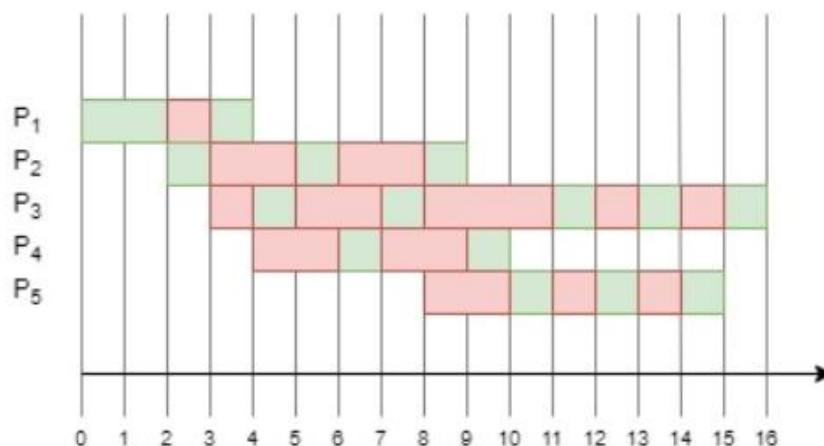
Il **Round Robin** è una politica di **scheduling preemptive** che assegna a ciascun processo una **quantità fissa di tempo**, detta **time slice** (σ), durante la quale può utilizzare la CPU. Se un processo non termina entro il proprio time slice, viene **sospeso (prelazonato)** e **reinserito in fondo alla coda dei processi ready**.

Nel momento in cui un nuovo processo arriva mentre un altro viene prelazonato, il nuovo processo può essere **inserito nella coda prima** del processo appena sospeso.

Processo	P_1	P_2	P_3	P_4	P_5
Tempo di ammissione	0	2	3	4	8
Tempo di servizio	3	3	5	2	3

Tempo di schedulazione	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	c	ta	w
Posizione dei processi nella ready queue (1 indica la testa della coda)	P_1	1	1	2	1												4	4	1.33
	P_2			1	3	2	1	3	2	1							9	7	2.33
	P_3				2	1	3	2	1	4	3	2	1	2	1	2	16	13	2.60
	P_4					3	2	1	3	2	1						10	6	3.00
	P_5									3	2	1	2	1		1	15	7	2.33
Processo schedulato	P_1	P_1	P_2	P_1	P_3	P_2	P_4	P_3	P_2	P_4	P_5	P_3	P_5	P_3	P_5	P_3			

$\bar{t}_0 = 7.4$ secondi, $\bar{w} = 2.32$ c : tempo di completamento di un processo



Questo si osserva chiaramente nel caso in cui **P3** (arrivato al tempo 3) viene posizionato prima di **P2**, che era stato appena prelazonato.

Come si nota dalla tabella e dal diagramma Gantt, la CPU viene assegnata ciclicamente ai processi **nell'ordine della ready queue**, e ciascun processo riceve un **time slice di 1 unità di tempo**. I processi sono gestiti in modo equo e ciclico, il che garantisce una buona reattività ma può comportare un **elevato overhead** dovuto alla frequente **commutazione di contesto** (context switch), soprattutto con molti processi attivi.

Scheduling Least Completed Next (LCN)

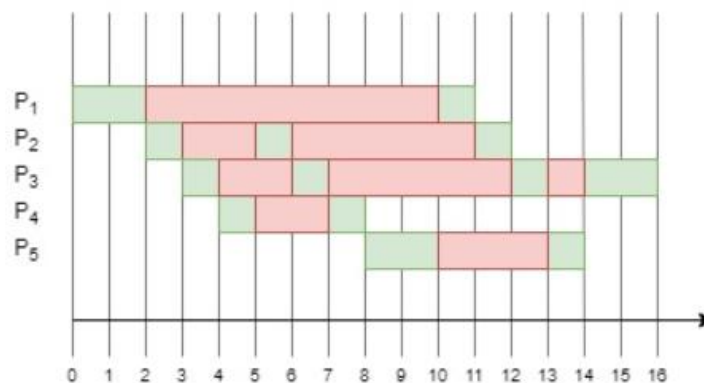
Lo **scheduling LCN (Least Completed Next)** assegna la CPU al processo che ha **utilizzato meno tempo di esecuzione fino a quel momento**. Questo criterio favorisce i processi che **sono stati meno serviti dalla CPU**, in particolare quelli appena arrivati, che **non hanno ancora utilizzato tempo di CPU**.

Tuttavia, questa strategia porta a una forte **starvation** per i processi più lunghi o quelli che, pur avendo già iniziato l'esecuzione, non sono riusciti a completarsi velocemente.

Processo	P_1	P_2	P_3	P_4	P_5
Tempo di ammissione	0	2	3	4	8
Tempo di servizio	3	3	5	2	3

Tempo di schedulazione	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	c	ta	w
P_1	0	1	2	2	2	2	2	2	2	2	2						11	11	3.67
P_2			0	1	1	1	2	2	2	2	2	2					12	10	3.33
P_3				0	1	1	1	2	2	2	2	2	2	3	4	5	16	13	2.60
P_4					0	1	1	1									8	4	2.00
P_5									0	1	2	2	2	2			14	6	2.00
Processo schedulato	P_1	P_1	P_2	P_3	P_4	P_2	P_3	P_4	P_5	P_5	P_1	P_2	P_3	P_5	P_3	P_3			

$\bar{t}_q = 8.8$ secondi, $\bar{w} = 2.72$ c : tempo di completamento di un processo



Nel diagramma, si osserva ad esempio il caso del processo **P1**, che inizia la sua esecuzione ma viene **interrotto dopo due unità di tempo**. In seguito, la CPU continua a essere assegnata a nuovi processi appena arrivati, che hanno completamento minore. Questo comporta che **P1 venga ripreso solo molto più tardi**, allungando il suo tempo di attesa in maniera significativa.

Il **fenomeno di starvation è evidente**, poiché LCN privilegia sempre i processi con minor utilizzo di CPU, anche se ciò penalizza quelli già in corso. Non è previsto alcun meccanismo di aging che riequilibri questa tendenza, rendendo questa politica **poco equa nei sistemi generali**.

Scheduling Shortest Time to Go (STG)

Lo scheduling **STG (Shortest Time to GO)** rappresenta la **versione con prelazione dello SJF**. La politica prevede che la CPU venga sempre assegnata al processo che ha il **minor tempo di esecuzione rimanente**, ovvero a quello che **sta per terminare**.

A ogni nuova schedulazione (o arrivo di un nuovo processo), lo scheduler confronta il **tempo rimanente** dei processi in esecuzione e pronti. Se un nuovo processo ha un tempo rimanente inferiore a quello del processo attualmente in esecuzione, quest'ultimo viene **prelazionato** e il nuovo processo prende il controllo della CPU.

Questa politica **non favorisce automaticamente i processi appena arrivati**, a meno che non abbiano un tempo rimanente inferiore rispetto a quelli già presenti.

Processo	P_1	P_2	P_3	P_4	P_5
Tempo di ammissione	0	2	3	4	8
Tempo di servizio	3	3	5	2	3

Tempo di schedulazione	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	c	ta	w
Richiesta di tempo di CPU rimanente di un processo	P_1	3	2	1													3	3	1.00
	P_2			3	3	2	2	2	1								8	6	2.00
	P_3				5	5	5	5	5	5	5	5	4	3	2	1	16	13	2.60
	P_4					2	1										6	2	1.00
	P_5								3	2	1						11	3	1.00
Processo schedulato	P_1	P_1	P_1	P_2	P_4	P_4	P_2	P_2	P_5	P_5	P_5	P_5	P_5	P_5	P_5	P_5			

$\bar{t}_a = 5.4$ secondi, $\bar{w} = 1.52$ c : tempo di completamento di un processo



Nel diagramma si osserva come questo approccio tenda a **schedulare prima i processi brevi o prossimi alla conclusione**, migliorando il tempo medio di completamento e garantendo un **elevato throughput**.

Tipi di Scheduler

In un sistema operativo esistono tre tipi principali di **scheduler**:

- **Scheduler a lungo termine** (long-term)
- **Scheduler a medio termine** (medium-term)
- **Scheduler a breve termine** (short-term)

Scheduler a lungo termine

Si occupa di **ammettere i processi** nel sistema. Decide quali e quanti processi possono entrare nella coda dei processi pronti (ready), controllando il **grado di multiprogrammazione**. È particolarmente importante nei sistemi con **risorse limitate** e nei sistemi **real-time**, dove è necessario mantenere l'equilibrio tra carico e risposta alle scadenze.

Scheduler a medio termine

Gestisce lo **swap-in** e **swap-out** dei processi, ovvero sposta temporaneamente in memoria secondaria i processi che non possono essere eseguiti subito, liberando spazio nella memoria principale. Successivamente, quando le risorse diventano disponibili, i processi vengono ripristinati. È utile per gestire il carico e mantenere efficiente l'utilizzo della memoria.

Scheduler a breve termine

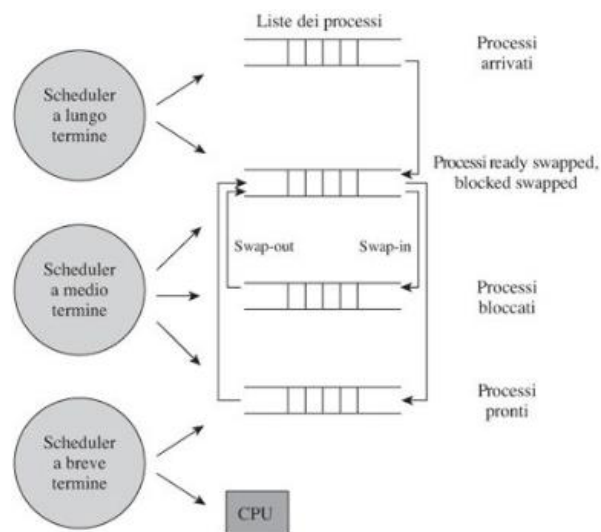
Seleziona i **processi dalla coda ready** per essere eseguiti dalla CPU. È anche detto **CPU scheduler**, ed è il più frequente e reattivo tra i tre.

Lavora in collaborazione con il **dispatcher**, che si occupa della **commutazione di contesto** (context switch).

Può anche decidere, in sistemi con **prelazione**, per quanto tempo un processo deve essere eseguito (ad esempio nei sistemi **time-sharing**).

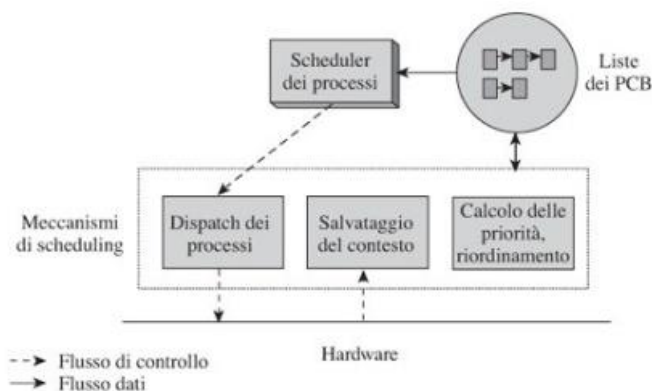
Quando non ci sono processi nella coda ready, il sistema esegue un **idle loop**, in attesa che arrivi un nuovo processo.

Esempio di flusso in un sistema time-sharing



1. I **processi sottomessi** vengono gestiti dallo scheduler a lungo termine, che li inserisce in una delle code (ready swapped o blocked swapped).
2. Lo **scheduler a medio termine** applica la sua politica di **swap-in** o **swap-out** dei processi in base alle risorse disponibili.
3. Infine, lo **scheduler a breve termine** seleziona i processi pronti per l'esecuzione e li invia al **dispatcher**, che li fa girare sulla CPU.

Funzionamento dello Scheduler a Breve Termine



Lo **scheduler a breve termine** lavora su **liste di PCB** (Process Control Block), la cui organizzazione dipende dalla **politica di scheduling** adottata.

Il suo compito è selezionare un processo tra quelli in stato ready e passare il suo **ID** al **dispatcher**.

NB: Se non ci sono processi ready il kernel esegue un loop idle (attesa attiva).

Il **dispatcher** si occupa della **commutazione del processo**, ovvero:

- Salva il **contesto** del processo in uscita,
- Carica i **registri PSW e GPR** (Program Status Word e General Purpose Registers) del processo in ingresso,
- Aggiorna lo **stato del processo**,
- E si **interfaccia** sia con lo scheduler (per ottenere l'ID del processo) sia con l'**hardware** (per il caricamento nei registri della CPU).

Una volta che il processo è stato selezionato e caricato, lo **scheduler ricalcola le priorità** delle richieste in lista, se necessario, e **riordina i PCB**. Questo ricalcolo può avvenire **periodicamente** oppure **su richiesta**, a seconda dell'algoritmo utilizzato.

Scheduling basato su priorità

Lo **scheduling basato su priorità** utilizza una **lista di code** per i processi in stato ready, in cui ogni coda corrisponde a un determinato **livello di priorità**.

Ogni **PCB** punta alla propria coda, e ogni coda è organizzata come una **lista concatenata**, dove il primo processo punta al successivo. Lo **scheduler** scorre le teste delle code, partendo da quella con **priorità più alta**, e seleziona il **primo processo disponibile** nella prima coda non vuota. In questo modo, il tempo richiesto per la selezione dipende dal **numero di livelli di priorità** e dalla **distribuzione dei processi**.

Problemi comuni

L'uso delle priorità può causare **starvation**, ovvero la condizione in cui un processo a bassa priorità **attende indefinitamente** perché viene continuamente prevaricato da processi a priorità più alta.

Per mitigare questo problema, si può applicare l'**aging**, una tecnica che **incrementa gradualmente la priorità** di un processo in base al tempo di attesa. Questo garantisce che, col passare del tempo, anche i processi inizialmente a bassa priorità abbiano la possibilità di essere schedulati.

Un altro fenomeno critico è l'**inversione di priorità**, che si verifica quando un processo ad alta priorità è **bloccato** da un processo a bassa priorità che detiene una risorsa condivisa. In questo caso, il processo a bassa priorità **impedisce** l'avanzamento di quello ad alta priorità, causando un **blocco indiretto**.

Per risolvere questo problema, si può applicare una tecnica detta **priority inheritance**: il processo a bassa priorità riceve temporaneamente una **priorità più alta**, esegue, rilascia la risorsa, e poi torna alla sua priorità originale. In questo modo, il processo ad alta priorità può proseguire appena la risorsa viene liberata.

Scheduling Round-Robin con Time Slice

Lo **scheduling Round-Robin (RR)** assegna a ciascun processo un **time slice**, cioè un intervallo fisso di tempo per l'uso della CPU. I processi ready sono gestiti tramite una **coda FIFO**. Lo scheduler seleziona il primo processo dalla coda e lo passa al **dispatcher** per l'esecuzione.

Se il processo **non termina** entro il time slice, viene **prelazionato** e **reinserito in fondo alla coda**.

Se invece richiede un'operazione di **I/O**, viene **sospeso (blocked)** e **rimosso temporaneamente** dalla coda ready; vi tornerà, sempre in fondo, solo dopo il completamento dell'I/O.

Se il processo **termina**, il suo **PCB viene rimosso definitivamente**.

Scheduling Multilivello

Lo **scheduling multilivello** combina lo **scheduling basato su priorità** con lo **scheduling Round-Robin**, al fine di bilanciare **prestazioni del sistema** e **tempi di risposta**.

A ogni **livello di priorità** è associata una propria **coda di processi ready** e un **time slice** specifico:

- I processi con **priorità alta** hanno **time slice brevi**, favorendo l'interattività,
- mentre quelli con **priorità bassa** hanno **time slice più lunghi**, riducendo l'overhead da commutazione di contesto.

Questo modello prevede una **classificazione statica** dei processi:

- i processi **I/O-bound** sono associati a **priorità alte**,
- i processi **CPU-bound** a **priorità basse**.

Tuttavia, questa assegnazione può rivelarsi **non sempre corretta**, perché un processo può cambiare **comportamento nel tempo**. Inoltre, l'uso di priorità fisse può generare **starvation** per i processi a bassa priorità e **non consente** l'applicazione della **tecnica dell'inversione di priorità**.

Scheduling Multilivello Adattivo

Lo **scheduling multilivello adattivo** risolve i limiti del modello statico, consentendo di **modificare dinamicamente la priorità dei processi**.

In base al comportamento osservato (se il processo è più **CPU-bound** o **I/O-bound**), il sistema adatta la **priorità** e assegna un **time slice** appropriato.

Grazie a questa **dinamicità**, è possibile gestire in modo più efficace l'allocazione della CPU e **applicare tecniche come l'inversione di priorità**, migliorando la reattività del sistema e prevenendo fenomeni di starvation.

Scheduling Multi-Level Feedback Queue (MLFQ)

Lo **scheduling MLFQ** (Multilevel Feedback Queue) è un algoritmo sofisticato di gestione della CPU, progettato per migliorare le prestazioni complessive del sistema in ambienti multitasking. La sua forza risiede nella capacità di **apprendere dal comportamento passato dei processi per fare previsioni sul futuro**, adattando dinamicamente le priorità per garantire **reattività, efficienza ed equa distribuzione** delle risorse.

L'MLFQ ha come principali obiettivi **l'ottimizzazione del tempo di turnaround**, dando priorità ai processi brevi, e la **minimizzazione dei tempi di risposta**, anche in assenza di una conoscenza preventiva sulla durata effettiva dei job.

Struttura delle code e regole operative

Il sistema è composto da **più code**, ciascuna associata a un **livello di priorità**. Un processo pronto all'esecuzione si trova sempre in **una sola coda** e viene selezionato, per l'esecuzione, tra quelli presenti nella coda a **priorità massima**.

Se più processi condividono la stessa coda, viene utilizzato lo **scheduling Round Robin (RR)** per gestirli equamente. Le regole fondamentali sono:

- Se **Priorità(A) > Priorità(B)**, il processo A viene eseguito.
- Se **Priorità(A) = Priorità(B)**, i processi A e B vengono gestiti tramite RR.

Adattamento dinamico: il cuore dell'MLFQ

Una delle caratteristiche distintive dell'MLFQ è la sua capacità di **modificare la priorità dei processi in base al comportamento osservato**:

- Un processo che **cede frequentemente la CPU** (tipico dei processi I/O-bound) **mantiene la sua priorità**.
- Un processo che **usa intensamente la CPU per lunghi periodi** (CPU-bound) **subisce un abbassamento di priorità**, scendendo nelle code inferiori.

Questa logica permette al sistema di privilegiare i processi interattivi, mantenendoli reattivi, e al tempo stesso spingere verso il basso quelli meno urgenti.

Esempio visivo

In uno scenario con code dalla Q8 (massima priorità) alla Q1 (minima), i processi A e B si trovano nella parte alta, essendo interattivi, mentre processi più pesanti come D si trovano in code inferiori. I processi migrano tra le code in base al loro comportamento: chi consuma troppa CPU scende, chi si comporta "bene" (rilasciando la CPU in anticipo) resta nella stessa posizione.

Meccanismo di modifica della priorità

L'algoritmo MLFQ implementa tre regole fondamentali:

- **Regola 3:** un nuovo processo viene sempre inserito nella coda con **priorità massima**.
- **Regola 4a:** se un processo consuma **interamente il proprio time slice**, la sua priorità viene **abbassata**.
- **Regola 4b:** se invece **rilascia la CPU prima del termine** del time slice, **mantiene la priorità attuale**.

Questo schema consente al sistema di **approssimare il comportamento dello Shortest Job First (SJF)**, favorendo i processi brevi senza conoscere a priori la loro durata.

Comportamenti tipici e interazioni

Un processo a **lunga esecuzione**, come mostrato in uno degli esempi, viene progressivamente degradato da Q2 a Q0 man mano che continua a occupare la CPU. Se durante la sua esecuzione arriva un processo **interattivo e breve** (es. 20ms di servizio), questo viene subito eseguito, dimostrando la reattività dell'MLFQ.

Quando nel sistema coesistono processi **CPU-bound** e **I/O bound** (es. un processo che occupa la CPU per 1ms e poi fa I/O), quest'ultimo viene **mantenuto costantemente nelle code alte**, assicurando una bassa latenza di risposta.

Problemi e criticità dell'MLFQ base

Nonostante i vantaggi, l'MLFQ base presenta **alcuni problemi**:

- **Starvation**: i processi a lunga durata rischiano di non ricevere mai tempo di CPU se il sistema è saturo di processi interattivi.
- **Manipolazione dello scheduler**: un processo può comportarsi in modo strategico, cedendo la CPU poco prima della fine del time slice, per **restare artificialmente nelle code alte**.
- **Evoluzione del comportamento**: un processo può **cambiare natura nel tempo**, passando da CPU-bound a I/O-bound, ingannando le logiche di feedback.

Strategie di correzione

Per contrastare la manipolazione, è stata proposta una **nuova versione della Regola 4**: una volta che un processo **usa tutto il suo allotment a un certo livello** (cioè un numero massimo cumulativo di time slice concessi in una coda), **la sua priorità viene comunque abbassata**, indipendentemente da quante volte ha effettivamente ceduto la CPU. Questo approccio rende l'algoritmo **tollerante alla manipolazione** e garantisce un comportamento più equo.

Un'ulteriore strategia, definita dalla **Regola 5**, consiste **nell'aumentare periodicamente la priorità di tutti i processi**. Dopo un certo intervallo temporale S , tutti i processi vengono spostati nella **coda con priorità più alta**, assicurando che anche i JOB a lunga durata possano ricevere tempo di CPU ed evitando starvation.

Regolazione dei time slice

Infine, per migliorare ulteriormente l'equità e l'efficienza, l'MLFQ può adottare **time slice di durata variabile** in base al livello di priorità:

- Le **code ad alta priorità** utilizzano **time slice molto brevi** (ad esempio 10ms o meno), favorendo la reattività dei processi interattivi.
- Le **code a bassa priorità** impiegano **time slice più lunghi** (es. 100ms), adattandosi a processi CPU-bound che necessitano di tempo esteso per completare il lavoro.

Questo modello consente un miglior **bilanciamento tra latenza e throughput**, riducendo l'overhead di contesti e massimizzando l'efficienza per ogni classe di processo.

Scheduling Real-Time

Lo scheduling implementato in un **sistema operativo real-time** deve rispettare due vincoli fondamentali:

1. **Rispetto delle deadline**: I processi devono essere eseguiti in modo da rispettare le deadline temporali imposte dall'applicazione.
2. **Periodicità dei processi**: I processi possono essere periodici (cioè generati a intervalli regolari), e ciascuna istanza deve completarsi entro una specifica scadenza (**deadline**).

Tipologie di sistemi real-time

In un sistema **hard real-time**, il rispetto della deadline è obbligatorio. Anche una sola violazione può compromettere la correttezza del sistema.

In un sistema **soft real-time**, invece, le deadline possono essere occasionalmente violate, senza compromettere la funzionalità generale. L'obiettivo è mantenere una qualità del servizio accettabile.

Dipendenze tra processi

I processi real-time possono avere dipendenze di esecuzione, che devono essere considerate dallo scheduler:

- Se un processo $P_i \rightarrow P_j$, significa che P_i deve completarsi prima che P_j inizi.
- Se $P_i \rightarrow P_j \rightarrow P_k$, si ha una dipendenza transitiva, indicata con **$P_i \rightarrow^* P_k$** (si dice che P_i precede indirettamente P_k).

Tali relazioni portano al concetto di **schedulazione ammissibile**: una schedulazione è ammissibile se tutte le deadline sono rispettate e le dipendenze tra processi sono mantenute.

Esempio pratico:

Processo	P_1	P_2	P_3	P_4	P_5
Tempo di servizio	3	3	2	4	5

La tabella mostra un'applicazione con 5 processi e un tempo massimo totale di 15 secondi per completarla. Poiché la somma dei tempi di servizio dei processi è **superiore a 15 secondi, non esiste una schedulazione ammissibile**. Se però un processo come P_5 non esistesse, oppure avesse un tempo di servizio minore, potrebbero esserci più schedulazioni ammissibili.

Approcci di Scheduling nei Sistemi Real-Time

Esistono **tre principali approcci** di scheduling utilizzati nei sistemi real-time:

Scheduling statico

Lo **scheduling è determinato prima dell'esecuzione**, costruendo una **tabella statica** che tiene conto di periodicità, vincoli temporali, dipendenze e risorse. Durante il runtime, il sistema consulta questa tabella per decidere quale processo eseguire.

Questo metodo presenta un **overhead minimo**, poiché non richiede decisioni dinamiche durante l'esecuzione, ma è **poco flessibile**: non è in grado di gestire guasti o variazioni dinamiche nel carico di lavoro, ed è adatto solo ad **ambienti deterministici e prevedibili**.

Scheduling basato su priorità

I processi ricevono una priorità (fissa o variabile), assegnata tramite analisi a priori. Durante l'esecuzione, lo scheduler usa un algoritmo dinamico, come Rate Monotonic (priorità fissa) o Earliest Deadline First (priorità variabile), che presenta un **overhead più elevato** rispetto allo statico, poiché richiede continui ricalcoli o riordini della coda dei processi ready.

È però più **flessibile** e consente l'adozione di meccanismi come **l'inversione di priorità**, utili per prevenire problemi di blocchi e starvation.

Scheduling dinamico

Le decisioni vengono prese **al momento della creazione di un processo**: lo scheduler valuta se il processo può essere completato entro la deadline.

Se la risposta è positiva, il processo viene accettato e schedulato; altrimenti viene rifiutato.

Questo approccio è **molto flessibile** e consente una gestione reattiva dei carichi, ma comporta un overhead elevato. Una variante **ottimistica** accetta tutti i processi anche se non garantisce il rispetto della deadline: è tipica dei sistemi **soft real-time**, dove è accettabile perdere occasionalmente alcune scadenze.

Scheduling con deadline

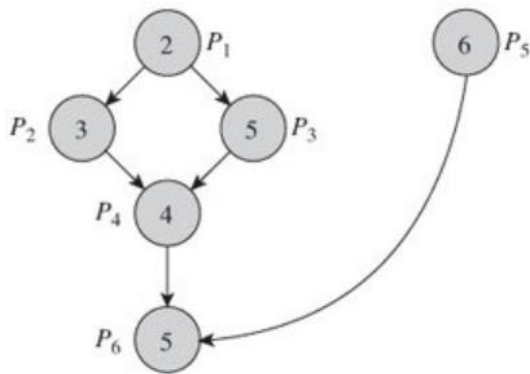
Per ogni processo possono essere specificati due tipi di **deadline** (scadenze): la **deadline di inizio** corrisponde al tempo entro il quale le operazioni devono iniziare; la **deadline di fine** corrisponde al tempo entro il quale le operazioni devono terminare.

La **deadline** di un processo è una **stima** che viene ricavata considerando le **precedenze** del processo, lavorando sulle **richieste di risposta dell'applicazione** e lavorando a ritroso sulle **singole deadline** dei processi discendenti. La formula che descrive tutto ciò è la seguente:

$$D_i = D_{app} - \sum x_k$$

Dove D_i è la deadline del processo che si sta considerando, D_{app} è la scadenza dell'applicazione e x_k è il tempo di servizio del processo P_k processo discendente da P_i .

Processo	P_1	P_2	P_3	P_4	P_5	P_6
Deadline	8	16	16	20	20	25



Tutto ciò si traduce nel grafo **Process Precedence Graph (PPG)**. Il caso richiede che l'applicazione deve produrre una risposta in 25 secondi (tempo di risposta dell'applicazione) e bisogna analizzare le **deadline di fine**. I numeri sui nodi del grafo sono i tempi di servizio dei processi.

Si parte dalla fine dell'applicazione, quindi dal caso in cui sono passati 25 secondi e il processo **P6** è terminato. La deadline di quest'ultimo è proprio di 25 secondi. Di conseguenza il processo **P5** termina dopo 25-5 secondi (deadline attuale – tempo di servizio del processo che sarà eseguito dopo). Anche **P4** deve terminare dopo 25-5 secondi, quindi a 20. I processi **P2** e **P3** terminano entrambi a 20-4 secondi, cioè 16. Il processo **P1** termina a 16-(5+3) secondi, cioè a 8.

Earliest Deadline First (EDF)

Lo **scheduling EDF** seleziona sempre il **processo con la deadline più prossima**. Affinché non ci siano sforamenti di deadline, la condizione da rispettare è:

$$\sum_{k: pos(P_k) \leq pos(P_i)} x_k \leq D_i$$

Ovvero, la **somma dei tempi di servizio dei processi che precedono** un certo processo P_i nella sequenza di esecuzione deve essere **minore o uguale alla sua deadline** D_i .

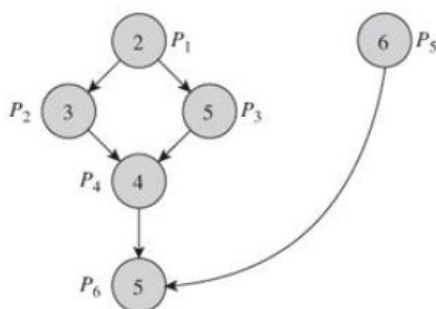
Questa condizione è sempre rispettata quando **esiste una schedulazione ammissibile**.

Uno dei principali vantaggi dell'EDF è la sua **semplicità** e l'**assenza di prelazione**, che implica un **overhead minimo**. È adatto a contesti di **scheduling statico**, in cui si conoscono i processi a priori e si vuole generare una sequenza schedulabile che rispetti le deadline.

Tuttavia, EDF è anche **compatibile con i sistemi soft real-time**, dove occasionali violazioni delle deadline possono essere tollerate.

Un limite di EDF è che **non è possibile prevedere con certezza** quanti processi non rispetteranno la deadline in casi di **schedulazione non ammissibile**.

Esempio 1 – Caso schedulabile



Processo	P_1	P_2	P_3	P_4	P_5	P_6
Deadline	8	16	16	20	20	25

Tempo	Processo completato	Deadline overrun	Processi nel sistema	Processo schedulato
0	–	0	$P_1 : 8, P_2 : 16, P_3 : 16, P_4 : 20, P_5 : 20, P_6 : 25$	P_1
2	P_1	0	$P_2 : 16, P_3 : 16, P_4 : 20, P_5 : 20, P_6 : 25$	P_2
5	P_2	0	$P_3 : 16, P_4 : 20, P_5 : 20, P_6 : 25$	P_3
10	P_3	0	$P_4 : 20, P_5 : 20, P_6 : 25$	P_4
14	P_4	0	$P_5 : 20, P_6 : 25$	P_5
20	P_5	0	$P_6 : 25$	P_6
25	P_6	0	–	–

Nel primo esempio, l'applicazione ha un **tempo di risposta massimo di 25**. I processi sono schedulati in ordine crescente di deadline: P_1, P_2, P_3, P_4, P_5 . Come si può osservare dalla tabella, **tutte le deadline vengono rispettate**.

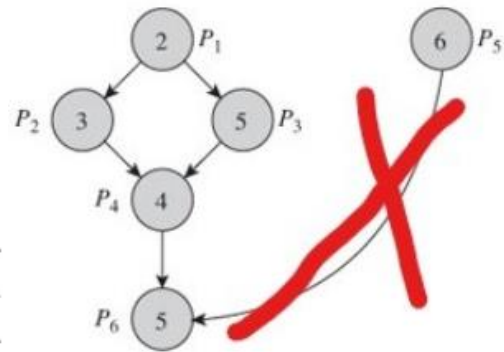
Esempio 2 – Caso non schedulabile

Nel secondo caso, abbiamo due applicazioni:

- La prima composta dai processi P1, P2, P3, P4, P6,
- La seconda composta solo da P5,

Con una deadline totale di 19 secondi per ciascuna.

Processo	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆
Deadline	2	10	10	14	19	19



In questa situazione non esiste alcuna sequenza schedulabile in grado di soddisfare entrambe le deadline. Infatti, EDF può selezionare ad esempio:

- La sequenza P1, P2, P3, P4, P6: in questo caso P5 non rispetta la deadline;
- Oppure la sequenza P1, P2, P3, P4, P5: in questo caso P6 non rispetta la deadline.

Ciò dimostra che quando non è garantita una schedulazione ammissibile, **EDF non è in grado di prevedere** quale o quante deadline verranno violate. Questa caratteristica lo rende **poco adatto a sistemi hard real-time** in condizioni di sovraccarico.

Scheduling Rate Monotonic (SRC)

Lo scheduling **Rate Monotonic** si applica a **processi periodici**, ovvero che si ripetono a intervalli regolari. Ogni processo è caratterizzato da un **periodo** (tempo tra due attivazioni consecutive) e da un **tempo di servizio** (tempo richiesto dalla CPU per ogni attivazione).

Questa politica di scheduling è tipica dei **sistemi hard real-time**, dove è fondamentale che ogni processo rispetti la sua deadline. Una schedulazione è garantita solo se:

$$\sum_{i=1}^n \left(\frac{C_i}{T_i} \right) \leq 1$$

Dove C_i è il tempo di servizio del processo P_i e T_i , il suo periodo.

Assegnazione della priorità

Nel Rate Monotonic, la **priorità è inversamente proporzionale al periodo**: processi con **periodo più breve** hanno **priorità più alta**.

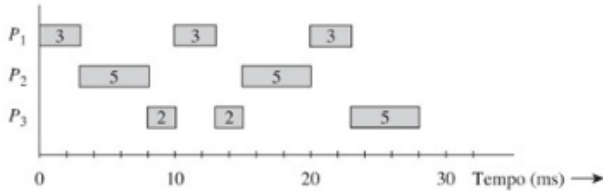
Il **rate** (frequenza di attivazione) viene calcolato come $rate = 1/T$, quindi un processo che si attiva più frequentemente ottiene priorità maggiore.

Processo	P ₁	P ₂	P ₃
Periodo (ms)	10	15	30
Tempo di servizio (ms)	3	5	9

Esempio:

L'ordine di esecuzione sarà quindi **P1 > P2 > P3**. Se due processi sono pronti contemporaneamente, verrà eseguito quello con priorità maggiore.

Esempio di esecuzione



Nel grafico viene mostrato come P1 e P2 vengano eseguiti interamente prima di P3. Tuttavia, se P3 dovesse avere un periodo più breve (es. 27ms), ma l'algoritmo mantiene le vecchie priorità, allora la nuova sequenza potrebbe non essere più schedulabile: la priorità **non può essere modificata dinamicamente**. Questo mostra uno dei **limiti principali** dello scheduling RM: **l'ordine di esecuzione è fisso** e non sempre garantisce il rispetto delle scadenze anche quando esisterebbero altre sequenze ammissibili.

Schedulabilità e limite di utilizzo CPU

È stato dimostrato che RM **non riesce a garantire la schedulabilità** se l'utilizzo della CPU supera:

$$U \leq m \cdot \left(2^{\frac{1}{m}} - 1\right)$$

Dove m è il numero di processi. Questo valore tende a circa **0.69** per $m \rightarrow \infty$, il che significa che **non è possibile sfruttare oltre il 69% della CPU** con garanzia di rispetto delle deadline. Per valori superiori, si potrebbero verificare **superamenti delle scadenze**.

Lo scheduling Rate Monotonic è efficiente e semplice da implementare, ma presenta due limiti principali:

1. **Priorità fissa non adattabile** → non gestisce bene i carichi variabili o dinamici.
2. **Utilizzo massimo della CPU limitato** → non adatto a sistemi ad alta densità di carico.

Per superare questi limiti si può usare EDF, che adatta le priorità dinamicamente e permette di sfruttare la CPU fino al 100%, evitando così il superamento delle deadline.

Capitolo 8 - DEADLOCK

Un **deadlock** corrisponde a uno **stallo** che si verifica quando due o più processi **sono in attesa** di un evento che deve essere generato da altri processi, i quali a loro volta **sono anch'essi in attesa** di eventi generati dai primi. In pratica, si crea una **catena circolare di attese**: ciascun processo sta aspettando qualcosa che **può essere prodotto solo da un altro processo bloccato** nella stessa catena.

Più formalmente, un deadlock si verifica quando **le seguenti due condizioni** sono entrambe soddisfatte:

- Tutti i processi di un insieme D sono bloccati in attesa di un evento e;
- L'evento atteso può essere generato solo da processi appartenenti all'insieme D.

Esistono diversi tipi di deadlock, ma i principali sono:

- **Deadlock di risorsa**, che si verifica quando i processi competono per l'**allocazione di risorse** (CPU, stampanti, memoria, ecc.). La gestione e prevenzione è **responsabilità del SO**.
- **Deadlock di sincronizzazione o comunicazione**, causato da **errori nella progettazione** della sincronizzazione tra processi (es. uso scorretto di semafori o attesa di segnali). In questo caso, la prevenzione è **a carico dello sviluppatore**.

Deadlock di risorsa

Durante l'esecuzione, un processo può richiedere risorse **hardware** (come memoria o dispositivi di I/O) o **software** (come file). Una risorsa è classificata in una **classe di risorse**, che può avere più **unità** (ad esempio, la classe stampante può avere più stampanti). Il kernel tiene traccia dello stato delle risorse tramite un'apposita **tabella di allocazione**.

Quando si **alloca una risorsa**, si eseguono tre operazioni, in questo ordine:

- **Richiesta:** il processo effettua una system call per richiedere una risorsa. Se disponibile, viene allocata; altrimenti, il processo entra nello stato **blocked**.
- **Allocazione:** se la risorsa viene concessa, il processo diventa **holder**, la tabella viene aggiornata, e il processo passa allo stato **ready**.
- **Rilascio:** quando un processo rilascia una risorsa (con un'altra system call), il **kernel decide** a chi assegnarla tra i processi in attesa (di norma secondo politiche come FCFS).

Condizioni per il deadlock di risorsa

Affinché un **deadlock si verifichi**, devono essere presenti **simultaneamente** le seguenti condizioni:

- **Risorse non condivisibili oppure mutua esclusione:** alcune risorse non possono essere condivise oppure un processo necessita dell'accesso ad una risorsa in mutua esclusione;
- **Niente prelazione:** una risorsa non può essere tolta da un processo, ma solo rilasciata volontariamente.
- **Hold and wait:** un processo mantiene una risorsa già allocata mentre attende un'altra.
- **Attesa circolare:** esiste una catena di processi in attesa reciproca. Es: $P_i \rightarrow P_j \rightarrow P_k \rightarrow P_i$.
- **Nessun ritiro delle richieste:** un processo che ha chiesto una risorsa non può annullare la richiesta mentre è in attesa (a meno che il SO non lo consenta espressamente).

Modello dello stato di allocazione delle risorse

Per consentire una gestione efficiente delle risorse, il kernel deve poter accedere in ogni momento allo stato di allocazione delle stesse. Questo stato viene rappresentato mediante modelli formali che descrivono quali risorse sono allocate a quali processi e quali risorse sono richieste. I due modelli principali utilizzati a questo scopo sono:

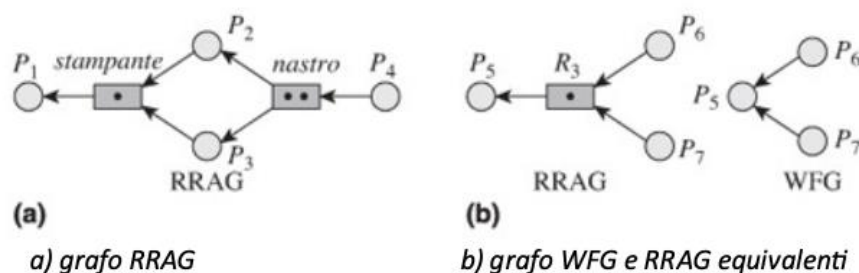
- Il **modello a grafo** (noto anche come **RRAG** – Resource Request and Allocation Graph- o **WGF**, Wait-for-Graph);
- Il **modello a matrice**.

Modello a Grafo

Il **modello a grafo RRAG** è utilizzato quando le **classi di risorse** possono avere **più unità**. In questo tipo di grafo troviamo:

- **Nodi processo** (cerchi), che rappresentano i processi;
- **Nodi risorsa** (rettangoli), che rappresentano le unità di risorsa;
- **Archi di allocazione** (da nodo risorsa a nodo processo), che indicano l'assegnazione di una risorsa a un processo;
- **Archi di richiesta** (da nodo processo a nodo risorsa), che indicano una richiesta di risorsa.

Il **modello a grafo WFG** è usato quando ogni classe di risorsa ha **una sola unità**. In questo grafo si hanno solo **nodi processo e archi**, che indicano che un processo è **bloccato** in attesa di un altro.



Questi grafi rappresentano le **relazioni di attesa** tra processi. Un **deadlock** si può riconoscere dalla presenza di un **ciclo** nel grafo. Tuttavia, **questa regola vale solo quando ogni risorsa ha una sola istanza**. Se le risorse hanno più unità, la presenza di un ciclo **non implica necessariamente** un deadlock: bisogna analizzare più attentamente la situazione (per esempio con il metodo del grafo ridotto).

Modello a Matrice

Il **modello a matrice** rappresenta lo stato di allocazione delle risorse tramite due matrici principali:

- La matrice delle **risorse allocate**, che indica quante unità di ciascuna risorsa sono attualmente assegnate a ogni processo;
- La matrice delle **risorse richieste**, che indica quante unità ciascun processo ha richiesto ma non ancora ottenuto.

Entrambe le matrici hanno dimensione $n \times r$, dove n è il numero di processi e r il numero di classi di risorse. A supporto, esistono anche **matrici ausiliarie** come:

- La matrice delle **risorse totali** disponibili nel sistema,
- La matrice delle **risorse libere**, ovvero quelle non attualmente allocate.

	Printer Tape			Printer Tape			Printer Tape	
P_i	0	1	P_i	1	0	Total resources	1	2
P_j	1	0	P_j	0	1	Free resources	0	0
P_k	0	1	P_k	0	0			
Allocated resources			Requested resources					

A differenza del **modello a grafo**, il modello a matrice **non rappresenta esplicitamente le relazioni di attesa** tra processi e risorse, ma è molto utile per implementazioni computazionali nei meccanismi di allocazione.

Gestione dei deadlock

Esistono essenzialmente tre approcci per gestire i deadlock:

- **Individuazione e risoluzione:**

Il Kernel analizza periodicamente lo stato delle risorse per verificare la presenza di un deadlock. In caso positivo, interrompe uno o più processi coinvolti e riassegna le risorse liberate ad altri processi, fino a quando il deadlock non viene eliminato. Questo metodo comporta un doppio costo: uno legato al rilevamento del deadlock e uno alla riesecuzione dei processi terminati forzatamente.

- **Prevenzione:**

Il kernel applica una politica di allocazione tale da impedire che si verifichino simultaneamente tutte e quattro le condizioni necessarie per un deadlock. I processi devono quindi rispettare vincoli ben precisi nell'uso delle risorse.

- **Evitamento:**

Il Kernel valuta in anticipo se l'assegnazione di una nuova risorsa potrebbe causare un deadlock, analizzando lo stato corrente del sistema. Se l'allocazione risulta "sicura", viene concessa; altrimenti la richiesta viene sospesa. Questo approccio richiede più controlli, ma consente di evitare situazioni di stallo.

Individuazione e risoluzione dei deadlock

Un processo bloccato **non è coinvolto in un deadlock** se la richiesta sulla quale è in attesa può ancora essere soddisfatta tramite una sequenza di eventi del tipo: **completamento di un altro processo** → **rilascio delle sue risorse** → **nuova allocazione**

Nel caso in cui le risorse abbiano una sola unità, è sufficiente controllare la presenza di **cicli nel grafo**. Tuttavia, se le risorse possiedono **più unità**, è preferibile utilizzare il **modello a matrice**, poiché il grafo diventerebbe troppo complesso.

Per rilevare un deadlock, il kernel costruisce **sequenze fittizie**, cioè simulazioni in cui si verifica se, seguendo un certo ordine, tutti i processi bloccati riescono comunque a ottenere le **risorse richieste**. Se in almeno una sequenza **tutti i processi** riescono a completare, **non c'è deadlock**. Se invece **esistono processi che restano bloccati** in tutte le possibili sequenze, allora il deadlock è **presente**.

	R_1		R_1		R_1
P_1	4	P_1	6	Risorse totali	10
P_2	4	P_2	2	Risorse libere	0
P_3	2	P_3	0		
Risorse allocate		Risorse richieste			

Nel caso mostrato nell'esempio, si parte con 0 risorse libere. Tuttavia, il processo P3 non ha richieste pendenti, quindi è in esecuzione, termina e libera 2 unità. A questo punto, il sistema riesce a soddisfare la richiesta di P2, che termina e rilascia 6 risorse. Con queste, viene soddisfatta la richiesta di P1, che termina e libera le ultime risorse. Poiché esiste almeno una sequenza in cui **tutti i processi completano, non si verifica un deadlock**.

Algoritmo Individuazione

INPUT

n: numero di processi
r: numero di classi di risorsa
Blocked: insieme dei processi bloccati
Running: insieme dei processi da eseguire
Risorse_libere[*r*]: risorse libere per ogni classe di risorsa
Risorse_allocate[*n*][*r*]: risorse allocate dai processi in quel momento
Risorse_richieste[*n*][*r*]: risorse richieste dai processi

Strutture dati

Exit: insieme dei processi terminati

```
1)  repeat until Running non è vuoto
2)    seleziona un processo  $P_i$  dall'insieme Running;
3)    cancella  $P_i$  dall'insieme Running e aggiungilo all'insieme Exit;
4)    for  $k = 1 \dots r$ 
5)      Risorse_libere[ $k$ ] += Risorse_allocate[ $i$ ][ $k$ ];
6)  while l'insieme Blocked contiene un processo  $P_l$  tale che con
     $k = 1 \dots r$  vale Risorse_richieste[ $l$ ][ $k$ ] ≤ Risorse_libere[ $k$ ]
7)    for  $k = 1 \dots r$ 
8)      Risorse_libere[ $k$ ] -= Risorse_richieste[ $l$ ][ $k$ ];
9)      Risorse_allocate[ $l$ ][ $k$ ] += Risorse_richieste[ $l$ ][ $k$ ];
10)   Rimuovi  $P_l$  da Blocked e aggiungilo a Running;
11)  if l'insieme Blocked non è vuoto then imposta i processi
    nell'insieme Blocked come processi in deadlock.
```

Questo algoritmo ha complessità $O(n^2 * r)$, dove *n* è il numero di processi e *r* il numero di classi di risorse. N quadro perché: il repeat until viene fatto al più *n* volte. All'interno del repeat until c'è un altro ciclo che viene fatto *n* volte e quindi complessità finale è *n* quadro.

Il cuore dell'algoritmo si trova **tra le righe 2 e 10**, dove si ripete finché ci sono processi da eseguire.

Quando l'insieme *Running* è vuoto, significa che non ci sono più processi eseguibili.

Nella **prima parte**, si seleziona un processo da *Running*, lo si sposta in *Exit* e si **rilasciano le sue risorse** aggiornando l'array delle risorse libere.

Dalla riga 6 in poi, si verifica se ci sono processi in *Blocked* le cui **richieste possono essere soddisfatte** con le risorse attualmente disponibili. Se almeno uno di questi processi può essere eseguito, le risorse vengono allocate (riga 9), aggiornando le strutture *Risorse_allocate* e *Risorse_libere*, e il processo viene spostato da *Blocked* a *Running* (riga 10), pronto per un nuovo ciclo.

Alla fine, se l'insieme *Blocked* **non è vuoto**, significa che ci sono processi bloccati le cui richieste **non possono essere soddisfatte** con le risorse attualmente disponibili: questi processi si trovano quindi in **deadlock**.

Funzionamento pratico dell'algoritmo

	R1	R2	R3
P1	2	1	0
P2	1	3	1
P3	1	1	1
P4	1	2	2

Risorse Allocate

	R1	R2	R3
P1	2	1	3
P2	1	4	0
P3	0	0	0
P4	1	0	2

Risorse Richieste

Risorse Libere		
R1	R2	R3
0	0	1

Quando P3 termina

R1	R2	R3
1	1	2

	R1	R2	R3
P1	2	1	0
P2	1	3	1
P3	1	1	1
P4	1+1	2	2+2

Risorse Allocate

	R1	R2	R3
P1	2	1	3
P2	1	4	0
P3	0	0	0
P4	1	0	2

Risorse Richieste

Risorse Libere		
R1	R2	R3
1	1	2

Esegui P4

Risorse Libere		
R1	R2	R3
0	1	0

Risorse Libere		
R1	R2	R3
2	3	4

Quando P4 termina

	R1	R2	R3
P1	2+2	1+1	0+3
P2	1	3	1
P3	1	1	1
P4	1+1	2	2+2

Risorse Allocate

	R1	R2	R3
P1	2	1	3
P2	1	4	0
P3	0	0	0
P4	1	0	2

Risorse Richieste

Risorse Libere		
R1	R2	R3
2	3	4

Esegui P1

Risorse Libere		
R1	R2	R3
0	2	1

Risorse Libere		
R1	R2	R3
4	4	4

Quando P1 termina

	R1	R2	R3
P1	2+2	1+1	0+3
P2	1+1	3+4	1
P3	1	1	1
P4	1+1	2	2+2

Risorse Allocate

	R1	R2	R3
P1	2	1	3
P2	1	4	0
P3	0	0	0
P4	1	0	2

Risorse Richieste

Risorse Libere		
R1	R2	R3
4	4	4

Esegui P2

Risorse Libere		
R1	R2	R3
3	0	4

Risorse Libere		
R1	R2	R3
5	7	5

Quando P2 termina

Risoluzione

Un meccanismo per risolvere un deadlock in corso consiste nel **terminare uno dei processi coinvolti e rilasciare le risorse** che ha in uso, in modo da renderle disponibili agli altri processi. Il processo che viene terminato prende il nome di **vittima della risoluzione del deadlock** e viene scelto in base a **criteri specifici**, come la **priorità**, la **quantità di risorse consumate**, il **tempo di esecuzione** o l'impatto sull'intero sistema.

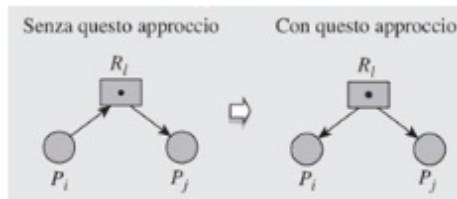
Nel **modello a grafo**, questa operazione corrisponde alla **rimozione del nodo processo** dal grafo e degli archi ad esso collegati.

Nel **modello a matrice**, equivale invece alla **cancellazione della riga** corrispondente al processo negli insiemi $R_{risorse_allocate}$ e $R_{risorse_richieste}$. Tutti gli altri insiemi e strutture collegate devono essere **aggiornati di conseguenza** per riflettere la nuova disponibilità delle risorse.

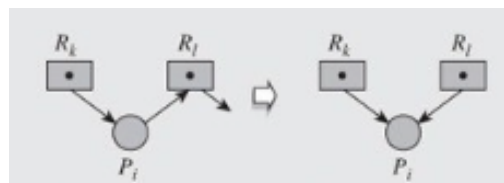
Prevenzione dei Deadlock

Affinché si verifichi un deadlock, devono essere soddisfatte contemporaneamente quattro condizioni: mutua esclusione, assenza di prelazione, hold and wait, e attesa circolare.

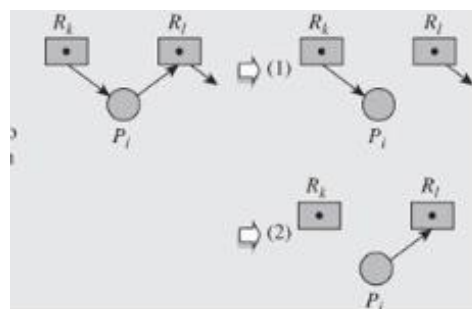
Esistono diversi approcci per prevenirlo, ciascuno mirato a rompere almeno una di queste condizioni:



- Un primo approccio consiste nel **condividere tutte le risorse**: in questo modo non si generano attese e quindi non si verificano cicli nel grafo delle allocazioni. Nel modello a grafo, ciò si riflette nell'assenza di **archi di richiesta**, in quanto non ci sono risorse da attendere. Nel caso di risorse non condivisibili, si possono usare tecniche come la **bufferizzazione dell'output** o l'uso di **istanze virtuali**.

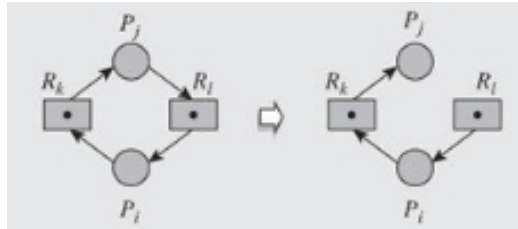


- Un secondo metodo è rendere le risorse **prelazionabili**, ovvero permettere che una risorsa già assegnata possa essere "presa in prestito" da un altro processo. In questo caso, è stata allocata una risorsa R_k al processo P_i che ha richiesto la risorsa R_l che è stata allocata a qualche altro processo. A questo punto è possibile prelazionare R_l dal processo al quale è stata allocata e far sì che venga assegnata al processo P_i . Il processo che la deteneva la perde e il deadlock viene evitato, rompendo la condizione di assenza di prelazione.



NB: L'**Hold and wait** è una situazione in cui un processo si blocca poiché possiede una risorsa, ma ne richiede un'altra che non è disponibile (allocata a qualche altro processo).

- Per evitare l'**hold and wait**, si può impedire ai processi di richiedere ulteriori risorse se ne possiedono già. In alternativa, si può permettere la richiesta solo **una volta (allocazione globale)**. In termini di grado RRAG, ciò si traduce in un singolo arco per ogni processo attivo. Nell'immagine, nel primo caso (primo modo) non si permette al processo P_i (che possiede già R_k) di richiedere la risorsa R_l . Nel secondo modo (immagine (2)) si consente al processo P_i di richiedere R_l , ma non si consente di mantenere la risorsa R_k .



- Per prevenire l'**attesa circolare**, si può introdurre un **vincolo di ordinamento (ranking) delle risorse**, rappresentato da una funzione booleana che valuta la validità dell'arco. Se l'arco può condurre a una situazione ciclica, viene rifiutato. Solo le richieste che non violano il vincolo vengono accettate.

Allocazione Simultanea o Globale

Questa tecnica di prevenzione del deadlock impone che un processo debba richiedere **tutte le risorse di cui ha bisogno in un'unica richiesta**.

In questo modo, se anche solo una risorsa non è disponibile, **nessuna** viene allocata e il processo resta in attesa. Così si evita la condizione di **hold and wait**, perché **nessun processo può detenere una risorsa e chiederne altre**. **Svantaggio**: l'allocazione potrebbe avvenire **prima del reale bisogno**, sprecando risorse e **riducendo efficienza e multiprogrammazione**.

Ranking delle risorse

Ad ogni classe di risorse viene associato un **rank**, ovvero un numero che ne indica la posizione in un ordine gerarchico predefinito. Quando un processo effettua una richiesta, il kernel valuta un **vincolo di validità** per decidere se soddisfare la richiesta.

Questo vincolo si basa sulla seguente regola:

*Se il **rank della risorsa richiesta** è **maggiore del rank delle risorse già possedute dal processo**, la richiesta è **valida (true)**, e quindi accettata (se la risorsa è disponibile). In caso contrario, il vincolo è **falso (false)**, e la richiesta viene **rifiutata**.*

Questa politica impone che un processo possa **richiedere solo risorse con rank superiore** a quelle che già possiede. Le richieste sono accettate solo se seguono un **ordine crescente** di rank, impedendo così la formazione di **cicli** tra i processi, ed eliminando quindi l'**attesa circolare**, una delle condizioni del deadlock.

Questo metodo funziona al meglio quando i processi seguono un ordine coerente e crescente nelle richieste di risorse. Tuttavia, in presenza di richieste "non valide" (che violano l'ordine), si possono applicare strategie alternative, come:

- **Richiedere subito tutte le risorse** a basso rank prima di iniziare (per rispettare l'ordine), ma ciò può comportare l'**allocazione immediata di risorse inutilizzate**, riducendo l'efficienza.

Nel **caso peggiore**, questa soluzione si riduce a una **allocazione globale**, dove un processo deve richiedere tutte le risorse di basso rank **simultaneamente**, prima di iniziare.

Esempio (attesa circolare evitata con il ranking)

Supponiamo di avere 3 processi e 3 risorse:

- **P1** ha **R1** e vuole **R2**;
- **P2** ha **R2** e vuole **R3**;
- **P3** ha **R3** e vuole **R1**;

Questo crea un ciclo: **P1 → P2 → P3 → P1**

Tutti aspettano una risorsa bloccata da un altro processo, creando un attesa circolare e causando un possibile deadlock.

Se però assegniamo un ranking:

- **R1 = 1**;
- **R2 = 2**;
- **R3 = 3**;

E si impone una regola semplice: **Un processo può chiedere solo risorse con rank più alto di quelle che già possiede**, allora:

- **P1** ha **R1** (rank 1) → può chiedere solo **R2** o **R3**, non **R1** o meno;
- **P2** ha **R2** (rank 2) → può chiedere solo **R3**, non **R1** o **R2**;

- P3 ha R3 (rank 3) → non può chiedere nulla, perché non esiste rank > 3

Quindi P3 non potrà mai chiedere R1, perché sarebbe come “tornare indietro” nell'ordine e quindi il ciclo viene rotto. La richiesta P3 → R1 viene bloccata dal sistema perché R1 ha un rank più basso di R3.

Questo metodo è **efficace** se i processi richiedono le risorse sempre **nello stesso ordine crescente**.

Evitamento dei deadlock (Algoritmo del banchiere)

L'**algoritmo del banchiere** è una tecnica di **evitamento del deadlock**, ispirata al comportamento delle banche quando devono decidere se concedere un prestito. Questo algoritmo assegna risorse ai processi solo se è certo di poter garantire il completamento di tutti, evitando così situazioni di stallo.

A differenza **dell'individuazione del deadlock**, che interviene solo dopo che il problema si è manifestato, l'algoritmo del banchiere opera in maniera **preventiva**, simulando l'evoluzione del sistema prima di concedere una risorsa. La sua efficacia si basa sul fatto che ogni processo, all'avvio, deve dichiarare **qual è il massimo numero di risorse** di ogni classe che potrebbe richiedere durante la sua esecuzione. Il sistema tiene traccia in ogni istante delle **risorse allocate**, delle **risorse ancora disponibili** e delle **richieste residue** (cioè quanto manca a ciascun processo per raggiungere il suo massimo dichiarato).

Quando un processo effettua una richiesta, l'algoritmo verifica innanzitutto se la richiesta è coerente con il **massimo dichiarato** e se ci sono **risorse sufficienti** per soddisfarla. Se queste due condizioni sono vere, il sistema effettua una **simulazione**: concede temporaneamente le risorse richieste e valuta se esiste una **sequenza sicura** di esecuzione in cui tutti i processi possano completare, rilasciando gradualmente le risorse. Se tale sequenza esiste, lo **stato è considerato sicuro** e la richiesta viene approvata; altrimenti, il sistema **rifiuta** la richiesta, evitando così di entrare in uno stato pericoloso che potrebbe portare a un deadlock.

Il punto centrale dell'algoritmo è quindi la capacità di **anticipare** situazioni di stallo mediante simulazioni. Solo se, in seguito alla concessione delle risorse, è ancora possibile garantire che tutti i processi possano raggiungere la conclusione, allora la richiesta viene soddisfatta realmente. In caso contrario, viene respinta. In questo modo il sistema si mantiene **sempre in uno stato sicuro**, prevenendo il deadlock **prima ancora che possa verificarsi**.

Schema dell'approccio

1. Quando un processo effettua una richiesta di risorse, il sistema simula lo stato di allocazione **come se la richiesta fosse soddisfatta**. Questo permette di valutare **preventivamente** se il sistema rimarrebbe in uno stato sicuro.
2. Se la simulazione restituisce uno **stato sicuro**, le risorse vengono effettivamente allocate e le strutture dati come `Allocated_resources` e `Total_alloc` vengono aggiornate. In caso contrario, la richiesta resta **in sospeso** finché non sarà possibile soddisfarla senza rischi.
La **verifica di sicurezza** consiste nel controllare se esiste una **sequenza di esecuzione** (completamento → rilascio → nuova allocazione) che consenta a tutti i processi di terminare correttamente. Si assume che un processo possa terminare solo se gli viene garantita la possibilità di ottenere **il massimo delle risorse di cui potrebbe aver bisogno**. La condizione formale è:

$$Total_resource_k - Total_alloc_k \geq Max_need_{ik} - Allocated_resource_{ik}$$

Per ogni **processo i** e **risorsa k**.

3. Quando un processo **rilascia una risorsa o termina**, il sistema esamina le richieste pendenti e valuta nuovamente se è possibile soddisfarle, **concedendo solo quelle che mantengono il sistema in uno stato sicuro**.

Algoritmo del banchiere

Input

n: numero di **processi**;
r: numero di **classi di risorse**;
Blocked: **insieme** di processi;
Running: **insieme** di processi;
P_r: processo che effettua la **nuova richiesta** di risorsa;
Risorse_max_processo[*n*][*r*]: **massime risorse** che un processo utilizza;
Risorse_allocate[*n*][*r*]: risorse **allocate dei processi** in quel momento;
Risorse_richieste[*n*][*r*]: risorse **richieste dai processi**;
Risorse_totali_allocate[*r*]: risorse totali **allocate in quel momento**;
Risorse_totali[*r*]: risorse **totali** nel sistema (allocate e non);

Strutture dati

Active: **insieme** di processi;
fattibile: boolean;
Nuova_richiesta[*r*]: **nuova richiesta** da controllare;
Risorse_allocate_simulate[*n*][*r*]: risorse **allocate dei processi** nella simulazione;
Risorse_totali_allocate_simulate[*r*]: risorse totali **allocate nella simulazione**;

1)Inizializzazioni

1. *Active* = unisci *Running* e *Blocked*;
2. **for** (*k* = 1...*r*)
3. *Nuova_richiesta*[*k*] = *Risorse_richieste*[*p_r*, *k*];

Il sistema costruisce **un insieme di processi attivi** (in *Active*) che comprende sia quelli in esecuzione sia quelli bloccati. Successivamente, viene prelevata la richiesta corrente del processo che ha fatto la nuova richiesta di risorse.

2)Simulazione

4. *Risorse_allocate_simulate* = *Risorse_allocate*;
5. **for** (*k* = 1...*r*)
6. *Risorse_allocate_simulate*[*p_r*, *k*] += *Nuova_richiesta*[*k*];
7. *Risorse_totali_allocate_simulate*[*k*] += *Nuova_richiesta*[*k*];

Il sistema copia le informazioni correnti sulle risorse allocate in **strutture dati di simulazione**. A queste strutture vengono poi aggiunte le risorse della nuova richiesta, come se fossero state già concesse, per simulare l'evoluzione del sistema.

3)Fattibilità

8. *fattibile* = **true**;
9. **for** *k* = 1...*r*
10. **if** (*Risorse_totali*[*k*] < *Risorse_totali_allocate_simulate*[*k*]) **then**
11. *fattibile* = **false**;

Prima di simulare le esecuzioni, il sistema verifica che le risorse totali del sistema siano sufficienti per soddisfare la nuova allocazione simulata. Se non lo sono, la simulazione è interrotta e la richiesta viene rifiutata.

4) Verifica allocazione sicura

```
12. if (fattibile = true) then  
13.   while (Active contiene un processo  $P_i$  tale che per ogni  $k = 1 \dots r$ ,  
-a.    $Risorse\_totali[k] - Risorse\_totali\_allocate\_simulate[k] \geq$   
-b.    $Risorse\_max\_processo[i, k] - Risorse\_allocate\_simulate[i, k]$ )  
14.     Rimuovi  $P_i$  da Active;  
15.     for ( $k = 1 \dots r$ )  
16.        $Risorse\_totali\_allocate\_simulate[k] -=$   
          $Risorse\_allocate\_simulate[i, k];$ 
```

Durante la verifica di allocazione sicura, il sistema controlla se tutti i processi possono terminare senza causare deadlock. Si simula l'esecuzione dei processi uno alla volta, verificando che le risorse ancora necessarie a ciascun processo possano essere soddisfatte con le risorse disponibili.

Se un processo soddisfa questa condizione, si simula il suo completamento e le sue risorse vengono restituite, aumentando così la disponibilità.

Il ciclo continua finché:

- **Tutti i processi terminano** (quindi lo stato è sicuro), oppure
- **Nessun altro processo può terminare**, e quindi si prevede un deadlock.

5) Allocazione effettiva

```
17. if (Active è vuoto) then  
18.   for ( $k = 1 \dots r$ )  
19.      $Risorse\_richieste[p_r, k] = 0;$   
20.   for ( $k = 1 \dots r$ )  
21.      $Risorse\_allocate[p_r, k] += Nuova\_richiesta[k];$   
22.      $Risorse\_totali\_allocate[k] += Nuova\_richiesta[k];$ 
```

Se la simulazione termina con tutti i processi completati (insieme Active vuoto), allora il sistema può concedere la richiesta in modo sicuro. Le risorse vengono effettivamente allocate, e la richiesta è rimossa dalla lista. Se invece la simulazione fallisce, la richiesta viene bloccata e non viene concessa.

Capitolo 9 – Gestione della memoria

La **gerarchia della memoria** comprende **cache**, **MMU**, **RAM** (detta memoria) e **disco**.

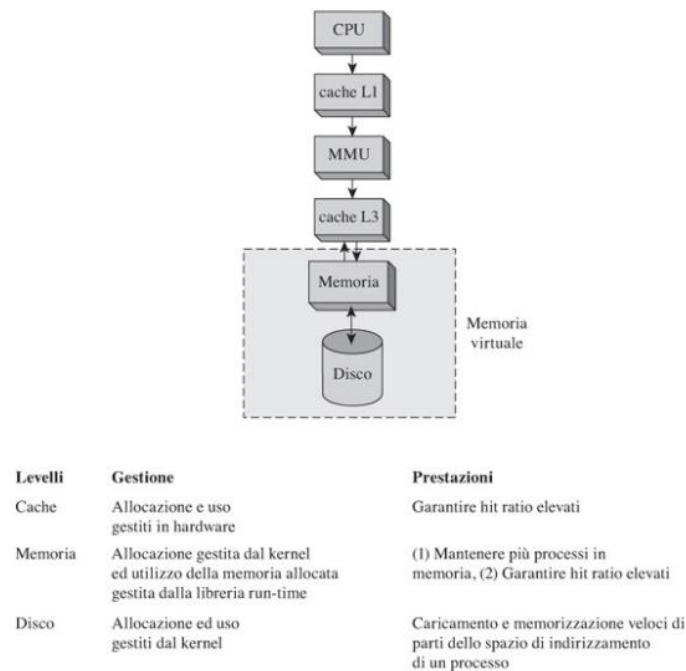
La **gestione della memoria** da parte del SO si divide nell'uso **efficiente della memoria** e nella **gestione della memoria virtuale**.

Il **memory binding** associa indirizzi di memoria con le istruzioni e dati di un programma sia **staticamente**, attraverso il compilatore/linker, prima che il programma cominci l'esecuzione, sia **dinamicamente**, durante l'esecuzione del programma, lasciando la gestione al SO (kernel).

Il kernel **assegna memoria ai processi** combinando binding statico e dinamico, mirando a **velocità** ed **efficienza**. Riutilizza la memoria rilasciata da un processo e la assegna ad altri processi che la richiedono, ma si deve affrontare il problema della **frammentazione della memoria**, che viene ridotto con tecniche come la **paginazione** e **segmentazione**.

Il **kernel** gestisce rapidamente la creazione e distruzione delle **strutture dati di controllo** come i **PCB**, sfruttando il fatto che le loro **dimensioni sono note a priori** per garantire, attraverso un insieme di tecniche che sfruttano queste informazioni, **allocazione/deallocazione veloce** e **uso efficiente della memoria**.

Gestione della Gerarchia di Memoria



La **Gerarchia di Memoria** include la **cache** (L1, L2, L3), **MMU**, **memoria** (RAM) e **disco**, ed è progettata per offrire l'illusione di una memoria **veloce** e **grande** a **basso costo**. La **CPU** accede prima alla **cache**; se i dati o le istruzioni non sono presenti, si passa progressivamente ai livelli inferiori. Le prestazioni dipendono dall'**hit ratio**, cioè la percentuale di accessi soddisfatti ai livelli più alti. Le cache sono gestite dall'hardware.

La **memoria** è gestita congiuntamente dal **kernel** e dalla **libreria run-time** del linguaggio di programmazione utilizzato per il codice del processo. Il **kernel** si occupa dell'**allocazione** della memoria ai processi, cercando di mantenere in memoria il maggior numero possibile di processi per migliorare le **prestazioni del sistema** e riutilizzando la memoria liberata al termine dell'esecuzione dei processi. Durante l'esecuzione, la gestione delle **strutture dati** (come creazione e distruzione) è affidata alla **libreria run-time**, che adotta tecniche di **riutilizzo efficiente** della memoria.

Per aumentare il numero di processi in memoria, il **kernel** può mantenere solo parte dello **spazio di indirizzamento** di ciascun processo, grazie alla **memoria virtuale** (memoria + disco). Le parti dello spazio di indirizzamento mancanti vengono caricate dal **disco** su richiesta. Anche qui, l'**hit ratio** è fondamentale per le prestazioni, e il kernel adotta strategie per garantirlo elevato, organizzando efficientemente i dati sul disco.

Allocazione Statica e Dinamica della memoria

L'**allocazione della memoria** fa parte del concetto più generale di **binding**, che comprende anche **linking** e **caricamento**. In un programma, ogni elemento (come una funzione o una variabile) ha attributi come **nome**, **tipo** e **indirizzo di memoria**, che vengono assegnati tramite il **binding**.

In particolare, il **memory binding** consiste nell'**assegnare un indirizzo di memoria** a una variabile o a un dato. Quando si parla di allocazione della memoria di un processo, si intende attribuire indirizzi sia alle **istruzioni** sia ai **dati** del programma.

Il **binding** può avvenire in due momenti diversi:

- **Early binding** (binding statico): avviene **prima** dell'esecuzione o dell'attivazione del programma.
- **Late binding** (binding dinamico): avviene **durante** l'esecuzione del programma.

L'**allocazione statica** della memoria viene effettuata dal **compilatore**, dal **linker** o dal **loader** prima che il programma venga eseguito. Funziona bene solo se le **dimensioni delle strutture dati** sono **note in anticipo**. Se la stima delle dimensioni è sbagliata, può portare a **sprechi di memoria** o **errori di esecuzione**.

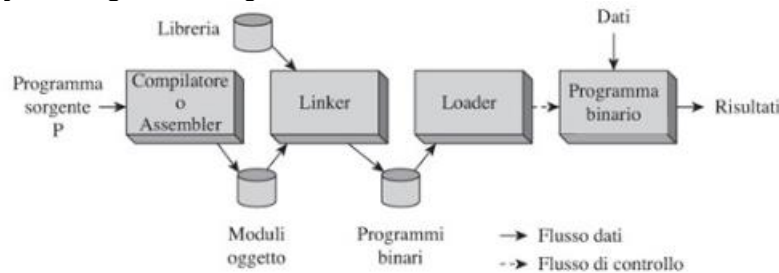
L'**allocazione dinamica**, invece, avviene **durante l'esecuzione**. Permette di adattarsi alle necessità reali del programma, come ad esempio gestire array di dimensione variabile, ma introduce **overhead** a causa delle operazioni di allocazione mentre il programma è in funzione.

I **Sistemi Operativi** scelgono tra allocazione statica e dinamica in base a quante informazioni hanno disponibili:

- Se le esigenze di memoria sono già note, si preferisce l'**allocazione statica** per avere maggiore **velocità**.
- Se invece non si conoscono in anticipo le necessità, si adotta l'**allocazione dinamica**, accettando un po' più di **overhead** pur di evitare **sprechi**.

Esecuzione dei Programmi

Un programma scritto in un linguaggio di alto livello deve essere trasformato prima di poter essere eseguito, tramite fasi successive di **memory binding**, che assegnano nuovi indirizzi a istruzioni e dati.



Le principali trasformazioni sono:

- **Compilazione o Assemblaggio**: un **traduttore** (compilatore o assembler) converte il programma in un **modulo oggetto** in linguaggio macchina. Durante questa fase vengono assegnati gli **indirizzi tradotti** (cioè gli indirizzi di memoria assegnati dal compilatore) a istruzioni e dati, a partire da un'**origine** specificata dall'utente o predefinita. Si definisce anche l'**indirizzo di avvio** dell'esecuzione.
- **Linking e Rilocazione**: Il **linker** unisce diversi **moduli oggetto** (come librerie) risolvendo i riferimenti esterni e assegnando **indirizzi linkati**. Durante il linking, se ci sono **conflitti** tra indirizzi dei moduli, il linker esegue la **rilocazione**, aggiornando gli indirizzi per evitare sovrapposizioni. Il risultato è un **programma binario**, memorizzato in una libreria.
- **Caricamento (Loader)**: il **loader** carica il **programma binario** in memoria. Se l'**origine di caricamento** (l'area di memoria dove caricare il programma) differisce dall'origine linkata del programma, il loader effettua un'ulteriore **rilocazione** degli indirizzi. Un **loader assoluto** gestisce questo processo automaticamente.

NB: Compilatori, linker e loader non fanno parte del Sistema Operativo.

Rilocazione

La **rilocazione** è il processo con cui si aggiornano gli **indirizzi di memoria** di un programma quando, al momento dell'esecuzione, il programma viene caricato in una posizione diversa da quella prevista.

Quando un programma viene compilato, gli vengono assegnati **indirizzi tradotti**, che dipendono **dall'origine specificata**. Tuttavia, se l'origine effettiva di caricamento è diversa, gli indirizzi che dipendono dalla posizione devono essere corretti, per garantire il corretto funzionamento del programma. Questa correzione degli indirizzi è appunto la **rilocazione**.

Gli indirizzi che devono essere modificati sono chiamati **address-sensitive**, perché il loro valore dipende direttamente dalla posizione del programma in memoria. Se il programma viene spostato, questi indirizzi devono essere aggiornati aggiungendo una **differenza** tra l'**origine di caricamento** e l'**origine tradotta**. Se, ad esempio, il programma era pensato per partire da 500 e viene invece caricato a 900, la differenza di 400 deve essere sommata agli indirizzi address-sensitive.

La rilocazione può essere effettuata in due modi. Si parla di **rilocazione statica** quando gli indirizzi vengono corretti **prima** dell'inizio dell'esecuzione del programma, direttamente durante il caricamento. In questo caso, il **loader** modifica fisicamente gli indirizzi nel programma prima che cominci a funzionare. Si parla invece di **rilocazione dinamica** quando la correzione degli indirizzi avviene **durante l'esecuzione**. In questo secondo caso, si utilizza un componente hardware speciale chiamato **registro di rilocazione**, che contiene il valore di scostamento tra il nuovo indirizzo di partenza e quello precedentemente linkato (es. indirizzo linkato: 50000; indirizzo nuovo: 70000; quindi

contenuto del registro di rilocalizzazione: $70000 - 50000 = 20000$). Questo valore viene poi automaticamente sommato ad ogni indirizzo address-sensitive durante l'esecuzione, cioè viene utilizzato come **offset**.

Il **registro di rilocalizzazione** è un registro della **CPU** che contiene il valore della differenza tra l'**origine di caricamento** e l'**origine tradotta**. Durante l'esecuzione di ogni istruzione, il contenuto del registro viene automaticamente **sommato** agli indirizzi utilizzati nel programma. In questo modo si ottiene l'**indirizzo effettivo** di memoria senza modificare fisicamente il codice del programma. La formula che riassume questo meccanismo è: **indirizzo effettivo = indirizzo dell'istruzione + contenuto del registro di rilocalizzazione**.

Linking

Il **linking** è il processo con cui un programma collega i **riferimenti esterni** (simboli definiti in altri moduli) agli **indirizzi corretti** prima o durante l'esecuzione. Nei programmi **assembly**, i simboli definiti all'interno del programma vengono indicati con l'istruzione **ENTRY**, mentre i simboli definiti all'esterno vengono dichiarati con **EXTRN**.

Durante il **linking**, il **linker** raccoglie tutte le informazioni sui simboli (entry point ed esterni) costruendo una **tabella** con i loro **indirizzi linkati**. Quando trova un riferimento esterno, sostituisce l'indirizzo mancante con quello corretto, permettendo al programma di accedere ai simboli esterni in modo corretto.

Nel caso del **linking statico**, questo collegamento avviene **prima** dell'esecuzione: tutti i moduli e riferimenti vengono risolti e il programma binario risultante non ha riferimenti esterni pendenti.

Nel **linking dinamico**, invece, il collegamento avviene **durante** l'esecuzione, quando il programma incontra un riferimento esterno non ancora risolto. Questo permette **maggiore flessibilità, condivisione della memoria e aggiornamenti delle librerie** senza dover ricompilare i programmi.

Allocazione della memoria a un processo

La memoria riservata a un processo è suddivisa in quattro aree principali: **codice**, **dati statici**, **stack** e **heap**. Il **codice** contiene le istruzioni del programma, mentre i **dati statici** includono variabili con dimensione fissa allocate in fase di compilazione.

Stack

Lo **stack** viene utilizzato per gestire automaticamente le **chiamate di funzione**. Ogni volta che una funzione viene chiamata, viene creato uno **stack frame** (o **activation record**) che contiene il valore del **frame base** precedente (serve per tornare al frame della funzione che ha chiamato), l'**indirizzo di ritorno** (dove riprendere l'esecuzione del codice una volta terminata la funzione), i **parametri** della funzione e i **dati locali**.

L'organizzazione dello stack segue il principio **last-in, first-out (LIFO)**: l'ultimo stack frame inserito è il primo a essere rimosso.

Lo stack è un'area di memoria contigua che cresce solitamente verso indirizzi più bassi. È gestito tramite tre puntatori:

- **Stack Base (SB)**: punta all'inizio fisso dello stack.
- **Top of Stack (TSO)**: punta all'ultimo elemento inserito nello stack.
- **Frame Base (FB)**: punta all'inizio del frame corrente, facilitando l'accesso ai dati della funzione attiva.

Durante le chiamate annidate di funzioni, ogni nuovo stack frame salva il valore del precedente **FB**, aggiornando anche **TSO**. Alla fine di ogni funzione, lo stack frame viene eliminato, ripristinando il **frame base precedente** e l'**indirizzo di ritorno**, permettendo al programma di continuare correttamente.

Le operazioni di gestione dello stack (**push** e **pop**) aggiornano i puntatori e gestiscono dinamicamente l'allocazione/deallocazione dei frame.

Heap

L'**heap** è l'area di memoria utilizzata per memorizzare i **dati dinamici** creati manualmente durante l'esecuzione del programma, tramite funzioni come **malloc** e **calloc** in C oppure **new** in C++ e Java. A differenza dello **stack**, che segue una gestione automatica **LIFO**, l'heap permette un'allocazione e deallocazione **casuale**, ed è il programmatore a dover gestire manualmente sia la creazione sia la liberazione della memoria.

Quando un processo richiede memoria dinamica, l'**allocatore dell'heap** riserva un'area e restituisce un **puntatore** che permette l'accesso a quella zona. Per liberare memoria, il programmatore deve chiamare una funzione apposita (come **free** in C), fornendo di nuovo il puntatore all'area da deallocare.

La memoria dell'heap cresce solitamente verso **indirizzi di memoria più alti**.

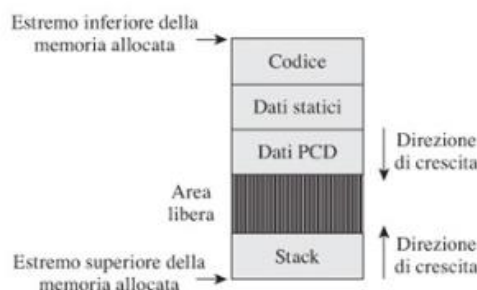
Una gestione tipica dell'heap può portare alla creazione di **buchi** nella memoria: questi si formano quando alcune aree vengono allocate e successivamente liberate, lasciando spazi vuoti che devono essere riutilizzati per future allocazioni. L'allocatore deve cercare di gestire e riutilizzare queste aree libere per evitare sprechi di memoria.

Lo **stack** garantisce gestione veloce e automatica delle chiamate di funzione, mentre lo **heap** offre maggiore flessibilità nella gestione dei dati dinamici, al costo di una maggiore complessità.

Il Modello di allocazione della memoria

Quando un utente avvia l'esecuzione di un programma, il **kernel** deve decidere quanta memoria assegnare a ciascun componente del processo: **codice**, **dati statici** e **dati dinamici controllati dal programma (PCD, ovvero heap e stack)**.

La **dimensione** del codice e dei dati statici può essere ottenuta dal file eseguibile, mentre le dimensioni dello **stack** e dei dati **PCD** possono variare durante l'esecuzione e non sono conosciute a priori. Per evitare sprechi di memoria o limiti rigidi sulle dimensioni massime di stack e heap, si adotta un **modello flessibile**.



In questo modello:

- Lo **stack** cresce da una estremità della memoria (normalmente dall'alto verso il basso).
- I dati **PCD** (heap) crescono dalla direzione opposta (dal basso verso l'alto).
- Tra stack e heap c'è un'**area libera** condivisa che viene occupata dinamicamente man mano che uno dei due componenti cresce.

Questo approccio permette al **kernel** di non dover prevedere rigidi limiti separati per stack e heap. L'area libera viene utilizzata da entrambi, a seconda delle necessità. Inoltre, la gestione dell'heap (creazione e distruzione dei dati PCD) è effettuata direttamente tramite **routine di libreria** del linguaggio di programmazione (come malloc, free, new, delete) e **non coinvolge direttamente il kernel**: è il programma a gestire l'allocazione e deallocazione dinamica della propria memoria.

Protezione della memoria

La **protezione della memoria** viene realizzata tramite due registri della **CPU**, chiamati **registro base** e **registro ampiezza**. Il **registro base** contiene l'**indirizzo di partenza** dell'area di memoria assegnata a un processo, mentre il **registro ampiezza** ne definisce la **dimensione massima**.

Durante l'esecuzione di un processo, l'**hardware di protezione** controlla che ogni indirizzo di memoria utilizzato rimanga all'interno dell'area consentita. Se un processo prova ad accedere a un indirizzo fuori dai limiti, viene generato un **interrupt di violazione di protezione della memoria**, e il **kernel** termina il processo.

I registri base e ampiezza fanno parte delle informazioni di **protezione della memoria (MPI)** contenute nella **program status word (PSW)**. Quando il kernel schedula un processo, carica i valori appropriati in questi registri. Inoltre, i **processi utente non possono modificare** questi registri: solo la CPU in modalità privilegiata può farlo, utilizzando istruzioni speciali.

Quando si usa un **registro di rilocalizzazione**, ogni programma viene scritto come se la sua memoria iniziasse sempre da **0**. In questo modo il programma utilizza solo **indirizzi virtuali**, che partono da 0 e vengono automaticamente spostati

nella giusta posizione della **memoria reale** grazie al registro di rilocalizzazione. Questo elimina il rischio che il programma acceda a indirizzi inferiori a 0.

La **protezione della memoria** diventa quindi più semplice ed efficiente: non serve più confrontare ogni indirizzo con un **registro base**, basta controllare che l'indirizzo richiesto sia minore del valore contenuto nel **registro ampiezza** (che indica la dimensione massima concessa al programma).

Insieme, **registro di rilocalizzazione** e **registro ampiezza** definiscono i limiti di memoria utilizzabile dal processo e permettono al processore di garantire una protezione veloce senza complicazioni.

Gestione dell'Heap

Riuso della memoria

La progettazione di un **allocatore di memoria** deve affrontare due problemi principali: garantire **velocità di allocazione** e assicurare **uso efficiente della memoria**.

L'allocazione basata su stack è molto efficiente per entrambi: allocare e deallocare memoria è velocissimo, perché basta aggiornare i puntatori **SB**, **FB** e **TSO**. Inoltre, la memoria liberata viene riutilizzata automaticamente per nuove allocazioni.

Tuttavia, l'allocazione su stack funziona bene solo quando l'ordine di allocazione e rilascio dei dati è ordinato (last-in, first-out). Non è adatta per dati che vengono allocati e rilasciati in ordine casuale.

Per gestire dati dinamici (**PCD**) e richieste di memoria del **kernel**, si utilizza invece l'**heap**, dove il **riuso della memoria non è automatico**. L'**allocatore dell'heap** deve cercare di riutilizzare le aree liberate, ma questo è complicato perché:

- La dimensione richiesta raramente corrisponde alla dimensione delle aree libere disponibili.
- Aree troppo piccole per soddisfare nuove richieste diventano **sprecate**.
- L'allocatore deve **selezionare con cura** le aree libere da riutilizzare, rallentando l'allocazione.

Inoltre, parte della memoria viene usata dalle stesse **strutture di gestione dell'heap**, riducendo ulteriormente l'efficienza. Di conseguenza, un **allocatore di heap** difficilmente può garantire la stessa **alta efficienza** che si ottiene con la gestione a stack.

Gestione con free list

Per garantire il **riuso efficiente** della memoria, il **kernel** utilizza una struttura chiamata **free list**, che contiene tutte le informazioni relative alle **aree di memoria libera** nel sistema. Quando un processo libera memoria, ad esempio rilasciando strutture dati o terminando la propria esecuzione, le informazioni sull'area liberata vengono inserite nella **free list**.

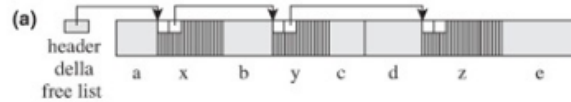
Quando arriva una nuova richiesta di memoria, il **kernel** seleziona dalla **free list** l'**area di memoria più adatta** per soddisfare la richiesta. Se una parte dell'area scelta rimane inutilizzata, quella parte viene nuovamente inserita nella free list, così da poter essere riutilizzata in futuro.

Per migliorare ulteriormente l'efficienza, il **kernel** prova anche a **unire** aree di memoria libere contigue, formando così **blocchi di memoria più grandi**. Questa strategia permette di ridurre la frammentazione e di soddisfare più facilmente richieste che necessitano di grandi quantità di memoria.

Le principali operazioni che il kernel svolge sono dunque **mantenere aggiornata la free list**, **selezionare l'area libera più adatta** per ogni nuova allocazione e **unire** le aree contigue per ottimizzare lo spazio disponibile.

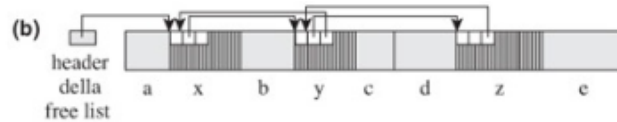
Mantenere una free list

Il **Kernel** deve mantenere, per ogni area di memoria libera, due **informazioni di controllo**: la **dimensione** dell'area e i **puntatori** utilizzati per costruire la free list. Per ridurre l'**overhead di memoria** dovuto a questi dati di controllo, il kernel li memorizza nei primi byte di ogni area libera.



La figura (a) mostra una **free list a singolo puntatore**, dove ogni area di memoria libera contiene la sua dimensione e un puntatore alla prossima area nella lista. Questa organizzazione è semplice, ma comporta più lavoro quando si deve inserire o cancellare un'area dalla lista. Per esempio, cancellare un'area di memoria richiede di aggiornare il puntatore dell'area precedente, mentre inserire una nuova area in una posizione specifica implica lo scorrimento della lista partendo dalla testa.

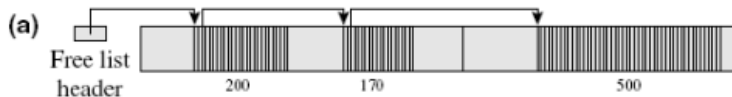
La **complessità** di queste operazioni è proporzionale a m , dove m è il numero di aree di memoria presenti nella lista.



Per rendere più efficienti le operazioni di inserimento e cancellazione, si può adottare una **free list a doppio puntatore**. In questo caso, ogni elemento della lista contiene due puntatori: uno alla prossima area di memoria e uno all'area precedente. Se si deve cancellare un'area di memoria, il kernel può aggiornare direttamente i puntatori delle aree precedente e successiva, senza dover scorrere tutta la lista. Questo riduce la complessità, rendendo il costo delle operazioni **costante**, indipendentemente dal numero di aree nella lista.

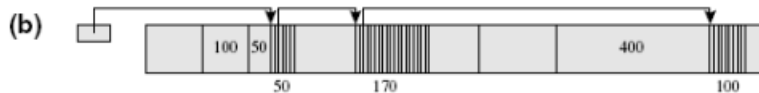
Tecniche di allocazione mediante free list

Per allocare memoria utilizzando una **free list**, possono essere adottate **tre principali tecniche**: **first-fit**, **best-fit** e **next-fit**.



L'esempio considera 100, 50 e 400 byte da allocare.

La **tecnica first-fit (b)** serve una richiesta di memoria cercando la **prima area libera** con dimensione **maggiore o uguale** ai byte richiesti. Una volta trovata, l'area viene **divisa** in due parti: una parte di dimensione esattamente pari alla richiesta viene allocata, mentre l'eventuale spazio rimanente viene **reinserito nella free list**. Questa tecnica è veloce, ma nel tempo porta alla **frammentazione** della memoria, perché le aree libere all'inizio della lista diventano sempre più piccole, rendendo difficile soddisfare richieste di grandi dimensioni.

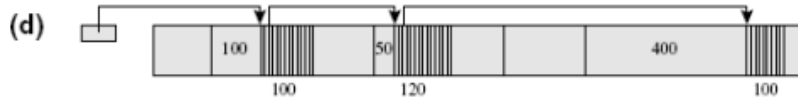


La **tecnica best-fit (c)** seleziona invece l'**area libera più piccola** che sia comunque sufficiente a soddisfare la richiesta. In questo modo cerca di **evitare di frammentare** grandi blocchi di memoria, ma tende comunque a creare **molte piccole aree residue** che, nel tempo, possono diventare inutilizzabili. Inoltre, questa tecnica genera **maggior overhead** perché richiede di **scorrere tutta la free list** oppure di **mantenere la lista ordinata per dimensione**.



La **tecnica next-fit (d)** si comporta in modo simile alla first-fit, ma con una differenza importante: **memorizza la posizione** dell'ultimo elemento utilizzato nella free list.

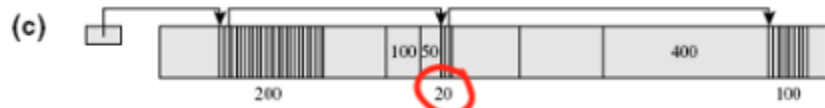
Quando arriva una nuova richiesta, la ricerca **riparte dall'elemento successivo** e non dalla testa della lista. In questo modo si **evita la divisione ripetuta** delle stesse aree di memoria, come accade con la first-fit, e si riduce l'**overhead** rispetto alla best-fit.



Frammentazione della memoria

La **frammentazione della memoria** è la presenza di **aree di memoria inutilizzabili** all'interno della memoria di un computer. Esistono due principali forme di frammentazione: **frammentazione esterna** e **frammentazione interna**.

La **frammentazione esterna** si verifica quando un'area di memoria rimane inutilizzata perché è troppo piccola per poter soddisfare una futura richiesta di allocazione. Un esempio tipico si osserva con la **tecnica di allocazione best-fit**, che può lasciare piccoli spazi liberi, come mostrato in figura.



Dopo un'allocazione rimane un'area libera di 20 byte, troppo piccola per essere utilizzata. Questo spazio risulta sprecato, causando inefficienza.

La **frammentazione interna** invece si verifica quando viene allocata **più memoria** rispetto a quella effettivamente richiesta dal processo. Questo accade, ad esempio, se un allocatore assegna 100 byte anche quando un processo ne richiederebbe solo 50. Il risultato è che una parte della memoria allocata resta **inutilizzata**. Questo problema si presenta quando un sistema usa blocchi di memoria di dimensione fissa per ridurre l'overhead di gestione.

Unione delle aree di memoria libera

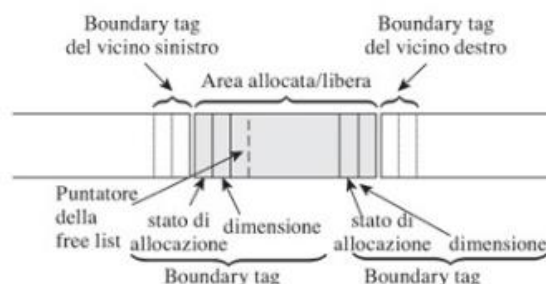
La **frammentazione esterna** può essere ridotta unendo più **aree di memoria libera** per formare **blocchi più grandi**. L'**unione** può essere tentata ogni volta che una nuova area di memoria viene **aggiunta alla free list**.

Un metodo semplice consiste nello **scorrere la free list** per verificare se esiste **un'area contigua** già presente. Se viene trovata, l'area nuova e quella contigua possono essere **unite** per creare una **singola area più grande**. Questo processo può essere ripetuto finché non ci sono più aree contigue da unire, e alla fine l'area risultante viene reinserita nella free list.

Tuttavia, questa tecnica è **costosa**, perché richiede di **cercare** nella free list ogni volta che viene liberata una nuova area di memoria. Ci sono **due** tecniche generali per **risolvere la frammentazione esterna**; quindi, unendo aree di memoria libere e sono il **Boundary tag** e la **Compattazione della memoria**.

Boundary Tag

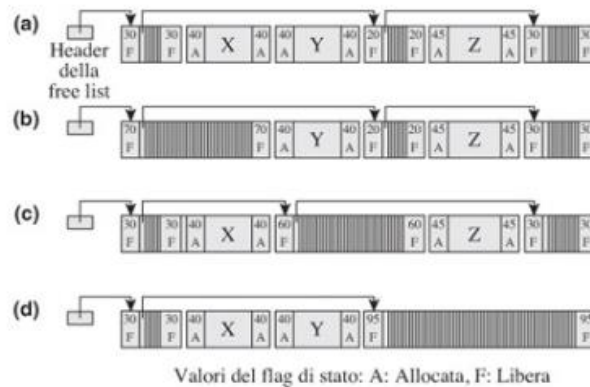
Un **boundary tag** è un **descrittore di stato** associato a un'area di memoria. Ogni area ha due boundary tag identici, uno all'inizio e uno alla fine, che indicano lo **stato di allocazione** (libera o occupata) e la sua **dimensione**. Se l'area è libera, il **puntatore della free list** può essere inserito dopo il boundary tag iniziale, come mostrato di seguito.



Quando **un'area di memoria** viene liberata, il **kernel** controlla i **boundary tag** delle aree vicine. Se una delle aree adiacenti è libera, viene **unita** con l'area appena rilasciata. Questo rende molto più semplice trovare aree contigue da unire, poiché i tag si trovano subito prima e subito dopo l'area.

Esempio

Di seguito è mostrato come vengono gestite tre aree X, Y, Z. Quando X viene liberata e il suo vicino di sinistra è libero, vengono unite. Poi Y viene liberata, e si unisce al suo vicino di destra. Infine, quando Z viene liberata e ha entrambi i vicini liberi, si crea un'unica grande area libera.



Questa tecnica permette un'unione efficiente delle aree libere, specialmente se la **free list** è mantenuta con una **lista a doppio puntatore**, che facilita aggiornamenti rapidi.

Quando si gestisce la memoria usando i **boundary tag**, si applica una relazione chiamata **regola del 50%**, che serve a stimare il numero medio di aree di memoria libera in un sistema. Questa regola si basa sul comportamento tipico della memoria durante le operazioni di **rilascio** e **unione** di aree.

Nel momento in cui con la tecnica del boundary tag si libera un blocco di memoria, nella lista dei blocchi liberi quello che succede è che la lista aumenta di 1 elemento o si decrementa di 1 elemento, oppure rimane invariata a seconda che il blocco che si va a liberare ha 0 (caso C), 1 (caso B), o 2 (caso A) vicini.

Esempio:



Number of allocated areas, $n = \#A + \#B + \#C$

Number of free areas, $m = \frac{1}{2}(2 \times \#A + \#B)$

In the steady state $\#A = \#C$. Hence $m = n/2$

Nel tempo, si osserva che in un sistema stabile il numero medio di blocchi liberati di tipo **A** tende a **essere uguale** al numero di quelli di tipo **C**, cioè a quelli che aumentano il numero complessivo di aree libere. Questo equilibrio statistico porta alla conclusione che, mediamente, **il numero di blocchi liberi m è circa la metà del numero totale di blocchi allocati, ovvero $m = n/2$** .

Questa regola si rivela molto utile anche per stimare la **quantità di memoria libera complessiva**. Se ogni area libera ha una dimensione media indicata con **sf**, e ci sono approssimativamente **$n/2$** blocchi liberi, allora la memoria totale disponibile sarà stimata come **$sf \times n/2$** . Questa informazione è particolarmente preziosa per algoritmi di allocazione come il **best-fit**, che necessitano di conoscere la dimensione media delle aree disponibili per funzionare in modo efficace.

Compattazione della memoria

La **compattazione della memoria** è una tecnica in cui i **memory binding** vengono modificati in modo da **unire tutte le aree di memoria libera** in un'unica grande area contigua. La compactazione della memoria serve a **risolvere la frammentazione esterna**, che si verifica quando la memoria ha abbastanza spazio libero totale, ma è spezzettata in tante aree piccole non contigue, che non possono essere usate per allocazioni di dimensioni maggiori. Come suggerisce il nome, la compactazione consiste nell' "**impacchettare**" tutte le aree di memoria occupate verso un'estremità della memoria, liberando l'altra estremità.

Tuttavia, la **compactazione** non è così semplice da realizzare come potrebbe sembrare. Infatti, richiede di **spostare fisicamente** codice e dati in memoria. Se, ad esempio, l'area b contiene un processo attivo, è necessario **rilocare** tale processo affinché possa continuare a funzionare correttamente nella nuova posizione di memoria.



La **rilocalizzazione** implica la **modifica di tutti gli indirizzi** utilizzati dal processo: questo include non solo gli indirizzi dei dati allocati sull'**heap**, ma anche gli **indirizzi nei registri** general purpose della CPU.

La **compattazione della memoria** diventa praticabile solo se il computer supporta l'uso di un **registro di rilocalizzazione**, perché in questo caso il processo lavora sempre con **indirizzi virtuali** che partono da 0, mentre l'indirizzo reale viene calcolato automaticamente come *indirizzo virtuale + valore del registro di rilocalizzazione*. Quando un processo viene spostato in memoria, non è necessario modificare manualmente tutti gli indirizzi presenti nel codice: basta semplicemente aggiornare il valore del **registro di rilocalizzazione**, e tutti gli indirizzi vengono adattati automaticamente alla nuova posizione di partenza.

Buddy System e allocatori potenza del 2

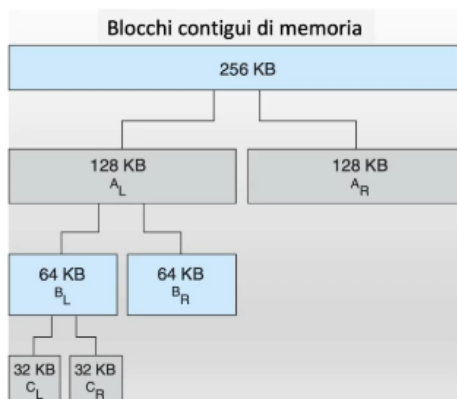
L'allocazione di memoria nell'**heap** può essere gestita con il **Buddy System** o con gli **allocatori a potenze di 2**. Entrambi suddividono la memoria in blocchi di dimensione fissa, in genere potenze di 2, e organizzano questi blocchi in **free list** separate per ciascuna dimensione. Questo consente di rendere l'**allocazione** e la **deallocazione** molto veloci, ma introduce il problema della **frammentazione interna**, perché ogni richiesta viene soddisfatta con un intero blocco, anche se in parte inutilizzato.

Nel **Buddy System**, un blocco troppo grande viene suddiviso ricorsivamente in due parti uguali, chiamate **buddy**, fino a ottenere la potenza di 2 più piccola che possa contenere la richiesta. Ogni livello della suddivisione è associato a una propria free list. Quando un blocco viene liberato, il sistema controlla se il suo buddy è anch'esso libero: in tal caso i due vengono fusi tramite **coalescenza**, ricostituendo il blocco originario, e questa operazione può ripetersi risalendo di livello. Se invece il buddy non è libero, il blocco viene semplicemente reinserto nella free list della sua dimensione. In questo modo il sistema riduce la frammentazione e mantiene una struttura ordinata.

Gli **allocatori a potenze di 2** adottano una logica simile, ma si limitano a gestire blocchi di dimensioni 2^k senza eseguire la fusione dei blocchi adiacenti al momento della deallocazione. Ogni blocco ha un **header** che indica la free list di appartenenza e, una volta liberato, viene semplicemente reinserto nella lista corrispondente. Questo approccio è ancora più veloce del buddy system, ma meno efficiente nell'uso della memoria perché non sfrutta la coalescenza.

Rispetto agli algoritmi tradizionali come **first-fit**, **best-fit** e **next-fit**, che soffrono di **frammentazione esterna** e richiedono di scorrere le liste per cercare un blocco adatto, buddy system e allocatori a potenze di 2 sono molto più rapidi. In cambio, accettano un po' di **frammentazione interna**, caratteristica che li rende adatti in contesti dove la **velocità** è prioritaria rispetto al massimo sfruttamento della memoria.

Esempio Buddy System



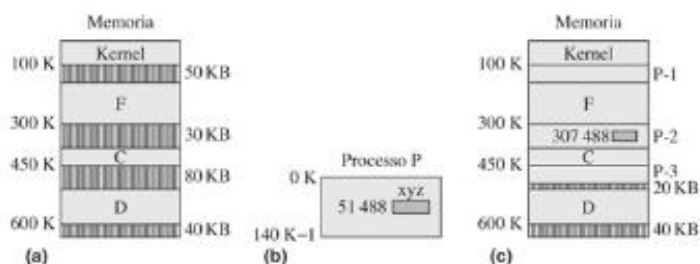
Allocazione contigua della memoria

L'**allocazione contigua della memoria** è un modello in cui a ciascun processo viene assegnata **un'unica area continua di memoria**, comprendente codice, dati, stack e PCD. Questo approccio semplifica la gestione ma può causare problemi di **frammentazione**, specialmente quella esterna. La **frammentazione interna** non può essere evitata, poiché il kernel non riesce a stimare con precisione le richieste di memoria di un processo. Per affrontare la frammentazione esterna si utilizza la **compattazione**, che consiste nello spostare i processi presenti in memoria per creare un'unica area libera abbastanza grande da contenere un nuovo processo. Questo meccanismo richiede però la **rilocalizzazione dinamica**, possibile solo se il sistema dispone di un **registro di rilocalizzazione**. Senza di esso, il kernel può soltanto riutilizzare aree già libere, con il rischio di rallentare l'avvio dei processi quando non ci sono spazi contigui sufficienti. In assenza di memoria contigua, anche se la memoria totale è sufficiente, un nuovo processo può subire ritardi nell'avvio. Un'altra tecnica legata alla gestione della memoria è lo **swapping**, che prevede lo spostamento su disco di un processo non in esecuzione, in un'apposita **area di swap**. Il processo viene ricaricato in RAM quando gli viene assegnato un burst di CPU. Tuttavia, se la memoria che occupava in precedenza è stata nel frattempo assegnata ad altri, sarà necessario rilocalarlo in una nuova area, operazione possibile solo con la **rilocalizzazione dinamica**.

Pertanto, sia la compactazione che lo swapping efficiente dipendono dalla presenza di un sistema capace di gestire dinamicamente la rilocalizzazione dei processi in memoria.

Allocazione non contigua della memoria

L'**allocazione non contigua della memoria** è una tecnica utilizzata dai moderni sistemi operativi in cui le varie parti di un processo non devono risiedere in un'unica area continua di memoria, ma possono essere distribuite in più blocchi sparsi. In questo modo, il **kernel** può riutilizzare spazi di memoria più piccoli e frammentati, riducendo il problema della **frammentazione esterna**. Il processo funziona correttamente grazie alla gestione dello **spazio di indirizzamento**, che mappa logicamente le parti del processo anche se fisicamente separate. L'allocazione non contigua consente quindi una maggiore efficienza nell'uso della memoria disponibile, soprattutto in presenza di frammentazione.



Nell'esempio mostrato, un processo P viene caricato in tre blocchi distinti di memoria (50KB, 30KB e 60KB), distribuiti tra aree libere non contigue, dimostrando come il sistema possa allocare dinamicamente i componenti del processo (P-1, P-2, P-3) in indirizzi fisici diversi. Questo tipo di gestione si basa sul concetto di **segmentazione o paginazione**, che permette di suddividere il processo in più segmenti o pagine, ognuna delle quali può essere caricata indipendentemente in una qualsiasi area libera della memoria fisica, evitando l'obbligo di uno spazio contiguo sufficientemente grande.

Indirizzo Logico, Fisico e Traduzione degli Indirizzi

Nell'**allocazione non contigua della memoria**, le parti di un processo non sono salvate tutte di seguito nella RAM, ma possono essere sparse in blocchi separati. Per gestire tutto questo, si fa una distinzione tra **indirizzi logici** e **indirizzi fisici**.

Un **indirizzo logico** è l'indirizzo usato dal programma. È un numero che indica dove si trova un'istruzione o un dato all'interno del processo, **senza sapere dove si trovi davvero in memoria**. L'insieme di tutti questi indirizzi forma lo **spazio di indirizzamento logico** del processo.

Un **indirizzo fisico**, invece, è l'indirizzo vero e proprio in **RAM** dove si trova quel dato o istruzione. L'insieme degli indirizzi fisici disponibili in RAM forma lo **spazio di indirizzamento fisico**.

Poiché le varie componenti di un processo (come P-1, P-2, P-3...) sono messe in punti diversi della memoria, serve una traduzione tra indirizzi logici e fisici. Questo lavoro lo fa la **MMU (Memory Management Unit)**.

Come funziona la traduzione dell'indirizzo

La CPU invia alla MMU un **indirizzo logico**, che è composto da due parti:

- $comp_i \rightarrow$ indica **quale componente** del processo (es. P-1, P-2, ecc.)
- $byte_i \rightarrow$ indica **il numero del byte** dentro quella componente.

Quindi ogni indirizzo logico è una coppia scritta così: $(comp_i, byte_i)$.

La MMU ha accesso a una **tabella** (gestita dal **kernel**) che dice dove è stata caricata ogni componente in memoria. Per calcolare l'indirizzo fisico, la MMU somma:

$$\text{indirizzo fisico} = \text{indirizzo di inizio della componente} + \text{byte interno alla componente}$$

Esempio pratico

Il processo P ha tre componenti:

- P-1 ha dimensione 50KB (cioè 51200 byte) ed è caricato dall'indirizzo 100K
- P-2 da 300K $\rightarrow$ cioè **307200**
- P-3 da 450K

L'istruzione vuole accedere al dato xyz che si trova al **byte 51488**.

Poiché P-1 ha dimensione 50KB (cioè 51200 e occupa da 0 a 51.199), xyz **non ci sta in P-1**.

Quindi si trova nella **seconda componente, P-2**.

Per sapere dove si trova all'interno di P-2 bisogna calcolare l'offset; quindi, bisogna sottrarre la dimensione di P-1: **51488 - 51200 = 288**.

Ciò significa che il byte che stiamo cercando è il byte numero 288 dentro P-2.

Quindi l'indirizzo logico è: **(P-2, 288)**.

La MMU guarda la tabella e sa che **P-2 è caricato a 307200**, quindi:

$$\text{indirizzo fisico} = \text{indirizzo di inizio di P} - 2 + \text{offset}$$

Quindi,

$$\text{indirizzo fisico} = 307200 + 288 = 307488$$

Il byte logico 51488 del processo P si trova all'indirizzo fisico 307488 che è quello reale in RAM dove si trova il dato.

Paginazione

La **paginazione** è una tecnica di gestione della memoria che divide lo **spazio degli indirizzi logici** di un processo in **pagine** tutte della stessa dimensione.

Ogni pagina ha s byte, dove s è una potenza di 2 (per esempio 1024 byte).

I processi lavorano sempre con **indirizzi logici** (numeri), che vengono suddivisi dalla **MMU (Memory Management Unit)** in due parti:

- p_i : il numero di pagina,
- b_i : l'offset, cioè il numero del byte all'interno della pagina.

In pratica, un indirizzo logico è la coppia (p_i, b_i) , con $0 \leq b_i < s$.

Le pagine logiche non corrispondono direttamente a blocchi contigui di RAM: possono essere collocate in **frame fisici** sparsi.

La memoria fisica è suddivisa in **frame**, anch'essi della stessa dimensione di una pagina.

Quando il kernel avvia un processo:

- assegna a ogni pagina logica un **frame libero** scelto dalla **lista dei frame liberi**,
- e aggiorna la **tabella delle pagine** (ogni processo ne ha una), che registra in quale frame è stata caricata ciascuna pagina.

Quando la MMU riceve un indirizzo logico (p_i, b_i) , consulta la tabella delle pagine, trova il frame q_i in cui si trova la pagina p_i , e calcola l'**indirizzo fisico** sommando l'indirizzo iniziale del frame con l'offset b_i .

Esempio

Supponiamo che un processo abbia dimensione **5500 byte** e che le pagine siano di **1 KB (1024 byte)**. Il processo richiede quindi **6 pagine**.

Se vogliamo accedere all'indirizzo logico **5248**:

- $p_i = 5248 / 1024 = 5$,
- $b_i = 5248 \bmod 1024 = 128$,
→ quindi l'indirizzo logico è (5, 128).

Se la quinta pagina ($p_i = 5$) è stata caricata nel frame 8, allora l'indirizzo fisico è:

$$8320 = (8 \times 1024) + 128$$

Vantaggi della paginazione

La **paginazione** introduce numerosi vantaggi rispetto all'allocazione contigua. Anzitutto, consente l'**eliminazione della frammentazione esterna**, poiché le pagine di un processo possono essere collocate in frame non adiacenti senza che i "buchi" liberi della RAM vadano sprecati. Inoltre, rende possibile la **memoria virtuale**, dato che non è necessario che l'intero processo risieda in RAM: solo le pagine effettivamente necessarie vengono caricate, mentre le altre restano su disco, permettendo di eseguire programmi più grandi della memoria fisica disponibile. Un ulteriore beneficio è la **protezione e l'isolamento**: ciascun processo dispone della propria tabella delle pagine, evitando accessi non autorizzati ad altre aree di memoria e aumentando la sicurezza complessiva del sistema. La paginazione favorisce anche la **condivisione della memoria**, ad esempio per librerie comuni utilizzate da più processi, riducendo gli sprechi. Infine, garantisce una **gestione flessibile**, in quanto il kernel può sostituire dinamicamente una pagina con un'altra grazie ai meccanismi di page replacement.

NB: Il vero vantaggio di dividere un processo in **pagine** e mapparle su **frame** è che così si elimina la **necessità di avere blocchi contigui di memoria fisica**, evitando la **frammentazione esterna**, e diventa possibile usare la **memoria virtuale**, caricando in RAM solo le pagine necessarie mentre le altre restano su disco.

Svantaggi della paginazione

Il principale svantaggio è la **frammentazione interna**: ogni pagina ha una dimensione fissa (ad esempio 4 KB) e, se un processo non utilizza interamente l'ultima pagina assegnata, lo spazio residuo rimane inutilizzato. Questo spreco cresce al crescere della dimensione delle pagine, perché aumenta la quantità media di spazio vuoto all'interno di esse.

Un secondo problema è l'**overhead di memoria**, legato alle **tabelle delle pagine**. Ogni processo ne possiede una propria e, più ampio è lo spazio di indirizzi logici, maggiore sarà la memoria necessaria per conservarle. Nei sistemi a 64 bit, questo costo può diventare particolarmente rilevante.

La paginazione introduce anche un **maggior tempo di accesso**: la traduzione di un indirizzo logico in indirizzo fisico richiede un passaggio aggiuntivo. Per ridurre questo ritardo si impiegano strutture cache dedicate come la **TLB (Translation Lookaside Buffer)**, ma in caso di mancato hit l'accesso alla memoria risulta sensibilmente più lento.

Un ulteriore svantaggio è la **maggior complessità di gestione**: il sistema operativo deve mantenere costantemente aggiornata la lista dei frame liberi, le tabelle delle pagine e, nei sistemi con memoria virtuale, deve occuparsi anche dei **page fault** e delle politiche di **page replacement**, aumentando la complessità complessiva.

Infine, vi è il rischio di **page fault e swapping**. Quando una pagina richiesta non è presente in RAM, deve essere caricata dal disco, operazione molto più lenta. Un numero eccessivo di page fault può portare al fenomeno del **thrashing**, in cui il sistema impiega la maggior parte del tempo a spostare pagine tra RAM e disco anziché eseguire i processi.

Notazione per la traduzione degli indirizzi:

s dimensione di una pagina

l_l lunghezza di un indirizzo logico (cioè, numero di bit)

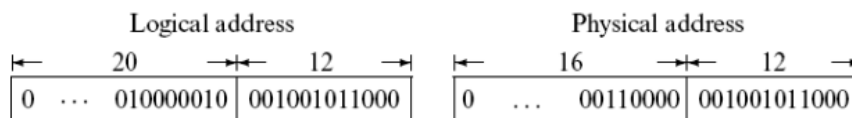
l_p lunghezza di un indirizzo fisico

n_b numero di bit usato per rappresentare il numero del byte in un indirizzo logico

n_p numero di bit usato per rappresentare il numero di pagina in un indirizzo logico

n_f numero di bit usato per rappresentare il numero di frame in un indirizzo fisico

Dal punto di vista della **rappresentazione binaria**, la traduzione degli indirizzi si basa su una scomposizione in bit: gli indirizzi logici e fisici hanno una lunghezza l_l e l_p , mentre n_b è il numero di bit necessario per rappresentare un byte (in un indirizzo logico) all'interno della pagina (dato che $s = 2^{n_b}$). I rimanenti $n_p = l_l - n_b$ bit identificano il numero di pagina. Allo stesso modo, per l'indirizzo fisico, si usano $n_f = l_p - n_b$ bit per il frame e gli n_b bit finali per il byte interno.



Ad esempio, in un sistema con indirizzi logici di 32 bit e pagine di 4KB, i 12 bit meno significativi rappresentano b_i , mentre i 20 più significativi rappresentano p_i . In una memoria fisica di 256MB, servono $l_p = 28 \text{ bit}$ per rappresentare un indirizzo fisico. Se la pagina $p_i = 130$ è associata al frame $q_i = 48$ e $b_i = 600$, la MMU concatena i 16 bit di q_i con i 12 bit di b_i per ottenere l'indirizzo fisico.

Segmentazione

La **segmentazione** è una tecnica di **gestione della memoria** che suddivide lo **spazio di indirizzamento** di un processo in entità logiche distinte, chiamate **segmenti**, che possono rappresentare **funzioni**, **strutture dati**, **moduli** o lo **stack**. Trattare queste parti come unità autonome permette di **caricarle**, **proteggerle** o **condividerle** in modo indipendente. Dal punto di **vista logico** un processo appare quindi composto da segmenti separati, mentre dal punto di **vista fisico** questi segmenti possono essere collocati in aree di memoria **non contigue**, rendendo più flessibile la condivisione e l'uso dinamico della memoria.

Uno dei principali vantaggi della segmentazione è che consente di **eliminare la frammentazione interna**, poiché ogni segmento occupa esattamente lo spazio necessario. Tuttavia, può introdurre **frammentazione esterna**, perché la memoria viene occupata da blocchi di dimensione variabile che possono lasciare spazi vuoti tra un segmento e l'altro.

Per ogni processo, il **kernel** mantiene una **tabella dei segmenti**, che associa a ciascun segmento il suo **nome** o **indice**, la sua **dimensione** in byte e l'**indirizzo fisico di partenza** nell'area di memoria. Questa tabella viene utilizzata dalla **MMU (Memory Management Unit)** per eseguire la traduzione degli indirizzi logici e per applicare i meccanismi di **protezione**, evitando accessi non consentiti.

L'**indirizzo logico** utilizzato da un programma ha la forma (s_i, b_i) , dove s_i è il numero del segmento e b_i è il byte all'interno del segmento, cioè l'offset. Quando il processo accede a una variabile o a un'istruzione, la **MMU** traduce questo indirizzo logico in un **indirizzo fisico** sommando l'**indirizzo base del segmento** e l'**offset**. A differenza della **paginazione**, dove la traduzione avviene per mezzo della **scomposizione binaria** dell'indirizzo, nella segmentazione si utilizza una semplice **somma aritmetica**.

Ad esempio, se l'istruzione `get_sample` si trova al **byte 232** del segmento `update`, e se questo segmento ha un **indirizzo iniziale** pari a **608**, l'indirizzo logico $(\text{update}, 232)$ viene tradotto in un **indirizzo fisico** pari a **840**.

L'**allocazione della memoria** per i segmenti può avvenire secondo **strategie** come il **first-fit** o il **best-fit**, che cercano di collocare ogni segmento nel primo o nel più adatto spazio disponibile. Tuttavia, questo approccio **può causare frammentazione esterna**, specialmente dopo molte operazioni di allocazione e rilascio. Per gestire questo problema, alcuni sistemi operativi impiegano tecniche di **compattazione**, spostando segmenti nella memoria per rendere lo spazio disponibile più continuo e sfruttabile.

La segmentazione, quindi, rappresenta un modello di gestione della memoria che **rispecchia più da vicino la struttura logica dei programmi**, offrendo maggiore **modularità**, **protezione** e **flessibilità**, pur introducendo sfide legate all'allocazione e alla frammentazione.

Vantaggi della segmentazione

La **segmentazione** presenta diversi vantaggi. Innanzitutto, rispecchia in modo naturale la **logica del programma**, poiché i segmenti corrispondono a funzioni, moduli, strutture dati o allo **stack**, risultando quindi più intuitiva per il programmatore. Un altro punto di forza è l'eliminazione della **frammentazione interna**, dato che ogni segmento occupa esattamente lo spazio necessario senza sprechi all'interno del blocco. La segmentazione facilita anche la **protezione**, perché la MMU può bloccare facilmente gli accessi che superano i limiti di un segmento. Inoltre, consente la **condivisione** di memoria, poiché segmenti come librerie comuni possono essere utilizzati da più processi, e favorisce la **crescita indipendente** delle aree di memoria, come **stack** e **heap**, che possono espandersi separatamente. Infine, garantisce **modularità e flessibilità**, dal momento che i segmenti possono essere caricati, rimossi o ricollocati in modo indipendente.

Svantaggi della segmentazione

Accanto a questi vantaggi, la segmentazione comporta anche alcuni svantaggi. Il più rilevante è la **frammentazione esterna**, dovuta al fatto che i segmenti hanno dimensione variabile e col tempo creano buchi di memoria non riutilizzabili. La gestione risulta inoltre **più complessa**, poiché richiede strategie di allocazione come **first-fit** o **best-fit** e il kernel deve mantenere costantemente aggiornata la tabella dei segmenti. A volte diventa necessaria la **compattazione** della memoria, un'operazione costosa che serve a ridurre la frammentazione spostando i segmenti. Anche l'**allocazione di segmenti grandi** può essere problematica: pur avendo RAM libera sufficiente, potrebbe non esserci un blocco contiguo di dimensioni adeguate. Infine, l'**accesso alla memoria** risulta meno efficiente rispetto alla paginazione, perché le dimensioni variabili dei segmenti rendono la gestione hardware più complicata.

Segmentazione con paginazione

La **segmentazione con paginazione** è una tecnica **ibrida** che unisce i **vantaggi** della **segmentazione**, che organizza logicamente un programma in moduli come funzioni o strutture dati, e della **paginazione**, che suddivide la memoria fisica in blocchi fissi (pagine). In questo modello, ogni **segmento** del processo viene **paginato separatamente**, ossia il suo contenuto viene suddiviso in pagine, anziché occupare un'area contigua di memoria.

Questa soluzione consente di eliminare la **frammentazione esterna**, poiché i segmenti non devono più essere memorizzati in blocchi consecutivi. Tuttavia, può introdurre **frammentazione interna**, limitata all'**ultima pagina di ciascun segmento**. In pratica, se un segmento non occupa interamente la sua ultima pagina, lo spazio rimanente in quella pagina va sprecato (per esempio, segmento di 10 KB con pagine da 4 KB → servono 3 pagine, ma l'ultima avrà 2 KB occupati e 2 KB sprecati). L'effetto è quindi molto più contenuto rispetto alla segmentazione pura: si elimina completamente la frammentazione esterna, ma rimane una piccola frammentazione interna dovuta solo alle ultime pagine dei segmenti.

La gestione si basa su due strutture: la **tabella dei segmenti**, che per ogni segmento memorizza l'**indice intero**, la **dimensione** e l'**indirizzo della tabella delle pagine associata**, e la **tabella delle pagine**, che contiene il mapping tra le pagine logiche del segmento e i frame fisici.

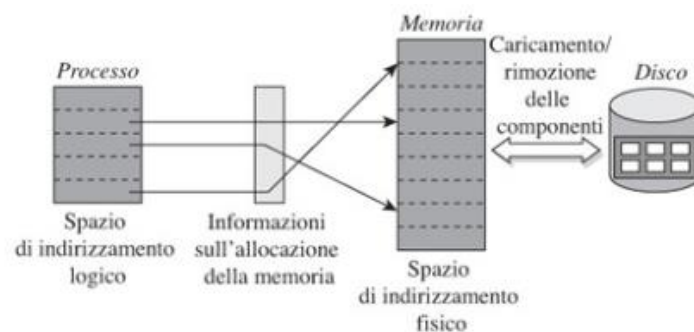
La **traduzione di un indirizzo logico** (s_i, b_i), avviene in due passaggi. Prima, si individua il **segmento** tramite l'indice s_i e si recupera l'indirizzo della sua tabella delle pagine. Poi, l'offset b_i viene scomposto in due parti: il **numero di pagina** all'interno del segmento e il **byte all'interno della pagina**. Infine, si accede alla tabella delle pagine, si recupera il numero del frame e si **calcola l'indirizzo fisico** concatenando il frame con l'offset. Questo meccanismo è lo stesso della paginazione classica.

La **segmentazione con paginazione** consente una **gestione flessibile e modulare della memoria**, facilitando la protezione, la condivisione e l'efficienza nell'uso dello spazio fisico.

Capitolo 10 – Memoria Virtuale

Introduzione

La **memoria virtuale** è una tecnica fondamentale nella gestione moderna delle risorse di calcolo, in quanto permette di creare l'**illusione** di una memoria più ampia rispetto a quella fisicamente disponibile. Essa si basa sull'integrazione tra **memoria principale** (RAM) e **memoria secondaria** (tipicamente un disco rigido), consentendo a un processo di eseguire anche se solo una parte del suo **spazio di indirizzamento** – ovvero **codice e dati** – risiede effettivamente in memoria in un dato momento. Questa illusione, percepita dall'**utente** e dalle **applicazioni** come uno spazio di memoria continuo e unificato, è il risultato di un complesso meccanismo di gestione interno al sistema operativo.



L'implementazione della memoria virtuale si fonda su un modello di **allocazione non contigua**, che consente di caricare in memoria solo le **componenti necessarie** di un processo, riducendo così la **frammentazione** e migliorando l'utilizzo della memoria fisica. Tale modello è gestito congiuntamente da una componente **software**, il **virtual memory manager**, e da una componente **hardware**, la **Memory Management Unit (MMU)**. Quest'ultima si occupa di accelerare la **traduzione degli indirizzi logici in indirizzi fisici**, mentre il gestore della memoria virtuale decide quali porzioni del processo devono essere mantenute in memoria.

Il **caricamento avviene su richiesta**, a partire dall'**indirizzo iniziale** del processo e proseguendo dinamicamente in base alle necessità. Per evitare un sovraccarico della memoria, le componenti non più utilizzate vengono rimosse e successivamente ricaricate solo se necessario. Le **prestazioni** della memoria virtuale dipendono dalla frequenza con cui queste operazioni di caricamento e rimozione avvengono. Per ottimizzarle, si fa affidamento al **principio della località dei riferimenti**, secondo cui un processo tende a riutilizzare dati e istruzioni referenziati di recente. In questo contesto, vengono adottati **algoritmi di sostituzione** e assegnate quantità di memoria adeguate, al fine di mantenere efficiente l'esecuzione dei processi.

Due approcci principali alla realizzazione della memoria virtuale sono la **paginazione** e la **segmentazione**. Con la paginazione, la memoria è suddivisa in **pagine** di dimensione uniforme, il che elimina la **frammentazione esterna**, ma può generare **frammentazione interna**. Al contrario, la segmentazione organizza la memoria in **segmenti logici**

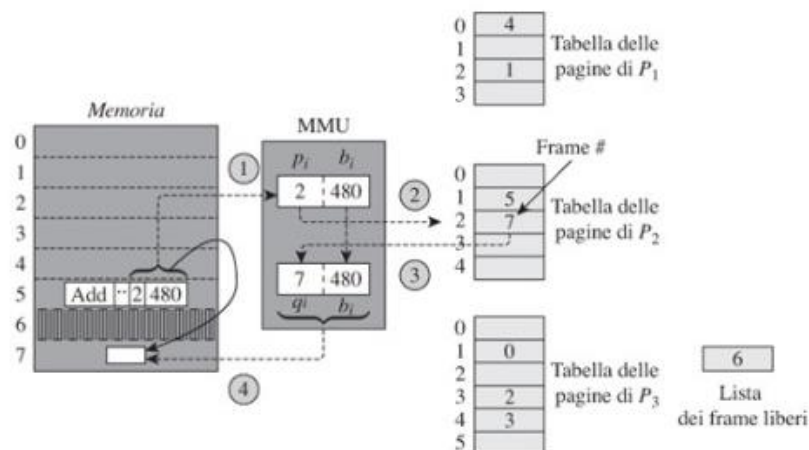
di dimensioni variabili, definiti dal **programmatore**, riducendo la frammentazione interna a scapito di quella esterna. Entrambi i modelli, sebbene diversi, supportano in vario grado la **condivisione della memoria** tra processi.

Paginazione su richiesta

La **paginazione su richiesta** è una tecnica impiegata nella gestione della **memoria virtuale**, in cui lo spazio di indirizzamento di un processo è suddiviso in **pagine** numerate sequenzialmente. Ogni pagina ha una **dimensione fissa** (tipicamente una potenza di due, ad esempio 1KB) ed è mappata su un **frame di pagina**, cioè un'area di memoria fisica di pari dimensione. I **frame** sono anch'essi numerati e costituiscono l'insieme degli spazi disponibili nella **memoria fisica**. A ogni istante, un frame può contenere una pagina oppure essere libero.

Un **indirizzo logico** generato da un processo è rappresentato da una **coppia (p_i, b_i)**, dove p_i è il numero di pagina e b_i è l'**offset** all'interno della pagina. L'**indirizzo fisico** corrispondente viene calcolato sommando l'indirizzo iniziale del frame che contiene la pagina con il valore dell'offset. Grazie alla scelta di una dimensione di pagina pari a una potenza di due, tale operazione può essere effettuata tramite **concatenazione di bit**, garantendo **efficienza** e rapidità nell'accesso alla memoria.

Le informazioni sull'allocazione delle pagine vengono memorizzate in una **tabella delle pagine**, che associa a ciascuna pagina logica il numero del frame in cui essa risiede. Ogni processo possiede una propria tabella delle pagine.



Nel modello di **paginazione su richiesta**, lo **spazio di indirizzamento logico** di un processo è suddiviso in **pagine** di dimensione fissa, ad esempio **1 KB** (1024 byte). La **memoria fisica** del sistema è composta da **frame** di pari dimensione, ognuno dei quali può contenere una sola pagina oppure essere libero. L'allocazione delle pagine nei frame è descritta tramite strutture chiamate **tabelle delle pagine**, specifiche per ogni processo. Ogni voce della tabella indica in quale frame risiede una data pagina.

Nel caso illustrato, i processi P_1 , P_2 e P_3 hanno alcune pagine attive in memoria, la quale dispone di **8 frame**, numerati da 0 a 7. In particolare, P_2 ha le sue **pagine 1 e 2** nei **frame 5 e 7**, P_1 ha le **pagine 0 e 2** nei **frame 4 e 1**, mentre P_3 ha le **pagine 1, 3 e 4** nei **frame 0, 2 e 3**. L'unico **frame libero** al momento è il numero **6**.

Quando il processo P_2 esegue l'istruzione relativa all'**indirizzo logico 2528**, la **Memory Management Unit (MMU)** effettua la traduzione in base alla tabella delle pagine di P_2 . La MMU vede l'indirizzo dell'operando 2528 come coppia **(2, 480)**: la **pagina 2** con **offset 480**. La MMU consulta la tabella e rileva che la pagina 2 si trova nel **frame 7**. Pertanto, l'**indirizzo fisico** corrispondente sarà:

$$7 \times 1024 + 480 = 7616$$

L'accesso alla memoria avviene quindi al **byte 480 nel frame 7**. Questo esempio mostra come la paginazione consenta di mantenere in memoria solo le porzioni necessarie all'esecuzione corrente, ottenendo un accesso **efficiente e trasparente** alla memoria fisica.

Concetti preliminari

Nel contesto della **paginazione su richiesta**, se un'istruzione del processo **P₂** fa riferimento a un byte contenuto nella **pagina 3**, il **gestore della memoria virtuale** provvede a caricare tale pagina in memoria e ad aggiornare la **tabella delle pagine** di **P₂**, inserendo il numero del **frame** assegnato nel corrispondente elemento. Questo meccanismo prende il nome di **paginazione su richiesta**, poiché le pagine vengono caricate solo nel momento in cui si rendono necessarie.

Per realizzare questo meccanismo, una copia dell'intero **spazio di indirizzamento logico** del processo è mantenuta su **disco**, in un'area specifica chiamata **spazio di swap**. All'avvio del processo, il gestore della memoria virtuale alloca tale spazio e vi copia codice e dati del processo. Durante l'esecuzione, ogni qualvolta il processo tenta di accedere a una **pagina non presente in memoria**, il sistema è in grado di rilevare tale richiesta e carica la pagina dallo spazio di swap: questa operazione è detta **page-in**.

Quando invece è necessario **liberare spazio in memoria**, il gestore può rimuovere una pagina, salvandola nuovamente nello spazio di swap del processo, nel caso in cui sia stata **modificata** dall'ultima volta in cui era stata caricata. Questa operazione è nota come **page-out**. Lo **spazio di swap** tiene quindi traccia di tutte le pagine del processo non attualmente in memoria. Se per caricare una nuova pagina è necessario liberare un frame già occupato, si ha una **sostituzione della pagina**, che può implicare sia un page-out della pagina da rimuovere sia un nuovo page-in della pagina richiesta.

Tabella delle pagine

La **tabella delle pagine** è una struttura fondamentale per la gestione della **memoria virtuale**, in quanto supporta la **traduzione degli indirizzi logici**, il **caricamento su richiesta** delle pagine e le operazioni di **sostituzione delle pagine**. Ogni processo possiede una propria tabella delle pagine, formata da più voci, ciascuna delle quali descrive lo stato di una specifica **pagina logica** del processo.

Bit di validità	Frame #	Informazioni varie			
		Prot info	Ref info	Modificato	Altre info

Un elemento della tabella, come illustrato nella **Figura**, include diversi **campi informativi**:

- Il **bit di validità** è un valore booleano che indica se la pagina è **presente in memoria** (1) o meno (0).
- Il **campo frame #** specifica quale **frame di memoria fisica** contiene la pagina, facilitando la traduzione logica-fisica degli indirizzi.
- Il campo **informazioni varie**, suddiviso in sottocampi:
 - Il **campo di protezione (prot info)** serve a specificare i **permessi di accesso** alla pagina (lettura, scrittura, esecuzione), impedendo usi impropri.
 - Il **campo ref info** registra se la pagina è stata recentemente **referenziata**, informazione utile per le strategie di **sostituzione** delle pagine.
 - Il **bit modificato**, o **dirty bit**, indica se la pagina è stata **modificata** da quando è stata caricata; questo consente di stabilire se sia necessario effettuare un **page-out** prima della sua rimozione dalla memoria.
 - Il campo **altre info** contiene dati aggiuntivi, come l'**indirizzo del blocco di swap** su disco dove è memorizzata una copia della pagina.

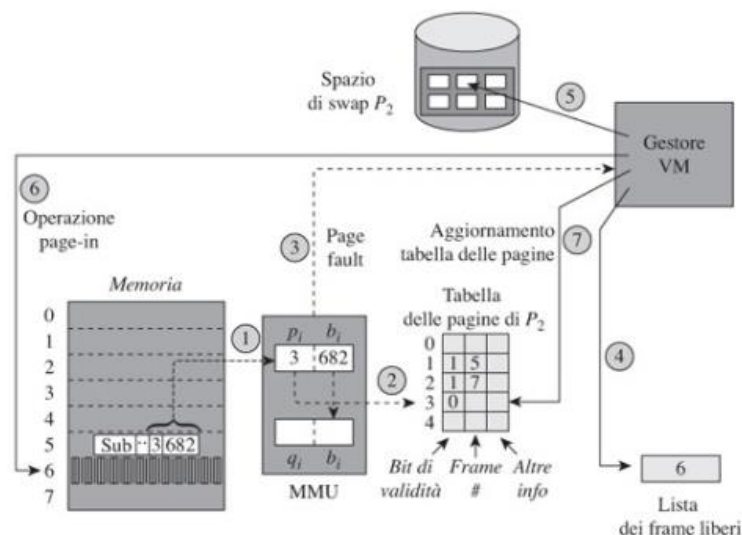
Tutti questi campi permettono al sistema operativo di gestire la memoria virtuale in modo **efficiente, sicuro e coerente**, garantendo che ogni pagina venga trattata correttamente secondo il suo stato e il suo utilizzo.

Page fault e caricamento su richiesta delle pagine

Nel processo di **traduzione degli indirizzi logici** da parte della **Memory Management Unit (MMU)**, può verificarsi un evento noto come **page fault**, ossia un'interruzione generata quando una **pagina richiesta non è presente in memoria**. Questo accade, ad esempio, quando il **bit di validità** nella **tabella delle pagine** del processo indica che la pagina è **assente**. La MMU, rilevando tale condizione, genera un **interrupt di programma**, detto anche **missing page interrupt**, attivando così la routine di gestione del page fault.

La procedura si articola nei seguenti passaggi fondamentali:

1. L'**indirizzo logico** viene interpretato come una coppia (p_i, b_i), dove p_i rappresenta il **numero della pagina** e b_i l'**offset** all'interno della pagina.
2. La MMU consulta la **tabella delle pagine** per verificarne il **bit di validità**.
3. Se tale bit è pari a 0, la pagina non è residente in memoria. La MMU genera quindi un **page fault** e il **processo viene sospeso**, passando nello stato **blocked**.
4. Il **gestore della memoria virtuale** interviene per effettuare l'**operazione di page-in**, caricando la pagina mancante dallo **spazio di swap**.
5. La tabella delle pagine viene aggiornata con le nuove informazioni: il bit di validità viene impostato a 1 e viene registrato il numero del frame assegnato.
6. La MMU calcola infine l'**indirizzo fisico effettivo**, combinando il numero del **frame assegnato** con l'offset b_i , e l'esecuzione del processo può riprendere.



La **figura** allegata illustra visivamente questi passaggi. Durante l'esecuzione dell'istruzione "Sub .. 682", la MMU identifica che si tratta della **pagina 3** con **offset 682**. Poiché la pagina non è presente in memoria (bit di validità = 0), si genera un **page fault**. Il **gestore della memoria virtuale** accede allo **spazio di swap** per recuperare la pagina e la carica nel **frame 6** disponibile. La **tabella delle pagine** viene aggiornata: il bit di validità è impostato a 1 e viene registrato il numero del frame. Infine, l'indirizzo fisico è calcolato come:

$$6 \times 1024 + 682 = 6826$$

L'operazione di **page-in** si conclude così, e il processo può riprendere l'esecuzione.

Questo meccanismo permette al sistema di **caricare solo le pagine realmente necessarie**, migliorando l'**efficienza nell'uso della memoria fisica** e consentendo l'esecuzione di **processi più ampi** rispetto alla dimensione della memoria disponibile.

Operazioni di page-in, page-out e sostituzione delle pagine

Come illustrato nella **Figura 12.4**, quando si verifica un **page fault** ma **non è disponibile alcun frame libero**, il **gestore della memoria virtuale** deve eseguire un'**operazione di sostituzione della pagina**. Questo significa che una **pagina attualmente in memoria** viene **rimossa** per far posto a quella richiesta, permettendo la continuazione del processo interrotto.

Il procedimento si articola in vari passaggi. Innanzitutto, il gestore utilizza un **algoritmo di sostituzione delle pagine** per scegliere quale pagina, attualmente residente in memoria, debba essere sostituita. Una volta individuata, il sistema accede alla relativa **entry** nella **tabella delle pagine**, marcandola come "**non presente**". Se il **bit Modificato** della pagina è impostato, indicando che è stata **alterata** durante la sua permanenza in memoria, viene eseguita un'**operazione di page-out**, ossia la scrittura della pagina nello **spazio di swap**.

Successivamente, si procede con il **page-in** della nuova pagina richiesta, che viene caricata nel **frame appena liberato**. Terminato il caricamento, la tabella delle pagine viene aggiornata: si registra il nuovo **numero di frame**, si imposta il **bit di validità**, e si ripristina l'esecuzione del processo. Quando quest'ultimo riprende, l'istruzione che aveva causato il page fault viene **rieseguita**, questa volta senza ulteriori interruzioni.

Le operazioni di **page-in** e **page-out** sono note come **page I/O**, in quanto implicano **movimenti di dati tra memoria e disco**. Il termine **traffico di pagine** viene utilizzato per indicare il flusso complessivo di queste operazioni all'interno e all'esterno della memoria fisica. Poiché il **page I/O** è differente dal normale **I/O di programma**, il **kernel** può mettere **in stato di blocco** il processo che ha causato il page fault e **passare la CPU ad altri processi**, garantendo così un utilizzo efficiente delle risorse del sistema.

Tempo di accesso effettivo alla memoria (EAT)

Il **tempo di accesso effettivo alla memoria** è il tempo medio che un processo impiega per accedere a un dato, tenendo conto sia dei casi in cui la pagina è già in memoria, sia dei casi in cui deve essere caricata a causa di un **page fault**.

La formula è la seguente:

$$EAT = (pr_1 \times 2 \times t_{mem}) + (1 - pr_1) \times (t_{men} + t_{pfh} + 2 \times t_{mem})$$

Dove:

- pr_1 = probabilità che la pagina sia già in memoria
- t_{mem} = tempo di accesso alla memoria
- t_{pfh} = tempo necessario per gestire un page fault (incluso il tempo di I/O)

Se la pagina è già in memoria si impiega $2 \times t_{mem}$ perché:

- il primo accesso serve a consultare la **tabella delle pagine**, che si trova in memoria;
 - il secondo accesso serve a leggere il **dato effettivo** nella pagina.
- Questo tempo è moltiplicato per pr_1 , cioè la probabilità di trovare la pagina in memoria.

Se la pagina non è in memoria si impiega un tempo maggiore:

- t_{mem} per accedere alla tabella delle pagine (dove si scopre che la pagina è assente),
 - t_{pfh} per gestire il page fault (caricamento della pagina da disco),
 - $2 \times t_{mem}$ per accedere nuovamente alla tabella e poi al dato richiesto.
- Il tutto è moltiplicato per $(1 - pr_1)$, ossia la probabilità di *non* trovare la pagina in memoria.

Per ottimizzare l'EAT, è fondamentale **ridurre il numero di page fault**, ovvero **aumentare pr_1** . Ciò si ottiene mantenendo in memoria **più pagine per ogni processo**, così da aumentare la probabilità che le pagine necessarie siano già presenti.

Per supportare questa gestione, ogni pagina dispone di:

- un **bit di riferimento**, che indica se la pagina è stata usata di recente;
- un **dirty bit**, che segnala se la pagina è stata modificata dall'ultima volta che è stata caricata in memoria.

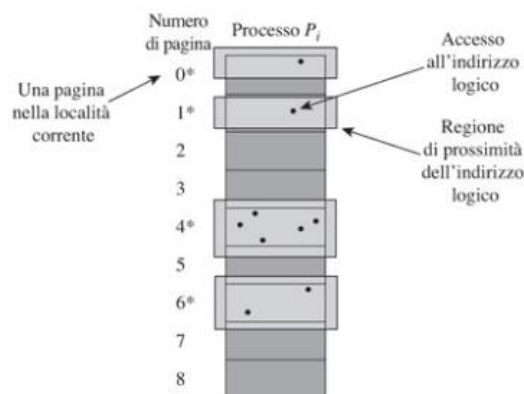
Questi bit aiutano il sistema a decidere **quali pagine mantenere** e **quali sostituire** in caso di necessità.

Sostituzione delle pagine e Località dei riferimenti

Quando si verifica un **page fault** e **non ci sono frame liberi**, il sistema deve **sostituire** una delle pagine già presenti in memoria. Per evitare che venga rimossa una pagina che il processo userà di nuovo a breve, è importante **scegliere una pagina che probabilmente non verrà più usata a breve**.

Per fare questa scelta, il sistema si basa sul **principio della località dei riferimenti**, che afferma che un processo tende a usare indirizzi di memoria **vicini tra loro** in un intervallo di tempo. Questo succede perché, normalmente, le istruzioni vengono eseguite in modo **sequenziale** e i dati elaborati si trovano in **blocchi contigui**, come array. Per questi motivi, i riferimenti a memoria che un processo fa in un certo momento sono spesso **molto simili o vicini** a quelli che ha fatto poco prima.

L'insieme delle pagine che un processo ha usato **di recente** è detto **località corrente**. Se una pagina appartiene a questa località, è molto probabile che il processo ne abbia ancora bisogno a breve. Per questo, il sistema cerca di **non rimuovere** queste pagine quando deve fare spazio in memoria.



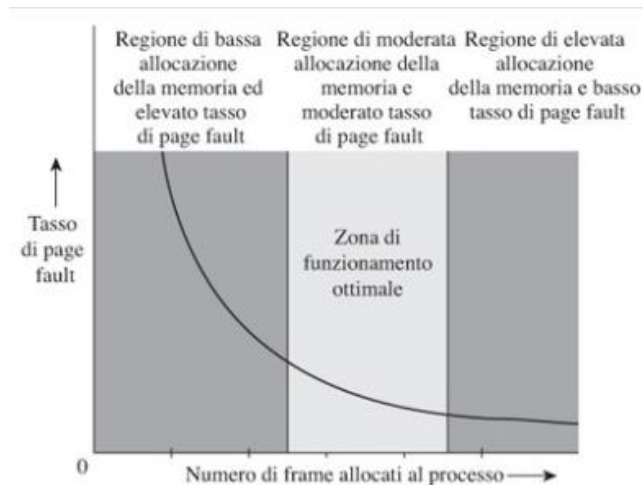
Un esempio è mostrato nella **Figura**, dove si vede che un processo accede più volte a indirizzi logici molto vicini. Questi accessi ricadono in **regioni di prossimità**, che possono anche attraversare più pagine, ovvero **0, 1, 3, 4, 5, 6 e 7**. In questo caso, la **località corrente** del processo include le pagine **{0, 1, 4, 6}**, che è meglio mantenere in memoria.

Per decidere **quale pagina rimuovere**, il sistema può valutare **da quanto tempo** una pagina non viene usata. Se una pagina non è stata usata da molto, è meno probabile che venga usata di nuovo a breve, quindi può essere sostituita. Questo è il principio della **politica LRU (Least Recently Used)**, cioè si rimuove la pagina **meno usata di recente**.

Allocazione della memoria per un processo

La **frequenza dei page fault** in un processo dipende dalla quantità di **memoria che gli viene allocata**. Quando un processo dispone di **pochi frame**, tende a generare **più page fault**; al contrario, se ha più memoria a disposizione, la percentuale di page fault diminuisce. Questo comportamento è utile perché consente al **gestore della memoria virtuale** di **monitorare l'efficienza** di ciascun processo e di intervenire, se necessario, per migliorarne le prestazioni.

Ad esempio, se un processo mostra un **tasso elevato di page fault**, il sistema può decidere di **aumentargli la memoria**. Questo intervento può ridurre i page fault e rendere l'esecuzione del processo più fluida. In particolare, la **politica LRU** di sostituzione delle pagine mostra che è più vantaggioso rimuovere **le pagine meno usate di recente**: se una pagina viene sostituita ma appartiene ancora alla **località corrente** del processo, è probabile che venga richiesta di nuovo, generando altri page fault. Al contrario, allocare più frame al processo **riduce il rischio** che le pagine rilevanti vengano rimosse.



La **Figura 12.6** illustra la relazione tra **numero di frame allocati** a un processo e il **tasso di page fault**. La curva mostra che, all'inizio, aumentando i frame, i page fault calano rapidamente. Dopo una certa soglia, però, **aggiungere ulteriori frame produce benefici minimi**: si entra nella **zona di funzionamento ottimale**, dove il processo ha memoria sufficiente per lavorare bene. Oltre questa zona, si parla di **sovra-allocazione**, che comporta uno spreco di risorse e una diminuzione del grado di multiprogrammazione con conseguente abbassamento del throughput. Al contrario, al di sotto della soglia ottimale, il processo subisce un **sotto-allocazione**, con elevate frequenze di page fault e prestazioni compromesse.

Stabilire **quanta memoria assegnare a un processo** non è semplice, perché il **comportamento del processo può variare** nel tempo. Anche eseguendo lo stesso processo più volte, il numero e la distribuzione dei page fault potrebbero cambiare. Per questo motivo, il sistema deve **adattarsi dinamicamente**, monitorando le esigenze del processo e la sua **località corrente**, come già mostrato nella Figura 12.5.

Thrashing

Il **thrashing** è una situazione in cui un processo, avendo **troppa poca memoria** a disposizione, genera una **frequenza molto alta di page fault**, come mostrato nella **Figura 12.6**. Quando ciò accade, il processo trascorre gran parte del tempo in **stato bloccato**, impegnato nel caricamento continuo di pagine da disco. In questo stato, la **CPU resta inutilizzata** o poco efficiente, perché il processo non riesce a eseguire istruzioni in modo continuo.

Questa condotta comporta un **elevato overhead**, dovuto non solo alla gestione frequente dei page fault, ma anche al continuo **passaggio di controllo tra processi**. Se molti processi del sistema si trovano nella stessa condizione, l'intero sistema finisce per passare più tempo a **gestire il traffico di pagine** che a svolgere computazioni utili. Il risultato è una **prestazione complessiva molto bassa**, sia in termini di tempo di risposta, sia in termini di throughput. Questa condizione prende il nome di **thrashing**, che viene definita come **una situazione in cui si verifica un traffico elevato di pagine accompagnato da una scarsa efficienza della CPU**.

È importante notare che **una bassa efficienza della CPU** può avere anche altre cause, ad esempio quando sono presenti in memoria **troppi pochi processi** o se **tutti i processi eseguono I/O** molto frequentemente. Tuttavia, nel **thrashing**, ciò che caratterizza il problema è che **tutti i processi rallentano** a causa dell'**elevata frequenza di page fault**.

Come evidenziato dalla Figura 12.6, la causa principale del thrashing è spesso un'**allocazione di memoria troppo bassa** per i processi attivi. La soluzione può consistere nell'**aumentare la memoria** assegnata ai processi oppure nel **ridurre il numero di processi in memoria**, diminuendo il grado di multiprogrammazione. Anche un singolo processo può entrare in thrashing se ha **una percentuale di page fault troppo alta**, ma il problema si aggrava quando ciò accade **a molti processi contemporaneamente**.

Dimensione di pagina ottimale

La **dimensione di una pagina** ha un impatto diretto sulle **prestazioni della memoria** e viene stabilita dall'**hardware**. Una pagina **troppo grande** occupa **più spazio**, aumenta la **frammentazione interna**, ma riduce i **page fault** perché si mantengono meno pagine in memoria. Al contrario, una pagina **troppo piccola** riduce la frammentazione e migliora la località, ma genera **più overhead** nella gestione della tabella delle pagine e può causare un **degrado delle prestazioni**, soprattutto se provoca **scambi frequenti con la memoria secondaria**.

Per questo, è necessario trovare un **compromesso** che riduca sia lo spreco di spazio sia i page fault. In generale, la **dimensione ottimale della pagina** è quella che minimizza la somma tra la memoria persa per la frammentazione e quella usata per la tabella delle pagine. Una stima empirica di questo valore è data dalla formula:

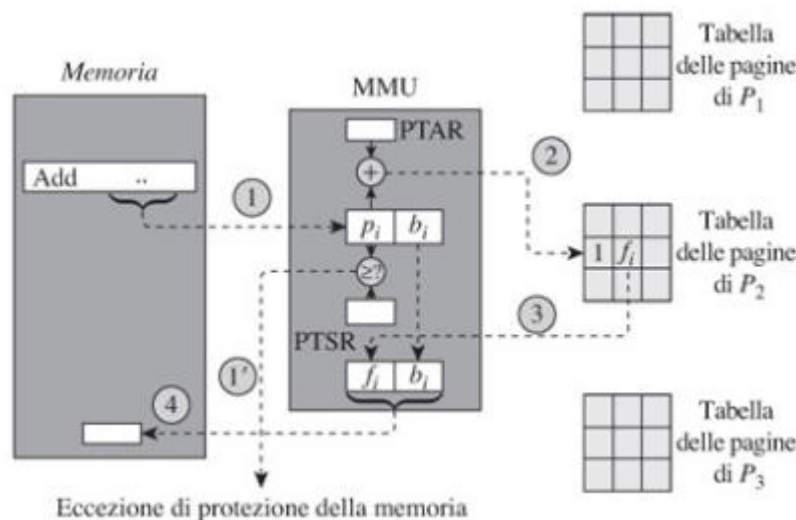
$$\text{Dimensione ottimale} = \sqrt{2 \times \text{dimensione del processo}}$$

Supporto hardware alla paginazione

Per gestire la **memoria virtuale** in modo efficiente e sicuro nei sistemi multi-programmati, l'hardware utilizza due registri fondamentali:

- **PTAR** (*Page Table Address Register*), che contiene l'indirizzo di inizio della **tabella delle pagine** del processo attivo;
- **PTSR** (*Page Table Size Register*), che indica **quante pagine** sono state allocate a quel processo.

Quando il **kernel** crea il **PCB (Process Control Block)** di un processo, inserisce in esso i valori da assegnare a PTAR e PTSR. Questi registri vengono poi **inizializzati** ogni volta che il processo viene schedato per l'esecuzione.



Quando la **MMU** riceve un **indirizzo logico** della forma (p_i, b_i) , calcola l'indirizzo dell'entry nella tabella delle pagine usando la formula:

$$\text{Indirizzo entry} = \text{PTAR} + p_i \times \text{IPT}_{\text{entry}}$$

dove $\text{IPT}_{\text{entry}}$ è la dimensione di una singola entry della tabella (solitamente una potenza di 2, per permettere un calcolo rapido tramite operazioni di shift). L'entry fornisce il **numero del frame fisico** associato alla pagina logica, al quale viene sommato l'**offset b_i** per ottenere l'**indirizzo fisico effettivo**.

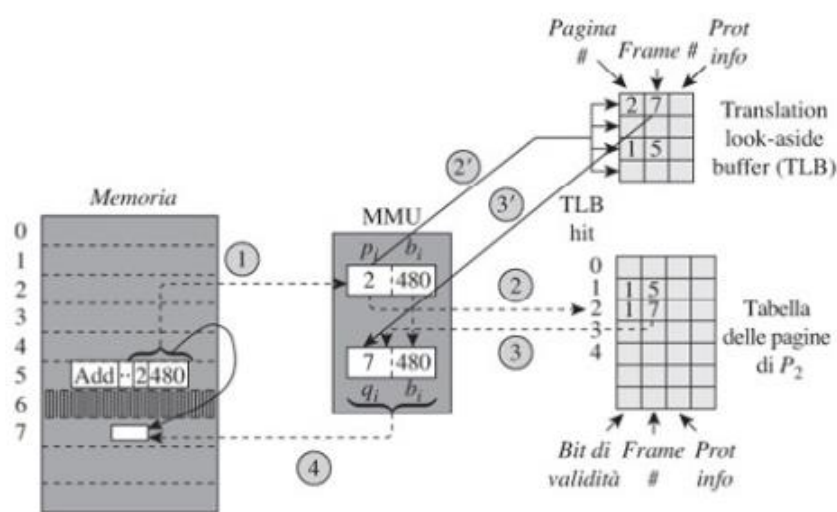
Prima di completare l'accesso, la MMU esegue un controllo di **protezione della memoria**: verifica che il valore di p_i non superi quello contenuto nel **PTSR**. Se ciò accade, significa che si sta cercando di accedere a una pagina non autorizzata, e viene generata una **violazione di protezione** (interrupt).

Inoltre, ogni entry può contenere un campo **prot info**, che specifica i **privilegi di accesso** alla pagina (lettura, scrittura, esecuzione). Se il processo tenta un'operazione non permessa, viene generata un'**eccezione di protezione**.

Infine, se la pagina richiesta **non è presente in memoria**, la MMU genera un **page fault**. Il **sistema operativo** si occupa allora di caricare la pagina dallo **spazio di swap**, aggiornare la tabella delle pagine e riprendere l'esecuzione del processo.

Questo meccanismo integrato tra hardware e software assicura una gestione della memoria **trasparente, protetta ed efficiente**, anche con molti processi in esecuzione.

Translation Look-aside Buffer (TLB)



Per migliorare l'**efficienza nella traduzione degli indirizzi**, i sistemi operativi utilizzano il **TLB (Translation Look-aside Buffer)**, una **piccola cache hardware** che memorizza le traduzioni recenti tra indirizzi logici e fisici. Invece di accedere ogni volta alla **tabella delle pagine** in memoria, la **MMU** consulta prima il TLB: se la traduzione è presente (**TLB hit**), l'accesso è molto rapido; altrimenti (**TLB miss**), viene usata la tabella delle pagine. Se la pagina non è nemmeno in memoria, si verifica un **page fault**, e il sistema operativo la carica dallo **spazio di swap**.

L'utilizzo del TLB riduce i tempi di accesso, ma introduce **problemi di sicurezza**: può contenere voci di un altro processo dopo un **context switch**. Per questo, il sistema può svuotare il TLB o associare a ogni voce un **identificativo di processo**.

Il **tempo di accesso effettivo alla memoria (EAT)** tiene conto di tre situazioni:

1. La pagina è nel TLB → accesso veloce.
2. La pagina non è nel TLB ma è in memoria → due accessi in memoria.
3. La pagina non è in memoria → si genera un page fault (caso più lento).

La **formula del tempo medio** è:

$$EAT = \underbrace{p_{r1} \cdot p_{r2} \cdot (t_{TLB} + t_{mem})}_{(1) \text{ Pagina nel TLB}} + \underbrace{p_{r1} \cdot (1 - p_{r2}) \cdot (t_{TLB} + 2 \cdot t_{mem})}_{(2) \text{ Pagina in memoria ma non nel TLB}} + \underbrace{(1 - p_{r1}) \cdot (t_{TLB} + t_{mem} + t_{pfh} + 2 \cdot t_{mem})}_{(3) \text{ Page fault (pagina non in memoria)}}$$

dove:

- p_{r1} = probabilità che la pagina sia in memoria,

- p_{r2} = probabilità che la pagina sia nel TLB,
- t_{mem} = tempo di accesso alla memoria,
- t_{tlb} = tempo di accesso al TLB,
- $t_p f_h$ = overhead di tempo per gestire un page fault.

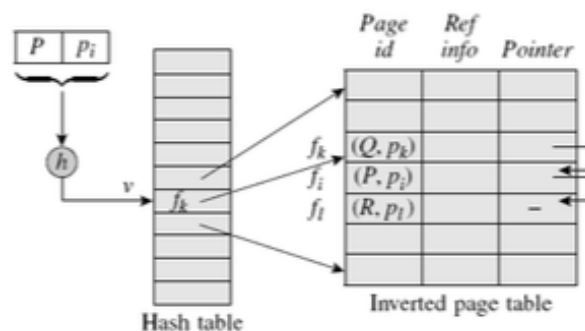
NB: Nei casi 2 e 3 compare $2 \times t_{mem}$ perché il primo accesso serve per consultare la tabella delle pagine e il secondo per accedere alla pagina di memoria vera e propria.

Per ottimizzare le prestazioni, si possono usare:

- **wired TLB entries**, voci fisse riservate al **kernel**,
- **superpagine**, cioè pagine più grandi che riducono il numero di traduzioni da memorizzare.

Implementazione tabelle delle pagine

Tabella delle pagine invertita



La **tabella delle pagine invertita** è una tecnica di gestione della **memoria virtuale** che sostituisce le tradizionali tabelle delle pagine per ogni processo con **una singola tabella globale**. In questa struttura, ogni entry rappresenta un **frame fisico di memoria** e contiene il **numero di pagina virtuale** e l'**ID del processo** a cui la pagina appartiene. Questo consente di ridurre significativamente l'occupazione di memoria, soprattutto in sistemi con **spazi di indirizzamento molto grandi** (es. 64 bit), poiché la tabella cresce in base alla **quantità di memoria fisica** e non al numero di pagine virtuali totali.

Per trovare il frame corrispondente a una certa pagina di un processo, si cerca nella tabella una voce che contenga la **coppia ID-processo/pagina**. Poiché una ricerca lineare sarebbe inefficiente, si utilizza una **funzione hash** applicata a questa coppia per accedere direttamente a una posizione nella tabella. In caso di **collisioni** (più coppie con lo stesso hash), si impiega una **lista concatenata**: ogni entry punta alla successiva con lo stesso valore hash, formando una catena che può essere percorsa fino a trovare la voce corretta.

Il **vantaggio principale** di questa struttura è il **notevole risparmio di memoria** e la **scalabilità**, mentre lo **svantaggio** è la maggiore **complessità nella ricerca** e nella gestione delle collisioni. Tuttavia, tecniche come l'hashing e l'uso di liste concatenate aiutano a mitigare questi problemi.

Nelle **tabelle delle pagine tradizionali**, la dimensione cresce con lo **spazio di indirizzamento virtuale**. Ad esempio, con **32 bit** e pagine da **4KB** (2^{12}), le pagine virtuali sono $\frac{2^{32}}{2^{12}} = 2^{20} \approx 1 \text{ milione di entry}$. Con **64 bit**, diventano $\frac{2^{64}}{2^{12}} = 2^{52} \approx 4,5 \times 10^{16}$ entry per processo, una dimensione ingestibile.

La **tabella delle pagine invertita**, invece, cresce con la **memoria fisica**. Con **8GB** (8×2^{30}) di RAM e pagine da **4KB** (2^{12}), i frame sono $\frac{8 \times 2^{30}}{2^{12}} = 8 \times 2^{18} = 2^{21} \approx 2 \text{ milioni di entry}$ per l'intero sistema, molto più gestibili.

🧠 Scenario di esempio

Immagina di avere:

- 3 **frame fisici** nella memoria principale: `f0`, `f1`, `f2`.
- 2 **processi**: `P1` e `P2`.
- Ogni processo ha delle **pagine virtuali** numerate: `P1` ha `p0`, `p1`; `P2` ha `p0`, `p1`.

Supponiamo che:

- La **pagina** `p0` di `P1` sia caricata nel **frame** `f0`
- La **pagina** `p1` di `P2` sia caricata nel **frame** `f1`
- La **pagina** `p1` di `P1` sia caricata nel **frame** `f2`

📄 Contenuto della tabella delle pagine invertita

Frame fisico (indice)	ID processo	Pagina virtuale
<code>f0</code>	<code>P1</code>	<code>p0</code>
<code>f1</code>	<code>P2</code>	<code>p1</code>
<code>f2</code>	<code>P1</code>	<code>p1</code>

Questa tabella ha una sola riga per ciascun frame fisico.

🔍 Esempio di accesso

Il processo `P1` vuole accedere alla **pagina virtuale** `p1`. Il sistema operativo:

1. Prende la **coppia** (`P1`, `p1`)
2. Applica una **funzione hash** → ad esempio, ottiene `indice = 2`
3. Va alla posizione `f2` nella tabella e verifica che lì ci sia esattamente (`P1`, `p1`)
4. Trova una **corrispondenza** e quindi accede al **frame fisico** `f2`

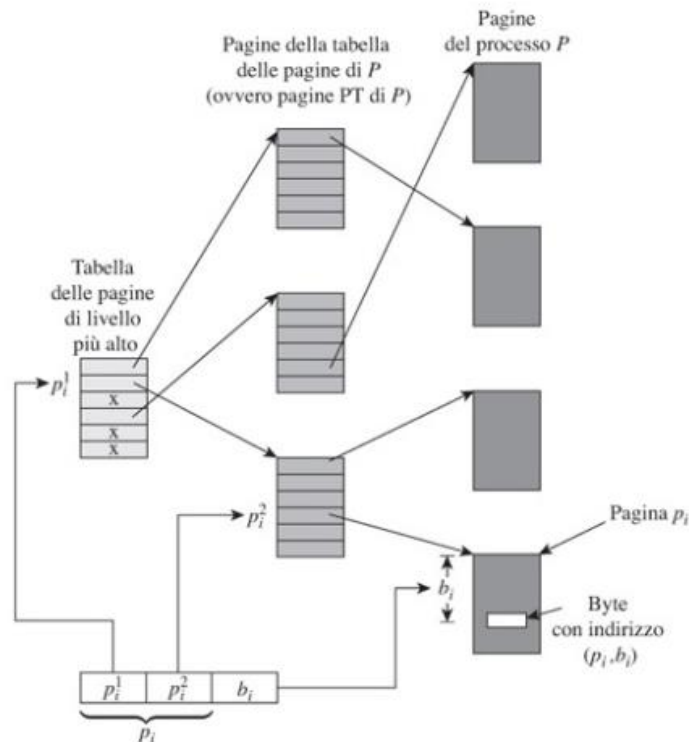
⚠️ Gestione delle collisioni

Supponiamo ora che anche la coppia (`P2`, `p0`) dia lo stesso hash di (`P1`, `p1`) → entrambi puntano a `indice = 2`.

In questo caso:

- Il sistema crea una **lista concatenata** alla voce `f2`.
- Se `P2` accede a `p0`, il sistema va a `f2`, trova (`P1`, `p1`), non corrisponde → segue il puntatore alla voce successiva della lista.
- Lì trova (`P2`, `p0`) → accesso corretto.

Tabella delle pagine multilivello



La **tabella delle pagine multilivello** è una tecnica per la gestione efficiente della **memoria virtuale**, utilizzata nei sistemi operativi quando lo spazio di indirizzamento è molto esteso, come nei sistemi a **32 o 64 bit**. L'idea principale è quella di **paginare più volte la tabella delle pagine**, ovvero suddividerla in più livelli per evitare che cresca troppo in dimensione.

Questo approccio prevede una **prima tabella**, detta di primo livello o **alto livello**, che contiene **puntatori ad altre tabelle** di secondo livello. Ciascuna di queste, a loro volta, può indicizzare direttamente le **pagine del processo** o ulteriori tabelle. Se la dimensione delle tabelle intermedie è ancora eccessiva, si aggiunge un altro livello, evitando però di andare troppo in profondità per **mantenere l'accesso efficiente**.

L'**indirizzo logico** viene quindi suddiviso in più campi (in questo caso tre campi distinti). In un sistema a due livelli, il primo campo, indicato come p_i^1 , seleziona l'entrata nella **tabella delle pagine di primo livello**. Il secondo campo, p_i^2 , identifica la **pagina specifica** all'interno della tabella inferiore (di secondo livello). L'ultima parte dell'indirizzo, b_i , rappresenta l'**offset** o le **informazioni del byte** da accedere all'interno della pagina.

Nel caso specifico di una **paginazione a due livelli**, un sistema a **32 bit** con **pagine da 4 KB** divide i 32 bit dell'indirizzo in **10 bit per la tabella di primo livello**, **10 bit per quella di secondo livello**, e **12 bit per l'offset**. Questo significa che lo **spazio virtuale di 4 GB** viene suddiviso in **2²⁰ pagine**, con ogni livello che contiene **1024 voci**. Quando un processo accede a una certa pagina, il sistema consulta prima la tabella superiore, poi quella inferiore, e infine accede al **frame fisico** corrispondente, sommando l'offset.

Questa struttura consente un **notevole risparmio di memoria**, perché vengono create e mantenute in memoria **solo le tabelle necessarie**, ovvero quelle che contengono pagine effettivamente usate. Le altre tabelle possono essere caricate **su richiesta**, sfruttando la **paginazione su domanda**. Se una tabella richiesta non si trova in memoria, viene generato un **page fault**, che forza il sistema operativo a caricarla dalla memoria secondaria.

L'unico svantaggio di questa tecnica è la **maggiore complessità nella traduzione degli indirizzi**, che richiede più accessi alla memoria. Tuttavia, grazie all'utilizzo della **TLB (Translation Lookaside Buffer)**, che memorizza le traduzioni più recenti, è possibile **mitigare il costo** di questi passaggi, mantenendo elevate le prestazioni.

Politiche di sostituzione delle pagine

Le politiche di sostituzione delle pagine regolano il meccanismo attraverso il quale il sistema operativo **seleziona quale pagina rimuovere dalla memoria fisica** quando quest'ultima è satura e si rende necessario caricare una nuova pagina.

L'analisi di tali politiche si fonda sul concetto di **stringa dei riferimenti alle pagine**, ovvero la sequenza ordinata delle richieste di accesso alle pagine generate da un processo durante la sua esecuzione. Tale sequenza rappresenta, pertanto, la traccia del comportamento di accesso alla memoria del programma.

Per ogni pagina è inoltre possibile associare una **stringa temporale dei riferimenti**, definita come l'insieme ordinato degli istanti in cui la pagina è stata referenziata. Questa informazione consente di caratterizzare con maggiore precisione il profilo di utilizzo delle pagine e di valutare l'efficacia delle diverse strategie di sostituzione.

Strategie di sostituzione: Ottimale, FIFO e LRU

La **strategia di sostituzione ottimale** consiste nello scegliere le pagine da sostituire in modo da ottenere **il minor numero possibile di page fault**, quindi selezionando, a ogni passo, la scelta teoricamente migliore. Naturalmente, non è possibile conoscere in anticipo tutte le decisioni alternative né gli accessi futuri del processo; per questo motivo, il gestore della memoria non può applicare questa strategia in modo pratico, e il suo utilizzo è limitato **a scopi analitici**. Belady ha proposto un'approssimazione di questa scelta ottimale: viene sostituita la pagina il cui **prossimo riferimento è il più lontano (ovvero non previsto negli istanti successivi)** nella stringa dei riferimenti del processo, oppure quella che non verrà più referenziata in futuro.

La **strategia FIFO** (First-In, First-Out) consiste nel sostituire la pagina che è **stata caricata per prima**. A tale scopo, il gestore della memoria memorizza per ogni pagina, **l'istante di caricamento** nel campo `ref_info`.

La **strategia LRU** (Least Recently Used) sfrutta il principio di **località dei riferimenti** e consiste nel sostituire la pagina **meno recentemente referenziata**. Per farlo, si registra l'istante in cui ogni pagina viene referenziata: questo valore, aggiornato a ogni accesso, è memorizzato nella tabella delle pagine.

Esempio

Consideriamo un processo con **2 pagine allocate** e la seguente stringa di riferimenti:

$t_1 : 0, \quad t_2 : 1, \quad t_3 : 0, \quad t_4 : 2, \quad t_5 : 0, \quad t_6 : 1, \quad t_7 : 2$

Istante	Rif. pagina	Ottimale			FIFO			LRU		
		Bit di validità	Ref info	Sostituzione	Bit di validità	Ref info	Sostituzione	Bit di validità	Ref info	Sostituzione
t_1	0	0 1 1 0 2 0		–	0 1 t_1 1 0 2 0		–	0 1 t_1 1 0 2 0		–
t_2	1	0 1 1 1 2 0		–	0 1 t_1 1 1 t_2 2 0		–	0 1 t_1 1 1 t_2 2 0		–
t_3	0	0 1 1 1 2 0		–	0 1 t_3 1 1 t_2 2 0		–	0 1 t_3 1 1 t_2 2 0		–
t_4	2	0 1 1 0 2 1		Sostituisci 1 con 2	0 0 1 1 t_2 2 1 t_4		Sostituisci 0 con 2	0 1 t_3 1 0 2 1 t_4		Sostituisci 1 con 2
t_5	0	0 1 1 0 2 1		–	0 1 t_5 1 0 2 1 t_4		Sostituisci 1 con 0	0 1 t_5 1 0 2 1 t_4		–
t_6	1	0 0 1 1 2 1		Sostituisci 0 con 1	0 1 t_5 1 1 t_6 2 0		Sostituisci 2 con 1	0 1 t_5 1 1 t_6 2 0		Sostituisci 2 con 1
t_7	2	0 0 1 1 2 1		–	0 0 1 1 t_6 2 1 t_7		Sostituisci 0 con 2	0 0 1 1 t_6 2 1 t_7		Sostituisci 0 con 2

In tutte e tre le strategie si verificano page fault agli istanti t_1 e t_2 , poiché le pagine referenziate non si trovano ancora in memoria.

- Nella **sostituzione ottimale**, al tempo t_4 , viene referenziata la pagina 2, che non è in memoria. Secondo la strategia, si sostituisce la pagina **il cui prossimo utilizzo è più tardivo**: in questo caso, la pagina 1. Al tempo t_7 , si deve inserire la pagina 2: viene quindi sostituita la pagina 0, il cui prossimo utilizzo non è previsto.
- Nella **sostituzione FIFO**, si sostituisce la **prima pagina caricata**. Dopo aver caricato 0 e 1, al tempo t_4 viene sostituito 0, poi al tempo t_5 1, e infine a t_6 2. A t_7 si riparte sostituendo nuovamente la pagina 0.
- Nella **sostituzione LRU**, viene sostituita la pagina **che non è stata referenziata da più tempo**.
 - A t_4 , viene sostituita la pagina 1 (la meno recente),
 - A t_6 , viene sostituita la pagina 2,
 - A t_7 , la pagina 0 (non usata da t_5) viene sostituita.

Risultati

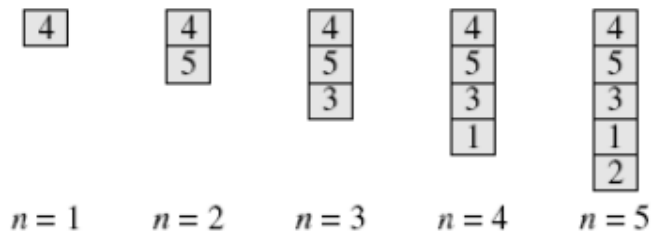
- Con la **sostituzione ottimale** si verificano **4 page fault**, di cui **2 causati dalla politica**.
- Con la **strategia FIFO** si verificano **6 page fault**, di cui **4 causati dalla politica**.
- Con la **strategia LRU** si verificano **5 page fault**, di cui **3 causati dalla politica**.

La strategia **LRU** funziona bene **solo quando i processi rispettano il principio di località dei riferimenti**, ovvero accedono in modo non casuale alle pagine.

Proprietà dello stack

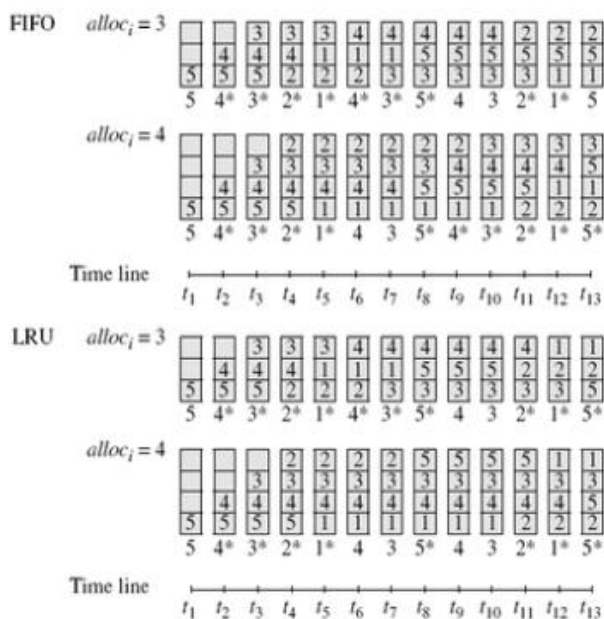
Una buona **politica di sostituzione delle pagine** ha come obiettivo quello di mantenere **basso** il numero di **page fault**. Ogni processo viene identificato con la notazione $\{p_i\}_n^k$, dove p_i rappresenta il processo, mentre $alloc_i = n$ indica il numero di **frame** di pagina allocati al processo all'istante temporale t_k^+ (dopo t_k ma prima di t_{k+1}).

Una politica efficace dovrebbe rispettare la **proprietà dello stack**, nota anche come **proprietà di inclusione**. Questa proprietà stabilisce che, se a un processo vengono assegnati n frame e successivamente $m > n$, allora **le pagine presenti in memoria con n devono essere un sottoinsieme di quelle presenti con m** . In termini pratici, le pagine presenti in memoria in un dato momento devono restare tali anche quando si aumenta il numero di frame disponibili, così da evitare che un aumento della memoria possa peggiorare le prestazioni.



Quando questa proprietà è rispettata, aggiungere più memoria a un processo consente di **mantenere o diminuire** il numero di page fault, ma mai di aumentarlo. Questo comportamento è rappresentato chiaramente nell'esempio illustrato, in cui si confrontano i contenuti di memoria per vari valori di **allocazione** da $n=1$ a $n=5$. In ogni istante t_k , le pagine presenti per un valore di n sono le stesse o un sottoinsieme di quelle presenti per $n+1$, quindi la **proprietà dello stack è rispettata**.

Esempio

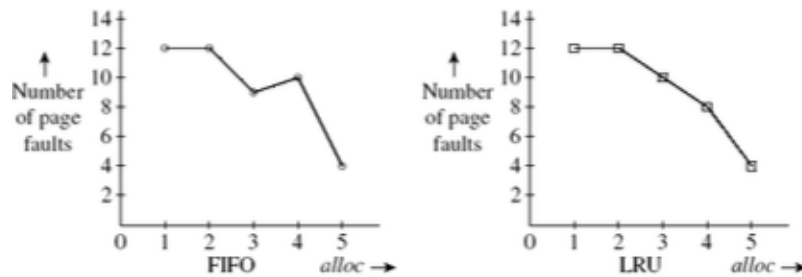


Nel caso specifico, si analizza una **stringa di riferimenti** alle pagine:

$t_1 = 5, t_2 = 4, t_3 = 3, t_4 = 2, t_5 = 1, t_6 = 4, t_7 = 3, t_8 = 5, t_9 = 4, t_{10} = 3, t_{11} = 2, t_{12} = 1, t_{13} = 5$.

Applicando questa stringa alle strategie **FIFO** e **LRU**, si osservano comportamenti differenti. La **strategia FIFO**, con allocazioni $alloc_i = 3$ e $alloc_i = 4$, **non rispetta la proprietà dello stack**: agli istanti t_8, t_9 e t_{12} , le pagine in memoria nel caso di quattro frame non comprendono tutte quelle presenti nel caso di tre. Questo porta a un comportamento paradossale, noto come **anomalia di Belady**, in cui l'aumento della memoria causa un **aumento dei page fault**, rendendo più difficile anche il controllo del fenomeno di **thrashing**.

Diversamente, la **strategia LRU** rispetta pienamente la **proprietà dello stack**. In tutti gli istanti osservati, le pagine in memoria con più frame includono sempre quelle dei casi con meno frame. Di conseguenza, il numero di page fault con LRU tende a diminuire o rimanere stabile all'aumentare della memoria allocata.



Nel grafico finale si osserva chiaramente che, mentre con LRU i **page fault diminuiscono** con l'aumento della memoria, con FIFO questo non accade, confermando che solo strategie come LRU sono in grado di **evitare l'anomalia di Belady** e garantire un comportamento prevedibile.

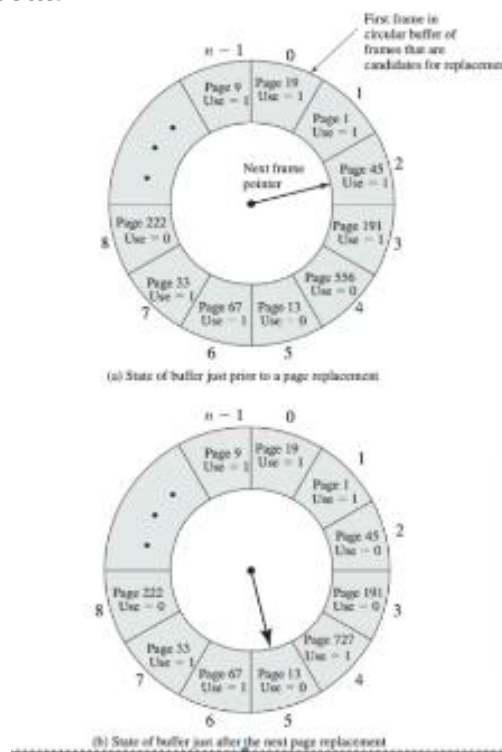
Strategia Not Recently Used (NRU)

Nel contesto della gestione della memoria virtuale, la strategia **LRU** risulta teoricamente molto efficace, poiché sostituisce la pagina meno recentemente usata. Tuttavia, nei sistemi reali, LRU non è facilmente implementabile quando si dispone solo di un **bit di riferimento** per ogni pagina (il **ref bit**), che indica semplicemente se la pagina è stata usata di recente. Questo bit non permette di rappresentare con precisione l'istante dell'ultimo utilizzo, rendendo **impossibile una vera politica LRU**.

Per questo motivo si utilizzano strategie approssimate come la **Not Recently Used (NRU)**. Questa strategia imposta il **ref bit** a **1** ogni volta che una pagina viene acceduta. Periodicamente, il sistema operativo azzerava tutti i bit di riferimento. In caso di **page fault**, viene selezionata per la sostituzione una pagina il cui bit sia **0**, ovvero che **non è stata usata di recente**. Se tutte le pagine hanno bit impostati a 1, questi vengono **azzerati** e la selezione avviene **casualmente**.

Per migliorare questa strategia si impiegano gli **algoritmi di clock**, che organizzano le pagine in una **lista circolare** e usano uno o due puntatori mobili per gestire la sostituzione.

Algoritmo di Clock ad una Lancetta

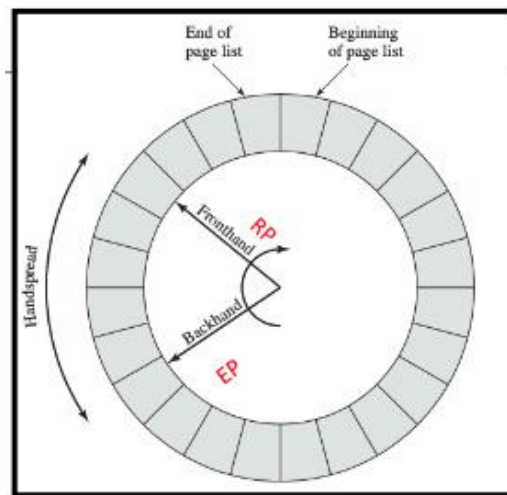


Il primo algoritmo, detto **Clock a una lancetta**, utilizza un solo **puntatore** che scorre ciclicamente tra i frame. **Quando serve sostituire una pagina**, il sistema controlla quella attualmente puntata. Se il suo **ref bit** è **1**, significa che è stata usata di recente: il bit viene **azzerato**, la pagina viene risparmiata e il puntatore avanza. Se invece il bit è già a **0**, allora la pagina viene **sostituita**. Questa tecnica dà alla pagina una "**seconda possibilità**" prima di essere eliminata.

Supponiamo, ad esempio, che il puntatore si trovi sul **frame 2** che contiene la **pagina 45**, e che il suo bit sia a 1. In presenza di un page fault (es. per la **pagina 727**), la pagina 45 non viene immediatamente sostituita: il suo bit viene resettato e il puntatore prosegue. Quando si raggiunge il **frame 4**, che contiene la **pagina 556** con bit già impostato a **0**, questa viene **sostituita** con la pagina 727 e il suo bit viene impostato a **1**.

Tuttavia, il **Clock a una lancetta** presenta alcune limitazioni. Poiché il reset del **ref bit** avviene durante la stessa scansione usata per scegliere la pagina da sostituire, possono verificarsi due situazioni problematiche. Se **tutti i bit** sono a **1**, il puntatore potrebbe dover compiere un giro completo per trovare una pagina candidabile, causando **ritardi**. Al contrario, se quasi tutti i bit sono a **0**, il sistema tende a sostituire pagine appena utilizzate, comportandosi in modo simile a una politica **FIFO**.

Algoritmo di Clock a due Lancette



Per superare i limiti del **clock a una lancetta**, si introduce l'**algoritmo di clock a due lancette**. In questo approccio si usano due puntatori: il **puntatore di reset (RP)**, che ha il compito di **azzerare i bit di riferimento**, e il **puntatore di controllo (EP)**, che invece li **esamina** per decidere se una **pagina** può essere **sostituita**. I due puntatori avanzano lungo la **lista circolare dei frame**, mantenendo tra loro una **distanza fissa**. Questa distanza definisce in pratica che cosa significa "**recentemente usata**": se una pagina viene **referenziata** dopo il passaggio di RP e prima di quello di EP, il suo **bit** sarà a **1** e quindi non verrà sostituita; se invece non viene più toccata in quell'intervallo, il suo bit resta a **0** e la pagina diventa un buon **candidato per la rimozione**.

Immaginiamo ad esempio di avere quattro pagine in memoria, etichettate **A, B, C e D**. RP passa su **B** e **azzer**a il suo bit di riferimento; poco dopo il **processo** accede nuovamente a **B**; quindi, il bit torna a **1** (è l'hardware che si occupa di alzare il bit a 1 quando la pagina viene referenziata da un processo). Quando **EP** arriva a **B**, vede il bit settato e la pagina viene **salvata**. Al contrario, **C** non viene più **utilizzata** dopo il passaggio di **RP**: il bit rimane **0**, e quando **EP** la incontra, **C** viene scelta per la **sostituzione**.

La scelta della **distanza** tra i due puntatori è **cruciale**. Una distanza troppo grande finisce per assomigliare al **clock classico**, con la necessità di **scansionare** molti frame prima di trovare una **vittima**, mentre una distanza troppo breve impedisce alle pagine di avere una reale **seconda possibilità** e l'algoritmo si riduce di fatto a **FIFO**. La distanza ottimale dipende molto dal **carico di lavoro** e dalla **località di riferimento** dei programmi: processi con forte **località temporale** beneficiano di una distanza più breve, mentre **accessi** più dispersi richiedono una distanza maggiore per evitare **rimpiazzi prematuri**.

In conclusione, sia il clock a una che a due lancette sono strategie **efficienti e realistiche** per la sostituzione delle pagine. La versione a due lancette, sebbene leggermente più complessa, offre un controllo più raffinato sulla **definizione di "recentemente usata"**, migliorando le prestazioni in molte situazioni pratiche.

Thread demoni dell'MMU

Nella gestione della **memoria virtuale**, oltre al normale **gestore dei page fault**, lavorano in background due **thread demoni** che si occupano delle operazioni di servizio necessarie al sistema. Il primo è il **gestore dei frame liberi**, che mantiene aggiornata la **lista dei frame disponibili**. Questo thread entra in azione solo quando i frame liberi scendono sotto una certa **soglia**: in quel momento applica la **politica di sostituzione delle pagine**, sceglie quali pagine rimuovere dalla memoria, aggiorna la **tabella delle pagine** e, se necessario, ordina la scrittura sul disco nel caso il **dirty bit** sia impostato. Quando invece i frame liberi sono già sufficienti, il thread rimane sospeso per non consumare risorse.

Il secondo è il **gestore dell'I/O di pagina**, che coordina le operazioni di **page-in** e **page-out** verso il disco di **swap**. Grazie a questo demone, le operazioni di trasferimento tra RAM e disco possono avvenire in parallelo rispetto all'esecuzione dei processi, evitando che questi restino bloccati più del necessario.

Il funzionamento combinato di questi thread consente di avere quasi sempre dei frame pronti per essere assegnati in caso di **page fault**. Se una pagina da caricare si trova già nella lista dei frame liberi e non necessita di un vero **page-in**, il sistema può riassegnarla immediatamente senza dover ricorrere al disco, ottenendo un notevole miglioramento delle **prestazioni** complessive.

Allocazione della memoria ad un processo

In un sistema operativo, ogni processo in esecuzione necessita di una certa quantità di **memoria fisica** per mantenere in RAM le pagine di cui ha bisogno. Questa memoria può essere assegnata secondo due strategie principali: una **allocazione fissa**, dove la quantità di memoria non cambia nel tempo, e una **allocazione variabile**, che si adatta dinamicamente al comportamento del processo.

Nel caso dell'**allocazione fissa**, il processo riceve una quantità predefinita di frame di memoria per tutta la durata della sua esecuzione. Questa strategia è semplice da gestire e ha un overhead contenuto, ma può risultare inefficiente. Infatti, se la quantità assegnata è **troppo piccola**, il processo può incorrere in **frequenti page fault**; se invece è **troppo grande**, si rischia uno **spreco di memoria**.

L'**allocazione a memoria variabile** consente invece di adattare dinamicamente la memoria assegnata a un processo, aumentando o riducendo il numero di frame in base alle sue **effettive necessità**. Per gestire questo tipo di allocazione, si fa uso del concetto di **working set**, che rappresenta l'insieme delle **pagine referenziate di recente**. Più precisamente, il working set di un processo in un dato istante è composto dalle pagine utilizzate nelle **ultime Δ istruzioni**, dove Δ (delta) è un parametro di sistema che definisce la **finestra del working set**.

Working Set

A ogni istante, per ogni processo, si può determinare il **working set $WS(t, \Delta)$** e la sua **dimensione $WSS(t, \Delta)$** , cioè il numero di pagine uniche referenziate in quella finestra. È possibile che una stessa pagina venga referenziata più volte, per cui WSS può essere anche molto più piccolo di Δ . Se un processo accede frequentemente alle stesse pagine, il suo working set sarà **piccolo**, mentre se accede a pagine diverse, la dimensione sarà **maggiore**.

Il sistema operativo può usare la dimensione del working set per determinare quanta memoria assegnare a ciascun processo, impostando il valore di **alloc** uguale a **WSS**. Se la memoria totale richiesta da tutti i working set supera il numero di frame disponibili, il sistema deve **abbassare il grado di multiprogrammazione**, ossia **sospendere uno o più processi** per liberare memoria. In questo caso, viene scelto un processo da sospendere, si esegue un **page-out** di tutte le sue pagine eventualmente modificate, e i suoi frame vengono liberati e resi disponibili per altri processi. Il valore di **alloc** per il processo sospeso viene impostato a zero, ma il suo working set rimane registrato per un possibile futuro ripristino.

Al contrario, se la somma dei working set dei processi attivi è **inferiore** alla memoria fisica disponibile, è possibile **aumentare il grado di multiprogrammazione**. In tal caso, si può riattivare un processo sospeso e assegnargli nuovamente una quantità di frame pari al suo WSS . Quando un processo viene riattivato, il sistema può decidere di **caricare immediatamente tutte le pagine del suo working set**, oppure solo la **pagina che ha causato il page fault**, e lasciare che le altre vengano caricate man mano che servono. Quest'ultima opzione consente di ridurre l'overhead iniziale e ottimizzare l'utilizzo della memoria.

Va sottolineato che calcolare $WS(t, \Delta)$ e alloc in tempo reale per ogni processo può essere **computazionalmente costoso**, soprattutto in sistemi con molti processi attivi. Per questo motivo, nella pratica il sistema operativo **calcola i working set periodicamente**, ad esempio ogni intervallo di tempo prestabilito. I valori ottenuti vengono poi utilizzati come riferimento per il successivo intervallo, fino al prossimo aggiornamento. Questo approccio riduce il carico computazionale e permette una gestione più efficiente.

L'utilizzo del modello del **working set** consente al sistema operativo di adattare dinamicamente l'allocazione della memoria, garantendo buoni livelli di **località dei riferimenti**, prevenendo il **thrashing** e migliorando l'efficienza del sistema. Allo stesso tempo, offre un controllo efficace sul **grado di multiprogrammazione**, bilanciando in modo intelligente le esigenze dei singoli processi con le risorse complessive del sistema.

Copy On Write

Il comando **fork()** crea un **processo figlio** che è, inizialmente, una **copia esatta del processo padre**, comprese le aree di memoria. Questo significa che la memoria viene **duplicata** e che entrambi i processi possono lavorare in modo **indipendente**. Tuttavia, questa duplicazione è costosa in termini di tempo e risorse, poiché richiede di copiare tutta la memoria anche se, nella pratica, spesso il figlio viene eseguito solo per lanciare un altro programma tramite **exec()**. In questi casi, la memoria del processo figlio verrebbe immediatamente **sovrascritta**, rendendo l'uso di **fork()** inefficiente e generando un overhead inutile. Per questo motivo, in passato si utilizzava una versione alternativa chiamata **vfork()**, che però è oggi **deprecata**.

I sistemi moderni adottano invece una strategia chiamata **Copy-on-Write (COW)**. Con questo meccanismo, quando viene creato un processo figlio, **non viene copiata immediatamente tutta la memoria del padre**. Inizialmente, padre e figlio **condividono la stessa area di memoria**, che viene marcata come **sola lettura**. La copia vera e propria viene eseguita **solo nel momento in cui uno dei due processi prova a modificare una pagina**. In quel caso, il sistema operativo crea una **copia privata** della pagina modificata per il processo che ha effettuato la scrittura.

Questo approccio consente un grande risparmio di memoria, poiché molte pagine potrebbero **non essere mai modificate**, e quindi non devono mai essere duplicate. Le pagine condivise sono gestite attraverso una **tabella delle pagine** in cui è presente un **flag copy-on-write**, che segnala al sistema che la pagina può essere condivisa fino a che non viene modificata.

Per chiarire il funzionamento, consideriamo due processi, A e B. Dopo una **fork()**, entrambi puntano alle **stesse pagine di memoria**. Le tabelle delle pagine di A e B indicano che queste pagine sono **condivise**, ad esempio con un flag **c**. Quando il processo A modifica una delle pagine, il sistema crea una **copia privata** della pagina solo per A. A quel punto, la pagina modificata viene **esclusa dalla condivisione** e il flag viene rimosso. Se anche il processo B effettua una modifica alla stessa pagina, verrà creata una **seconda copia privata**, questa volta per B. Nel frattempo, la pagina originaria che era condivisa tra padre e figlio può essere **marcata come libera**, perché non è più utilizzata.

Questo comportamento assicura che ogni processo lavori in modo **isolato**, pur evitando la duplicazione inutile della memoria. In pratica, appena un processo effettua una scrittura su una pagina condivisa, questa viene **copiata solo per lui**, lasciando inalterata la copia usata dagli altri processi. Il grande vantaggio del Copy-on-Write è che la **memoria viene realmente duplicata solo quando serve**, ovvero al momento della modifica.

In conclusione, il Copy-on-Write è una tecnica fondamentale per ottimizzare l'uso della memoria nei sistemi operativi moderni. Consente di **ridurre i tempi e il costo della fork()**, evitando copie superflue, e garantisce al tempo stesso che ogni processo abbia **la propria copia privata dei dati modificati**, preservando l'**integrità** e l'**isolamento** tra padre e figlio.

Capitolo 11 - File System

Introduzione

Il **file system** rappresenta un'**astrazione fondamentale** dei sistemi operativi per la gestione della **memoria secondaria**, finalizzata alla **memorizzazione a lungo termine** delle informazioni. Questa astrazione consente di organizzare, accedere e condividere dati in maniera strutturata, garantendo che essi siano **permanenti, accessibili e condivisibili** da più processi, anche simultaneamente. In termini pratici, il file system espone un insieme di **API** che permettono un **accesso conveniente** e veloce ai file, rendendo più agevole la gestione dei dati da parte delle applicazioni.

Un file system non si limita a scrivere dati su disco, ma garantisce la **memorizzazione affidabile**, minimizzando il rischio di **corruzione** dei dati e supportando la **condivisione tra utenti autorizzati**. Per assicurare queste proprietà, il file system lavora in stretta sinergia con un'altra astrazione: l'**Input/Output Control System (IOCS)**. Mentre il file system gestisce gli aspetti logici e strutturali dei dati (come directory, permessi e metadati), l'IOCS si occupa della gestione **efficiente dei dispositivi di I/O**, ottimizzando le operazioni di lettura e scrittura a basso livello.

Il modello a strati evidenzia come il file system sia solo una delle componenti che interagiscono con i dispositivi fisici: sopra di esso troviamo applicazioni, librerie e meccanismi di cache; sotto, invece, interfacce a blocchi, **driver di dispositivo** e il livello fisico vero e proprio. Questo approccio modulare permette di **separare gli aspetti logici** da quelli **fisici**, migliorando sia le prestazioni che la manutenzione del sistema.

Dal punto di vista funzionale, il file system considera un file come una **collezione di dati di proprietà dell'utente**, strutturata e accessibile nel tempo, mentre l'IOCS vede lo stesso file come un **archivio di dati** da gestire in modo rapido ed efficiente attraverso le risorse hardware disponibili. I dati trattati dal file system comprendono sia il **contenuto informativo** (dati nel file), sia i **metadati** (informazioni di controllo come permessi, timestamp, dimensioni, ecc.).

File

Un **file** è una collezione di **dati identificata da un nome**, gestita dal sistema operativo attraverso una struttura organizzata nota come **file system**. Questa struttura organizza i dati in una **gerarchia di file e directory**, mantenendo la **coerenza** e l'**integrità** delle informazioni memorizzate. Ogni file è composto da una **parte di dati** (contenuto effettivo) e una **parte di metadati**, cioè informazioni aggiuntive come dimensione, proprietà, permessi di accesso e timestamp, che vengono utilizzate dal sistema operativo per gestire il file stesso.

Il **file system** si occupa della **manipolazione dei file** da parte dei processi tramite apposite **API**, che fungono da interfaccia tra i programmi e le operazioni di I/O. Esistono due principali **tipi di file**: i **file strutturati**, composti da una **collezione di record** organizzati in campi, dove uno dei campi agisce da **chiave primaria**; e gli **stream di byte**, che rappresentano una sequenza lineare di byte senza struttura interna, utilizzati tipicamente per dati binari o testuali non organizzati.

L'**accesso ai dati** può avvenire in maniera **sequenziale** o **casuale**, in base alla natura del dispositivo di memorizzazione e al tipo di organizzazione del file. I dispositivi di I/O determinano in larga parte l'efficienza di un certo tipo di accesso: ad esempio, un **nastro magnetico** è adatto all'accesso sequenziale, mentre un **disco** consente sia l'accesso sequenziale sia quello casuale.

Nel contesto del file system, si distinguono tre **organizzazioni fondamentali** dei file:

1. **Organizzazione sequenziale**: i record sono memorizzati in ordine (ascendente o discendente) rispetto a una **chiave**. Le operazioni sono limitate alla lettura o salto del record successivo o precedente. È particolarmente efficiente per file **preordinati** o per **stream di byte**.
2. **Organizzazione diretta**: consente l'accesso **casuale** ai record mediante il calcolo dell'indirizzo fisico dal valore della chiave. I file sono chiamati file ad accesso diretto. Un comando read/write indica il valore del campo chiave. Il valore chiave è usato per generare l'indirizzo di un record sul dispositivo fisico. Pur garantendo efficienza nei casi in cui l'accesso non segue una sequenza, presenta alcuni **svantaggi** come l'utilizzo di **record fittizi** per chiavi non utilizzate e **sovraccarico di CPU** per il calcolo dell'indirizzo.
3. **Organizzazione indicizzata**: utilizza una **struttura di indice** per stabilire la posizione del record. Si distingue tra **indicizzazione pura** (chiave → indirizzo disco) e **indicizzazione sequenziale**, in cui un indice punta a una sezione del disco e la ricerca all'interno di quella sezione avviene in modo sequenziale.

Nel ciclo di vita di un file all'interno di un programma, sono previste una serie di **operazioni standard**: **apertura, lettura/scrittura, chiusura, copia, rinomina, cancellazione** e **specificazione dei permessi di accesso**. Tali operazioni sono gestite dal **modulo del file system** e dal **modulo IOCS**, che collaborano per eseguire le istruzioni applicative e per interagire con il dispositivo fisico.

Questa architettura permette al sistema operativo di **astrarre** i dettagli fisici dell'archiviazione e presentare ai processi una **visione logica uniforme e affidabile** dei dati, garantendo **efficienza, affidabilità e sicurezza** nell'accesso e nella gestione dei file.

Directory

Nel contesto dei sistemi operativi, una **directory** è una struttura fondamentale del **file system** che organizza i file all'interno di una **struttura ad albero**. Ogni directory può contenere file e altre sottodirectory, configurandosi come un **grafo aciclico** in cui ogni nodo rappresenta una directory o un file. L'elemento di radice è la **directory madre** (o root), da cui si diramano le **directory figlie**. Questa struttura consente la **coesistenza di file con lo stesso nome**, purché collocati in directory diverse.

A livello implementativo, una directory è una **tabella** i cui elementi contengono informazioni come il **nome del file**, il **tipo e la dimensione**, la **posizione sul disco** (location info), i **permessi di accesso** (protection info), il **conteggio dei processi attivi** su quel file (open count), e altri attributi (flag, lock, info varie). In ambienti Unix, queste informazioni sono memorizzate tramite riferimenti all'**i-node**, struttura dati che descrive un file o una directory.

Ogni utente ha una **home directory**, che rappresenta il punto di ingresso personale nel file system e garantisce **isolamento e sicurezza**, evitando accessi non autorizzati. L'utente può interagire con file e directory tramite **pathname assoluti** (che indicano il percorso completo a partire dalla root) o **pathname relativi** (che partono dalla directory corrente).

Per supportare la **condivisione dei file tra utenti**, il file system può usare collegamenti detti **link simbolici**, i quali permettono di accedere a file o directory presenti in percorsi differenti senza duplicare i dati. Tuttavia, se il file di destinazione viene rimosso o spostato, il link perde validità. Per evitare ridondanze e facilitare la navigazione e il riutilizzo dei file condivisi, è utile rappresentare la struttura delle directory come un **grafo aciclico diretto** (DAG), dove i link rappresentano archi orientati tra nodi esistenti.

La **protezione dei file** è gestita mediante il campo **protection info**, che contiene informazioni sui privilegi di accesso. Tali permessi sono tipicamente organizzati in una **Access Control List (ACL)**, ovvero una lista che associa ciascun utente o gruppo di utenti a un insieme di privilegi. I permessi standard previsti sono **read**, **write** ed **execute**, e il loro corretto controllo è essenziale per garantire la **sicurezza e la condivisione controllata** dei file nel sistema.

Questa struttura organizzativa consente agli utenti di avere **libertà di assegnare nomi, organizzare gerarchicamente i dati**, e **gestire in modo sicuro e flessibile l'accesso ai file** da parte di più utenti o processi.

Allocazione di Spazio su Disco

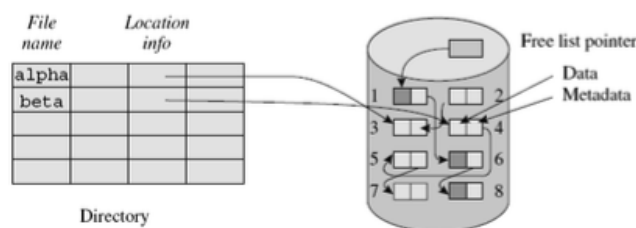
L'**allocazione sul disco** rappresenta una delle funzionalità centrali del **file system**, responsabile dell'assegnazione efficiente dello spazio fisico per i file all'interno di una **partizione**. Ogni partizione può contenere un **file system**

indipendente, e l'allocazione dello spazio è vincolata a scelte architetturali che influenzano le prestazioni, la gestione dello spazio libero e la frammentazione.

I sistemi storici facevano uso di **allocazione contigua**, che però introduceva gravi problemi di **frammentazione esterna**, oggi evitati grazie all'impiego di modelli di **allocazione non contigua**. Quest'ultima pone attenzione a tre problematiche principali:

1. **Gestione dello spazio libero**, per cui si impiegano strutture come la **free-list** (lista concatenata di blocchi liberi) o la **Disk Status Map (DSM)**, anche detta *bit map*, che associa a ogni blocco del disco un bit indicante se è **libero** o **occupato**.
2. **Riduzione dei movimenti della testina** nei dischi magnetici, che è una componente meccanica costosa in termini di tempo. Per ottimizzare l'accesso, si cerca di allocare blocchi **contigui logicamente**, formando **cluster** o gruppi di cilindri.
3. **Accesso ai dati**, che dipende dal tipo di allocazione adottata: **concatenata** o **indicizzata**.

Allocazione Concatenata



Nell'**allocazione concatenata**, ogni blocco di file contiene, oltre ai dati, anche l'**indirizzo del blocco successivo**, realizzando così una sorta di lista concatenata. Questo approccio è **semplice da implementare** e comporta un **basso overhead**, risultando efficiente per file **sequenziali**. Tuttavia, è **poco affidabile** in caso di corruzione dei metadati, poiché la perdita di un solo puntatore può compromettere l'intero file.

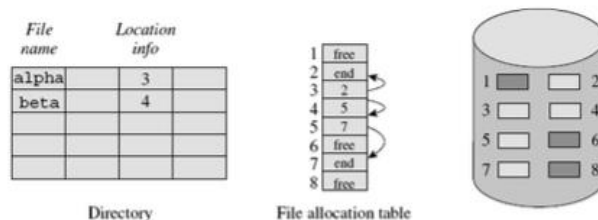
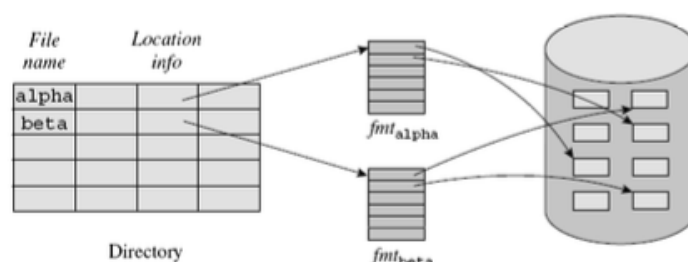


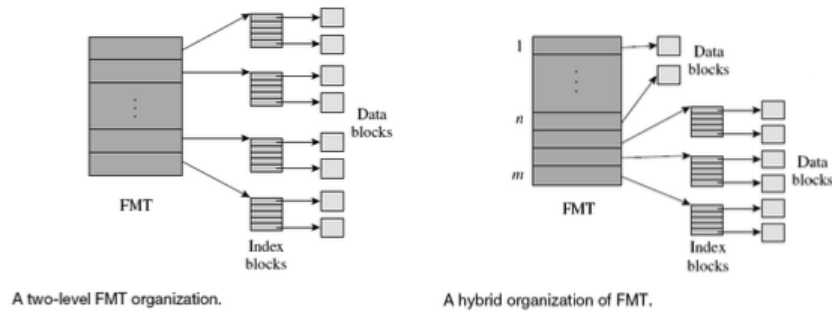
Figure 13.14 File Allocation Table (FAT).

Una variante di questo schema è l'**allocazione concatenata con File Allocation Table (FAT)**, dove i **metadati sono separati dai dati** e memorizzati in una tabella centrale (FAT) che contiene, per ogni blocco, l'indirizzo del **blocco successivo**. In tal caso, la **Location Info** nella directory punta a un **indice** della FAT, che permette di ricostruire l'intera catena. Per evitare rallentamenti, la FAT viene spesso **mantenuta in memoria**.

Allocazione indicizzata



Nell'**allocazione indicizzata**, si utilizza una **File Map Table (FMT)**, una struttura che memorizza direttamente gli **indirizzi dei blocchi** assegnati al file.



La forma più semplice della FMT è un array, ma per file di dimensioni elevate si impiegano **FMT a più livelli**, dove le entry di primo livello puntano a tabelle di secondo livello e così via. Esiste anche una **FMT ibrida**, in cui i file piccoli sono gestiti direttamente con un solo livello, mentre per i file più grandi si ricorre a livelli multipli. Questa organizzazione garantisce un **accesso diretto ed efficiente** anche per file non sequenziali, a scapito di una maggiore complessità strutturale.

In sintesi, il file system moderno adotta **allocazioni non contigue** per affrontare le sfide di efficienza, affidabilità e gestione dello spazio, scegliendo tra approcci concatenati o indicizzati in base alle caratteristiche dei file e dei dispositivi di memorizzazione.

Affidabilità del file system

L'**affidabilità del file system** rappresenta la capacità di garantire l'**integrità dei dati e dei metadati** anche in presenza di **guasti** o **malfunzionamenti** del sistema. Un file system affidabile deve quindi assicurare che operazioni come **creazione, cancellazione o aggiornamento dei file** avvengano in modo corretto e che i dati non vadano persi o corrotti in seguito a eventi imprevisti, come crash di sistema o mancanza di alimentazione.

Due sono gli approcci fondamentali per garantire affidabilità:

1. **Recovery**: mira a **ripristinare lo stato precedente** del file system, ovvero quello consistente prima del guasto. Questo si ottiene mediante l'utilizzo di **backup**, che possono essere completi o **incrementali**. Il backup incrementale consente di salvare solo i blocchi modificati rispetto all'ultimo salvataggio, riducendo l'**overhead di creazione e rielaborazione**.
2. **Tolleranza ai guasti**: mira a **prevenire** la perdita di dati o l'**incoerenza dei metadati**. Tra le principali strategie adottate vi è la **memorizzazione stabile**, in cui si mantengono **due copie dei dati** o dei metadati, rendendoli accessibili in qualsiasi momento. L'aggiornamento avviene solo dopo aver verificato che entrambe le copie siano coerenti. Un altro approccio è l'utilizzo di **azioni atomiche**, ispirate al modello transazionale: o tutte le modifiche vengono completate, oppure nessuna. Queste operazioni sono tracciate tramite **intention list** e **commit flag**, che definiscono quando un'azione è stata effettivamente portata a termine.

Un meccanismo moderno che unisce i due approcci è il **journaling file system**, che registra in un **log (journal)** tutte le operazioni effettuate, prima che vengano applicate ai dati effettivi. Questo sistema consente un **ripristino rapido** in caso di interruzione improvvisa, garantendo coerenza. Esistono diversi livelli di journaling: il **write-behind** protegge solo i metadati, il **journal ordered** garantisce la scrittura ordinata dei dati e metadati, mentre il **full data journaling** protegge entrambi.

La **perdita di consistenza** si verifica quando i metadati risultano parzialmente aggiornati a causa di un guasto durante una scrittura, ad esempio se viene interrotta l'aggiunta di un blocco a un file. Ciò può rendere i dati **inaccessibili** o causare **scambi tra file**. Per ridurre queste possibilità, è essenziale che il file system implementi meccanismi di protezione proattivi.

Unix File System (UFS)

Il **Unix File System (UFS)** è uno dei file system più longevi e influenti, basato su una struttura fortemente **modulare e indicizzata** che consente un'efficiente gestione dei file, sia in termini di accesso sia di allocazione dello spazio. Le **strutture dati principali** del file system Unix includono:

- **Directory entries**, che contengono esclusivamente il **nome del file**, separando quindi l'identificazione simbolica dalla gestione fisica.
- **i-node**, struttura centrale che memorizza **attributi del file** come dimensione, ID dell'utente proprietario, permessi, timestamp e **informazioni sull'allocazione dei blocchi**.
- **File structure**, che mantiene lo **stato di un file aperto** durante l'esecuzione (offset e puntatore all'i-node).
- **File descriptors**, tabelle per processo che contengono riferimenti ai file aperti.

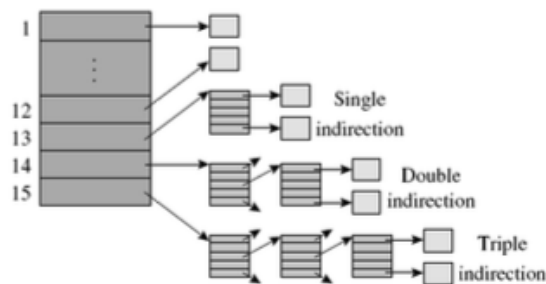


Figure 13.33 Unix file allocation table.

Uno degli aspetti più rilevanti di UFS è l'**allocazione indicizzata a tre livelli**: i primi blocchi dell'i-node contengono puntatori diretti ai blocchi dati; successivamente, si trovano puntatori a blocchi di indirizzamento **singolo**, **doppio** e **triplo** (indirection), che permettono di gestire file di dimensioni crescenti in modo scalabile.

Dal punto di vista della **condivisione dei file**, il modello Unix adotta una **semantica immediata**, dove ogni scrittura su un file da parte di un processo diventa istantaneamente **visibile agli altri processi** che stanno accedendo allo stesso file, garantendo coerenza globale tra i processi concorrenti.

Berkeley Fast File System (FFS)

Il **Berkeley Fast File System (FFS)** è una variante evoluta di UFS, progettata per **superare le limitazioni del file system s5fs**. FFS introduce **nomi di file più lunghi**, **supporto per link simbolici**, e un insieme di **ottimizzazioni per l'allocazione dei blocchi** su disco. In particolare, FFS:

- Utilizza **blocchi di grandi dimensioni** (fino a 8KB) per ridurre l'overhead di gestione;
- Adotta **gruppi di cilindri** per minimizzare il movimento della testina nei dischi magnetici;
- Implementa strategie per **ridurre la latenza di rotazione**, migliorando le performance in letture sequenziali.

Insieme, UFS e FFS rappresentano modelli altamente efficienti per la gestione dei file in ambienti multiutente e multitasking, garantendo una combinazione di **flessibilità strutturale**, **robustezza nell'accesso concorrente** e **ottimizzazione a basso livello per le prestazioni fisiche** del disco.

Capitolo 12 – Input/Output Control System

Introduzione

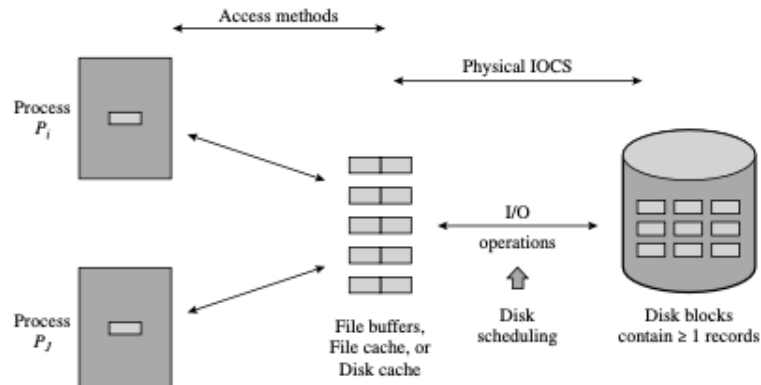


Figure 14.1 Implementation of file operations by the IOCS.

L'**IOCS (Input/Output Control System)** è la componente del sistema operativo che **realizza effettivamente** le operazioni di I/O richieste dal file system, interagendo direttamente con i dispositivi fisici di memorizzazione. La sua funzione è quella di garantire che l'accesso ai file avvenga in modo **efficiente, sicuro e ottimizzato**. L'architettura dell'IOCS è organizzata in **strati**, ciascuno con compiti distinti ma integrati:

- Lo **strato del metodo di accesso** si occupa dell'apertura e chiusura dei file, nonché della gestione di operazioni di lettura e scrittura. Questo strato ottimizza le performance mediante tecniche come **buffering** e **caching**, impiegando strutture come il **file cache** o il **disk cache**.
- Lo **strato fisico dell'IOCS** è responsabile dell'avvio delle operazioni di I/O, del tracciamento del loro stato, del recupero da errori e della **programmazione del disco** (disk scheduling). In quest'ambito, vengono adottate politiche per ottimizzare l'uso della testina di lettura/scrittura nei dispositivi meccanici.

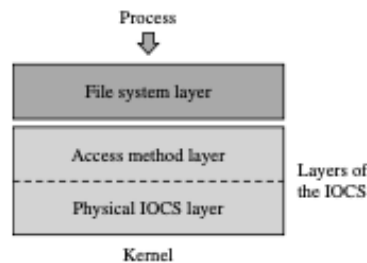
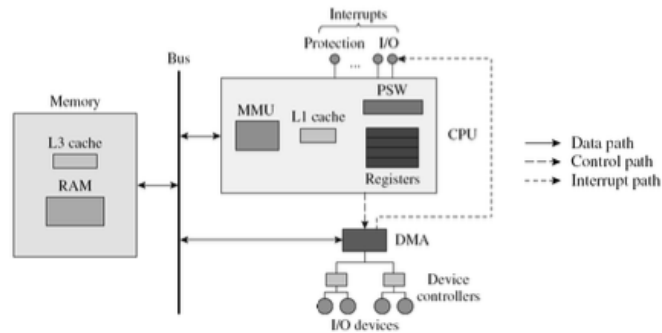


Figure 14.2 Layers of the file system and the IOCS.

L'interazione tra processi utente e dispositivi fisici avviene tramite il kernel, che intermedia le operazioni attraverso questi livelli, come mostrato dalla figura a strati: file system $\rightarrow$ access methods $\rightarrow$ physical IOCS. In questo modello, l'**accesso ai dati** su disco si realizza passando per **buffer, cache, scheduler** e infine per i **blocchi del disco**, che contengono uno o più **record**.

Organizzazione dell'I/O



La **gestione dell'I/O** è implementata tramite un sistema complesso che coinvolge più componenti hardware. La **CPU** comunica con i dispositivi di I/O attraverso un **controller DMA (Direct Memory Access)**, che consente il trasferimento diretto dei dati tra memoria e periferiche senza intervento diretto della CPU, liberando così risorse computazionali. Ogni **dispositivo** è associato a un **controller**, e l'unità identificativa di ciascun dispositivo è rappresentata dalla coppia (**controller ID, device ID**). Il **DMA** e il **controller di dispositivo** cooperano per gestire le operazioni in modo asincrono, attivando un **interrupt** a fine trasferimento.

L'IOCS è quindi una **struttura modulare** che si colloca tra il file system e l'hardware, garantendo **astrazione, efficienza nell'accesso, ottimizzazione dell'I/O e gestione degli errori**. La sua architettura stratificata permette di separare la **logica di accesso ai file** dalla **gestione fisica delle risorse**, favorendo sia la portabilità che le performance complessive del sistema.

Operazioni di I/O

Le **operazioni di I/O** rappresentano il meccanismo attraverso il quale un processo interagisce con un dispositivo esterno per leggere o scrivere dati. Ogni operazione di I/O è definita da un insieme di parametri fondamentali: l'**operazione da eseguire** (es. read, write), l'**indirizzo del dispositivo I/O** (identificato dalla coppia <controller_id, device_id>), il **numero di byte da trasferire**, e gli **indirizzi delle aree di memoria coinvolte**.

La **CPU** ha il compito di **iniziare l'operazione** eseguendo un'istruzione I/O che punta a una zona di memoria contenente un **insieme di comandi**. Tuttavia, l'effettivo trasferimento dei dati è gestito da un meccanismo dedicato: il **Direct Memory Access (DMA)**. Questo consente alla CPU di **liberarsi dal carico operativo**, continuando a elaborare altre istruzioni mentre l'I/O viene completato in parallelo.

Nel dettaglio, l'esecuzione di un'operazione I/O tramite **DMA di terza parte (Third Party DMA)** segue i seguenti passaggi:

1. Il **controller del DMA** trasmette i parametri dell'operazione al **controller del dispositivo**.
2. Il controller del dispositivo, una volta pronto, genera un segnale di **DMA request**.
3. Il **DMA riceve il segnale**, prende il controllo del **bus** e comunica un segnale di **DMA ack** al controller del dispositivo.
4. Il **controller del dispositivo** effettua il trasferimento dei dati verso o dalla **memoria**.
5. Terminato il trasferimento, il DMA emette un **interrupt di completamento I/O**, contenente l'indirizzo del dispositivo.
6. La **routine di gestione degli interrupt** identifica il dispositivo che ha completato l'operazione e attiva le **azioni appropriate** nel sistema operativo o nell'applicazione.

Un esempio di operazione di lettura può essere descritto con l'istruzione:

```
I/O-init(controller_id, device_id), I/O_command_addr
```

dove `I/O_command_addr` è l'indirizzo della memoria che contiene i comandi per:

1. Posizionare la **testina del disco** sulla traccia `track_id`.
2. Leggere il **record** `record_id` e trasferirlo in memoria a partire dall'indirizzo `memory_add`.

Questa architettura **disaccoppia il controllo** dell'I/O dalla CPU, permettendo una gestione **efficiente, asincrona e parallela** delle operazioni di input/output. L'uso del **DMA** in combinazione con controller specializzati garantisce un'elevata **performance di sistema** e una **riduzione del carico computazionale** centrale.

Dispositivi di I/O

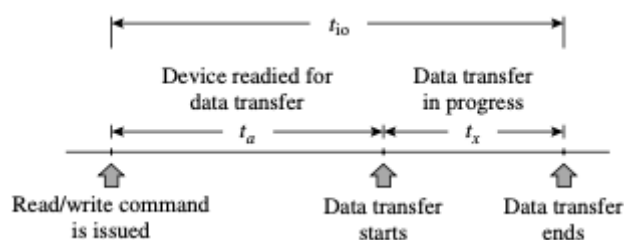
I **dispositivi di I/O** (Input/Output) rappresentano l'interfaccia tra il sistema di elaborazione e il mondo esterno, e si distinguono per **scopo** (input, output, memorizzazione), **natura dell'accesso** (sequenziale o casuale), e **modalità di trasferimento dei dati** (a caratteri o a blocchi). La classificazione e il comportamento di un dispositivo di I/O dipendono dai **principi fisici** con cui operano e soprattutto dalla loro **velocità di trasferimento**.

I **dispositivi lenti**, come tastiere, mouse o stampanti, operano in **modalità a carattere**, trasferendo un carattere alla volta tra memoria e periferica. Tali dispositivi dispongono di un **buffer interno**, il cui riempimento o svuotamento genera un **interrupt** al sistema. Questi dispositivi sono spesso **collegati direttamente al bus** attraverso un controller e non necessitano di canali dedicati di trasferimento ad alta velocità.

I **dispositivi veloci**, come dischi e nastri, operano in **modalità a blocchi**, trasferendo un intero blocco di dati per ogni operazione. In questo caso, la gestione del trasferimento è affidata a un **controller DMA**, che utilizza un **buffer dedicato** per garantire un **tasso di trasferimento costante**, necessario per evitare perdite o errori. I dati sono trasferiti in modo continuo tra la memoria e la periferica attraverso il buffer del DMA, garantendo alta efficienza.

La **prestazione** di un'operazione di I/O è quantificata attraverso due parametri principali:

- Il **tempo di accesso** t_a , che rappresenta l'intervallo tra il comando (es. read/write) e l'inizio effettivo del trasferimento;
- Il **tempo di trasferimento** t_x , ovvero il tempo necessario a completare il trasferimento dei dati (dal primo all'ultimo byte).

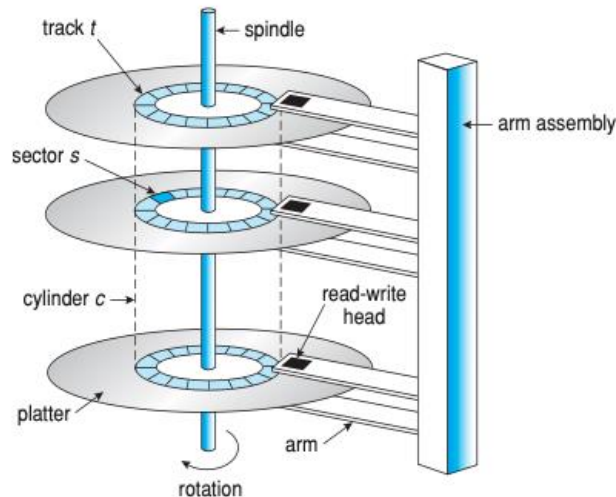


La somma di questi due tempi determina il **tempo totale di I/O** (intervallo tra esecuzione istruzione inizio I/O e completamento operazione):

$$t_{io} = t_a + t_x$$

Un'efficiente gestione di tali dispositivi richiede che il sistema operativo sia in grado di coordinare l'accesso al bus, le richieste di trasferimento, gli interrupt e la sincronizzazione tra memoria e periferiche. La conoscenza del tipo di dispositivo coinvolto è cruciale per ottimizzare la sequenza delle operazioni e garantire la **qualità del servizio I/O**.

Hard Disk



L'**hard disk** è un dispositivo di memorizzazione di massa basato sul principio dell'**elettromagnetismo**. È composto da più **piatti magnetici sovrapposti** che ruotano attorno a un asse comune. Ogni superficie dei piatti è dotata di una **testina di lettura-scrittura**, collegata a un braccio mobile, che si muove **radialmente** per accedere alle diverse tracce.

Ogni piatto è suddiviso in **tracce circolari**, le quali a loro volta sono organizzate in **settori**, ovvero porzioni di dimensione fissa destinate a contenere i dati. Le tracce che si trovano alla stessa altezza su tutti i piatti costituiscono un **cilindro**. Tale struttura è utile perché tutte le testine montate su un singolo braccio possono leggere contemporaneamente le tracce di un cilindro, **minimizzando i movimenti** della testina.

L'**indirizzamento di un record** avviene specificando **numero di cilindro, superficie e numero del record**. Questa organizzazione è pensata per ottimizzare l'accesso ai dati riducendo il tempo necessario per il posizionamento meccanico della testina.

Il **tempo di accesso** ai dati su disco è dato dalla somma di due componenti principali:

- Il **tempo di ricerca** t_s , necessario a posizionare la testina sulla traccia desiderata.
- La **latenza rotazionale** t_r , tempo richiesto affinché il settore cercato passi sotto la testina (in media 3–4 ms).

Quindi, il tempo di accesso totale è espresso come:

$$t_a = t_s + t_r$$

L'**organizzazione dei dati sul disco** è un elemento fondamentale per garantire l'efficienza dell'accesso. I **settori** sono disposti secondo tecniche che minimizzano lo **spreco di capacità** e l'**overhead meccanico**, come l'**hard sectoring** (hardware) o il **soft sectoring** (software). L'uso di **tecniche di distribuzione dei dati** e di **algoritmi di disk scheduling** consente di ridurre il numero di movimenti della testina e ottimizzare i tempi di accesso complessivi.

Per aumentare la capacità del disco, vengono montati più piatti: ogni superficie dispone di una propria testina, e tutte le testine sono montate su un unico **attuatore** che consente movimenti coordinati.

Scheduling del Disco

L'**organizzazione dei dati su disco** ha lo scopo di garantire un **accesso efficiente**, soprattutto nei sistemi dove l'**I/O su disco rappresenta un collo di bottiglia**. Le prestazioni dipendono fortemente dall'ordine in cui vengono soddisfatte le richieste, ed è qui che entrano in gioco gli **algoritmi di scheduling della testina**.

Un disco magnetico è suddiviso in **tracce** e **settori**, e le testine devono muoversi per posizionarsi sulla traccia corretta prima di poter leggere o scrivere. Il **tempo di movimento della testina** (seek time) è una delle principali cause di

ritardo, e la sua ottimizzazione è cruciale. Il tempo necessario per completare uno spostamento è calcolato con la formula:

$$T_{hm} = T_{const} + |track_1 - track_2| \times T_{pt}$$

dove T_{const} è un tempo fisso dovuto ai meccanismi del disco e T_{pt} rappresenta il tempo per il passaggio da una traccia all'altra.

I principali **algoritmi di scheduling del disco** sono:

- **First-Come, First-Served (FCFS)**: esegue le richieste in ordine di arrivo, semplice ma inefficiente in presenza di richieste sparse.
- **Shortest Seek Time First (SSTF)**: seleziona la richiesta che richiede il minor spostamento della testina, ottimizzando il tempo medio ma può causare **starvation** per richieste lontane.
- **SCAN**: la testina si muove in una direzione servendo tutte le richieste e poi inverte il senso una volta raggiunto il bordo, paragonabile a un "ascensore".
- **C-SCAN (Circular SCAN)**: come SCAN, ma la testina torna rapidamente all'inizio senza servire richieste nel ritorno, garantendo **latenza uniforme**.
- **LOOK e C-LOOK**: versioni ottimizzate di SCAN e C-SCAN, in cui la testina si ferma sull'ultima richiesta utile invece di arrivare fino all'estremo.

La **selezione dell'algoritmo** dipende dal carico del sistema e dal pattern delle richieste. **SSTF** è frequentemente usato per la sua semplicità ed efficacia, mentre **SCAN** e **C-SCAN** sono preferiti in sistemi ad alto carico perché offrono una **gestione più equa delle richieste**.

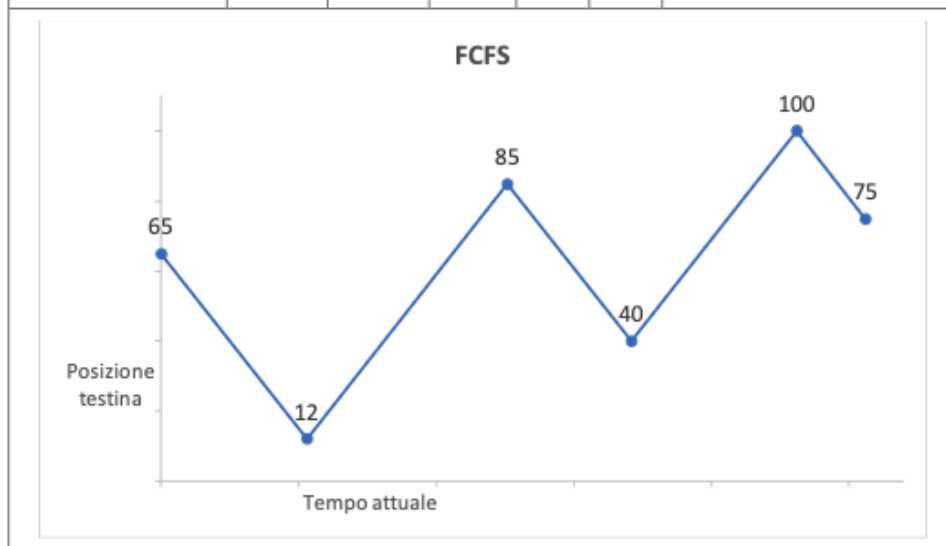
Idealmente, l'algoritmo di scheduling dovrebbe essere **modulare**, cioè separato dal resto del sistema operativo, in modo da poter essere aggiornato o sostituito in base alle esigenze del carico. Un default ragionevole può essere **SSTF** o **LOOK**.

Esempio scheduling del Disco

First-Come, First Served

Parametri	$T_{const} = 0 - T_{pt} = 1$ - traccia 65 - tempo attuale 160ms				
Numero seriale	1	2	3	4	5
Numero traccia	12	85	40	100	75
Tempo di arrivo	65	80	110	120	175

Dettagli	Decisioni di scheduling					Totale tempo di ricerca
	1	2	3	4	5	
Tempo attuale	160	213	286	331	391	226
Richieste in sospeso	1, 2, 3, 4	2, 3, 4, 5	3, 4, 5	4, 5	5	
Posizione testina	65	12	85	40	100	
Richiesta selezionata	1	2	3	4	5	
Tempo di ricerca	53	73	45	60	25	

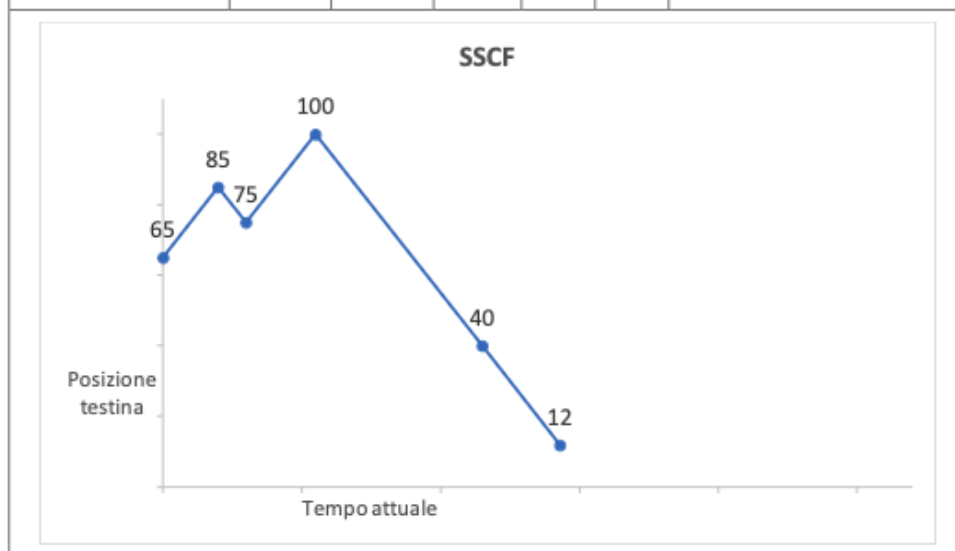


Al tempo 160 ci sono le richieste 1, 2, 3 e 4. Quella che è arrivata prima è la richiesta 1, quindi la testina si sposta da 65 a 12 con un tempo di ricerca $65 - 12 = 53$, di conseguenza allo step successivo saranno passati quindi $160 + 53 = 213ms$. La seconda richiesta ad essere servita è la 2, quindi la testina si sposta da 12 a 85 con tempo di ricerca $85 - 12 = 73$ e il prossimo tempo sarà di $213 + 73 = 286ms$. Così via fino alla fine. Il tempo di ricerca totale si ottiene sommando i vari tempi di ricerca e indica quante volte la testina si è spostata.

Shortest Seek Time First

Parametri	$T_{const} = 0 - T_{pt} = 1$ - traccia 65 - tempo attuale 160ms				
Numero seriale	1	2	3	4	5
Numero traccia	12	85	40	100	75
Tempo di arrivo	65	80	110	120	175

Dettagli	Decisioni di scheduling					Totale tempo di ricerca
	1	2	3	4	5	
Tempo attuale	160	180	190	215	275	143
Richieste in sospeso	1, 2, 3, 4	1, 3, 4, 5	1, 3, 4	1, 3	1	
Posizione testina	65	85	75	100	40	
Richiesta selezionata	2	5	4	3	1	
Tempo di ricerca	20	10	25	60	28	

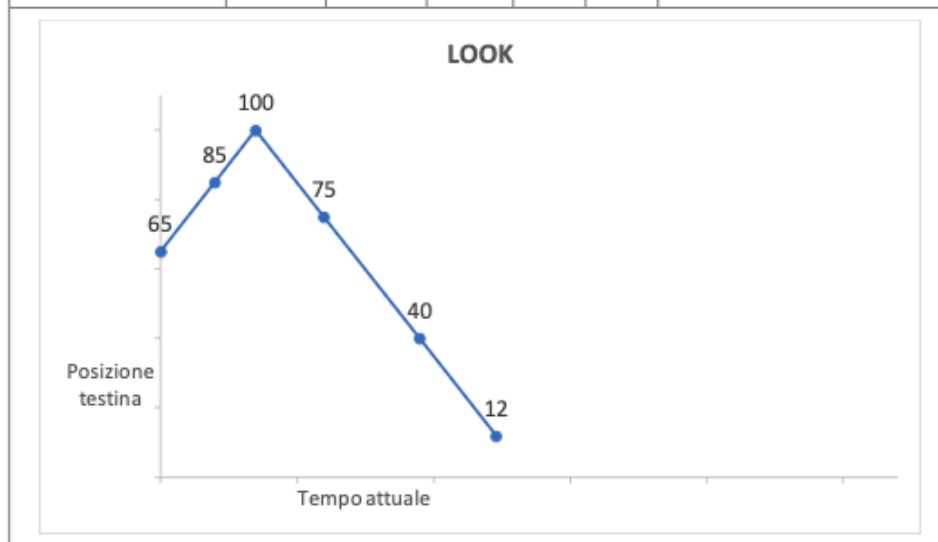


Al tempo 160 ci sono le richieste 1, 2, 3 e 4 e la testina si trova su 65. La richiesta la cui traccia si trova più vicino alla testina è la 2, quindi la testina passa da 65 a 85 con tempo di ricerca $85 - 65 = 20$ e prossimo tempo di $160 + 20 = 180ms$. La seconda richiesta ad essere servita è la 5 (arrivata al tempo 175), quindi la testina si sposta da 85 a 75 con tempo di ricerca $85 - 75 = 10$ e prossimo tempo di $180 + 10 = 190ms$. Così via fino alla fine. Il tempo di ricerca totale si ottiene sommando i vari tempi di ricerca e indica quante volte la testina si è spostata. Da notare che la richiesta 1 ha sofferto di starvation in quanto la posizione della testina è sempre distante.

LOOK

Parametri	$T_{const} = 0 - T_{pt} = 1$ - traccia 65 - tempo attuale 160ms				
Numero seriale	1	2	3	4	5
Numero traccia	12	85	40	100	75
Tempo di arrivo	65	80	110	120	175

Dettagli	Decisioni di scheduling					Totale tempo di ricerca
	1	2	3	4	5	
Tempo attuale	160	180	195	220	225	123
Richieste in sospeso	1, 2, 3, 4	1, 3, 4, 5	1, 3, 5	1, 3	1	
Posizione testina	65	85	100	75	40	
Richiesta selezionata	2	4	5	3	1	
Tempo di ricerca	20	15	25	35	28	

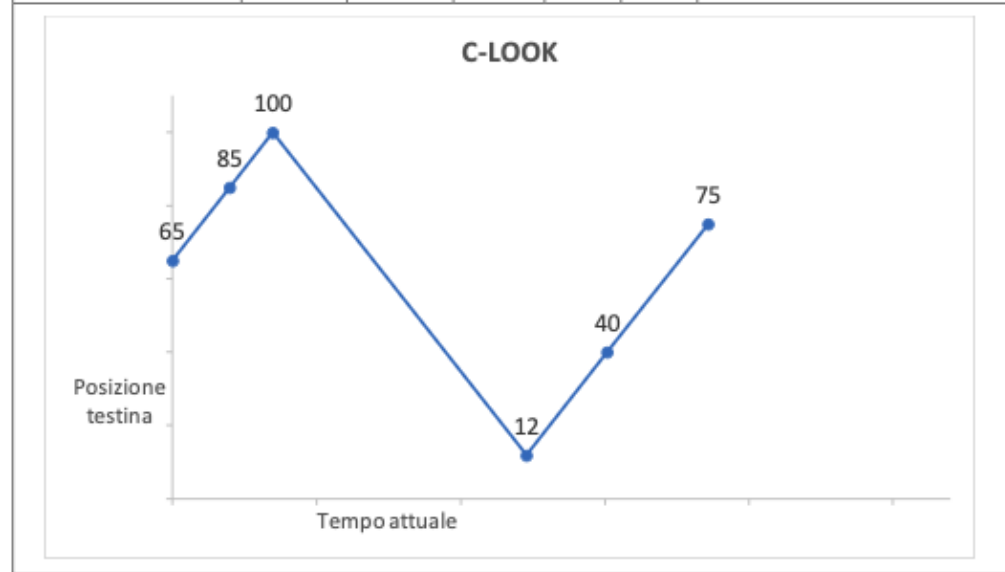


La testina procede verso i numeri di traccia più alti fino al raggiungimento dell'ultima richiesta "più alta", poi torna indietro. Al tempo 160 la testina si trova sulla traccia 65 e la richiesta che viene servita è la 2, quindi la testina si sposta da 65 a 85 con un tempo di ricerca $85 - 65 = 20$, di conseguenza allo step successivo saranno passati quindi $160 + 20 = 180ms$. La seconda richiesta ad essere servita è la 4, quindi la testina si sposta da 85 a 100 con tempo di ricerca $100 - 85 = 15$ e il prossimo tempo sarà di $180 + 15 = 195ms$. La terza richiesta ad essere servita è la 5 in quanto la testina torna indietro, quindi la testina si sposta da 100 a 75 con tempo di ricerca $100 - 75 = 25$ e prossimo tempo di $195 + 25 = 220ms$. Così via fino alla fine. Il tempo di ricerca totale si ottiene sommando i vari tempi di ricerca e indica quante volte la testina si è spostata.

C-LOOK

Parametri	$T_{const} = 0 - T_{pt} = 1$ - traccia 65 - tempo attuale 160ms				
Numero seriale	1	2	3	4	5
Numero traccia	12	85	40	100	75
Tempo di arrivo	65	80	110	120	175

Dettagli	Decisioni di scheduling					Totale tempo di ricerca
	1	2	3	4	5	
Tempo attuale	160	180	195	283	311	186
Richieste in sospeso	1, 2, 3, 4	1, 3, 4, 5	1, 3, 5	3, 5	5	
Posizione testina	65	85	100	12	40	
Richiesta selezionata	2	4	1	3	5	
Tempo di ricerca	20	15	88	28	35	



Stesso ragionamento di tutti gli altri ma la testina non effettua richieste mentre torna indietro. Osservando il grafico si nota che il punto più basso si ha proprio quando la testina si riposiziona all'inizio.